Mục lục

0 Lời nói đầu 6

1 Các khái niệm cơ bản 8

1.1 Khái niệm học máy 8

1.2 Dữ liệu 10

1.3 Các bài toán cơ bản trong học máy 11

1.4 Phân nhóm các thuật toán học máy 18

1.5 Tham số mô hình và quá trình tối ưu hàm mất mát 25

1.6 Những thách thức chính của học máy 28

1.7 Sự đánh đổi giữa thiên lệch và phương sai 30

1.8 Quy trình thực hiện một dự án học máy 34

2 Các kỹ thuật xây dựng đặc trưng 37

2.1 Giới thiệu 37

2.2 Mô hình chung cho các bài toán học máy 38

2.3 Một số kỹ thuật trích chọn đặc trưng 41

2.4 Chuẩn hóa vector đặc trưng 51

2.5 Kết luận 55

2.6 Câu hỏi ôn tập 56

3 Gradient Descent 58

3.1 Giới thiệu 58

3.2 GD cho hàm một biến 60

3.3 Hạ gradient cho hàm nhiều biến 64

3.4 Các biến thể của Gradient Descent 67

3.5 Kết luận 70

3.6 Câu hỏi ôn tập 70

4 Hồi quy tuyến tính 72

4.1 Phát biểu bài toán 72

4.2 Xây dựng và tối ưu hàm mất mát 73

4.3 Ví dụ trên Python 77

4.4 Thảo luận 80

4.5 Hồi quy Ridge và Lasso 82

4.6 Kết luận 85

4.7 Câu hỏi ôn tập 85

5 *K* láng giềng gần nhất 87

5.1 Giới thiệu 87

5.2 Phân tích toán học 89

5.3 Thực hành phân loại hoa IRIS 93

5.4 Ưu điểm và hạn chế của thuật toán KNN 98

5.5 Kết luận 100

5.6 Câu hỏi ôn tập 100

6 Bộ phân loại Naïve Bayes 102

6.1 Giới thiệu 102
6.2 Bài toán phân loại trong Naïve Bayes 105
6.3 Thực hành 112
6.4 Kết luận 115
6.5 Câu hỏi ôn tập 115

7 Thuật toán học Perceptron 118
7.1 Giới thiệu 118
7.2 Thuật toán học perceptron 119
7.3 Ví dụ và minh hoạ trên Python 122
7.4 Mô hình mạng neuron đầu tiên 125
7.5 Kết luận 127
7.6 Câu hỏi ôn tập 127

8 Cây quyết định 129
8.1 Khái niệm 130
8.2 Khái niệm về độ vẩn đục và độ tinh khiết 131
8.3 Các thuật toán xây dựng cây quyết định 135
8.4 Ví dụ xây dựng cây quyết định bằng Entropy và Information Gain 139
8.5 Thực hành: Xây dựng mô hình cây quyết định bằng Python 143
8.6 Kết luận 148
8.7 Câu hỏi ôn tập 149

9 Hồi quy logistic 151

9.1 Giới thiệu 151

9.2 Hàm mất mát và phương pháp tối ưu 153

9.3 Triển khai thuật toán trên Python 156

9.4 Tính chất của hồi quy logistic 159

9.5 Bài toán phân biệt hai chữ số viết tay 161

9.6 Bài toán phân loại đa lớp 162

9.7 Thảo luận 164

9.8 Kết luận 168

9.9 Câu hỏi ôn tập 168

10 Máy vector hỗ trợ - SVM 170

10.1 Mô hình máy vector hỗ trợ 170

10.2 Thực hành: Xây dựng bộ phân loại SVM trên Python 178

10.3 Kết luận 180

10.4 Câu hỏi ôn tập 181

11 Phân cụm *K*-means 183

11.1 Giới thiệu 183

11.2 Phân tích toán học 185

11.3 Lựa chọn số lượng cụm tối ưu 187

11.4 Thực hành với Python 190

11.5 Thực hành phân vùng ảnh bằng thuật toán K-means 195

Bảng 0.1: BẢNG CÁC KÝ HIỆU SỬ DỤNG TRONG GIÁO TRÌNH

Ký hiệu	Ý nghĩa
x, y, N, k	in nghiêng, thường hoặc hoa, là các số vô hướng
$\mathbf{x}, \mathbf{y}$	in đậm, chữ thường, là các vector
$\mathbf{X}, \mathbf{Y}$	in đậm, chữ hoa, là các ma trận
$\mathbb{R}$	tập hợp các số thực
$\mathbb{N}$	tập hợp các số tự nhiên
$\mathbb{C}$	tập hợp các số phức
$\mathbb{R}^m$	tập hợp các vector thực có m phần tử
$\mathbb{R}^{m\times n}$	tập hợp các ma trận thực có m hàng, n cột
$\mathbb{S}^n$	tập hợp các ma trận vuông đối xứng bậc n
$\mathbb{S}^n_+$	tập hợp các ma trận nửa xác định dương bậc n
$\mathbb{S}^n_{++}$	tập hợp các ma trận xác định dương bậc n
$\in$	phần tử thuộc tập hợp
$\exists$	tồn tại
$\forall$	mọi
$\triangleq$	ký hiệu là/bởi. Ví dụ $a \triangleq f(x)$ nghĩa là "ký hiệu $f(x)$ bởi a".
x_i	phần tử thứ i (tính từ 1) của vector $\mathbf{x}$
$\text{sgn}(x)$	hàm xác định dấu. Bằng 1 nếu $x \geq 0$, bằng -1 nếu $x < 0$.
$\exp(x)$	e^x
$\log(x)$	logarit *tự nhiên* của số thực dương x
$\underset{x}{\text{argmin}}\, f(x)$	giá trị của x để hàm $f(x)$ đạt giá trị nhỏ nhất
$\underset{x}{\text{argmax}}\, f(x)$	giá trị của x để hàm $f(x)$ đạt giá trị lớn nhất
a_{ij}	phần tử hàng thứ i, cột thứ j của ma trận $\mathbf{A}$
$\mathbf{A}^T$	chuyển vị của ma trận $\mathbf{A}$
$\mathbf{A}^H$	chuyển vị liên hợp (Hermitian) của ma trận phức $\mathbf{A}$
$\mathbf{A}^{-1}$	nghịch đảo của ma trận vuông $\mathbf{A}$, nếu tồn tại
$\mathbf{A}^\dagger$	giả nghịch đảo của ma trận không nhất thiết vuông $\mathbf{A}$
$\mathbf{A}^{-T}$	chuyển vị của nghịch đảo của ma trận $\mathbf{A}$, nếu tồn tại
$\|\mathbf{x}\|_p$	ℓ_p norm của vector $\mathbf{x}$
$\|\mathbf{A}\|_F$	Frobenius norm của ma trận $\mathbf{A}$
$\text{diag}(\mathbf{A})$	đường chéo chính của ma trận $\mathbf{A}$
$\text{trace}(\mathbf{A})$	trace của ma trận $\mathbf{A}$
$\det(\mathbf{A})$	định thức của ma trận vuông $\mathbf{A}$
$\text{rank}(\mathbf{A})$	hạng của ma trận $\mathbf{A}$
o.w	*otherwise* – trong các trường hợp còn lại
$\frac{\partial f}{\partial x}$	đạo hàm của hàm số f theo $x \in \mathbb{R}$
$\nabla_{\mathbf{x}} f$	gradient của hàm số f theo $\mathbf{x}$ ($\mathbf{x}$ là vector hoặc ma trận)
$\nabla^2_{\mathbf{x}} f$	gradient bậc hai của hàm số f theo $\mathbf{x}$, còn được gọi là *Hesse*
$\odot$	Hadamard product (elemenwise product). Phép nhân từng phần tử của hai vector hoặc ma trận cùng kích thước.
$\propto$	tỉ lệ với
———	đường nét liền
- - - - -	đường nét đứt
.....	đường nét chấm (đường chấm chấm)
—·—·—	đường chấm gạch
	nền chấm
	nền sọc chéo

Chương 0

Lời nói đầu

Trong bối cảnh chuyển đổi số và sự phát triển mạnh mẽ của Trí tuệ nhân tạo (AI), Học máy (Machine Learning) đã trở thành một trong những lĩnh vực cốt lõi, giữ vai trò nền tảng cho nhiều hướng nghiên cứu và ứng dụng hiện đại. Từ học sâu (Deep Learning), xử lý ngôn ngữ tự nhiên (Natural Language Processing), thị giác máy tính (Computer Vision) cho đến trí tuệ nhân tạo tạo sinh (Generative AI), tất cả đều được xây dựng dựa trên các nguyên lý và mô hình cơ bản của học máy. Vì vậy, việc nắm vững kiến thức nền tảng về học máy không chỉ có ý nghĩa học thuật mà còn mang tính định hướng lâu dài cho quá trình nghiên cứu và phát triển công nghệ.

Trong kỷ nguyên Dữ liệu lớn (Big Data), khối lượng dữ liệu được tạo ra mỗi ngày tăng với tốc độ rất lớn, đòi hỏi những phương pháp phân tích thông minh và có khả năng thích nghi cao. Học máy cung cấp cơ chế để hệ thống có thể tự động học từ dữ liệu, phát hiện các quy luật tiềm ẩn, xây dựng mô hình dự đoán và tối ưu hóa quyết định mà không cần lập trình tường minh cho từng tình huống cụ thể. Nhờ đặc tính này, học máy đã vượt ra khỏi phạm vi nghiên cứu thuần túy và trở thành công cụ thiết yếu trong nhiều lĩnh vực như công nghiệp, tài chính, y tế, giáo dục và quản trị.

Không chỉ dừng lại ở khả năng dự đoán, học máy còn góp phần hình thành tư duy phân tích dựa trên dữ liệu (data-driven approach), giúp người học hiểu rõ bản chất của quá trình mô hình hóa, cách đo lường sai số thông qua hàm mất mát (loss function), kiểm soát độ phức tạp của mô hình và đảm bảo khả năng tổng quát hóa (generalization) trên dữ liệu mới. Do đó, học máy không chỉ là một môn học kỹ thuật, mà còn là nền tảng phương pháp luận quan trọng cho việc phát triển các hệ thống AI hiện đại.

Cuốn giáo trình này được biên soạn với mục tiêu cung cấp cho người học một nền tảng lý thuyết vững chắc và có hệ thống về học máy. Nội dung giáo trình tập

trung trình bày các mô hình học máy cơ bản như hồi quy tuyến tính, hồi quy logistic, phân loại và phân cụm; đồng thời làm rõ các khái niệm trọng tâm như hàm mất mát, tối ưu hóa, đánh giá mô hình, hiện tượng quá khớp (overfitting) và thiếu khớp (underfitting). Các nội dung được trình bày theo lộ trình từ trực quan đến biểu diễn toán học, giúp người học hiểu sâu bản chất thay vì chỉ dừng lại ở thao tác công cụ.

Bên cạnh phần lý thuyết, giáo trình chú trọng đến tính ứng dụng thông qua các bài toán thực tiễn được thiết kế xuyên suốt trong từng chương. Mỗi nội dung đều đi kèm ví dụ minh họa và mã nguồn Python thực hành, giúp sinh viên và người đọc có thể triển khai, kiểm nghiệm và điều chỉnh mô hình trên dữ liệu thực tế. Thông qua quá trình thực hành này, người học có thể nắm được toàn bộ quy trình xây dựng một dự án học máy hoàn chỉnh, từ xử lý dữ liệu, xây dựng mô hình, huấn luyện, đánh giá cho đến phân tích và diễn giải kết quả.

Một số ứng dụng tiêu biểu của Học máy hiện nay có thể khái quát như sau:

- **Phân tích và dự đoán dữ liệu trong kinh tế, tài chính, thương mại điện tử và quản trị doanh nghiệp.**
- **Nhận dạng hình ảnh, tiếng nói và xử lý ngôn ngữ trong các hệ thống thông minh.**
- **Hỗ trợ ra quyết định và tự động hóa trong y tế, giao thông thông minh và công nghiệp 4.0.**

Giáo trình được xây dựng hướng tới sinh viên ngành Công nghệ Thông tin, Khoa học Máy tính và các lĩnh vực liên quan, đồng thời cũng phù hợp với những độc giả mong muốn tiếp cận học máy một cách bài bản và có hệ thống. Nhóm biên soạn đã nỗ lực trình bày nội dung với tinh thần khoa học, chính xác và cập nhật; tuy nhiên khó tránh khỏi những hạn chế nhất định. Chúng tôi rất mong nhận được các ý kiến đóng góp từ quý độc giả và đồng nghiệp để giáo trình ngày càng được hoàn thiện hơn trong các lần tái bản tiếp theo.

Thái Nguyên, tháng 02 năm 2026
Nhóm biên soạn

Chương 1

Các khái niệm cơ bản

1.1. Khái niệm học máy

Học máy là một môn khoa học nghiên cứu cách xây dựng và lập trình các hệ thống máy tính để chúng có khả năng học từ dữ liệu. Thay vì lập trình tường minh mọi quy tắc xử lý, học máy cho phép hệ thống tự suy ra mô hình và quy luật thông qua quá trình phân tích dữ liệu quan sát được.

Hay theo một số định nghĩa mang tính kỹ thuật hơn:

Khái niệm 1

Học máy là lĩnh vực nghiên cứu nhằm giúp máy tính có khả năng học mà không cần lập trình một cách tường minh.

Khái niệm 2

Học máy là quá trình xây dựng các hệ thống có khả năng tự động cải thiện hiệu năng của mình thông qua việc tích lũy và khai thác kinh nghiệm từ dữ liệu theo thời gian.

Khái niệm 3

Một chương trình máy tính được gọi là "học tập" từ kinh nghiệm E để hoàn thành nhiệm vụ T với hiệu quả được đo bằng phép đánh giá P, nếu hiệu quả của nó khi thực hiện nhiệm vụ T, khi được đánh giá bởi P, cải thiện theo kinh nghiệm E.

Định nghĩa trên làm rõ ba thành phần cơ bản của một hệ thống học máy: *nhiệm vụ* T mà hệ thống cần thực hiện; *phép đánh giá* P dùng để đo lường mức độ hoàn thành nhiệm vụ; và *kinh nghiệm* E, chính là dữ liệu mà hệ thống sử dụng để học. Việc xác định đúng ba yếu tố này có ý nghĩa quyết định trong quá trình thiết kế và xây dựng mô hình.

Nói chung, một thuật toán học máy là một thuật toán có khả năng học tập từ dữ liệu. Để xây dựng một chương trình máy tính có khả năng học, ta cần xác định rõ ba yếu tố: nhiệm vụ, phép đánh giá và đặc biệt là kinh nghiệm E, tức nguồn dữ liệu huấn luyện. Nếu không có dữ liệu và thuật toán học thì không thể xây dựng một chương trình học máy.

Về mặt bản chất, học máy có thể được xem là quá trình tìm kiếm một mô hình hoặc một hàm ánh xạ từ không gian đầu vào $\mathcal{X}$ sang không gian đầu ra $\mathcal{Y}$ sao cho sai số dự đoán được giảm thiểu.

Giả sử ta có tập dữ liệu huấn luyện:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}, \tag{1.1}$$

trong đó x_i là dữ liệu đầu vào và y_i là giá trị thực tế tương ứng. Ký hiệu mô hình cần học là $f_\theta(x)$ với θ là tập tham số. Khi đó, bài toán học máy có thể được phát biểu dưới dạng:

$$\theta^* = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\big(y_i, f_\theta(x_i)\big), \tag{1.2}$$

trong đó $\mathcal{L}$ là hàm mất mát đo lường mức độ sai lệch giữa giá trị dự đoán và giá trị thực tế. Công thức (1.2) cho thấy học máy về bản chất là một bài toán tối ưu hóa tham số mô hình.

Ví dụ minh họa trong thực tế: Giả sử ta muốn xây dựng một hệ thống dự đoán doanh thu bán hàng của một cửa hàng dựa trên dữ liệu lịch sử như số lượng khách mỗi ngày và chi phí quảng cáo. Mỗi ngày quan sát được tạo thành một cặp dữ liệu (x_i, y_i), trong đó x_i chứa thông tin đầu vào (khách, quảng cáo) và y_i là doanh thu thực tế.

Ban đầu, mô hình dự đoán có thể sai lệch đáng kể so với doanh thu thật. Thông qua việc điều chỉnh các tham số θ nhằm làm giảm giá trị trung bình của hàm mất mát trong công thức (1.2), sai số dự đoán dần được thu hẹp. Khi giá trị này đạt mức nhỏ nhất, ta thu được mô hình có khả năng dự đoán phù hợp nhất với dữ liệu đã quan sát.

1.2. Dữ liệu

Các *nhiệm vụ* trong học máy được mô tả thông qua cách một hệ thống xử lý một điểm dữ liệu đầu vào để tạo ra đầu ra mong muốn. Về mặt toán học, một mô hình học máy có thể được xem như một hàm

$$f_\theta : \mathcal{X} \to \mathcal{Y}, \tag{1.3}$$

trong đó $\mathcal{X}$ là không gian dữ liệu đầu vào, $\mathcal{Y}$ là không gian đầu ra, và θ là tập các tham số của mô hình. Đầu vào của hàm là dữ liệu quan sát được, và đầu ra là thông tin hoặc giá trị mà ta mong muốn suy ra từ dữ liệu đó.

Một điểm dữ liệu có thể là một bức ảnh, một đoạn âm thanh, một văn bản, hoặc một tập các hành vi của người dùng trên Internet. Tuy nhiên, để chương trình máy tính có thể xử lý và học được, các điểm dữ liệu thường được biểu diễn dưới dạng tập hợp các con số, trong đó mỗi con số được gọi là một *đặc trưng*.

Do đó, về mặt tính toán, mọi loại dữ liệu đều có thể được biểu diễn dưới dạng *vector đặc trưng*, ma trận hoặc tensor. Vector đặc trưng của một điểm dữ liệu thường được ký hiệu là

$$\mathbf{x} = (x_1, x_2, \ldots, x_d)^\top \in \mathbb{R}^d, \tag{1.4}$$

trong đó d là số lượng đặc trưng. Các mảng nhiều chiều về bản chất có thể được xem là đã được *vector hoá* thành mảng một chiều để thuận tiện cho việc biểu diễn và xử lý bằng các phép toán đại số tuyến tính. Kỹ thuật xây dựng vector đặc trưng cho dữ liệu sẽ được trình bày cụ thể hơn trong Chương 2.

Kinh nghiệm trong học máy là bộ dữ liệu được sử dụng để xây dựng mô hình. Giả sử ta có tập dữ liệu tổng thể

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \tag{1.5}$$

trong đó $\mathbf{x}_i$ là vector đặc trưng và y_i là đầu ra tương ứng. Trong thực tế, để đánh giá và điều chỉnh mô hình một cách khách quan, bộ dữ liệu thường được chia thành ba tập con không giao nhau: tập huấn luyện $\mathcal{D}_{\text{train}}$, tập xác thực $\mathcal{D}_{\text{val}}$ và tập kiểm tra $\mathcal{D}_{\text{test}}$, sao cho:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}, \quad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{val}} \cap \mathcal{D}_{\text{test}} = \varnothing. \tag{1.6}$$

Tập huấn luyện được sử dụng để ước lượng các tham số θ của mô hình, tập xác thực được dùng để điều chỉnh các siêu tham số và lựa chọn mô hình phù hợp, trong khi tập kiểm tra đóng vai trò đánh giá cuối cùng nhằm ước lượng khả năng tổng quát hóa của mô hình trên dữ liệu chưa từng quan sát.

Tập huấn luyện (training set) bao gồm các điểm dữ liệu được sử dụng trực tiếp trong việc xây dựng mô hình, tức là để ước lượng các tham số θ thông qua việc tối thiểu hóa hàm mất mát trên tập dữ liệu này.

Tập kiểm tra (test set) gồm các dữ liệu được dùng để đánh giá hiệu quả cuối cùng của mô hình. Để đảm bảo tính khách quan và khả năng tổng quát hóa, dữ liệu kiểm tra tuyệt đối không được sử dụng trong quá trình xây dựng hay điều chỉnh mô hình. Điều kiện cần để một mô hình được xem là hiệu quả là kết quả đánh giá trên cả tập huấn luyện và tập kiểm tra đều đạt giá trị cao và không chênh lệch quá lớn. Tập kiểm tra đại diện cho dữ liệu mà mô hình chưa từng quan sát trước đó, tương ứng với các tình huống có thể xuất hiện khi triển khai mô hình trong thực tế.

Để tăng hiệu quả của mô hình trên dữ liệu kiểm tra, người ta thường sử dụng thêm một tập dữ liệu trung gian được gọi là *tập xác thực* (validation set). Tập xác thực được dùng trong quá trình lựa chọn mô hình và điều chỉnh các siêu tham số, chẳng hạn như tốc độ học, độ phức tạp của mô hình hoặc các tham số điều chuẩn.

Quy trình xây dựng mô hình khi đó có thể được mô tả như sau: trước hết, mô hình được huấn luyện trên tập huấn luyện; tiếp theo, hiệu năng của mô hình được đánh giá trên tập xác thực để lựa chọn cấu hình tối ưu; cuối cùng, mô hình đã được lựa chọn sẽ được đánh giá một lần duy nhất trên tập kiểm tra để ước lượng khả năng tổng quát hóa thực sự.

Việc phân chia dữ liệu thành ba tập riêng biệt giúp đảm bảo rằng quá trình xây dựng mô hình không vô tình khai thác thông tin từ dữ liệu kiểm tra, qua đó duy trì tính khách quan và độ tin cậy của kết quả đánh giá.

1.3. Các bài toán cơ bản trong học máy

Nhiều bài toán phức tạp trong thực tiễn có thể được giải quyết hiệu quả bằng các phương pháp học máy. Những bài toán này thường có đặc điểm chung là dữ liệu đầu vào có kích thước lớn, cấu trúc phức tạp hoặc mối quan hệ giữa các biến không thể mô tả chính xác bằng các quy tắc lập trình tường minh.

Thay vì xây dựng các tập luật cố định, học máy cho phép mô hình tự động học ra quy luật từ dữ liệu quan sát được, từ đó đưa ra dự đoán, phân loại hoặc hỗ trợ ra quyết định. Chính nhờ khả năng thích nghi với dữ liệu và tự cải thiện hiệu năng theo thời gian, học máy đặc biệt phù hợp với các bài toán mà lời giải phụ thuộc mạnh vào mẫu dữ liệu thực tế.

Dưới đây là một số dạng bài toán phổ biến trong học máy:

1.3.1. Bài toán phân lớp

Phân lớp (classification) là một trong những bài toán trung tâm và được nghiên cứu nhiều nhất trong học máy. Trong bài toán này, hệ thống được yêu cầu xác định *nhãn* của một điểm dữ liệu trong số C nhãn khả dĩ.

Mỗi điểm dữ liệu được biểu diễn dưới dạng cặp $(\mathbf{x}, \mathbf{y})$, trong đó:

$$\mathbf{x} \in \mathbb{R}^d \tag{1.7}$$

là vector đặc trưng và

$$\mathbf{y} \in \mathcal{Y} \tag{1.8}$$

là nhãn tương ứng với:

$$\mathcal{Y} = \{1, 2, \ldots, C\}. \tag{1.9}$$

Việc xây dựng mô hình phân lớp tương đương với việc tìm một hàm ánh xạ:

$$\mathbf{f} : \mathbb{R}^d \to \mathcal{Y} \tag{1.10}$$

sao cho:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}). \tag{1.11}$$

Hàm $\mathbf{f}$ được học từ dữ liệu huấn luyện và sau đó được áp dụng lên các dữ liệu mới nhằm xác định nhãn với sai số chấp nhận được.

Ví dụ 1: Phát hiện email rác

Bài toán xác định email rác là một ví dụ của phân lớp nhị phân với:

$$C = 2. \tag{1.12}$$

Trong đó:

- Nhiệm vụ T là xác định một email mới có phải là "rác" hay không.
- Phép đánh giá P có thể là tỉ lệ dự đoán đúng trên tập kiểm tra.
- Kinh nghiệm E là tập các cặp (email, nhãn) đã được gán nhãn trước đó để huấn luyện mô hình và xây dựng hàm $\mathbf{f}(\mathbf{x})$.

Phân lớp xác suất

Trong nhiều trường hợp, mô hình không xuất trực tiếp một nhãn rời rạc mà xuất ra một vector xác suất như sau:

$$\mathbf{p}(\mathbf{x}) = (p_1, p_2, \ldots, p_C). \tag{1.13}$$

Trong đó:

$$p_c = \mathbb{P}(\mathbf{y} = c \mid \mathbf{x}) \tag{1.14}$$

và:

$$\sum_{c=1}^{C} p_c = 1. \tag{1.15}$$

Nhãn dự đoán được xác định như sau:

$$\hat{\mathbf{y}} = \arg\max_{c \in \mathcal{Y}} p_c. \tag{1.16}$$

Biểu diễn xác suất cho phép đánh giá mức độ tin cậy của mô hình. Nếu xác suất lớn nhất vượt trội so với các xác suất còn lại, mô hình được xem là có độ chắc chắn cao. Ngược lại, nếu các xác suất gần nhau, khả năng phân loại nhầm sẽ cao hơn.

1.3.2. Hồi quy

Nếu tập đích $\mathcal{Y}$ gồm các giá trị là các số thực thì bài toán được gọi là *hồi quy*. Trong bài toán này, mục tiêu là xây dựng một hàm số ánh xạ từ không gian đặc trưng sang tập các giá trị thực.

Cụ thể, ta cần tìm một hàm:

$$\mathbf{f} : \mathbb{R}^d \to \mathbb{R} \tag{1.17}$$

sao cho:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}), \tag{1.18}$$

trong đó $\mathbf{x} \in \mathbb{R}^d$ là vector đặc trưng và $\mathbf{y} \in \mathbb{R}$ là giá trị đầu ra mong muốn dự đoán.

Khác với bài toán phân lớp, đầu ra của hồi quy là một đại lượng liên tục. Do đó, mô hình không đưa ra một nhãn rời rạc mà dự đoán một giá trị số phản ánh mức độ hoặc cường độ của một đại lượng quan tâm.

Ví dụ: Xét bài toán ước lượng giá của một căn nhà dựa trên các đặc trưng như diện tích x (m^2), số phòng ngủ y và khoảng cách đến trung tâm thành phố z (km). Khi đó, mỗi điểm dữ liệu có thể được biểu diễn dưới dạng:

$$\mathbf{x} = (x, y, z)^\top \in \mathbb{R}^3. \tag{1.19}$$

Ta mong muốn xây dựng một hàm dự đoán như sau:

$$\mathbf{f} : \mathbb{R}^3 \to \mathbb{R} \tag{1.20}$$

sao cho:

$$\text{Giá} = \mathbf{f}(x, y, z). \tag{1.21}$$

Bài toán hồi quy có thể được mở rộng sang trường hợp dự đoán đồng thời nhiều đại lượng đầu ra. Khi đó, hàm cần tìm có dạng:

$$\mathbf{f} : \mathbb{R}^d \to \mathbb{R}^m, \tag{1.22}$$

trong đó m là số lượng biến đầu ra.

Ví dụ, trong bài toán tạo ảnh độ phân giải cao từ một ảnh có độ phân giải thấp, mô hình cần dự đoán giá trị cường độ của nhiều điểm ảnh cùng một lúc. Khi đó, đầu ra là một vector hoặc một tensor nhiều chiều, và bài toán được xem là hồi quy nhiều đầu ra.

1.3.3. Máy dịch

Trong bài toán *máy dịch*, chương trình máy tính được yêu cầu dịch một đoạn văn từ một ngôn ngữ nguồn sang một ngôn ngữ đích. Về bản chất, đây là một bài toán học ánh xạ giữa hai không gian biểu diễn ngôn ngữ.

Ta có thể mô hình hóa bài toán như sau:

$$\mathbf{f} : \mathcal{X} \to \mathcal{Y}, \tag{1.23}$$

trong đó:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}), \tag{1.24}$$

với $\mathbf{x}$ là câu hoặc đoạn văn trong ngôn ngữ nguồn và $\mathbf{y}$ là bản dịch tương ứng trong ngôn ngữ đích.

Dữ liệu huấn luyện là tập các cặp văn bản song ngữ:

$$\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{n}, \tag{1.25}$$

trong đó mỗi cặp $(\mathbf{x}_i, \mathbf{y}_i)$ gồm một câu gốc và bản dịch tương ứng. Các văn bản này có thể chỉ gồm hai ngôn ngữ đang xét hoặc có thêm các ngôn ngữ trung gian nhằm cải thiện chất lượng dịch.

Ví dụ 1: Hệ thống dịch thông báo tại Trường Đại học Công nghệ Thông tin và Truyền thông Thái Nguyên (ICTU)

Giả sử tại ICTU muốn xây dựng một hệ thống tự động dịch các thông báo từ tiếng Việt sang tiếng Anh để phục vụ sinh viên quốc tế.

Một mẫu dữ liệu huấn luyện có thể là:

$$\mathbf{x} = \text{“Sinh viên ICTU nộp học phí trước ngày 15/09.”} \tag{1.26}$$

và:

$$\mathbf{y} = \text{“ICTU students must pay tuition before September 15.”} \tag{1.27}$$

Sau khi huấn luyện, mô hình học được hàm:

$$\mathbf{f} : \mathcal{X} \to \mathcal{Y} \tag{1.28}$$

sao cho:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}). \tag{1.29}$$

Nếu ICTU đăng một thông báo mới như:

$$\mathbf{x}_{\text{new}} = \text{“Lịch thi học kỳ sẽ được công bố vào tuần tới.”} \tag{1.30}$$

Hệ thống có thể dự đoán bản dịch:

$$\hat{\mathbf{y}} = \text{“The semester exam schedule will be announced next week.”} \tag{1.31}$$

Ví dụ này cho thấy hệ thống máy dịch phải học được cấu trúc câu, ngữ cảnh và các thuật ngữ chuyên ngành để tạo ra bản dịch tự nhiên và chính xác.

Ví dụ 2: Dịch trong ứng dụng thực tế

Trong các ứng dụng như Google Translate hoặc các ứng dụng học ngoại ngữ, người dùng nhập một đoạn văn tiếng Việt:

$$\mathbf{x} = \text{“Hôm nay trời rất đẹp.”} \tag{1.32}$$

Hệ thống sẽ xuất ra bản dịch tiếng Anh:

$$\mathbf{y} = \text{“The weather is very nice today.”} \tag{1.33}$$

Ở đây, mô hình không chỉ dịch từng từ riêng lẻ mà còn phải hiểu ngữ cảnh và cấu trúc câu để tạo ra bản dịch tự nhiên và đúng ngữ pháp. Điều này cho thấy bài toán máy dịch là một bài toán phức tạp, trong đó đầu ra là một chuỗi ký tự có độ dài biến đổi và phụ thuộc mạnh vào ngữ cảnh của đầu vào.

Ví dụ Google translate là một ứng dụng (tất nhiên có nhiều kỹ thuật học sâu khác được áp dụng) học máy trong lĩnh vực máy dịch.

1.3.4. Phân cụm

Phân cụm (clustering) là bài toán chia tập dữ liệu $\mathcal{X}$ thành các nhóm nhỏ hơn dựa trên mức độ tương đồng giữa các điểm dữ liệu. Khác với bài toán phân lớp,

trong phân cụm dữ liệu huấn luyện không có nhãn. Mô hình phải tự động phát hiện cấu trúc tiềm ẩn và chia dữ liệu thành các cụm sao cho các điểm trong cùng một cụm có mức độ tương đồng cao hơn so với các điểm thuộc cụm khác.

Giả sử tập dữ liệu gồm n điểm:

$$\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}, \tag{1.34}$$

mục tiêu của phân cụm là tìm một phép gán nhãn cụm:

$$\mathbf{g} : \mathcal{X} \to \{1, 2, \ldots, K\}, \tag{1.35}$$

trong đó K là số lượng cụm cần chia.

Ví dụ: Phân cụm khách hàng trong thương mại điện tử

Xét một trang web thương mại điện tử theo dõi hành vi của người dùng như số lần xem sản phẩm, loại sản phẩm quan tâm, giá trị đơn hàng trung bình hoặc tần suất mua sắm. Mỗi người dùng được biểu diễn bởi một vector đặc trưng $\mathbf{x}$ phản ánh hành vi mua hàng của họ.

Sau khi áp dụng thuật toán phân cụm, hệ thống có thể chia người dùng thành các nhóm như:

- Nhóm khách hàng thường xuyên mua sản phẩm công nghệ.
- Nhóm khách hàng quan tâm đến thời trang và mỹ phẩm.
- Nhóm khách hàng chỉ mua khi có khuyến mãi.

Việc phân cụm này giúp doanh nghiệp hiểu rõ hơn về cấu trúc tập khách hàng mà không cần nhãn có sẵn. Từ đó, hệ thống có thể đề xuất sản phẩm hoặc quảng cáo phù hợp với từng nhóm người dùng dựa trên hành vi chung của các thành viên trong cùng cụm.

Bài toán phân cụm vì vậy đóng vai trò quan trọng trong khai phá dữ liệu, phân tích hành vi người dùng và hỗ trợ ra quyết định trong nhiều lĩnh vực thực tế.

1.3.5. Hoàn thiện dữ liệu

Hoàn thiện dữ liệu là bài toán dự đoán các trường dữ liệu còn thiếu trong dữ liệu đích. Nhiệm vụ của bài toán này là khai thác mối tương quan giữa các điểm dữ liệu để ước lượng những giá trị chưa được quan sát.

Giả sử ta có một tập dữ liệu được biểu diễn dưới dạng ma trận:

$$\mathbf{R} \in \mathbb{R}^{m \times n}, \tag{1.36}$$

trong đó phần tử R_{ij} biểu thị mức độ tương tác giữa đối tượng thứ i và mục thứ j (ví dụ: số sao đánh giá). Một số phần tử của $\mathbf{R}$ chưa được biết và cần được dự đoán.

Bài toán hoàn thiện dữ liệu có thể được mô tả như sau:

$$\hat{R}_{ij} = \mathbf{f}(i, j), \tag{1.37}$$

trong đó $\hat{R}_{ij}$ là giá trị ước lượng thay cho phần tử còn thiếu.

Ví dụ: Hệ thống khuyến nghị

Xét một hệ thống đánh giá phim với 3 người dùng và 3 bộ phim. Ma trận đánh giá có dạng:

$$\mathbf{R} = \begin{pmatrix} 5\ 4\ ? \\ 4\ ?\ 2 \\ ?\ 3\ 1 \end{pmatrix}. \tag{1.38}$$

Dấu "?" biểu thị các giá trị còn thiếu. Hệ thống cần dự đoán các giá trị này dựa trên mối tương quan giữa các người dùng hoặc giữa các bộ phim.

Ví dụ, nếu hai người dùng có xu hướng đánh giá giống nhau ở nhiều bộ phim, hệ thống có thể ước lượng rằng họ cũng sẽ có đánh giá tương tự cho bộ phim chưa xem. Khi đó, giá trị dự đoán có thể được xác định như sau:

$$\hat{R}_{ij} = \frac{\sum_{k \in \mathcal{N}(i)} w_{ik} R_{kj}}{\sum_{k \in \mathcal{N}(i)} w_{ik}}, \tag{1.39}$$

trong đó $\mathcal{N}(i)$ là tập các người dùng có hành vi tương tự người dùng i và w_{ik} là hệ số thể hiện mức độ tương đồng giữa hai người dùng.

Các ứng dụng khác

Ngoài hệ thống khuyến nghị, hoàn thiện dữ liệu còn xuất hiện trong:

- Khôi phục ảnh cũ hoặc ảnh bị hỏng bằng cách dự đoán các điểm ảnh bị thiếu.
- Tô màu ảnh đen trắng dựa trên học từ các ảnh màu tương tự.

Bên cạnh đó, học máy còn bao gồm nhiều dạng bài toán khác như: *xếp hạng*, *giảm chiều dữ liệu*, *phát hiện bất thường*, tùy thuộc vào yêu cầu đầu ra, đặc điểm dữ liệu và khả năng giải quyết của thuật toán.

1.4. Phân nhóm các thuật toán học máy

Phân nhóm dựa theo phương thức học có các loại thuật toán học máy sau:

1.4.1. Học có giám sát

Một thuật toán học máy được gọi là *học có giám sát* nếu việc xây dựng mô hình được thực hiện dựa trên các cặp dữ liệu đầu vào và đầu ra đã biết trước. Nói cách khác, trong tập huấn luyện, mỗi điểm dữ liệu đều có nhãn tương ứng.

Giả sử tập huấn luyện gồm các cặp:

$$\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), \ldots, (\mathbf{x}_N, \mathbf{y}_N)\}, \tag{1.40}$$

trong đó $\mathbf{x}_i$ là dữ liệu đầu vào và $\mathbf{y}_i$ là đầu ra mong muốn.

Mục tiêu của học có giám sát là xây dựng một hàm dự đoán:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}), \tag{1.41}$$

sao cho với các dữ liệu trong tập huấn luyện, giá trị $\mathbf{f}(\mathbf{x}_i)$ gần với $\mathbf{y}_i$ nhất có thể. Sau khi huấn luyện, mô hình được sử dụng để dự đoán đầu ra cho các dữ liệu mới chưa từng xuất hiện trong tập huấn luyện.

Bản chất của quá trình huấn luyện là điều chỉnh các tham số của mô hình nhằm làm giảm sai số giữa giá trị dự đoán và giá trị thực tế. Nếu mô hình học tốt, nó không chỉ khớp với dữ liệu huấn luyện mà còn dự đoán chính xác trên dữ liệu mới.

Ví dụ 1 : Dạy em bé nhận biết chữ cái

Học có giám sát có thể được hiểu tương tự như quá trình dạy một em bé học bảng chữ cái.

Giả sử ta đưa cho em bé xem một hình ảnh chữ cái và nói rõ đó là chữ gì. Ví dụ:

- Hình ảnh chữ "A" → nói với bé: "Đây là chữ A".
- Hình ảnh chữ "B" → nói với bé: "Đây là chữ B".
- Hình ảnh chữ "C" → nói với bé: "Đây là chữ C".

Trong học máy, mỗi hình ảnh được xem như một đầu vào $\mathbf{x}$ và chữ cái tương ứng là nhãn $\mathbf{y}$.

Tập huấn luyện khi đó có dạng:

$$\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), \ldots, (\mathbf{x}_N, \mathbf{y}_N)\}, \tag{1.42}$$

trong đó mỗi $\mathbf{x}_i$ là một hình ảnh chữ cái và $\mathbf{y}_i \in \{A, B, C, \ldots\}$ là nhãn đúng đã biết.

Quá trình học diễn ra như sau: em bé nhìn chữ, đoán tên chữ; nếu đoán sai, người lớn sẽ sửa lại. Việc sửa sai này chính là cơ chế "giám sát".

Tương tự, trong học máy, mô hình đưa ra dự đoán:

$$\hat{\mathbf{y}} = \mathbf{f}(\mathbf{x}), \tag{1.43}$$

sau đó so sánh với nhãn đúng $\mathbf{y}$ để điều chỉnh tham số bên trong.

Sau khi đã được dạy nhiều lần với các ví dụ có nhãn, khi gặp một hình ảnh chữ mới mà chưa từng thấy, em bé có thể nhận ra đúng chữ đó. Đây chính là khả năng tổng quát hóa của học có giám sát.

Ví dụ 2 về học máy có giám sát

Giả sử ta muốn xây dựng một hệ thống dự đoán kết quả "Đạt" hay "Không đạt" của sinh viên ICTU trong một học phần dựa trên hai thông tin:

- x_1: số buổi đi học,
- x_2: điểm giữa kỳ.

Trong quá trình huấn luyện, ta đã có dữ liệu của các sinh viên khóa trước, với mỗi sinh viên đều biết rõ kết quả cuối cùng. Tập dữ liệu có dạng:

$$\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), \ldots, (\mathbf{x}_N, \mathbf{y}_N)\}, \tag{1.44}$$

trong đó:

- $\mathbf{x}_i = (x_{1i}, x_{2i})$ là thông tin đầu vào,
- $\mathbf{y}_i \in \{\text{Đạt}, \text{Không đạt}\}$ là kết quả thực tế đã biết.

Vì mỗi điểm dữ liệu đều đã có nhãn $\mathbf{y}_i$, mô hình có thể so sánh giữa dự đoán và kết quả thật để điều chỉnh tham số của mình.

Sau khi huấn luyện, với một sinh viên mới:

$$\mathbf{x}_{\text{new}} = (x_1, x_2), \tag{1.45}$$

mô hình sẽ dự đoán:

$$\hat{\mathbf{y}} = \mathbf{f}(\mathbf{x}_{\text{new}}). \tag{1.46}$$

Điểm mấu chốt ở đây là: trong quá trình học, mô hình luôn được "giám sát" bởi các nhãn đúng đã biết trước. Nếu dự đoán sai so với nhãn thực tế, mô hình sẽ điều chỉnh tham số để giảm sai số. Chính sự tồn tại của nhãn đầu ra trong tập huấn luyện làm nên bản chất của học có giám sát.

1.4.2. Học không giám sát

Một thuật toán học máy được gọi là *học không giám sát* nếu dữ liệu huấn luyện không chứa nhãn đầu ra. Nói cách khác, mô hình chỉ được cung cấp các điểm dữ liệu đầu vào và phải tự khám phá cấu trúc hoặc quy luật tiềm ẩn bên trong dữ liệu đó.

Giả sử tập dữ liệu có dạng:

$$\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}, \tag{1.47}$$

trong đó không tồn tại các nhãn $\mathbf{y}_i$ tương ứng.

Mục tiêu của học không giám sát là tìm ra một cấu trúc ẩn, một cách biểu diễn mới, hoặc một cách phân nhóm dữ liệu sao cho các điểm có tính chất tương đồng được đặt gần nhau hơn trong cùng một nhóm.

Ví dụ: Phân nhóm sinh viên ICTU theo hành vi học tập

Giả sử ta thu thập dữ liệu về sinh viên ICTU gồm các thông tin:

- Số giờ tự học mỗi tuần,
- Số buổi đi học,
- Số lần truy cập hệ thống học trực tuyến.

Mỗi sinh viên được biểu diễn bởi một vector:

$$\mathbf{x} = (x_1, x_2, x_3). \tag{1.48}$$

Trong trường hợp này, ta không biết trước sinh viên thuộc nhóm nào (ví dụ: chăm học, học trung bình hay ít tham gia). Thuật toán học không giám sát sẽ tự động phân chia sinh viên thành các nhóm dựa trên mức độ tương đồng của các đặc trưng.

Kết quả có thể cho thấy:

- Một nhóm sinh viên có số giờ tự học cao và tham gia đầy đủ,

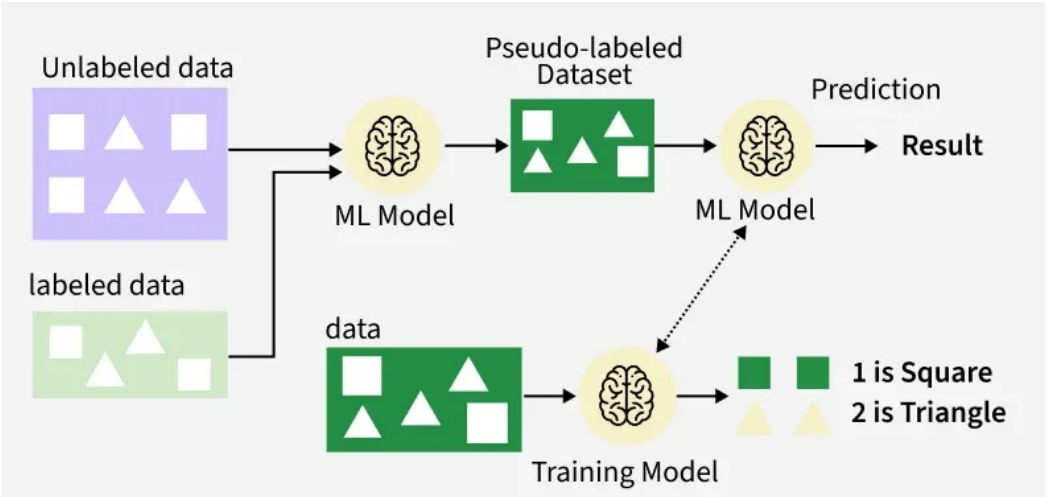
- Một nhóm có mức độ tham gia trung bình,
- Một nhóm ít tham gia học tập.

Ở đây, không có nhãn đúng được cung cấp trước. Mô hình tự tìm ra cấu trúc dữ liệu mà không có sự "giám sát" từ nhãn đầu ra.

1.4.3. Học bán giám sát

Học bán giám sát là nhóm phương pháp học máy trong đó tập huấn luyện bao gồm đồng thời hai loại dữ liệu: một phần dữ liệu có nhãn (biết trước đầu ra mong muốn) và một phần dữ liệu không có nhãn. Nói cách khác, mô hình được cung cấp một số ví dụ đã có câu trả lời đúng để học, đồng thời phải tận dụng thêm một lượng lớn dữ liệu chưa được gán nhãn nhằm cải thiện hiệu năng dự đoán.

Trong thực tế, học bán giám sát ngày càng trở nên quan trọng vì việc thu thập và gán nhãn dữ liệu thường tốn kém về thời gian, chi phí và nguồn lực chuyên môn. Trong khi đó, dữ liệu thô lại rất phong phú và dễ thu thập. Việc khai thác hiệu quả nguồn dữ liệu chưa gán nhãn giúp mô hình học tốt hơn mà không cần tăng đáng kể chi phí gán nhãn.



Hình 1.1. Minh họa mô hình học bán giám sát: kết hợp dữ liệu có nhãn và không có nhãn trong quá trình huấn luyện.

Hình minh họa cho thấy quy trình học bán giám sát thông qua việc kết hợp dữ liệu có nhãn và dữ liệu không có nhãn trong cùng một hệ thống.

Ban đầu, ta có hai nguồn dữ liệu:

- *Labeled data*: tập dữ liệu đã có nhãn, ví dụ các hình vuông và tam giác đã biết rõ thuộc lớp nào.

- *Unlabeled data*: tập dữ liệu chưa có nhãn, tức chỉ có hình dạng nhưng chưa biết đó là hình vuông hay tam giác.

Bước đầu tiên, mô hình học máy được huấn luyện trên phần dữ liệu có nhãn. Sau khi học được một mức độ nhất định, mô hình này được sử dụng để dự đoán nhãn cho phần dữ liệu chưa có nhãn. Kết quả của bước này tạo ra một tập dữ liệu mới gọi là *pseudo-labeled dataset*, tức là dữ liệu được gán nhãn tạm thời dựa trên dự đoán của mô hình.

Tiếp theo, tập dữ liệu gán nhãn tạm thời này được kết hợp với tập dữ liệu có nhãn ban đầu để huấn luyện lại mô hình. Quá trình này giúp mô hình tận dụng thêm thông tin từ dữ liệu chưa có nhãn, từ đó cải thiện khả năng phân loại.

Trong ví dụ minh họa ở hình, mô hình cuối cùng có thể đưa ra kết quả dự đoán như:

- Nhãn 1 tương ứng với hình vuông.

- Nhãn 2 tương ứng với hình tam giác.

Điểm quan trọng của học bán giám sát thể hiện rõ qua hình: mặc dù ban đầu chỉ có một số ít dữ liệu được gán nhãn, mô hình vẫn có thể khai thác cấu trúc của toàn bộ tập dữ liệu để nâng cao độ chính xác. Việc sử dụng dữ liệu không có nhãn làm tăng lượng thông tin sẵn có mà không cần tốn chi phí gán nhãn thủ công.

Ví dụ: Ảnh y học trong chẩn đoán bệnh

Giả sử một bệnh viện thu thập được hàng nghìn ảnh chụp X-quang phổi. Tuy nhiên, chỉ một phần nhỏ trong số đó được bác sĩ chuyên khoa gán nhãn là "bình thường" hoặc "có dấu hiệu bệnh". Việc gán nhãn đòi hỏi chuyên môn cao và nhiều thời gian, nên không thể thực hiện cho toàn bộ dữ liệu.

Trong trường hợp này, mô hình học bán giám sát sẽ:

- Sử dụng các ảnh đã có nhãn để học mối quan hệ giữa đặc trưng hình ảnh và tình trạng bệnh.

- Đồng thời khai thác cấu trúc của tập ảnh chưa có nhãn để nhận diện các mẫu tương đồng, từ đó cải thiện khả năng phân loại.

Nhờ kết hợp cả dữ liệu có nhãn và không có nhãn, mô hình có thể đạt độ chính xác cao hơn so với việc chỉ sử dụng lượng nhỏ dữ liệu đã được gán nhãn.

Học bán giám sát vì vậy đóng vai trò quan trọng trong các lĩnh vực mà dữ liệu dồi dào nhưng nhãn lại khan hiếm, như y tế, xử lý ảnh, nhận dạng tiếng nói và phân tích hành vi người dùng.

1.4.4. Học tăng cường

Bên cạnh học có giám sát và học không giám sát, tồn tại một nhóm thuật toán học máy khác không yêu cầu tập dữ liệu huấn luyện cố định. Thay vào đó, mô hình học cách ra quyết định thông qua quá trình tương tác trực tiếp với môi trường. Nhóm phương pháp này được gọi là *học củng cố* (reinforcement learning).

Trong học củng cố, tại mỗi thời điểm, hệ thống (gọi là tác nhân) quan sát trạng thái hiện tại của môi trường và thực hiện một hành động. Sau hành động đó, môi trường phản hồi lại bằng một giá trị đánh giá gọi là *điểm thưởng* và chuyển sang trạng thái mới. Quá trình này lặp lại nhiều lần, và mô hình dần điều chỉnh chiến lược hành động để đạt kết quả tốt hơn.

Mục tiêu của học củng cố là tìm một chính sách ra quyết định sao cho tổng điểm thưởng tích lũy là lớn nhất. Nếu ký hiệu r_t là điểm thưởng tại thời điểm t, thì tổng điểm thưởng trong một quá trình có thể được biểu diễn như sau:

$$G = \sum_{t=0}^{T} r_t, \tag{1.49}$$

trong đó T là số bước tương tác.

Khi đó, mục tiêu học tập có thể được diễn đạt như sau:

$$\max_{\pi} \mathbb{E}[G], \tag{1.50}$$

trong đó π là chính sách ra quyết định và $\mathbb{E}[G]$ là kỳ vọng của tổng điểm thưởng.

Khác với học có giám sát, trong học củng cố không tồn tại nhãn đúng cho từng hành động riêng lẻ. Thay vào đó, mô hình chỉ nhận được phản hồi thông qua điểm thưởng, và phải tự khám phá chiến lược tối ưu thông qua quá trình thử và sai.

Ví dụ minh họa

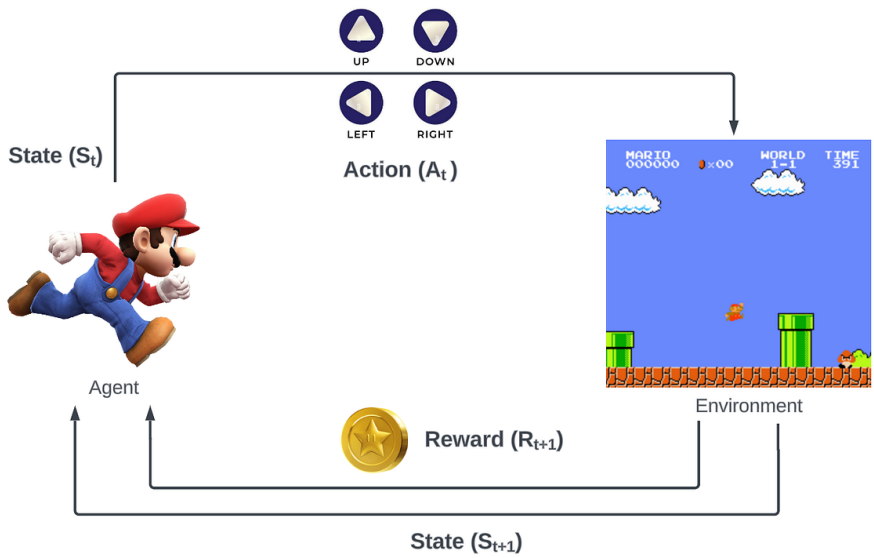
Ví dụ 1: Chơi cờ hoặc trò chơi điện tử

Một chương trình chơi cờ không được cung cấp trước nước đi đúng cho từng tình huống. Thay vào đó, nó thử nhiều nước đi khác nhau, và sau mỗi ván cờ sẽ nhận

được điểm thưởng (thắng, hòa hoặc thua). Qua nhiều lần chơi, hệ thống học được chiến lược giúp tăng xác suất chiến thắng.

Ví dụ 2: Xe tự lái

Trong bài toán xe tự lái, tác nhân phải quyết định tăng tốc, phanh hay rẽ hướng dựa trên trạng thái hiện tại của môi trường. Nếu xe di chuyển an toàn và đúng lộ trình, hệ thống nhận điểm thưởng dương; nếu va chạm hoặc đi sai đường, điểm thưởng sẽ thấp hoặc âm. Qua quá trình tương tác liên tục, mô hình học được chính sách lái xe tối ưu.



Hình 1.2. Minh họa học tăng cường thông qua trò chơi Mario: tác nhân tương tác với môi trường và nhận điểm thưởng.

Hình 1.2 minh họa cơ chế học tăng cường thông qua trò chơi Mario. Trong ví dụ này, Mario đóng vai trò là *tác nhân* (agent), còn thế giới trò chơi là *môi trường* (environment).

Tại mỗi thời điểm, Mario quan sát *trạng thái* hiện tại (vị trí của mình, vị trí ống nước, kẻ địch, hố sâu phía trước) và phải lựa chọn một *hành động*: nhảy, đi trái, đi phải hoặc đứng yên.

Sau khi thực hiện hành động, môi trường phản hồi lại bằng:

- Điểm thưởng (reward): ví dụ nhảy qua hố an toàn được cộng điểm.
- Trạng thái mới: Mario tiến gần hơn đến cột cờ hoặc... rơi xuống hố.

Nếu Mario nhảy đúng lúc và vượt qua chướng ngại vật, mô hình nhận điểm thưởng dương. Nếu Mario va phải kẻ địch hoặc rơi xuống hố, điểm thưởng có thể rất thấp hoặc âm (và người chơi thì hơi "đau tim").

Điều quan trọng là: không ai nói cho Mario biết "bây giờ phải nhảy". Không tồn tại nhãn đúng cho từng hành động cụ thể. Mario chỉ biết hành động nào tốt hay xấu thông qua điểm thưởng nhận được sau đó.

Qua hàng nghìn, thậm chí hàng triệu lần chơi, mô hình học được chiến lược tối ưu: khi thấy hố thì nhảy, khi thấy nấm thì ăn, khi thấy kẻ địch thì tránh hoặc dẫm lên.

Đây chính là bản chất của học tăng cường: học thông qua thử và sai, và dần dần tối đa hóa tổng điểm thưởng thay vì học từ các cặp dữ liệu đã gán nhãn trước.

Các cách phân loại khác

Ngoài cách phân loại theo mức độ giám sát, các thuật toán học máy còn có thể được phân loại theo nhiều tiêu chí khác:

- *Học dựa trên mẫu* và *học dựa trên mô hình*: phân biệt giữa phương pháp lưu trữ trực tiếp dữ liệu huấn luyện và phương pháp xây dựng mô hình tổng quát.
- *Học theo batch* và *học trực tuyến*: phân biệt giữa huấn luyện trên toàn bộ dữ liệu cùng một lúc và cập nhật mô hình dần dần khi dữ liệu mới xuất hiện.

1.5. Tham số mô hình và quá trình tối ưu hàm mất mát

Mỗi mô hình học máy được đặc trưng bởi một tập hợp các đại lượng gọi là *tham số mô hình* (model parameters). Các tham số này quyết định cách mô hình ánh xạ dữ liệu đầu vào thành đầu ra dự đoán. Ví dụ, trong mô hình hồi quy tuyến tính, các hệ số của biến đầu vào chính là các tham số của mô hình; trong mạng nơ-ron nhân tạo, các trọng số và độ lệch của các lớp mạng đóng vai trò là các tham số cần được học.

Mục tiêu cốt lõi của một thuật toán học máy là xác định bộ tham số sao cho mô hình có khả năng dự đoán tốt nhất đối với dữ liệu. Nói cách khác, thuật toán cần tìm ra các giá trị tham số giúp mô hình đưa ra dự đoán gần với giá trị thực tế của dữ liệu quan sát.

Để định lượng mức độ "tốt" của một mô hình, người ta sử dụng một hàm toán học gọi là *hàm mất mát* (loss function). Hàm mất mát đo lường mức độ sai khác giữa giá trị dự đoán của mô hình và giá trị thực của dữ liệu. Giá trị của hàm mất mát càng nhỏ thì mô hình được xem là hoạt động càng hiệu quả.

Trong các bài toán phân loại, một mô hình tốt là mô hình có tỷ lệ dự đoán sai thấp. Trong khi đó, đối với các bài toán hồi quy, mô hình được đánh giá dựa trên mức độ sai lệch giữa giá trị dự đoán và giá trị thực của biến đầu ra.

Giả sử tập hợp các tham số của mô hình được ký hiệu bởi θ và hàm mất mát của mô hình được ký hiệu là $\mathcal{L}(\theta)$ hoặc $J(\theta)$. Khi đó, quá trình huấn luyện mô hình có thể được diễn đạt như một bài toán tối ưu hóa:

$$\theta^* = \underset{\theta}{\operatorname{argmin}}\, \mathcal{L}(\theta). \tag{1.51}$$

Ký hiệu $\operatorname{argmin}_\theta$ biểu thị giá trị của θ làm cho hàm số $\mathcal{L}(\theta)$ đạt giá trị nhỏ nhất. Nói cách khác, θ^* là bộ tham số tối ưu của mô hình.

Như vậy, quá trình xây dựng một mô hình học máy thực chất là quá trình giải một bài toán tối ưu. Trong quá trình này, thuật toán liên tục điều chỉnh các tham số của mô hình nhằm giảm dần giá trị của hàm mất mát. Quá trình đó chính là quá trình "học" của mô hình.

Ví dụ về hàm mất mát

Xét một bài toán hồi quy đơn giản trong đó ta muốn dự đoán giá trị đầu ra y dựa trên một biến đầu vào x. Mô hình hồi quy tuyến tính có dạng:

$$\hat{y} = wx + b \tag{1.52}$$

trong đó:

- w là hệ số của biến đầu vào,
- b là hệ số chệch (bias),
- $\hat{y}$ là giá trị dự đoán của mô hình.

Hai đại lượng w và b chính là các tham số của mô hình. Để đánh giá chất lượng của mô hình, ta có thể sử dụng hàm mất mát *sai số trung bình bình phương* (Mean Squared Error – MSE):

$$\mathcal{L}(w, b) = \frac{1}{2N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \tag{1.53}$$

trong đó:

- N là số lượng mẫu dữ liệu,
- y_i là giá trị thực của mẫu thứ i,
- $\hat{y}_i$ là giá trị dự đoán của mô hình.

Hệ số $\frac{1}{2}$ thường được thêm vào để thuận tiện cho việc tính đạo hàm trong các thuật toán tối ưu như Gradient Descent.

Mục tiêu của quá trình huấn luyện là tìm các giá trị w và b sao cho hàm mất mát $\mathcal{L}(w, b)$ đạt giá trị nhỏ nhất. Khi đó, đường thẳng hồi quy sẽ xấp xỉ tốt nhất mối quan hệ giữa biến đầu vào x và biến đầu ra y.

Sai số huấn luyện

Sai số huấn luyện (training error) phản ánh mức độ sai khác giữa giá trị dự đoán của mô hình và giá trị thực trên *tập dữ liệu huấn luyện*. Trong nhiều trường hợp, sai số huấn luyện chính là giá trị của hàm mất mát được tính trên toàn bộ tập dữ liệu huấn luyện:

$$\text{Training Error} = \frac{1}{2N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (y_i - \hat{y}_i)^2 \tag{1.54}$$

Giá trị này cho biết mô hình khớp với dữ liệu huấn luyện tốt đến mức nào.

Sai số kiểm tra

Sau khi quá trình huấn luyện hoàn tất, mô hình cần được đánh giá trên một tập dữ liệu độc lập gọi là *tập kiểm tra* (test set). Tập dữ liệu này không được sử dụng trong quá trình huấn luyện nhằm đảm bảo việc đánh giá là khách quan.

Sai số kiểm tra được xác định bởi:

$$\text{Test Error} = \frac{1}{2N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (y_i - \hat{y}_i)^2 \tag{1.55}$$

Việc lấy giá trị trung bình là cần thiết vì số lượng dữ liệu trong tập huấn luyện và tập kiểm tra có thể khác nhau. Sai số kiểm tra giúp đánh giá khả năng *tổng quát hóa* (generalization) của mô hình đối với dữ liệu chưa từng xuất hiện trong quá trình huấn luyện.

1.6. Những thách thức chính của học máy

Mặc dù học máy đã đạt được nhiều thành tựu đáng kể trong các lĩnh vực như thị giác máy tính, xử lý ngôn ngữ tự nhiên và phân tích dữ liệu lớn, việc xây dựng một hệ thống học máy hiệu quả vẫn đối mặt với nhiều thách thức. Các thách thức này thường xuất phát từ hai nguồn chính: *dữ liệu* và *thuật toán*.

Trong thực tế, chất lượng của một hệ thống học máy phụ thuộc rất lớn vào dữ liệu huấn luyện và khả năng của thuật toán trong việc học được các quy luật từ dữ liệu đó. Nếu dữ liệu không phù hợp hoặc thuật toán không đủ khả năng biểu diễn các mối quan hệ trong dữ liệu, mô hình sẽ khó đạt được hiệu quả cao và có thể dẫn đến các dự đoán sai lệch.

1.6.1. Các thách thức liên quan đến dữ liệu

Thiếu dữ liệu huấn luyện

Nhiều thuật toán học máy, đặc biệt là các mô hình học sâu, yêu cầu một lượng lớn dữ liệu để có thể học được các mẫu và cấu trúc tiềm ẩn trong dữ liệu. Nếu số lượng dữ liệu quá ít, mô hình sẽ không đủ thông tin để học được quy luật tổng quát của bài toán.

Ví dụ, trong các hệ thống nhận diện hình ảnh, các mô hình hiện đại thường cần hàng trăm nghìn hoặc hàng triệu ảnh để đạt được độ chính xác cao. Khi dữ liệu huấn luyện hạn chế, mô hình dễ rơi vào tình trạng học không đầy đủ và cho kết quả dự đoán kém chính xác.

Dữ liệu không mang tính đại diện

Một tập dữ liệu huấn luyện cần phản ánh được các đặc điểm của dữ liệu mà hệ thống sẽ gặp trong thực tế. Nếu dữ liệu huấn luyện chỉ bao gồm một số trường hợp đặc biệt hoặc bị thiên lệch, mô hình có thể học được những quy luật sai lệch và hoạt động kém khi gặp dữ liệu mới.

Ví dụ, nếu một hệ thống nhận diện khuôn mặt được huấn luyện chủ yếu trên dữ liệu của một nhóm người nhất định, hệ thống có thể hoạt động kém đối với các nhóm khác. Hiện tượng này thường được gọi là *thiên lệch dữ liệu* (data bias).

Dữ liệu nhiễu và sai lệch

Trong nhiều ứng dụng thực tế, dữ liệu thường chứa lỗi đo lường, giá trị ngoại lai hoặc các thông tin không chính xác. Những yếu tố này có thể làm sai lệch quá trình học của mô hình và khiến mô hình khó phát hiện được các cấu trúc thực sự tồn tại trong dữ liệu.

Do đó, một bước quan trọng trong nhiều dự án học máy là *tiền xử lý dữ liệu* (data preprocessing), bao gồm các công việc như phát hiện ngoại lai, xử lý giá trị thiếu và chuẩn hóa dữ liệu.

Các đặc trưng không phù hợp

Chất lượng của dữ liệu không chỉ phụ thuộc vào số lượng mẫu mà còn phụ thuộc vào các đặc trưng (features) được sử dụng để mô tả dữ liệu. Nếu các đặc trưng không liên quan đến bài toán, mô hình sẽ khó có thể học được các quy luật có ý nghĩa.

Vì vậy, việc xây dựng các đặc trưng phù hợp, thường được gọi là *thiết kế đặc trưng* (feature engineering), đóng vai trò quan trọng trong quá trình xây dựng mô hình học máy. Quá trình này có thể bao gồm:

- Lựa chọn các đặc trưng quan trọng từ tập dữ liệu ban đầu.
- Kết hợp các đặc trưng hiện có để tạo ra các đặc trưng mới có ý nghĩa hơn.
- Thu thập thêm dữ liệu để bổ sung các đặc trưng cần thiết.

1.6.2. Các thách thức liên quan đến mô hình và thuật toán

Hiện tượng quá khớp (Overfitting)

Quá khớp xảy ra khi mô hình học quá chi tiết các đặc điểm của dữ liệu huấn luyện, bao gồm cả nhiễu và các biến động ngẫu nhiên. Khi đó, mô hình có thể đạt kết quả rất tốt trên tập huấn luyện nhưng lại hoạt động kém trên dữ liệu mới.

Hiện tượng này thường xảy ra khi mô hình quá phức tạp so với lượng dữ liệu huấn luyện. Một số phương pháp phổ biến để giảm quá khớp bao gồm:

- Giảm độ phức tạp của mô hình.
- Thu thập thêm dữ liệu huấn luyện.
- Áp dụng các kỹ thuật điều chuẩn (regularization).

Hiện tượng dưới khớp (Underfitting)

Ngược lại với quá khớp, dưới khớp xảy ra khi mô hình quá đơn giản và không đủ khả năng học được cấu trúc của dữ liệu. Khi đó, mô hình sẽ cho kết quả kém cả trên tập huấn luyện và tập kiểm tra.

Để khắc phục hiện tượng này, ta có thể:

- Sử dụng mô hình phức tạp hơn.
- Cung cấp thêm các đặc trưng có ý nghĩa cho mô hình.
- Giảm mức độ điều chuẩn của mô hình.

Lựa chọn và điều chỉnh siêu tham số

Ngoài các tham số được học trong quá trình huấn luyện, nhiều thuật toán còn có các *siêu tham số* (hyperparameters). Đây là các tham số được thiết lập trước khi quá trình huấn luyện bắt đầu và có ảnh hưởng lớn đến hiệu quả của mô hình.

Ví dụ, trong thuật toán Gradient Descent, tốc độ học (learning rate) là một siêu tham số quan trọng. Nếu tốc độ học quá lớn, quá trình tối ưu có thể không hội tụ; nếu quá nhỏ, quá trình học sẽ diễn ra rất chậm.

Việc lựa chọn siêu tham số phù hợp thường đòi hỏi nhiều thử nghiệm và được coi là một bước quan trọng trong quá trình xây dựng hệ thống học máy.

1.6.3. Ảnh hưởng của các thách thức trong học máy

Những thách thức nêu trên không chỉ ảnh hưởng đến độ chính xác của mô hình mà còn có thể gây ra các hệ quả đáng kể trong các ứng dụng thực tế. Nếu dữ liệu huấn luyện bị thiên lệch hoặc mô hình không được huấn luyện đúng cách, hệ thống có thể đưa ra các dự đoán sai lệch hoặc thiếu công bằng.

Trong các lĩnh vực quan trọng như y tế, tài chính hoặc giao thông tự động, những sai lệch này có thể dẫn đến các quyết định không chính xác và gây ra những hậu quả nghiêm trọng. Vì vậy, việc hiểu rõ các thách thức trong học máy và áp dụng các phương pháp phù hợp để khắc phục chúng là một bước quan trọng trong quá trình phát triển các hệ thống trí tuệ nhân tạo đáng tin cậy.

1.7. Sự đánh đổi giữa thiên lệch và phương sai

Một trong những vấn đề trung tâm trong việc xây dựng mô hình học máy là tìm được sự cân bằng hợp lý giữa khả năng học của mô hình trên dữ liệu huấn luyện và khả năng khái quát hóa trên dữ liệu mới. Hai khái niệm quan trọng giúp giải thích hiện tượng này là *thiên lệch* (bias) và *phương sai* (variance).

1.7.1. Thiên lệch (Bias)

Thiên lệch phản ánh mức độ sai khác giữa mô hình dự đoán trung bình và mối quan hệ thực sự tồn tại trong dữ liệu. Một mô hình có thiên lệch cao thường quá đơn giản và không đủ khả năng biểu diễn các cấu trúc phức tạp của dữ liệu.

Ví dụ, nếu dữ liệu thực tế có quan hệ phi tuyến nhưng ta sử dụng một mô hình tuyến tính để xấp xỉ, mô hình sẽ không thể mô tả chính xác mối quan hệ giữa các biến. Khi đó, sai số của mô hình sẽ lớn ngay cả trên tập dữ liệu huấn luyện. Hiện tượng này thường dẫn đến *dưới khớp* (underfitting).

Các mô hình có thiên lệch cao thường có các đặc điểm sau:

- Sai số huấn luyện cao.
- Sai số kiểm tra cũng cao.
- Mô hình quá đơn giản để học được cấu trúc của dữ liệu.

1.7.2. Phương sai (Variance)

Phương sai phản ánh mức độ nhạy cảm của mô hình đối với sự thay đổi của dữ liệu huấn luyện. Một mô hình có phương sai cao thường học quá chi tiết các đặc điểm của dữ liệu huấn luyện, bao gồm cả nhiễu và các biến động ngẫu nhiên.

Khi đó, mô hình có thể đạt sai số huấn luyện rất thấp nhưng lại hoạt động kém trên dữ liệu mới. Hiện tượng này chính là *quá khớp* (overfitting).

Các mô hình có phương sai cao thường có các đặc điểm sau:

- Sai số huấn luyện rất thấp.
- Sai số kiểm tra cao.
- Mô hình quá phức tạp so với dữ liệu huấn luyện.

1.7.3. Sự đánh đổi giữa thiên lệch và phương sai

Trong thực tế, việc giảm thiên lệch thường làm tăng phương sai và ngược lại. Ví dụ, khi ta sử dụng một mô hình phức tạp hơn để mô tả dữ liệu, mô hình có thể học được nhiều cấu trúc hơn và giảm thiên lệch. Tuy nhiên, điều này cũng khiến mô hình dễ bị ảnh hưởng bởi nhiễu trong dữ liệu và làm tăng phương sai.

Ngược lại, khi ta sử dụng một mô hình đơn giản hơn, phương sai của mô hình sẽ giảm nhưng thiên lệch lại tăng.

Do đó, mục tiêu của quá trình xây dựng mô hình học máy là tìm được một điểm cân bằng hợp lý giữa hai yếu tố này. Điểm cân bằng đó giúp mô hình vừa học được các quy luật chính của dữ liệu vừa duy trì được khả năng khái quát hóa tốt trên dữ liệu chưa từng xuất hiện.

1.7.4. Ví dụ minh họa

Xét một tập dữ liệu gồm các điểm quan sát trong đó đầu ra phụ thuộc phi tuyến vào biến đầu vào. Nếu ta sử dụng một mô hình tuyến tính để xấp xỉ dữ liệu, mô hình sẽ không thể biểu diễn được cấu trúc phi tuyến và dẫn đến sai số lớn. Đây là trường hợp mô hình có *thiên lệch cao*.

Ngược lại, nếu ta sử dụng một đa thức bậc rất cao để khớp với dữ liệu huấn luyện, mô hình có thể đi qua gần như tất cả các điểm dữ liệu. Tuy nhiên, mô hình này có thể dao động mạnh giữa các điểm dữ liệu và cho dự đoán kém trên dữ liệu mới. Đây là trường hợp mô hình có *phương sai cao*.

Hình 1.3 minh họa mối quan hệ giữa độ phức tạp của mô hình và sai số dự đoán. Trục hoành biểu diễn độ phức tạp của mô hình (model complexity), trong khi trục tung biểu diễn sai số dự đoán (prediction error). Khi độ phức tạp của mô hình thay đổi, các thành phần của sai số dự đoán cũng thay đổi theo những xu hướng khác nhau.

Giả sử dữ liệu được sinh ra theo mô hình

$$y = f(x) + \epsilon \tag{1.56}$$

trong đó $f(x)$ là hàm thực sự mô tả mối quan hệ giữa biến đầu vào và đầu ra, còn ϵ là nhiễu ngẫu nhiên với $\mathbb{E}[\epsilon] = 0$ và $\mathrm{Var}(\epsilon) = \sigma^2$.

Một mô hình học máy sau khi được huấn luyện sẽ xây dựng một hàm dự đoán $\hat{f}(x)$. Sai số dự đoán trung bình của mô hình tại một điểm x có thể được phân rã thành ba thành phần chính theo công thức

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\big[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\big]}_{\text{Variance}} + \sigma^2. \tag{1.57}$$

Trong đó, thành phần thứ nhất $(\mathbb{E}[\hat{f}(x)] - f(x))^2$ được gọi là *bình phương thiên lệch* (Bias^2), phản ánh mức độ sai lệch giữa mô hình trung bình và hàm thực sự của dữ liệu. Thành phần thứ hai là *phương sai* (Variance), thể hiện mức độ nhạy cảm của mô hình đối với sự thay đổi của dữ liệu huấn luyện. Thành phần σ^2 là nhiễu không thể tránh khỏi của dữ liệu.

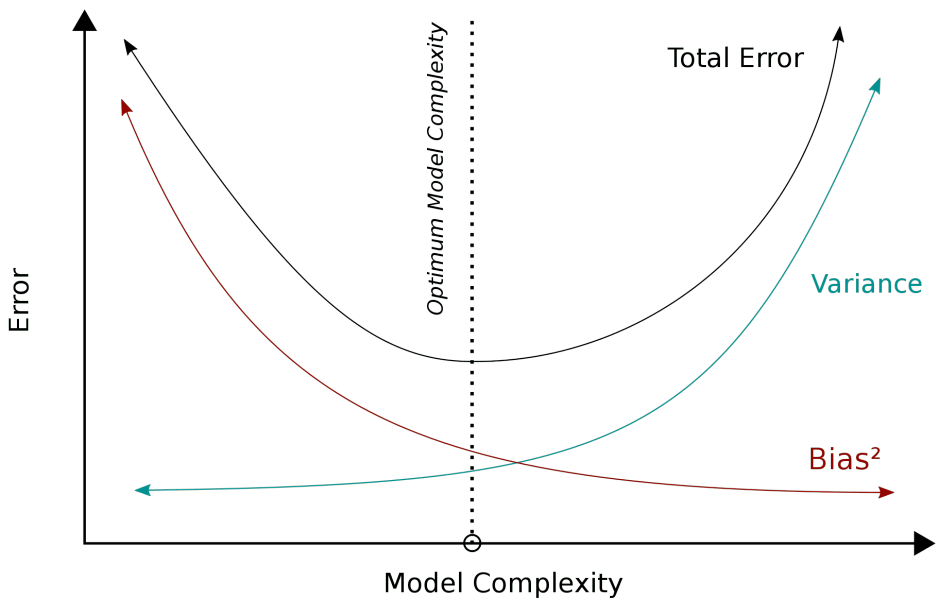
Trong hình, đường màu đỏ biểu diễn sự thay đổi của Bias^2. Khi mô hình có độ phức tạp thấp, Bias^2 thường lớn vì mô hình quá đơn giản để mô tả cấu trúc thực

của dữ liệu. Khi độ phức tạp của mô hình tăng lên, Bias^2 giảm dần do mô hình có khả năng biểu diễn dữ liệu tốt hơn.

Ngược lại, đường màu xanh biểu diễn phương sai của mô hình. Khi mô hình đơn giản, phương sai thường nhỏ vì mô hình ít phụ thuộc vào từng điểm dữ liệu cụ thể. Tuy nhiên, khi độ phức tạp của mô hình tăng lên, mô hình có xu hướng học cả các nhiễu trong dữ liệu huấn luyện, làm cho phương sai tăng nhanh.

Sai số tổng thể của mô hình, được biểu diễn bởi đường cong màu đen, có dạng chữ U vì nó là tổng của hai thành phần Bias^2 và Variance (ngoài ra còn có thành phần nhiễu không đổi). Khi mô hình quá đơn giản, sai số lớn do Bias^2 cao; khi mô hình quá phức tạp, sai số lại tăng do Variance lớn. Giữa hai vùng này tồn tại một điểm tối ưu, tại đó sai số tổng thể đạt giá trị nhỏ nhất. Điểm này được biểu diễn trong hình bởi đường thẳng đứng nét đứt và được gọi là *độ phức tạp tối ưu của mô hình* (optimal model complexity).

Việc lựa chọn mô hình trong học máy thực chất là tìm ra mức độ phức tạp phù hợp để đạt được sự cân bằng giữa thiên lệch và phương sai, từ đó giúp mô hình vừa học được các quy luật của dữ liệu huấn luyện vừa duy trì khả năng khái quát hóa tốt đối với dữ liệu mới.



Hình 1.3. Minh họa sự đánh đổi giữa thiên lệch và phương sai

1.7.5. Ý nghĩa trong thực tế

Hiểu được sự đánh đổi giữa thiên lệch và phương sai giúp chúng ta lựa chọn mô hình phù hợp và thiết kế các chiến lược huấn luyện hiệu quả hơn. Nhiều kỹ thuật

trong học máy hiện đại, chẳng hạn như điều chuẩn (regularization), cắt tỉa cây quyết định, hay phương pháp ensemble, đều nhằm mục tiêu kiểm soát sự cân bằng giữa hai yếu tố này.

Trong các chương tiếp theo, chúng ta sẽ thấy rằng nhiều thuật toán tối ưu, chẳng hạn như Gradient Descent và các kỹ thuật điều chuẩn, được thiết kế nhằm giảm sai số tổng thể của mô hình bằng cách kiểm soát sự đánh đổi giữa thiên lệch và phương sai.

1.8. Quy trình thực hiện một dự án học máy

Một dự án học máy thường bao gồm nhiều bước khác nhau, từ việc xác định bài toán đến triển khai hệ thống trong thực tế. Mặc dù quy trình có thể thay đổi tùy thuộc vào từng ứng dụng, phần lớn các dự án học máy đều trải qua các bước cơ bản sau:

a. Xác định bài toán và mục tiêu của hệ thống

b. Thu thập và hiểu dữ liệu

c. Khám phá và phân tích dữ liệu

d. Chuẩn bị dữ liệu cho quá trình huấn luyện

e. Huấn luyện và lựa chọn mô hình học máy phù hợp

f. Tinh chỉnh mô hình và đánh giá hiệu năng

g. Triển khai và giám sát hệ thống trong thực tế

Các bước trên không phải lúc nào cũng được thực hiện theo một thứ tự cố định. Trong nhiều trường hợp, sau khi đánh giá mô hình, ta có thể quay lại các bước trước đó để cải thiện dữ liệu hoặc lựa chọn mô hình tốt hơn.

Để hiểu rõ hơn quy trình này, ta xét một ví dụ cụ thể về bài toán dự đoán giá nhà.

Ví dụ: Bài toán dự đoán giá nhà

Giả sử một công ty bất động sản muốn xây dựng một hệ thống có khả năng ước lượng giá bán của một căn nhà dựa trên các đặc trưng của căn nhà đó.

a. Xác định bài toán

Bài toán được mô hình hóa như một bài toán hồi quy có giám sát. Đầu vào của mô hình là các đặc trưng của căn nhà, còn đầu ra là giá bán của căn nhà đó.

b. Thu thập dữ liệu

Dữ liệu có thể được thu thập từ các giao dịch bất động sản trước đó. Mỗi mẫu dữ liệu có thể bao gồm các thuộc tính như:

- Diện tích căn nhà
- Số phòng ngủ
- Khoảng cách tới trung tâm thành phố
- Năm xây dựng
- Giá bán thực tế

c. Khám phá dữ liệu

Ở bước này ta tiến hành phân tích dữ liệu để hiểu rõ đặc điểm của tập dữ liệu, ví dụ như phân phối của giá nhà, các điểm dữ liệu ngoại lai hoặc mối tương quan giữa các thuộc tính.

d. Chuẩn bị dữ liệu

Dữ liệu cần được xử lý trước khi đưa vào mô hình, chẳng hạn như:

- Xử lý các giá trị bị thiếu
- Chuẩn hóa các đặc trưng
- Chuyển các biến dạng hạng mục thành dạng số

Sau đó dữ liệu được chia thành tập huấn luyện và tập kiểm tra.

e. Huấn luyện mô hình

Giả sử mỗi căn nhà được biểu diễn bởi một vector đặc trưng

$$\mathbf{x} = (x_1, x_2, x_3) \tag{1.58}$$

trong đó

- x_1 là diện tích căn nhà

- x_2 là số phòng ngủ
- x_3 là khoảng cách tới trung tâm thành phố

Một mô hình hồi quy tuyến tính đơn giản có thể được sử dụng để dự đoán giá nhà:

$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b \tag{1.59}$$

trong đó $\hat{y}$ là giá nhà dự đoán, w_1, w_2, w_3 là các tham số của mô hình và b là hệ số chệch.

Các tham số của mô hình được học từ dữ liệu huấn luyện bằng cách tối thiểu hóa hàm mất mát, ví dụ như sai số trung bình bình phương:

$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \tag{1.60}$$

f. Tinh chỉnh mô hình

Sau khi huấn luyện, các siêu tham số của mô hình có thể được điều chỉnh để cải thiện hiệu năng. Ngoài ra, ta cũng có thể thử nhiều mô hình khác nhau như cây quyết định, rừng ngẫu nhiên hoặc mạng nơ-ron để tìm ra mô hình phù hợp nhất.

g. Triển khai hệ thống

Khi mô hình đạt chất lượng mong muốn, hệ thống có thể được triển khai trong ứng dụng thực tế. Ví dụ, một trang web bất động sản có thể sử dụng mô hình để ước lượng giá của một căn nhà khi người dùng nhập các thông tin như diện tích, số phòng và vị trí của căn nhà đó.

Chương 2

Các kỹ thuật xây dựng đặc trưng

Các kỹ thuật xây dựng đặc trưng (feature engineering) đóng vai trò quan trọng trong quá trình xây dựng mô hình học máy. Mục tiêu của các kỹ thuật này là tạo ra các đặc trưng mới từ dữ liệu ban đầu nhằm giúp mô hình học được những thông tin quan trọng hơn từ dữ liệu. Việc xây dựng đặc trưng phù hợp có thể cải thiện đáng kể hiệu quả dự đoán của mô hình, thậm chí trong nhiều trường hợp còn quan trọng hơn việc lựa chọn thuật toán học máy.

2.1. Giới thiệu

Trong các bài toán học máy, mỗi mẫu dữ liệu thường được biểu diễn bằng một vector số học gọi là *vector đặc trưng* (feature vector). Vector này chứa các thuộc tính quan trọng dùng để mô tả đối tượng cần phân tích. Nếu một mẫu dữ liệu có d đặc trưng thì nó có thể được biểu diễn dưới dạng

$$\mathbf{x} = (x_1, x_2, \ldots, x_d) \in \mathbb{R}^d \tag{2.1}$$

trong đó mỗi thành phần x_i biểu diễn một đặc trưng của dữ liệu. Trong cùng một mô hình học máy, các mẫu dữ liệu cần có cùng số lượng đặc trưng để có thể thực hiện các phép toán đại số tuyến tính như phép nhân ma trận hoặc phép nhân vô hướng.

Trong nhiều ứng dụng thực tế, dữ liệu ban đầu thường tồn tại dưới dạng thô và không phù hợp để đưa trực tiếp vào các thuật toán học máy. Ví dụ, các ảnh có thể có kích thước khác nhau, hoặc các văn bản có độ dài không giống nhau. Ngoài ra, dữ liệu cũng có thể có số chiều rất lớn, gây khó khăn cho việc lưu trữ và tính toán.

Do đó, trước khi xây dựng mô hình học máy, dữ liệu thường cần được chuyển đổi thành các vector đặc trưng có kích thước phù hợp. Quá trình này thường biến đổi dữ liệu ban đầu thành một ma trận dữ liệu:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} \in \mathbb{R}^{N \times d} \tag{2.2}$$

trong đó N là số lượng mẫu dữ liệu và d là số đặc trưng của mỗi mẫu.

Quá trình xác định và xây dựng các đặc trưng có ý nghĩa từ dữ liệu ban đầu được gọi là *kỹ thuật trích chọn đặc trưng* (feature extraction). Mục tiêu của bước này là biểu diễn dữ liệu dưới dạng các vector đặc trưng có kích thước phù hợp, đồng thời giữ lại những thông tin quan trọng phục vụ cho quá trình học của mô hình.

Để có cái nhìn tổng quan, chúng ta cần đặt bước trích chọn đặc trưng này trong cả quy trình xây dựng một mô hình học máy .

2.2. Mô hình chung cho các bài toán học máy

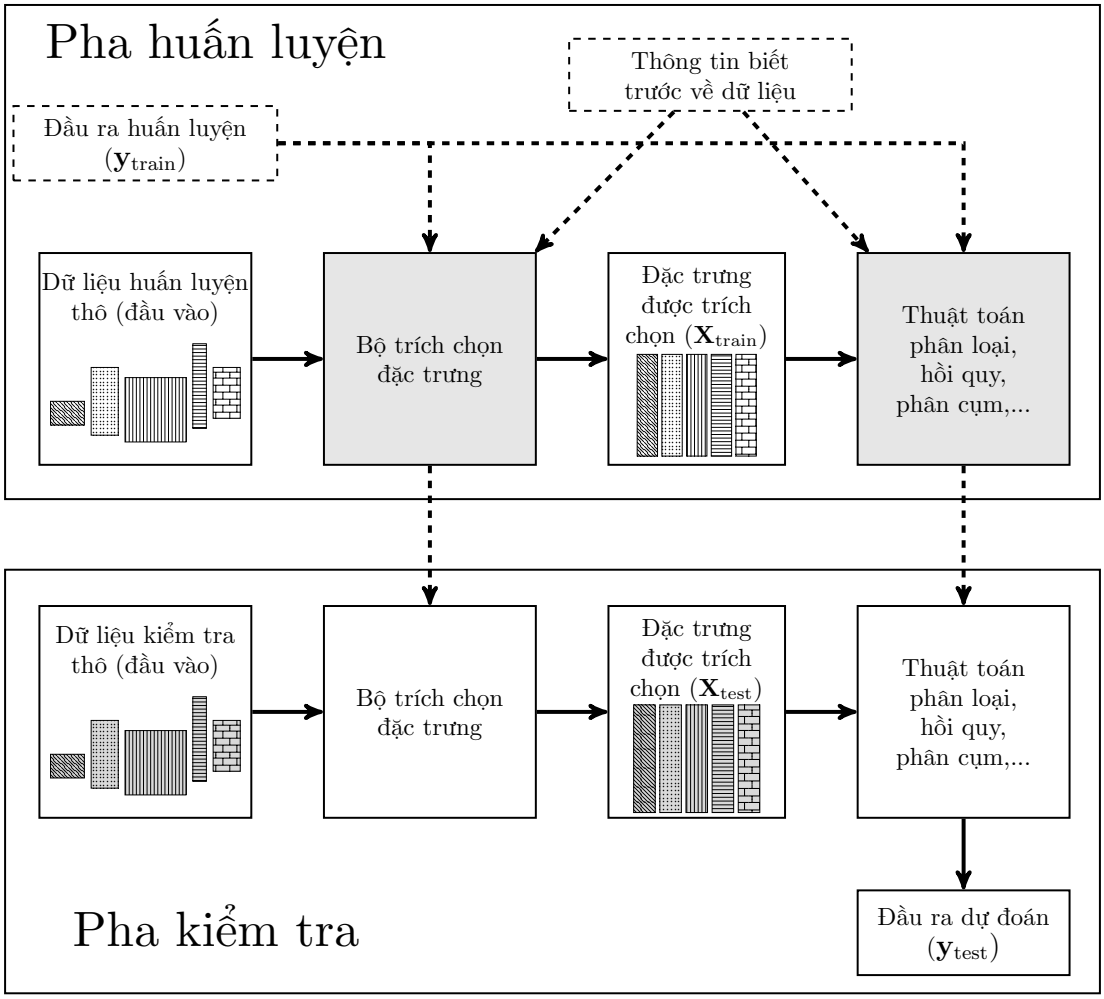
Phần lớn các mô hình học máy có thể được minh hoạ trong Hình 2.1. Có hai pha lớn trong mỗi bài toán học máy là *pha huấn luyện* và *pha kiểm tra*. Pha huấn luyện xây dựng mô hình dựa trên dữ liệu huấn luyện. Dữ liệu kiểm tra được sử dụng để đánh giá hiệu quả mô hình.

2.2.1. Pha huấn luyện

Trong mô hình học máy, pha huấn luyện là giai đoạn quan trọng nhất trong quá trình xây dựng hệ thống. Mục tiêu của pha này là sử dụng dữ liệu huấn luyện để học ra một mô hình có khả năng mô tả mối quan hệ giữa dữ liệu đầu vào và đầu ra mong muốn.

Quá trình huấn luyện bắt đầu từ tập dữ liệu huấn luyện thô. Các dữ liệu này thường chưa ở dạng phù hợp để đưa trực tiếp vào các thuật toán học máy. Vì vậy, bước đầu tiên là thực hiện trích chọn đặc trưng. Bộ trích chọn đặc trưng sẽ biến đổi dữ liệu thô thành các vector đặc trưng có kích thước cố định. Tập hợp các vector đặc trưng của dữ liệu huấn luyện được ký hiệu là

$$\mathbf{X}_{\text{train}} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} \tag{2.3}$$



Hình 2.1. Mô hình chung trong các bài toán học máy

trong đó N là số lượng mẫu huấn luyện và mỗi vector $\mathbf{x}_i \in \mathbb{R}^d$ biểu diễn d đặc trưng của mẫu dữ liệu thứ i.

Bên cạnh các đặc trưng đầu vào, trong các bài toán học có giám sát ta còn có các nhãn tương ứng với từng mẫu dữ liệu. Tập hợp các nhãn này được ký hiệu là

$$\mathbf{y}_{\text{train}} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \tag{2.4}$$

Sau khi có được các đặc trưng $\mathbf{X}_{\text{train}}$ và nhãn $\mathbf{y}_{\text{train}}$, các dữ liệu này sẽ được đưa vào một thuật toán học máy, chẳng hạn như hồi quy, phân loại hoặc phân cụm.

Thuật toán sẽ tìm các tham số của mô hình sao cho mô hình có thể dự đoán đầu ra từ các đặc trưng đầu vào một cách chính xác nhất.

Một cách tổng quát, mô hình học máy có thể được biểu diễn dưới dạng một hàm ánh xạ

$$\hat{\mathbf{y}} = f(\mathbf{X}; \boldsymbol{\theta}) \tag{2.5}$$

trong đó $\boldsymbol{\theta}$ là tập hợp các tham số của mô hình. Nhiệm vụ của pha huấn luyện là tìm ra các tham số tối ưu $\boldsymbol{\theta}^*$ sao cho sai số giữa đầu ra dự đoán $\hat{\mathbf{y}}$ và đầu ra thực $\mathbf{y}_{\text{train}}$ là nhỏ nhất. Quá trình này thường được thực hiện bằng cách tối thiểu hóa một hàm mất mát.

Ngoài ra, trong quá trình huấn luyện, hệ thống cũng có thể sử dụng những thông tin được biết trước về dữ liệu, chẳng hạn như kiến thức chuyên môn của lĩnh vực ứng dụng. Những thông tin này có thể hỗ trợ việc thiết kế bộ trích chọn đặc trưng hoặc lựa chọn mô hình học máy phù hợp.

Kết quả của pha huấn luyện là một mô hình đã được học với các tham số tối ưu. Mô hình này sau đó sẽ được sử dụng trong pha kiểm tra để dự đoán đầu ra cho các dữ liệu mới chưa từng xuất hiện trong quá trình huấn luyện.

2.2.2. Pha kiểm tra

Sau khi mô hình đã được huấn luyện và các tham số tối ưu $\boldsymbol{\theta}^*$ đã được xác định, hệ thống bước sang *pha kiểm tra* (testing phase). Mục tiêu của pha này là đánh giá khả năng khái quát hóa của mô hình trên các dữ liệu mới chưa từng được sử dụng trong quá trình huấn luyện.

Trong pha kiểm tra, hệ thống nhận vào một tập dữ liệu kiểm tra thô. Tương tự như dữ liệu huấn luyện, các dữ liệu này trước hết cũng phải được đưa qua bộ trích chọn đặc trưng để chuyển đổi thành các vector đặc trưng có cùng dạng với dữ liệu huấn luyện. Tập hợp các vector đặc trưng của dữ liệu kiểm tra được ký hiệu là

$$\mathbf{X}_{\text{test}} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_M \end{bmatrix} \tag{2.6}$$

trong đó M là số lượng mẫu trong tập kiểm tra.

Các vector đặc trưng $\mathbf{X}_{\text{test}}$ sau đó được đưa vào mô hình đã được huấn luyện ở pha trước. Mô hình sử dụng các tham số đã học $\boldsymbol{\theta}^*$ để tạo ra các giá trị dự đoán cho dữ liệu kiểm tra:

$$\hat{\mathbf{y}}_{\text{test}} = f(\mathbf{X}_{\text{test}}; \boldsymbol{\theta}^*) \tag{2.7}$$

Kết quả dự đoán $\hat{\mathbf{y}}_{\text{test}}$ sau đó được so sánh với các giá trị thực của dữ liệu kiểm tra $\mathbf{y}_{\text{test}}$ nhằm đánh giá chất lượng của mô hình. Tùy thuộc vào loại bài toán, các thước đo đánh giá khác nhau có thể được sử dụng, chẳng hạn như độ chính xác (accuracy) trong bài toán phân loại hoặc sai số trung bình bình phương trong bài toán hồi quy.

Pha kiểm tra đóng vai trò rất quan trọng vì nó cho phép đánh giá khả năng hoạt động của mô hình trên các dữ liệu chưa từng xuất hiện trong quá trình huấn luyện. Nếu mô hình hoạt động tốt trên tập kiểm tra, ta có thể kỳ vọng rằng mô hình cũng sẽ hoạt động hiệu quả khi được áp dụng vào các dữ liệu thực tế.

2.3. Một số kỹ thuật trích chọn đặc trưng

Trong các hệ thống học máy, dữ liệu ban đầu thường tồn tại dưới dạng thô và không phù hợp để đưa trực tiếp vào các thuật toán học máy. Vì vậy, một bước quan trọng là chuyển đổi dữ liệu ban đầu thành các vector đặc trưng có ý nghĩa. Quá trình này được gọi là *trích chọn đặc trưng* (feature extraction).

Giả sử mỗi mẫu dữ liệu ban đầu được biểu diễn bởi một vector

$$\mathbf{x} \in \mathbb{R}^D \tag{2.8}$$

Sau khi trích chọn đặc trưng, dữ liệu được biểu diễn trong không gian đặc trưng mới:

$$\mathbf{z} = \phi(\mathbf{x}) \in \mathbb{R}^d \tag{2.9}$$

trong đó $\phi(\cdot)$ là phép biến đổi đặc trưng và thường $d \leq D$. Mục tiêu của quá trình này là giữ lại các thông tin quan trọng của dữ liệu trong khi loại bỏ những thành phần dư thừa.

Sau đây là một số kỹ thuật trích chọn đặc trưng phổ biến:

2.3.1. Vector hóa dữ liệu thô

Một phương pháp đơn giản để tạo vector đặc trưng là trực tiếp chuyển dữ liệu thô thành một vector số học.

Ví dụ, xét một ảnh xám kích thước $m \times n$. Ảnh có thể được biểu diễn bởi một ma trận cường độ điểm ảnh. Bằng cách xếp chồng các cột hoặc các hàng của ma trận này, ta thu được vector đặc trưng:

$$\mathbf{x} = (x_1, x_2, \ldots, x_{mn}) \tag{2.10}$$

trong đó mỗi phần tử biểu diễn giá trị cường độ của một điểm ảnh.

Ví dụ.

Xét ảnh 2×2

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Vector đặc trưng tương ứng có thể là

$$\mathbf{x} = (10, 30, 20, 40)$$

Phương pháp này rất đơn giản nhưng không giữ được thông tin về quan hệ không gian giữa các điểm ảnh.

2.3.2. Lựa chọn đặc trưng

Trong nhiều bài toán, dữ liệu có thể chứa rất nhiều thuộc tính, nhưng chỉ một số trong đó thực sự hữu ích cho mô hình. Khi đó ta có thể chọn ra một tập con các đặc trưng quan trọng.

Giả sử dữ liệu ban đầu có dạng

$$\mathbf{x} = (x_1, x_2, \ldots, x_D) \tag{2.11}$$

Sau khi lựa chọn đặc trưng, ta giữ lại d đặc trưng quan trọng nhất:

$$\mathbf{z} = (x_{i_1}, x_{i_2}, \ldots, x_{i_d}) \tag{2.12}$$

Ví dụ.

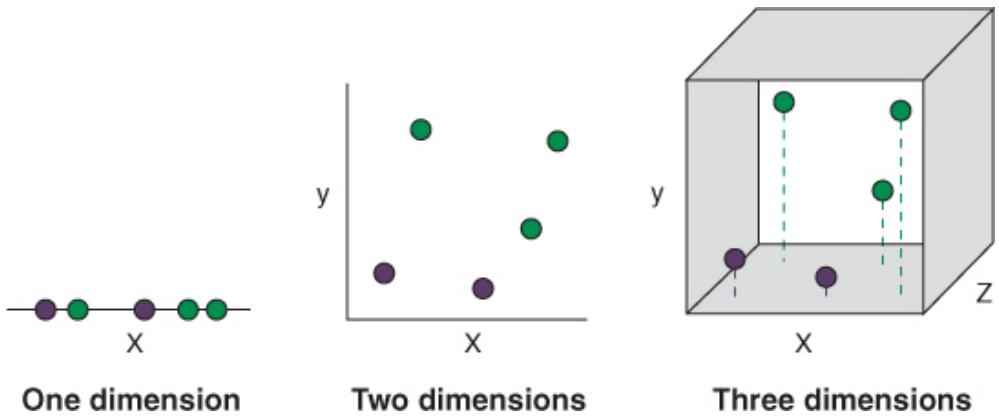
Trong bài toán dự đoán giá nhà, dữ liệu ban đầu có thể gồm:

- Diện tích
- Số phòng ngủ
- Khoảng cách đến trung tâm
- Màu sơn nhà
- Số cây trong sân

Một số đặc trưng như màu sơn có thể ít ảnh hưởng đến giá nhà và có thể bị loại bỏ.

2.3.3. Giảm chiều dữ liệu

Trong nhiều bài toán học máy, dữ liệu ban đầu có thể có số chiều rất lớn. Khi số chiều của dữ liệu tăng lên, nhiều vấn đề có thể xảy ra, hiện tượng này thường được gọi là *lời nguyền về chiều dữ liệu* (*curse of dimensionality*). Khi đó, không gian dữ liệu trở nên rất thưa, khoảng cách giữa các điểm dữ liệu trở nên kém ý nghĩa và các thuật toán học máy thường cần nhiều dữ liệu hơn để học được các quy luật trong dữ liệu. Ngoài ra, chi phí lưu trữ và tính toán cũng tăng lên đáng kể khi số chiều của dữ liệu lớn. Hình 2.2 minh họa một hiện tượng quan trọng trong học máy được gọi là *lời nguyền về chiều dữ liệu* (*curse of dimensionality*). Khi số chiều của dữ liệu tăng lên, không gian dữ liệu trở nên rất lớn và các điểm dữ liệu trở nên ngày càng thưa thớt. Điều này khiến việc học các quy luật từ dữ liệu trở nên khó khăn hơn.



Hình 2.2. Minh họa sự gia tăng không gian dữ liệu khi số chiều tăng

Trong hình, khi dữ liệu chỉ có một chiều, các điểm dữ liệu nằm trên một trục số. Khi số chiều tăng lên hai hoặc ba chiều, dữ liệu phải được phân bố trong một không gian lớn hơn nhiều. Nếu số lượng điểm dữ liệu không tăng tương ứng, các điểm sẽ trở nên thưa và khoảng cách giữa các điểm trở nên lớn hơn.

Ta có thể thấy rõ hiện tượng này thông qua một ví dụ đơn giản. Giả sử mỗi chiều của dữ liệu được chia thành k khoảng bằng nhau. Khi đó, số ô nhỏ trong không gian dữ liệu sẽ là

$$N = k^d \tag{2.13}$$

trong đó d là số chiều của dữ liệu.

Ví dụ, nếu mỗi chiều được chia thành 10 khoảng, ta sẽ có

$$d = 1 \Rightarrow N = 10$$

$$d = 2 \Rightarrow N = 10^2 = 100$$

$$d = 3 \Rightarrow N = 10^3 = 1000$$

Như vậy, khi số chiều tăng lên, số vùng trong không gian dữ liệu tăng theo cấp số mũ. Để duy trì mật độ dữ liệu tương tự, số lượng mẫu dữ liệu cũng phải tăng theo cấp số mũ.

Hiện tượng này dẫn đến một số hệ quả quan trọng:

- Dữ liệu trở nên rất thưa trong không gian nhiều chiều.
- Khoảng cách giữa các điểm dữ liệu trở nên kém ý nghĩa.
- Các thuật toán học máy cần nhiều dữ liệu hơn để học được các quy luật trong dữ liệu.
- Chi phí tính toán và lưu trữ tăng lên đáng kể.

Chính vì vậy, trong nhiều bài toán học máy, người ta thường áp dụng các kỹ thuật *giảm chiều dữ liệu* để biểu diễn dữ liệu trong một không gian có số chiều nhỏ hơn nhưng vẫn giữ được phần lớn thông tin quan trọng của dữ liệu.

Một cách phổ biến để giải quyết vấn đề này là *giảm chiều dữ liệu* (dimensionality reduction). Ý tưởng chính là biến đổi dữ liệu từ không gian có số chiều lớn sang một không gian có số chiều nhỏ hơn nhưng vẫn giữ được phần lớn thông tin quan trọng của dữ liệu.

Giả sử dữ liệu ban đầu được biểu diễn bởi vector

$$\mathbf{x} \in \mathbb{R}^D \tag{2.14}$$

Ta sử dụng một ma trận chiếu

$$\mathbf{A} \in \mathbb{R}^{d \times D} \tag{2.15}$$

để thu được vector đặc trưng mới

$$\mathbf{z} = \mathbf{A}\mathbf{x}, \quad d < D \tag{2.16}$$

trong đó $\mathbf{z}$ là biểu diễn mới của dữ liệu trong không gian có số chiều thấp hơn.

Một trong những phương pháp giảm chiều phổ biến nhất là *Principal Component Analysis* (PCA). Phương pháp này tìm một hệ trục mới sao cho phương sai của dữ liệu theo các trục đó là lớn nhất. Nói cách khác, PCA chiếu dữ liệu lên các hướng mang nhiều thông tin nhất của dữ liệu.

Ví dụ. Giả sử dữ liệu có hai đặc trưng (x_1, x_2) và hai đặc trưng này có tương quan mạnh. Khi đó, PCA có thể tìm một trục mới

$$z = a_1 x_1 + a_2 x_2 \tag{2.17}$$

sao cho dữ liệu có phương sai lớn nhất trên trục này. Khi đó dữ liệu ban đầu trong không gian hai chiều được biểu diễn lại trong không gian một chiều mà vẫn giữ được phần lớn thông tin của dữ liệu.

2.3.4. Túi từ (Bag of Words)

Trong xử lý ngôn ngữ tự nhiên, văn bản cần được chuyển đổi thành dạng số để có thể sử dụng trong các thuật toán học máy. Một trong những phương pháp phổ biến nhất để biểu diễn văn bản là kỹ thuật *túi từ* (Bag of Words).

Ý tưởng chính của phương pháp này là xây dựng một *từ điển* (dictionary) gồm tất cả các từ xuất hiện trong tập dữ liệu. Mỗi văn bản sau đó được biểu diễn

bởi một vector có số chiều bằng kích thước của từ điển. Giá trị của mỗi phần tử trong vector tương ứng với số lần xuất hiện của từ đó trong văn bản.

Giả sử từ điển có V từ khác nhau. Khi đó, mỗi văn bản được biểu diễn bởi vector đặc trưng

$$\mathbf{x} = (c_1, c_2, \ldots, c_V) \tag{2.18}$$

trong đó c_i là số lần xuất hiện của từ thứ i trong văn bản.

Ví dụ.

Xét hai câu sau:

- "Sinh viên ICTU học trí tuệ nhân tạo"
- "Sinh viên ICTU học khoa học dữ liệu"

Từ hai câu trên, ta xây dựng từ điển gồm các từ khác nhau xuất hiện trong tập văn bản:

```
["Sinh", "viên", "ICTU", "học", "trí", "tuệ",
"nhân", "tạo", "khoa", "dữ", "liệu"]
```

Mỗi câu sẽ được biểu diễn bằng một vector đặc trưng có số chiều bằng kích thước của từ điển. Khi đó ta thu được

$$\mathbf{x}_1 = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0]$$

$$\mathbf{x}_2 = [1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1]$$

Trong đó mỗi phần tử biểu thị số lần xuất hiện của từ tương ứng trong văn bản.

Một hạn chế quan trọng của phương pháp túi từ là nó không lưu giữ thông tin về thứ tự của các từ trong câu. Do đó, hai câu có cùng tập từ nhưng khác thứ tự vẫn có thể được biểu diễn bởi cùng một vector đặc trưng, mặc dù ý nghĩa của chúng có thể khác nhau.

2.3.5. Mã hóa biến phân loại bằng One-Hot Encoding

Trong nhiều bài toán học máy, dữ liệu đầu vào có thể chứa các biến phân loại (categorical variables), chẳng hạn như loại động vật, loại phương tiện hoặc loại sản phẩm. Tuy nhiên, các thuật toán học máy thường yêu cầu dữ liệu ở dạng số. Vì vậy, các biến phân loại cần được chuyển đổi sang dạng vector số.

Một phương pháp phổ biến để thực hiện điều này là *One-Hot Encoding*. Ý tưởng của phương pháp này là biểu diễn mỗi giá trị của biến phân loại bằng một vector nhị phân. Trong vector này, chỉ có một phần tử bằng 1 tương ứng với giá trị của biến, các phần tử còn lại bằng 0.

Giả sử một biến phân loại có K giá trị khác nhau. Khi đó mỗi giá trị được biểu diễn bởi một vector trong không gian $\mathbb{R}^K$:

$$\mathbf{x} = (x_1, x_2, \ldots, x_K) \tag{2.19}$$

trong đó

$$x_i = \begin{cases} 1 & \text{nếu biến nhận giá trị thứ } i \\ 0 & \text{ngược lại} \end{cases}$$

Ví dụ.

Xét bài toán phân loại động vật với ba lớp:

$$\{\text{Mèo},\ \text{Chó},\ \text{Chim}\}$$

Ta có thể mã hóa các lớp này bằng One-Hot Encoding như sau:

$$\text{Mèo} \to (1, 0, 0)$$

$$\text{Chó} \to (0, 1, 0)$$

$$\text{Chim} \to (0, 0, 1)$$

Như vậy, mỗi lớp động vật được biểu diễn bằng một vector nhị phân có độ dài bằng số lớp. Cách biểu diễn này giúp các thuật toán học máy có thể xử lý các biến phân loại một cách hiệu quả.

2.3.6. TF–IDF

Trong xử lý ngôn ngữ tự nhiên, phương pháp túi từ (Bag of Words) chỉ dựa trên số lần xuất hiện của các từ trong văn bản. Tuy nhiên, một số từ xuất hiện rất thường xuyên trong nhiều văn bản và không mang nhiều thông tin phân biệt. Vì vậy, phương pháp *Term Frequency – Inverse Document Frequency* (TF–IDF) được sử dụng để đánh giá mức độ quan trọng của một từ trong một văn bản.

Giả sử ta có tập văn bản gồm N tài liệu và xét một từ t trong tài liệu d.

Term Frequency (TF)

Term Frequency đo tần suất xuất hiện của từ t trong tài liệu d:

$$TF(t,d) = \frac{f_{t,d}}{\sum_k f_{k,d}} \tag{2.20}$$

trong đó $f_{t,d}$ là số lần từ t xuất hiện trong tài liệu d.

Inverse Document Frequency (IDF)

Inverse Document Frequency đo mức độ hiếm của từ trong toàn bộ tập tài liệu:

$$IDF(t) = \log \frac{N}{n_t} \tag{2.21}$$

trong đó n_t là số tài liệu có chứa từ t.

TF–IDF

Giá trị TF–IDF được tính bằng tích của hai đại lượng trên:

$$TFIDF(t,d) = TF(t,d) \times IDF(t) \tag{2.22}$$

Ví dụ.

Xét hai câu sau:

- "Sinh viên ICTU học trí tuệ nhân tạo"

- "Sinh viên ICTU học khoa học dữ liệu"

Các từ như "Sinh", "viên", "học" xuất hiện trong cả hai câu nên có giá trị IDF nhỏ. Trong khi đó các từ như "nhân", "tạo", "dữ", "liệu" xuất hiện ít hơn nên có giá trị TF–IDF cao hơn. Do đó các từ này được xem là mang nhiều thông tin hơn trong việc phân biệt các văn bản.

2.3.7. Các kỹ thuật trích chọn đặc trưng hiện đại

Trong các hệ thống học máy hiện đại, đặc biệt là học sâu, việc trích chọn đặc trưng thường được thực hiện tự động bởi mô hình.

Embedding

Các đối tượng như từ hoặc người dùng có thể được biểu diễn bởi các vector trong không gian liên tục:

$$\mathbf{z} = f(\mathbf{x}) \tag{2.23}$$

trong đó $\mathbf{z}$ là vector embedding.

Ví dụ.

Trong mô hình Word2Vec, từ "king" có thể được biểu diễn bởi vector

$$(0.25, 0.81, -0.12, \dots)$$

Các từ có ý nghĩa gần nhau sẽ có vector gần nhau trong không gian.

Autoencoder

Autoencoder là một mạng nơ-ron được huấn luyện để nén dữ liệu vào một biểu diễn có số chiều nhỏ hơn:

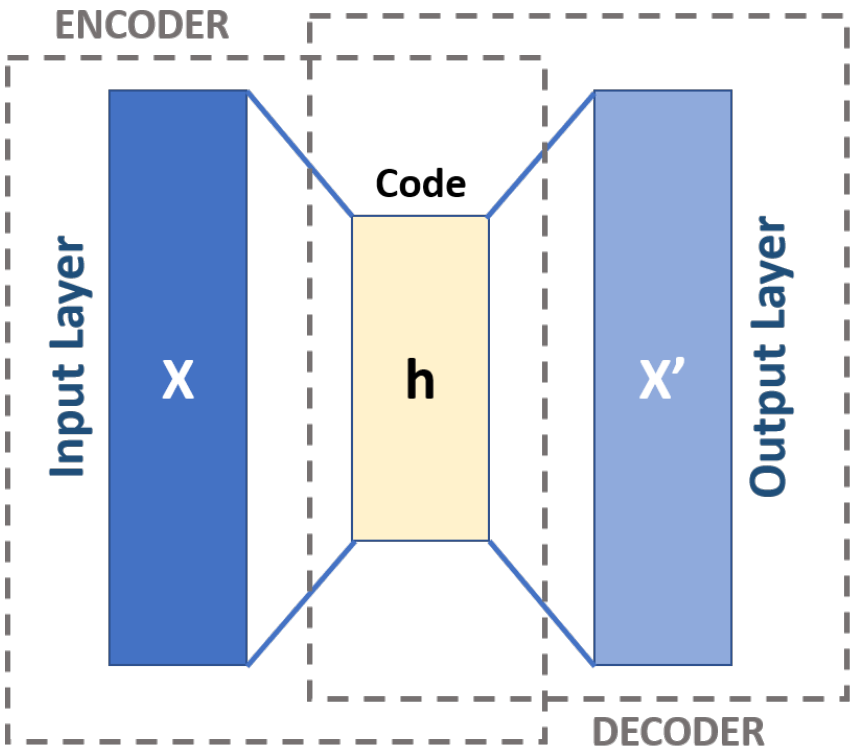
$$\mathbf{z} = Encoder(\mathbf{x}) \tag{2.24}$$

$$\hat{\mathbf{x}} = Decoder(\mathbf{z}) \tag{2.25}$$

Vector $\mathbf{z}$ chính là đặc trưng mới của dữ liệu.

Ví dụ.

Một ảnh 28×28 pixel có 784 đặc trưng ban đầu có thể được nén xuống còn 32 đặc trưng bởi autoencoder.



Hình 2.3. Cấu trúc cơ bản của mô hình Autoencoder

Hình 2.3 minh họa cấu trúc cơ bản của một *Autoencoder*. Đây là một mô hình mạng nơ-ron được sử dụng để học biểu diễn đặc trưng của dữ liệu một cách tự động. Autoencoder bao gồm hai thành phần chính: *encoder* và *decoder*.

Ở phía bên trái của hình là lớp đầu vào $\mathbf{X}$, biểu diễn dữ liệu ban đầu. Encoder có nhiệm vụ biến đổi dữ liệu đầu vào thành một biểu diễn có số chiều thấp hơn gọi là *vector ẩn* (latent representation) $\mathbf{h}$. Quá trình này có thể được biểu diễn bằng một phép biến đổi:

$$\mathbf{h} = f(\mathbf{X}) = \sigma(\mathbf{W}_e\mathbf{X} + \mathbf{b}_e) \tag{2.26}$$

trong đó $\mathbf{W}_e$ và $\mathbf{b}_e$ lần lượt là ma trận trọng số và vector bias của encoder, còn $\sigma(\cdot)$ là hàm kích hoạt.

Vector $\mathbf{h}$ nằm ở phần trung tâm của mô hình và thường có số chiều nhỏ hơn dữ liệu ban đầu. Vì vậy, $\mathbf{h}$ có thể được coi là một biểu diễn nén của dữ liệu đầu vào và thường được sử dụng như vector đặc trưng mới của dữ liệu.

Sau khi thu được vector ẩn $\mathbf{h}$, decoder sẽ cố gắng tái tạo lại dữ liệu ban đầu từ biểu diễn này. Quá trình giải mã được mô tả bởi

$$\mathbf{X}' = g(\mathbf{h}) = \sigma(\mathbf{W}_d\mathbf{h} + \mathbf{b}_d) \tag{2.27}$$

trong đó $\mathbf{X}'$ là dữ liệu được tái tạo từ mô hình.

Mục tiêu của quá trình huấn luyện Autoencoder là làm cho dữ liệu tái tạo $\mathbf{X}'$ càng giống dữ liệu ban đầu $\mathbf{X}$ càng tốt. Điều này thường được thực hiện bằng cách tối thiểu hóa một hàm mất mát dạng

$$\mathcal{L} = \|\mathbf{X} - \mathbf{X}'\|^2 \tag{2.28}$$

Khi mô hình được huấn luyện tốt, vector $\mathbf{h}$ sẽ chứa những thông tin quan trọng nhất của dữ liệu đầu vào. Vì vậy, Autoencoder thường được sử dụng như một phương pháp *giảm chiều dữ liệu* hoặc *trích chọn đặc trưng* trong các hệ thống học máy hiện đại.

2.4. Chuẩn hóa vector đặc trưng

Trong nhiều trường hợp, các đặc trưng của dữ liệu có đơn vị đo khác nhau hoặc có phạm vi giá trị rất khác nhau. Điều này có thể ảnh hưởng đến quá trình huấn luyện mô hình. Vì vậy, ta cần chuẩn hóa các đặc trưng trước khi đưa vào thuật toán học máy.

2.4.1. Chuyển khoảng giá trị

Một phương pháp phổ biến là đưa các đặc trưng về cùng một khoảng giá trị, thường là $[0, 1]$.

$$x_i' = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)} \tag{2.29}$$

Ví dụ.

Nếu diện tích nhà nằm trong khoảng $[50, 200]$ m^2 và ta có nhà 120 m^2, giá trị chuẩn hóa sẽ là

$$\frac{120 - 50}{200 - 50} = 0.47$$

2.4.2. Chuẩn hóa theo phân phối chuẩn

Phương pháp này đưa dữ liệu về phân phối có trung bình bằng 0 và độ lệch chuẩn bằng 1.

$$x_i' = \frac{x_i - \bar{x}_i}{\sigma_i} \tag{2.30}$$

Ví dụ.

Nếu tuổi trung bình của khách hàng là 40 và độ lệch chuẩn là 10, một người 50 tuổi sẽ có giá trị chuẩn hóa

$$\frac{50 - 40}{10} = 1$$

2.4.3. Chuẩn hóa theo norm

Một phương pháp khác là chuẩn hóa vector để có độ dài bằng 1 (theo ℓ_2 norm):

$$\mathbf{x}' = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} \tag{2.31}$$

Ví dụ. Nếu $\mathbf{x} = (3, 4)$ thì $\|\mathbf{x}\|_2 = 5$ và vector sau khi chuẩn hóa là $\mathbf{x}' = (0.6, 0.8)$.
Lưu ý. Trong toán học, *norm* là một hàm dùng để đo độ lớn hoặc độ dài của một vector. Với một vector $\mathbf{x} = (x_1, x_2, \ldots, x_n)$, norm thường được ký hiệu là $\|\mathbf{x}\|$.

Một dạng tổng quát của norm thường được sử dụng là ℓ_p norm:

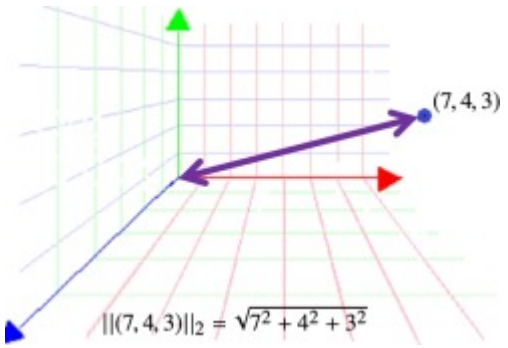
$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{\frac{1}{p}} \tag{2.32}$$

Một số norm phổ biến bao gồm:

- ℓ_1 norm (Manhattan norm): $\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$
- ℓ_2 norm (Euclid norm): $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$

- ℓ_∞ norm (max norm): $\|\mathbf{x}\|_\infty = \max_i |x_i|$

Trong học máy, ℓ_2 norm được sử dụng phổ biến nhất để chuẩn hóa vector đặc trưng, tuy nhiên ℓ_1 norm cũng thường xuất hiện trong các bài toán tối ưu và điều chuẩn (regularization).



Hình 2.4. Minh họa độ dài của một vector trong không gian ba chiều

Hình 2.4 minh họa khái niệm *norm* của một vector trong không gian ba chiều. Vector trong hình được biểu diễn bởi $\mathbf{x} = (7, 4, 3)$.

Mũi tên màu tím biểu diễn vector $\mathbf{x}$ bắt đầu từ gốc tọa độ và kết thúc tại điểm $(7, 4, 3)$ trong không gian. Độ dài của vector này chính là ℓ_2 norm của vector.

Các mũi tên màu đỏ, xanh lá cây và xanh da trời biểu diễn các thành phần của vector theo các trục tọa độ. Cụ thể, vector $\mathbf{x}$ có ba thành phần tương ứng với ba trục không gian là $x = 7$, $y = 4$ và $z = 3$. Các thành phần này cho biết mức độ đóng góp của từng trục vào vị trí cuối cùng của vector.

Độ dài của vector được tính theo công thức chuẩn Euclid:

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + x_3^2} \tag{2.33}$$

Với vector trong hình ta có:

$$\|(7, 4, 3)\|_2 = \sqrt{7^2 + 4^2 + 3^2} \tag{2.34}$$

Giá trị này biểu diễn khoảng cách từ gốc tọa độ đến điểm $(7, 4, 3)$ trong không gian ba chiều. Trong học máy, ℓ_2 norm thường được sử dụng để đo độ lớn của vector và để chuẩn hóa các vector đặc trưng trước khi đưa vào mô hình.

2.4.4. Học đặc trưng tự giám sát (Self-Supervised Feature Learning)

Trong nhiều bài toán học máy hiện đại, dữ liệu gán nhãn thường rất hạn chế trong khi dữ liệu chưa gán nhãn lại rất phong phú. Vì vậy, một hướng nghiên cứu quan trọng là *self-supervised learning*, trong đó mô hình học các đặc trưng của dữ liệu bằng cách tự tạo ra các nhiệm vụ huấn luyện từ chính dữ liệu đó.

Ý tưởng chung là xây dựng một nhiệm vụ dự đoán trung gian (*pretext task*) từ dữ liệu chưa gán nhãn, ví dụ như dự đoán một phần của dữ liệu từ phần còn lại. Thông qua quá trình này, mô hình sẽ học được các biểu diễn đặc trưng hữu ích cho nhiều bài toán khác.

Giả sử dữ liệu đầu vào là $\mathbf{x}$ và mô hình học được một biểu diễn đặc trưng

$$\mathbf{z} = f(\mathbf{x}) \tag{2.35}$$

trong đó $f(\cdot)$ là mạng nơ-ron học biểu diễn. Vector $\mathbf{z}$ sau đó có thể được sử dụng làm đặc trưng cho các bài toán khác như phân loại hoặc phát hiện đối tượng.

Một ví dụ nổi tiếng của self-supervised learning trong xử lý ngôn ngữ tự nhiên là mô hình BERT, trong đó mạng được huấn luyện bằng nhiệm vụ dự đoán các từ bị che trong câu.

2.4.5. Contrastive Learning

Một hướng tiếp cận rất phổ biến trong self-supervised learning hiện nay là *contrastive learning*. Ý tưởng của phương pháp này là học một không gian đặc trưng sao cho các mẫu dữ liệu tương tự nhau có biểu diễn gần nhau, trong khi các mẫu khác nhau có biểu diễn cách xa nhau.

Giả sử hai mẫu dữ liệu $\mathbf{x}_i$ và $\mathbf{x}_j$ là hai phiên bản biến đổi của cùng một dữ liệu ban đầu. Mô hình sẽ học các biểu diễn

$$\mathbf{z}_i = f(\mathbf{x}_i), \quad \mathbf{z}_j = f(\mathbf{x}_j) \tag{2.36}$$

Mục tiêu là làm cho hai vector đặc trưng này gần nhau trong không gian đặc trưng. Điều này thường được thực hiện bằng một hàm mất mát dạng contrastive loss:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_k \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)} \tag{2.37}$$

trong đó $\text{sim}(\cdot, \cdot)$ là hàm đo độ tương tự (ví dụ cosine similarity) và τ là một tham số nhiệt độ.

Một số mô hình nổi tiếng sử dụng contrastive learning bao gồm *SimCLR* trong thị giác máy tính và *CLIP* trong các hệ thống học đa phương thức (kết hợp hình ảnh và văn bản).

Các phương pháp này đã chứng minh rằng mô hình có thể học được các đặc trưng rất mạnh từ dữ liệu chưa gán nhãn và sau đó chỉ cần một lượng nhỏ dữ liệu gán nhãn để đạt được hiệu suất cao trong các bài toán học máy.

2.5. Kết luận

Trong chương này, chúng ta đã nghiên cứu các kỹ thuật xây dựng và trích chọn đặc trưng (feature engineering và feature selection), một bước quan trọng trong quá trình xây dựng mô hình học máy. Chất lượng của các đặc trưng đầu vào có ảnh hưởng trực tiếp đến hiệu quả và khả năng dự đoán của mô hình.

Trước hết, chương đã trình bày khái niệm đặc trưng và vai trò của đặc trưng trong các bài toán học máy. Một bộ đặc trưng tốt giúp mô hình học được các quy luật quan trọng trong dữ liệu, từ đó cải thiện độ chính xác và khả năng tổng quát hóa của mô hình.

Tiếp theo, chúng ta đã tìm hiểu các kỹ thuật xây dựng đặc trưng (feature construction), bao gồm việc tạo ra các đặc trưng mới từ các đặc trưng ban đầu thông qua các phép biến đổi toán học hoặc các phép kết hợp đặc trưng. Những kỹ thuật này giúp mô hình có thể biểu diễn tốt hơn các mối quan hệ phức tạp trong dữ liệu.

Chương cũng đã trình bày các phương pháp trích chọn đặc trưng (feature selection), nhằm lựa chọn một tập con các đặc trưng quan trọng nhất từ tập đặc trưng ban đầu. Việc loại bỏ các đặc trưng không cần thiết giúp giảm độ phức tạp của mô hình, giảm hiện tượng quá khớp và cải thiện tốc độ huấn luyện.

Ngoài ra, chương còn giới thiệu một số kỹ thuật giảm chiều dữ liệu như Principal Component Analysis (PCA), giúp biểu diễn dữ liệu trong không gian có số chiều thấp hơn nhưng vẫn giữ được phần lớn thông tin của dữ liệu ban đầu.

Nhìn chung, việc xây dựng và lựa chọn đặc trưng là một bước rất quan trọng trong quy trình xây dựng mô hình học máy. Trong nhiều trường hợp, việc lựa chọn đặc trưng phù hợp có thể mang lại hiệu quả cải thiện mô hình đáng kể, thậm chí quan trọng hơn cả việc lựa chọn thuật toán học máy.

2.6. Câu hỏi ôn tập

Câu hỏi lý thuyết

a. Trình bày khái niệm đặc trưng (feature) trong học máy. Tại sao đặc trưng lại đóng vai trò quan trọng trong việc xây dựng mô hình học máy?

b. Phân biệt hai khái niệm: xây dựng đặc trưng (feature engineering) và trích chọn đặc trưng (feature selection).

c. Nêu một số phương pháp phổ biến để xây dựng đặc trưng mới từ dữ liệu ban đầu.

d. Trình bày mục tiêu của việc trích chọn đặc trưng. Những lợi ích nào có thể đạt được khi giảm số lượng đặc trưng của dữ liệu?

e. Phân biệt ba nhóm phương pháp trích chọn đặc trưng:

- Filter methods
- Wrapper methods
- Embedded methods

f. Trình bày ý tưởng cơ bản của phương pháp Principal Component Analysis (PCA) trong giảm chiều dữ liệu.

g. Nêu một số khó khăn có thể gặp phải khi làm việc với dữ liệu có số chiều rất lớn.

Bài tập thực hành

h. Cho tập dữ liệu gồm hai đặc trưng x_1 và x_2. Hãy đề xuất ít nhất ba đặc trưng mới có thể được tạo ra từ hai đặc trưng ban đầu này.

i. Cho tập dữ liệu gồm năm đặc trưng:

$$x_1, x_2, x_3, x_4, x_5$$

Giả sử kết quả đánh giá mức độ quan trọng của các đặc trưng như sau:

$$\begin{aligned} &score(x_1) = 0.9, \quad score(x_2) = 0.7, \\ &score(x_3) = 0.2, \quad score(x_4) = 0.6, \\ &score(x_5) = 0.1 \end{aligned}$$

a. Nếu chọn ba đặc trưng quan trọng nhất, hãy xác định tập đặc trưng được giữ lại.

b. Giải thích vì sao việc loại bỏ các đặc trưng không quan trọng có thể cải thiện hiệu quả mô hình.

j. Cho tập dữ liệu gồm hai đặc trưng x_1 và x_2 có tương quan mạnh với nhau.

a. Giải thích tại sao việc tồn tại các đặc trưng tương quan cao có thể gây khó khăn cho một số mô hình học máy.

b. Hãy đề xuất một phương pháp giảm chiều dữ liệu để giải quyết vấn đề này.

Chương 3

Gradient Descent

Chương này trình bày về thuật toán Gradient Descent (Hạ đạo hàm), một phương pháp tối ưu được sử dụng rộng rãi trong học máy. Thuật toán này tìm cực tiểu của hàm mất mát bằng cách cập nhật các tham số theo hướng ngược với đạo hàm của hàm mục tiêu. Để thuận tiện chúng ta sẽ ký hiệu thuật toán này có tên là GD.

3.1. Giới thiệu

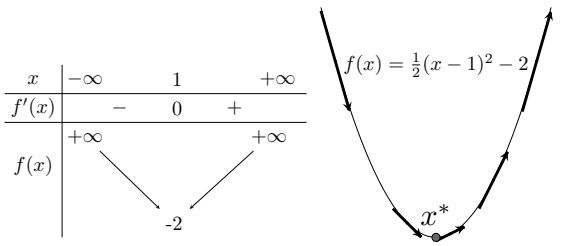
Xét một hàm số $f : \mathbb{R}^d \to \mathbb{R}$ với tập xác định $\mathcal{D}$,

- Điểm $\mathbf{x}^* \in \mathcal{D}$ được gọi là *cực tiểu toàn cục* (tương ứng *cực đại toàn cục*) nếu $f(\mathbf{x}) \geq f(\mathbf{x}^*)$ (tương ứng $f(\mathbf{x}) \leq f(\mathbf{x}^*)$) với mọi $\mathbf{x}$ trong tập xác định $\mathcal{D}$. Các điểm cực tiểu toàn cục và cực đại toàn cục được gọi chung là *cực trị toàn cục*.

- Điểm $\mathbf{x}^* \in \mathcal{D}$ được gọi là *cực tiểu địa phương* (tương ứng *cực đại địa phương*) nếu tồn tại $\varepsilon > 0$ sao cho $f(\mathbf{x}) \geq f(\mathbf{x}^*)$ (tương ứng $f(\mathbf{x}) \leq f(\mathbf{x}^*)$) với mọi $\mathbf{x}$ nằm trong lân cận $\mathcal{V}(\varepsilon) = \{\mathbf{x} : \mathbf{x} \in \mathcal{D}, d(\mathbf{x}, \mathbf{x}^*) \leq \varepsilon\}$. Ở đây $d(\mathbf{x}, \mathbf{x}^*)$ ký hiệu khoảng cách giữa hai vector $\mathbf{x}$ và $\mathbf{x}^*$, thường là khoảng cách Euclid. Các điểm cực tiểu địa phương và cực đại địa phương được gọi chung là *cực trị địa phương*. Các điểm cực tiểu/cực đại/cực trị toàn cục cũng là các điểm cực tiểu/cực đại/cực trị địa phương.

Giả sử ta đang quan tâm đến một hàm liên tục một biến có đạo hàm mọi nơi, xác định trên $\mathbb{R}$. Cùng nhắc lại một vài điểm cơ bản:

- Điểm cực tiểu địa phương x^* của hàm số là điểm có đạo hàm $f'(x^*)$ bằng không. Hơn nữa, trong lân cận của nó, đạo hàm của các điểm phía bên trái x^* là không dương, đạo hàm của các điểm phía bên phải x^* là không âm.



Hình 3.1. Khảo sát sự biến thiên của một đa thức bậc hai.

- Đường tiếp tuyến với đồ thị hàm số đó tại một điểm bất kỳ có hệ số góc bằng đạo hàm của hàm số tại điểm đó.

Hình 3.1 mô tả sự biến thiên của hàm số $f(x) = \frac{1}{2}(x-1)^2 - 2$. Điểm $x^* = 1$ là một điểm cực tiểu toàn cục của hàm số này. Các điểm bên trái của $\mathbf{x}^*$ có đạo hàm âm, các điểm bên phải có đạo hàm dương. Với hàm số này, càng xa về phía trái của $\mathbf{x}^*$ thì đạo hàm càng âm, càng xa về phía phải thì đạo hàm càng dương.

Trong học máy nói riêng và toán tối ưu nói chung, chúng ta thường xuyên phải tìm các cực tiểu toàn cục của một hàm số. Nếu chỉ xét riêng các hàm khả vi, việc giải phương trình đạo hàm bằng không có thể phức tạp hoặc có vô số nghiệm. Thay vào đó, người ta thường tìm các điểm cực tiểu địa phương, và coi đó là một nghiệm cần tìm của bài toán trong những trường hợp nhất định.

Các điểm cực tiểu địa phương là nghiệm của phương trình đạo hàm bằng không (ta vẫn đang giả sử rằng các hàm này liên tục và khả vi). Nếu tìm được toàn bộ (hữu hạn) các điểm cực tiểu địa phương, ta chỉ cần thay từng điểm đó vào hàm số để suy ra điểm cực tiểu toàn cục. Tuy nhiên, trong hầu hết các trường hợp, việc giải phương trình đạo hàm bằng không là bất khả thi. Nguyên nhân có thể đến từ sự phức tạp của đạo hàm, từ việc các điểm dữ liệu có số chiều lớn hoặc từ việc có quá nhiều điểm dữ liệu. Thực tế cho thấy, trong nhiều bài toán machine learning, các điểm cực tiểu địa phương thường cho kết quả tốt, đặc biệt là trong các mạng neuron nhân tạo.

Một hướng tiếp cận phổ biến để giải quyết các bài toán tối ưu là dùng một phép toán. Đầu tiên, chọn một *điểm xuất phát* rồi tiến dần đến *đích* sau mỗi vòng lặp. Hạ Gradient - Gradient descent (GD) và các biến thể của nó là một trong những phương pháp được dùng nhiều nhất.

Chú ý: Khái niệm *nghiệm* của một bài toán tối ưu được sử dụng không hẳn để chỉ cực tiểu toàn cục. Nó được sử dụng theo nghĩa là kết quả của quá trình tối ưu. Kết quả ở một vòng lặp trung gian được gọi là *vị trí của nghiệm*. Nói cách khác, *nghiệm* có thể được hiểu là giá trị hiện tại của tham số cần tìm trong quá trình tối ưu.

3.2. GD cho hàm một biến

Xét các hàm số một biến $f : \mathbb{R} \to \mathbb{R}$. Quay trở lại Hình 3.1 và một vài quan sát đã nêu. Giả sử x_t là điểm tìm được sau vòng lặp thứ t. Ta cần tìm một thuật toán để đưa x_t về càng gần x^* càng tốt. Có hai quan sát sau đây:

- Nếu đạo hàm của hàm số tại x_t là dương ($f'(x_t) > 0$) thì x_t nằm về bên phải so với x^*, và ngược lại. Để điểm tiếp theo x_{t+1} gần với x^* hơn, ta cần di chuyển x_t về bên trái, tức về phía *âm*. Nói các khác, *ta cần di chuyển x_t ngược dấu với đạo hàm*:
$$x_{t+1} = x_t + \Delta. \tag{3.1}$$
Trong đó Δ là một đại lượng ngược dấu với đạo hàm $f'(x_t)$.

- x_t càng xa x^* về bên phải thì $f'(x_t)$ càng lớn (và ngược lại). Một cách tự nhiên nhất, ta chọn lượng di chuyển Δ tỉ lệ thuận với $-f'(x_t)$.

Từ hai nhận xét trên, ta có công thức cập nhật đơn giản là:

$$\boxed{x_{t+1} = x_t - \eta f'(x_t)} \tag{3.2}$$

Trong đó η là một số dương được gọi là *tốc độ học* (*learning rate*). Dấu trừ thể hiện việc x_t cần *đi ngược* với đạo hàm $f'(x_t)$. Tên gọi *Hạ gradient* xuất phát từ đây. Mặc dù các quan sát này không đúng trong mọi trường hợp, chúng vẫn là nền tảng cho rất nhiều phương pháp tối ưu.

3.2.1. Ví dụ đơn giản với Python

Xét hàm số $f(x) = x^2 + 5\sin(x)$ với đạo hàm $f'(x) = 2x + 5\cos(x)$. Giả sử xuất phát từ một điểm x_0, quy tắc cập nhật tại vòng lặp thứ t là:

$$x_{t+1} = x_t - \eta(2x_t + 5\cos(x_t)). \tag{3.3}$$

Khi thực hiện trên Python, ta cần viết các hàm số

a. `grad` để tính đạo hàm.

b. `cost` để tính giá trị của hàm số. Ta không sử dụng hàm này trong thuật toán cập nhật nghiệm. Tuy nhiên, nó vẫn đóng vai trò quan trọng trong việc kiểm tra tính chính xác của đạo hàm và sự biến thiên của hàm số sau mỗi vòng lặp.

c. `myGD1` là phần chính thực hiện thuật toán GD. Đầu vào của hàm số này là điểm xuất phát `x0` và tốc độ học `eta`. Đầu ra là nghiệm của bài toán. Thuật toán dừng lại khi đạo hàm đủ nhỏ.

```
def grad(x):
    return 2*x+ 5*np.cos(x)

def cost(x):
    return x**2 + 5*np.sin(x)
```

```
def myGD1(x0, eta):
    x = [x0]
    for it in range(100):
        x_new = x[-1] - eta*grad(x[-1])
        if abs(grad(x_new)) < 1e-3: # just a small number
            break
        x.append(x_new)
    return (x, it)
```

Điểm xuất phát khác nhau

Sau khi đã có các hàm cần thiết, chúng ta thử tìm nghiệm với các điểm xuất phát khác nhau là $x_0 = -5$ và $x_0 = 5$ với cùng tốc độ học $\eta = 0.1$.

```
(x1, it1) = myGD1(-5, .1)
(x2, it2) = myGD1(5, .1)
print('Solution x1 = %f, cost = %f, after %d iterations'\
      %(x1[-1], cost(x1[-1]), it1))
print('Solution x2 = %f, cost = %f, after %d iterations'\
      %(x2[-1], cost(x2[-1]), it2))
```

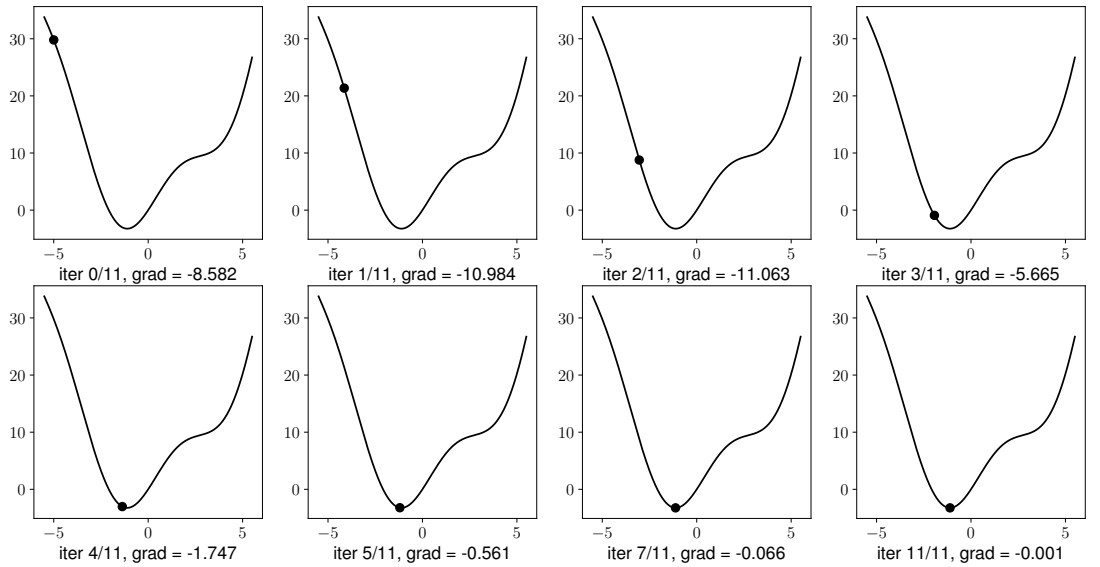
Kết quả:

```
Solution x1 = -1.110667, cost = -3.246394, after 11 iterations
Solution x2 = -1.110341, cost = -3.246394, after 29 iterations
```

Như vậy, thuật toán trả về kết quả gần giống nhau với các điểm xuất phát khác nhau, nhưng tốc độ hội tụ khác nhau. Hình 3.2 và Hình 3.3 thể hiện vị trí của x_t và đạo hàm qua các vòng lặp với cùng tốc độ học $\eta = 0.1$ nhưng điểm xuất phát khác nhau tại -5 và 5.

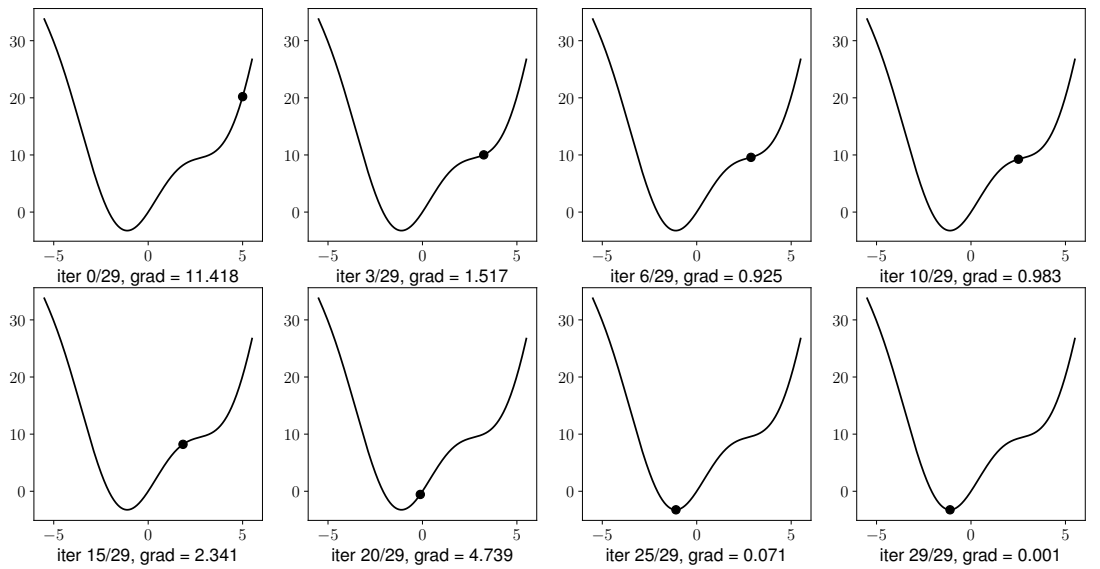
Hình 3.2 tương ứng với $x_0 = -5$, thuật toán hội tụ nhanh hơn. Hơn nữa, đường đi tới đích khá suôn sẻ với đạo hàm luôn âm và trị tuyệt đối của đạo hàm nhỏ dần khi x_t tiến gần tới đích.

Hình 3.3 tương ứng với $x_0 = 5$, đường đi của x_t chứa một khu vực có đạo hàm khá nhỏ gần điểm có hoành độ bằng 2.5. Điều này khiến thuật toán la cà ở đây khá lâu. Khi vượt qua được điểm này thì mọi việc diễn ra tốt đẹp. Các điểm không phải là điểm cực tiểu nhưng có đạo hàm gần bằng không rất dễ gây ra hiện tượng x_t bị *bẫy* vì đạo hàm nhỏ khiến nó không thay đổi nhiều ở vòng lặp



.

Hình 3.2. Kết quả tìm được qua các vòng lặp với $x_0 = -5, \eta = 0.1$



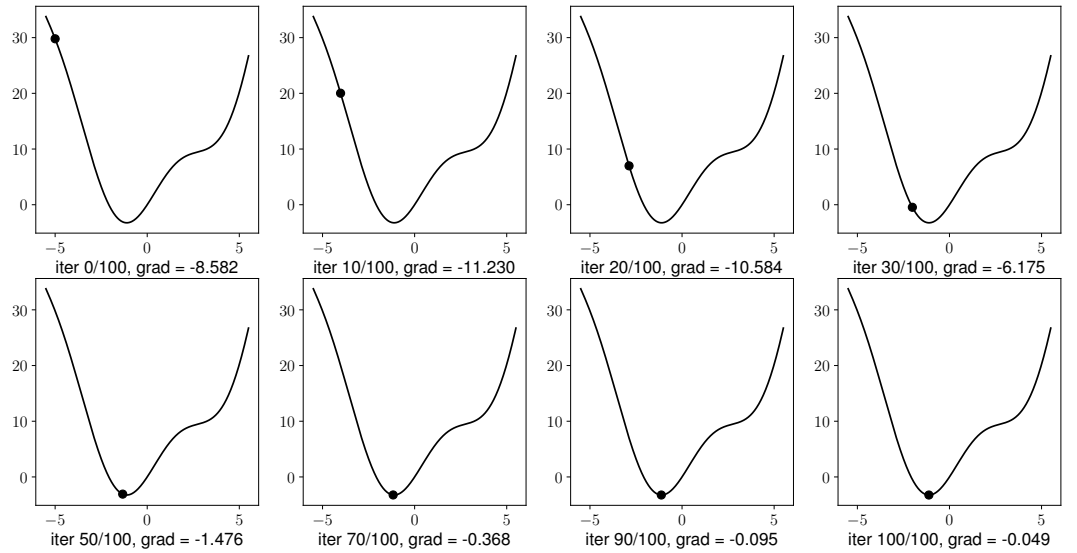
.

Hình 3.3. Kết quả tìm được qua các vòng lặp với $x_0 = 5, \eta = 0.1$

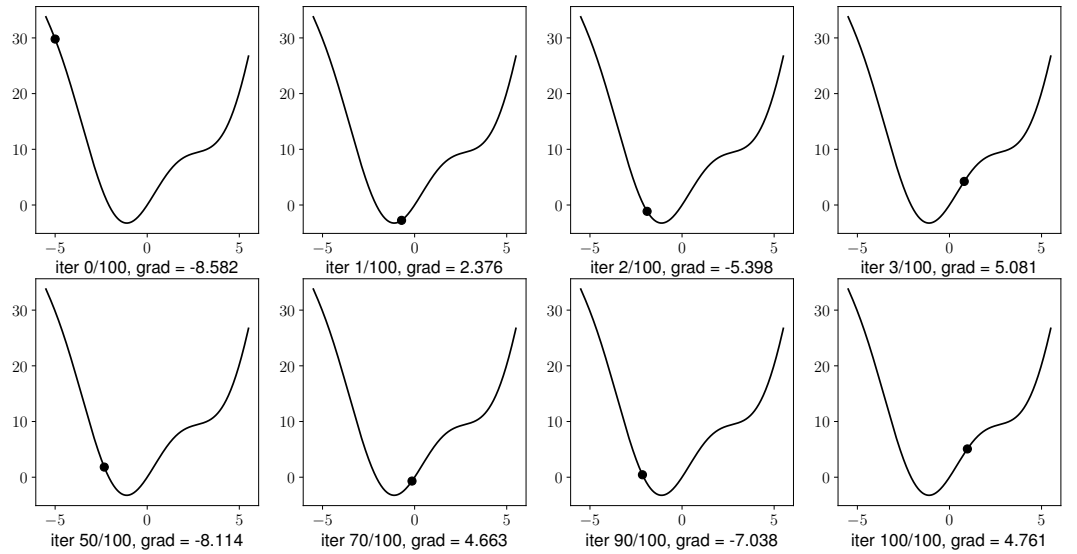
tiếp theo. Chúng ta sẽ thấy một kỹ thuật khác giúp thuật toán *thoát* những chiếc bẫy này.

Tốc độ học khác nhau

Tốc độ hội tụ của GD không những phụ thuộc vào điểm xuất phát mà còn phụ thuộc vào tốc độ học. Hình 3.4 và Hình 3.5 thể hiện vị trí của x_t qua các vòng



Hình 3.4. Kết quả tìm được qua các vòng lặp với $x_0 = -5$, $\eta = 0.01$.



.

Hình 3.5. Kết quả tìm được qua các vòng lặp với $x_0 = -5, \eta = 0.5$

lặp với cùng điểm xuất phát $x_0 = -5$ nhưng tốc độ học khác nhau. Ta quan sát thấy hai điều:

- Với tốc độ học nhỏ $\eta = 0.01$ (Hình 3.4), tốc độ hội tụ rất chậm. Trong ví dụ này ta chọn tối đa 100 vòng lặp nên thuật toán dừng lại trước khi tới đích, mặc dù đã rất gần. Trong thực tế, khi việc tính toán trở nên phức tạp, tốc độ học quá thấp sẽ ảnh hưởng nhiều tới tốc độ của thuật toán. Thậm chí x_t có thể không bao giờ tới được đích.

- Với tốc độ học lớn $\eta = 0.5$ (Hình 3.5), x_t tiến nhanh tới gần đích sau vài vòng lặp. Tuy nhiên, thuật toán không hội tụ được vì sự thay đổi vị trí của x_t sau mỗi vòng lặp là quá lớn, khiến x_t dao động quanh đích nhưng không tới được đích.

Việc lựa chọn tốc độ học rất quan trọng. Tốc độ học thường được chọn thông qua các thí nghiệm. Ngoài ra, GD có thể làm việc hiệu quả hơn bằng cách chọn tốc độ học khác nhau ở mỗi vòng lặp. Trên thực tế, một kỹ thuật thường được sử dụng có tên là *suy giảm tốc độ học* (*learning rate decay*). Trong kỹ thuật này, tốc độ học được giảm đi sau một vài vòng lặp để nghiệm không bị dao động mạnh khi gần đích hơn.

3.3. Hạ gradient cho hàm nhiều biến

Giả sử ta cần tìm cực tiểu toàn cục cho hàm $f(\theta)$ trong đó θ là tập hợp các tham số cần tối ưu. Gradient của hàm số đó tại một điểm θ bất kỳ được ký hiệu là $\nabla_\theta f(\theta)$. Tương tự như hàm một biến, thuật toán GD cho hàm nhiều biến cũng bắt đầu bằng một điểm dự đoán θ_0, sau đó sử dụng quy tắc cập nhật:

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta f(\theta_t)} \tag{3.4}$$

Hoặc viết dưới dạng đơn giản hơn: $\theta \leftarrow \theta - \eta \nabla_\theta f(\theta)$.

Minh họa Gradient Descent với một hàm mất mát dạng bình phương

Để minh họa cách hoạt động của thuật toán hạ dốc, ta xét một bài toán đơn giản với hàm mất mát có dạng bình phương. Giả sử ta có một tập dữ liệu gồm N điểm dữ liệu với vector đặc trưng $\mathbf{x}_i \in \mathbb{R}^d$ và giá trị mục tiêu tương ứng y_i. Gọi $\mathbf{X}$ là ma trận dữ liệu và $\mathbf{y}$ là vector chứa các giá trị mục tiêu.

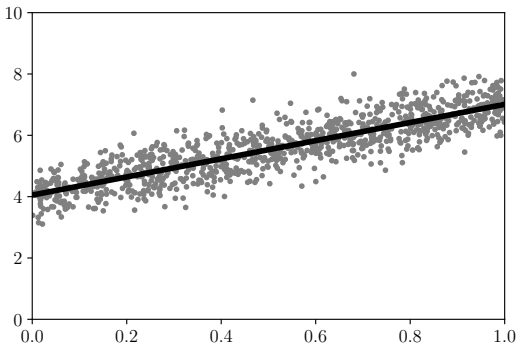
Ta xét hàm mất mát có dạng:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}^T \mathbf{w}\|_2^2 \tag{3.5}$$

Hàm mất mát này đo lường sai khác giữa giá trị dự đoán và giá trị thực tế của dữ liệu. Đạo hàm của hàm mất mát theo vector tham số $\mathbf{w}$ được tính như sau:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}(\mathbf{X}^T \mathbf{w} - \mathbf{y}). \tag{3.6}$$

Từ đạo hàm này, ta có thể áp dụng thuật toán hạ dốc để cập nhật các tham số $\mathbf{w}$ theo công thức:



Hình 3.6. Nghiệm của bài toán hồi quy tuyến tính (đường thẳng màu đen) tìm được bằng thư viện scikit-learn.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \tag{3.7}$$

trong đó η là tốc độ học (learning rate).

Ví dụ trên Python

Trước tiên, chúng ta tạo 1000 điểm dữ liệu gần đường thẳng $y = 4 + 3x$ rồi dùng thư viện scikit-learn để tìm nghiệm cho hồi quy tuyến tính:

```
from sklearn.linear_model import LinearRegression
X = np.random.rand(1000)
y = 4 + 3 * X + .5*np.random.randn(1000) # noise added
model = LinearRegression()
model.fit(X.reshape(-1, 1), y.reshape(-1, 1))
w, b = model.coef_[0][0], model.intercept_[0]
sol_sklearn = np.array([b, w])
print(sol_sklearn)
```

Kết quả:

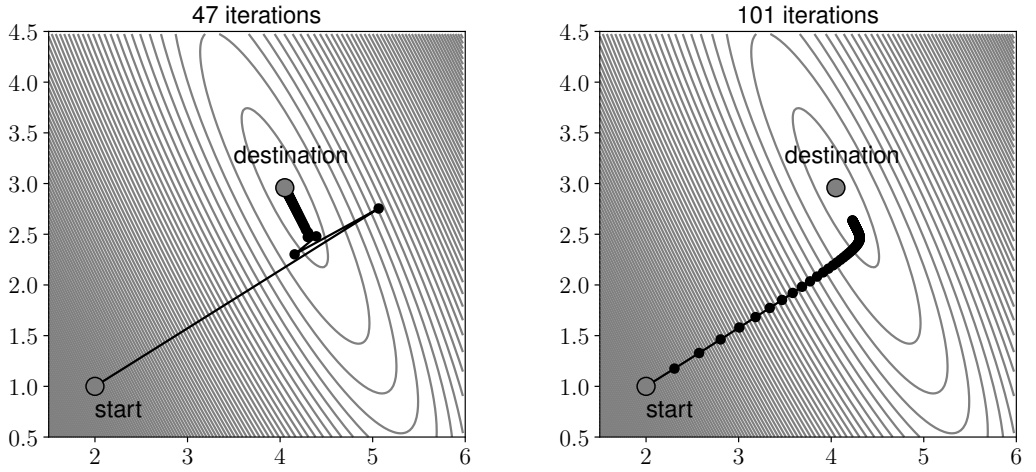
```
Solution found by sklearn: [ 3.94323245  3.12067542]
```

Các điểm dữ liệu và đường thẳng tìm được bằng hồi quy tuyến tính có phương trình $y \approx 3.94 + 3.12x$ được minh hoạ trong Hình 3.6. Nghiệm tìm này được rất gần với mong đợi.

Tiếp theo, ta sẽ thực hiện tìm nghiệm bằng GD. Ta cần viết hàm mất mát và gradient theo $\mathbf{w}$. Chú ý rằng ở đây $\mathbf{w}$ đã bao gồm hệ số điều chỉnh b.

```
def grad(w):
    N = Xbar.shape[0]
    return 1/N * Xbar.T.dot(Xbar.dot(w) - y)

def cost(w):
    N = Xbar.shape[0]
    return .5/N*np.linalg.norm(y - Xbar.dot(w))**2
```



(a) $\eta = 1$. **(b)** $\eta = 0.1$.

Hình 3.7. Đường đi nghiệm của hồi quy tuyến tính với các tốc độ học khác nhau.

```
def myGD(w_init, grad, eta):
    w = [w_init]
    for it in range(100):
        w_new = w[-1] - eta*grad(w[-1])
        if np.linalg.norm(grad(w_new))/len(w_new) < 1e-3:
            break
        w.append(w_new)
    return w, it

one = np.ones((X.shape[0],1))
Xbar = np.concatenate((one, X.reshape(-1, 1)), axis = 1)
w_init = np.array([[2], [1]])
w1, it1 = myGD(w_init, grad, 1)
print('Sol found by GD: w = ', w1[-1].T, ', after %d iterations.' %(it1+1))
```

Kết quả:

```
Sol found by GD: w =  [ 3.99026984  2.98702942] , after 49 iterations.
```

Thuật toán hội tụ tới kết quả khá gần với nghiệm tìm được theo scikit-learn sau 49 vòng lặp. Hình 3.7 mô tả đường đi của $\mathbf{w}$ với cùng điểm xuất phát nhưng tốc độ học khác nhau. Các điểm được đánh dấu 'start' là các điểm xuất phát. Các điểm được đánh dấu 'destination' là nghiệm tìm được bằng thư viện scikit-learn. Các điểm hình tròn nhỏ màu đen là vị trí của $\mathbf{w}$ qua các vòng lặp trung gian. Ta thấy rằng khi $\eta = 1$, thuật toán hội tụ tới rất gần đích theo thư viện sau 49 vòng lặp. Với tốc độ học nhỏ hơn, $\eta = 0.1$, nghiệm vẫn còn cách xa đích sau hơn 100 vòng lặp. Như vậy, việc chọn tốc độ học hợp lý là rất quan trọng.

Tóm tắt Gradient Descent (một biến và đa biến)

Gradient Descent *là thuật toán tối ưu được sử dụng để tìm giá trị nhỏ nhất của một hàm số bằng cách cập nhật tham số theo hướng ngược lại với đạo hàm (hoặc gradient) của hàm mất mát.*
Trường hợp một biến. *Với hàm mục tiêu $f(w)$, đạo hàm theo w là $f'(w)$. Quy tắc cập nhật của Gradient Descent được viết:*

$$w \leftarrow w - \eta f'(w) \tag{3.8}$$

trong đó $\eta > 0$ là tốc độ học (learning rate).
Trường hợp nhiều biến. *Với hàm mục tiêu $f(\mathbf{w})$ và vector tham số $\mathbf{w} = [w_1, w_2, \ldots, w_d]^T$, ta sử dụng gradient:*

$$\nabla f(\mathbf{w}) = \left[\frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}, \ldots, \frac{\partial f}{\partial w_d}\right]^T \tag{3.9}$$

Quy tắc cập nhật tham số:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla f(\mathbf{w}) \tag{3.10}$$

Một số chú ý quan trọng:

- *Tốc độ học η ảnh hưởng lớn tới quá trình hội tụ. Nếu η quá lớn, thuật toán có thể dao động hoặc không hội tụ. Nếu η quá nhỏ, quá trình tối ưu sẽ rất chậm.*
- *Gradient Descent thường được lặp lại nhiều lần cho tới khi gradient gần bằng 0 hoặc khi hàm mất mát thay đổi rất nhỏ giữa hai bước cập nhật liên tiếp.*
- *Trong nhiều bài toán học máy, hàm mục tiêu là hàm lồi, khi đó Gradient Descent có thể tìm được nghiệm tối ưu toàn cục.*

3.4. Các biến thể của Gradient Descent

Trong thực tế, khi làm việc với các tập dữ liệu lớn, việc tính gradient trên toàn bộ dữ liệu ở mỗi bước cập nhật có thể rất tốn kém về mặt tính toán. Vì vậy, nhiều biến thể của Gradient Descent đã được đề xuất nhằm tăng tốc quá trình học và cải thiện khả năng hội tụ. Ba biến thể phổ biến nhất là *Batch Gradient Descent*, *Stochastic Gradient Descent* và *Mini-batch Gradient Descent*.

3.4.1. Batch Gradient Descent

Batch Gradient Descent là dạng chuẩn của Gradient Descent. Trong mỗi bước cập nhật, gradient được tính dựa trên toàn bộ tập dữ liệu huấn luyện.

Giả sử tập dữ liệu gồm N điểm dữ liệu $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)$. Gradient của hàm mất mát được tính trên toàn bộ dữ liệu:

$$\nabla J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \nabla J_i(\mathbf{w}) \tag{3.11}$$

Sau đó các tham số được cập nhật theo quy tắc:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w}) \tag{3.12}$$

Ưu điểm của Batch Gradient Descent là quá trình tối ưu ổn định vì gradient được tính chính xác trên toàn bộ dữ liệu. Tuy nhiên, khi tập dữ liệu rất lớn, việc tính gradient trên toàn bộ dữ liệu ở mỗi bước cập nhật sẽ làm cho quá trình huấn luyện trở nên chậm.

3.4.2. Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) khắc phục nhược điểm trên bằng cách cập nhật tham số dựa trên *một điểm dữ liệu duy nhất* tại mỗi bước lặp.

Cụ thể, tại bước lặp thứ t, ta chọn ngẫu nhiên một điểm dữ liệu $(\mathbf{x}_i, y_i)$ và cập nhật tham số theo công thức:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J_i(\mathbf{w}) \tag{3.13}$$

Trong đó $\nabla J_i(\mathbf{w})$ là gradient của hàm mất mát tương ứng với điểm dữ liệu thứ i.

Ưu điểm của SGD là mỗi bước cập nhật rất nhanh và thuật toán có thể bắt đầu học ngay cả khi chưa xử lý toàn bộ dữ liệu. Tuy nhiên, do gradient được tính trên một điểm dữ liệu nên quá trình hội tụ thường dao động mạnh quanh nghiệm tối ưu.

3.4.3. Mini-batch Gradient Descent

Mini-batch Gradient Descent là một phương pháp trung gian giữa hai cách tiếp cận trên. Thay vì sử dụng toàn bộ dữ liệu hoặc chỉ một điểm dữ liệu, thuật toán sử dụng một tập con nhỏ gồm m điểm dữ liệu gọi là *mini-batch.*

Gradient được tính trên mini-batch:

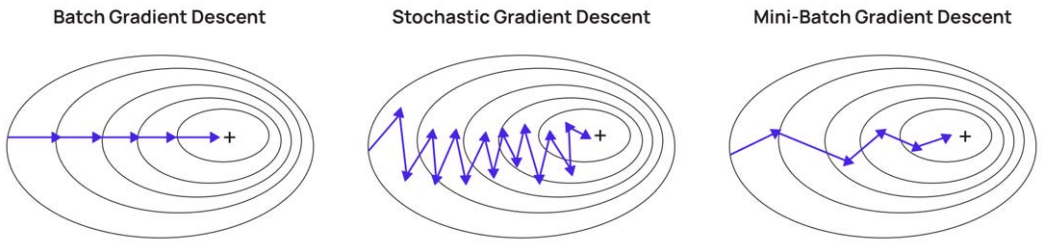
$$\nabla J(\mathbf{w}) = \frac{1}{m} \sum_{i \in B} \nabla J_i(\mathbf{w}) \tag{3.14}$$

trong đó B là tập các chỉ số của mini-batch.

Sau đó tham số được cập nhật theo:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w}) \tag{3.15}$$

Mini-batch Gradient Descent thường được sử dụng nhiều nhất trong các hệ thống học sâu hiện đại vì nó cân bằng được giữa độ ổn định của Batch Gradient Descent và tốc độ của Stochastic Gradient Descent. Các giá trị mini-batch phổ biến thường nằm trong khoảng từ 16 đến 256.



Hình 3.8. So sánh ba biến thể của Gradient Descent: Batch GD, Stochastic GD và Mini-batch GD.

Hình 3.8 minh họa sự khác nhau trong quá trình cập nhật tham số của ba biến thể phổ biến của thuật toán Gradient Descent. Các đường cong trong hình biểu diễn các đường đồng mức của hàm mất mát, trong khi các mũi tên thể hiện quỹ đạo di chuyển của nghiệm trong quá trình tối ưu.

Trong *Batch Gradient Descent*, gradient được tính dựa trên toàn bộ tập dữ liệu huấn luyện ở mỗi bước lặp. Do đó, hướng di chuyển của nghiệm thường ổn định và đi tương đối thẳng về phía điểm cực tiểu của hàm mất mát. Như minh họa ở hình bên trái, quỹ đạo hội tụ khá mượt và ít dao động. Tuy nhiên, khi tập dữ liệu lớn, việc tính gradient trên toàn bộ dữ liệu ở mỗi bước cập nhật sẽ tốn nhiều chi phí tính toán và làm cho quá trình huấn luyện trở nên chậm.

Trong *Stochastic Gradient Descent* (SGD), tham số được cập nhật dựa trên từng điểm dữ liệu riêng lẻ. Vì mỗi điểm dữ liệu chỉ cung cấp một ước lượng xấp xỉ của gradient thật sự, hướng cập nhật có thể thay đổi mạnh giữa các bước lặp. Điều này dẫn đến quỹ đạo tối ưu có dạng zig-zag như minh họa ở hình giữa. Mặc dù

SGD có thể hội tụ nhanh trong giai đoạn đầu, nhưng nghiệm thường dao động quanh điểm tối ưu thay vì hội tụ một cách ổn định.

Mini-batch Gradient Descent là phương pháp trung gian giữa hai cách tiếp cận trên. Thay vì sử dụng toàn bộ dữ liệu hoặc một điểm dữ liệu duy nhất, gradient được tính trên một nhóm nhỏ các điểm dữ liệu gọi là mini-batch. Như thể hiện ở hình bên phải, quỹ đạo hội tụ ổn định hơn so với SGD nhưng vẫn nhanh hơn Batch Gradient Descent. Trong thực tế, Mini-batch Gradient Descent là phương pháp được sử dụng phổ biến nhất trong các hệ thống học máy và học sâu hiện đại.

3.5. Kết luận

Trong chương này, chúng ta đã tìm hiểu về thuật toán hạ dốc (*Gradient Descent*), một phương pháp tối ưu cơ bản và được sử dụng rộng rãi trong học máy. Ý tưởng chính của thuật toán là cập nhật các tham số của mô hình theo hướng ngược với đạo hàm (gradient) của hàm mất mát nhằm tìm giá trị nhỏ nhất của hàm mục tiêu.

Chúng ta đã trình bày quy tắc cập nhật tham số trong trường hợp hàm một biến và trường hợp nhiều biến, đồng thời minh họa quá trình hội tụ của thuật toán thông qua các đường đồng mức của hàm mất mát. Vai trò của tốc độ học (learning rate) cũng được phân tích để cho thấy ảnh hưởng của nó đến tốc độ và độ ổn định của quá trình tối ưu.

Ngoài ra, chương cũng đã giới thiệu ba biến thể phổ biến của Gradient Descent là Batch Gradient Descent, Stochastic Gradient Descent và Mini-batch Gradient Descent. Mỗi phương pháp có đặc điểm riêng về chi phí tính toán, tốc độ cập nhật và độ ổn định trong quá trình hội tụ. Trong thực tế, Mini-batch Gradient Descent thường được sử dụng nhiều nhất trong các hệ thống học máy và học sâu hiện đại.

3.6. Câu hỏi ôn tập

a. Trình bày ý tưởng cơ bản của thuật toán Gradient Descent. Vì sao việc cập nhật tham số theo hướng ngược với đạo hàm của hàm mất mát lại giúp tìm được cực tiểu của hàm?

b. Viết công thức cập nhật tham số của Gradient Descent trong trường hợp hàm mục tiêu một biến.

c. Với hàm mục tiêu nhiều biến $f(\mathbf{w})$, hãy viết công thức gradient và quy tắc cập nhật tham số của Gradient Descent.

d. Đường đồng mức của một hàm số là gì? Giải thích vai trò của đường đồng mức trong việc minh họa quá trình hội tụ của Gradient Descent.

e. Tốc độ học (learning rate) là gì? Điều gì xảy ra nếu giá trị của tốc độ học quá lớn hoặc quá nhỏ?

f. Phân biệt Batch Gradient Descent, Stochastic Gradient Descent và Mini-batch Gradient Descent.

g. (Thực hành) Viết chương trình Python sử dụng Gradient Descent để tìm giá trị nhỏ nhất của hàm một biến

$$f(x) = x^2 + 4x + 4.$$

Vẽ đồ thị của hàm và minh họa quá trình hội tụ của thuật toán.

h. (Thực hành) Sử dụng thư viện `scikit-learn` hoặc viết cài đặt từ đầu để huấn luyện một mô hình hồi quy tuyến tính bằng Gradient Descent trên một bộ dữ liệu đơn giản (ví dụ: dự đoán giá nhà theo diện tích). So sánh kết quả khi sử dụng Batch Gradient Descent và Stochastic Gradient Descent.

Chương 4

Hồi quy tuyến tính

Hồi quy tuyến tính (linear regression) là một phương pháp học có giám sát được sử dụng trong các bài toán hồi quy, trong đó mối quan hệ giữa biến đầu vào và biến đầu ra được mô hình hóa bằng một hàm tuyến tính. Đây là một trong những mô hình cơ bản và đơn giản nhất trong học máy, đồng thời cũng là nền tảng cho nhiều phương pháp hồi quy phức tạp hơn.

4.1. Phát biểu bài toán

Xét bài toán dự đoán giá của một căn nhà dựa trên một số đặc trưng cơ bản của nó. Giả sử mỗi căn nhà được mô tả bởi ba đặc trưng: diện tích x_1 m^2, số phòng ngủ x_2 và khoảng cách tới trung tâm thành phố x_3 km. Giả sử ta có một tập dữ liệu gồm 10000 căn nhà trong cùng một thành phố, trong đó mỗi mẫu dữ liệu bao gồm các đặc trưng của căn nhà và giá bán tương ứng.

Khi có một căn nhà mới với các đặc trưng x_1, x_2 và x_3, mục tiêu của chúng ta là xây dựng một hàm dự đoán

$$y = f(\mathbf{x}) \tag{4.1}$$

trong đó vector đặc trưng đầu vào

$$\mathbf{x} = [x_1, x_2, x_3]^T \tag{4.2}$$

là vector cột chứa các biến đầu vào, còn y là giá bán của căn nhà.

Quan sát thực tế cho thấy giá nhà thường có xu hướng tăng khi diện tích lớn hơn, số phòng ngủ nhiều hơn và khoảng cách tới trung tâm thành phố nhỏ hơn.

Do đó, một giả thiết hợp lý là giá nhà có thể được xấp xỉ bởi một hàm tuyến tính của các đặc trưng đầu vào. Khi đó mô hình dự đoán có dạng

$$y \approx \hat{y} = f(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 \tag{4.3}$$

trong đó w_0 là *hệ số chệch* (bias) và $\mathbf{w} = [w_1, w_2, w_3]^T$ là *vector trọng số* cần được ước lượng từ dữ liệu.

Để viết gọn hơn, ta có thể mở rộng vector đặc trưng bằng cách thêm một phần tử bằng 1:

$$\tilde{\mathbf{x}} = [1, x_1, x_2, x_3]^T \tag{4.4}$$

và vector trọng số

$$\tilde{\mathbf{w}} = [w_0, w_1, w_2, w_3]^T. \tag{4.5}$$

Khi đó mô hình hồi quy tuyến tính có thể được viết gọn dưới dạng

$$\hat{y} = \tilde{\mathbf{x}}^T \tilde{\mathbf{w}}. \tag{4.6}$$

Lưu ý. Thành phần w_0 được gọi là *hệ số chệch* (*bias*) của mô hình. Nếu không có thành phần này, mô hình tuyến tính buộc phải đi qua gốc tọa độ, tức là khi $\mathbf{x} = \mathbf{0}$ thì $\hat{y} = 0$. Tuy nhiên, trong nhiều bài toán thực tế giả thiết này không hợp lý. Việc bổ sung hệ số chệch giúp mô hình linh hoạt hơn bằng cách cho phép đường hồi quy dịch chuyển song song với trục tung, từ đó mô tả dữ liệu tốt hơn.

4.2. Xây dựng và tối ưu hàm mất mát

Tổng quát, giả sử mỗi điểm dữ liệu được mô tả bởi một vector đặc trưng d chiều $\mathbf{x} \in \mathbb{R}^d$. Khi đó mô hình hồi quy tuyến tính có thể được biểu diễn dưới dạng

$$y = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d. \tag{4.7}$$

Đặt $\mathbf{w} = [w_1, w_2, \ldots, w_d]^T$ là vector trọng số, ta có thể viết gọn mô hình trên dưới dạng tích vô hướng

$$y = w_0 + \mathbf{x}^T \mathbf{w}. \tag{4.8}$$

4.2.1. Sai số dự đoán

Sau khi xây dựng được mô hình dự đoán đầu ra như (4.7), bước tiếp theo là xác định một hàm đánh giá mức độ sai khác giữa giá trị dự đoán của mô hình và giá trị thực của dữ liệu. Trong các bài toán hồi quy, mục tiêu của mô hình là làm cho sai số giữa giá trị thực y và giá trị dự đoán $\hat{y}$ càng nhỏ càng tốt.

Gọi e là sai số dự đoán:

$$e = y - \hat{y}. \tag{4.9}$$

Một cách phổ biến để đo lường sai số này là sử dụng *sai số bình phương*. Khi đó hàm sai số được viết dưới dạng

$$\frac{1}{2}e^2 = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(y - \mathbf{x}^T\mathbf{w})^2. \tag{4.10}$$

Việc sử dụng bình phương sai số có hai mục đích. Thứ nhất, sai số $e = y - \hat{y}$ có thể nhận giá trị âm hoặc dương, nên nếu chỉ cộng trực tiếp các sai số thì các giá trị này có thể triệt tiêu lẫn nhau. Bình phương sai số giúp đảm bảo mọi sai số đều được tính như một đại lượng không âm.

Thứ hai, một cách tiếp cận khác để tránh việc sai số âm là sử dụng trị tuyệt đối của sai số:

$$|e| = |y - \hat{y}|. \tag{4.11}$$

Tuy nhiên, trong thực tế phương pháp này ít được sử dụng trong hồi quy tuyến tính. Nguyên nhân là hàm trị tuyệt đối không khả vi tại điểm $e = 0$. Điều này gây khó khăn cho các thuật toán tối ưu dựa trên đạo hàm, chẳng hạn như gradient descent, vốn cần tính gradient của hàm mất mát theo các tham số của mô hình.

Ngoài ra, việc đưa hệ số $\frac{1}{2}$ vào trước bình phương sai số chỉ nhằm mục đích thuận tiện trong tính toán. Khi lấy đạo hàm của hàm mất mát theo các tham số $\mathbf{w}$, hệ số này sẽ triệt tiêu với số 2 sinh ra từ phép đạo hàm của bình phương, giúp biểu thức gradient trở nên gọn gàng hơn..

4.2.2. Hàm mất mát

Điều tương tự cũng áp dụng cho toàn bộ các cặp dữ liệu huấn luyện $(\mathbf{x}_i, y_i)$ với $i = 1, 2, \ldots, N$, trong đó N là số lượng mẫu trong tập huấn luyện. Với mỗi mẫu dữ liệu, sai số dự đoán được xác định bởi sự khác biệt giữa giá trị thực y_i và giá trị dự đoán $\hat{y}_i = \mathbf{x}_i^T\mathbf{w}$.

Để đánh giá chất lượng của mô hình trên toàn bộ tập dữ liệu, người ta thường sử dụng trung bình sai số bình phương của tất cả các mẫu. Khi đó hàm mất mát của mô hình hồi quy tuyến tính được định nghĩa như sau:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - \mathbf{x}_i^T\mathbf{w})^2. \tag{4.12}$$

Hàm số $\mathcal{L}(\mathbf{w})$ đo mức độ sai khác trung bình giữa giá trị dự đoán của mô hình và giá trị thực của dữ liệu. Trong bối cảnh này, $\mathbf{w}$ đóng vai trò là tham số của mô hình, tức là $\theta = \mathbf{w}$.

Mục tiêu của quá trình huấn luyện là tìm bộ tham số $\mathbf{w}$ sao cho hàm mất mát đạt giá trị nhỏ nhất. Bài toán học mô hình do đó có thể được biểu diễn như một bài toán tối ưu:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}}\, \mathcal{L}(\mathbf{w}). \tag{4.13}$$

Nghiệm $\mathbf{w}^*$ chính là bộ tham số tối ưu của mô hình, tức là bộ trọng số làm cho sai số dự đoán trên tập huấn luyện được giảm xuống mức nhỏ nhất.

Trung bình sai số

Trong nhiều mô hình học máy, hàm mất mát trên tập huấn luyện thường được định nghĩa là trung bình của sai số tại từng mẫu dữ liệu. Với tập dữ liệu gồm N mẫu $(\mathbf{x}_i, y_i)$, hàm mất mát trong hồi quy tuyến tính thường được viết dưới dạng

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - \mathbf{x}_i^T\mathbf{w})^2. \tag{4.14}$$

Về mặt toán học, hệ số $\frac{1}{2N}$ không làm thay đổi nghiệm tối ưu của bài toán tối ưu hóa, vì việc nhân hàm mất mát với một hằng số dương không ảnh hưởng đến vị trí của điểm cực tiểu. Tuy nhiên, việc lấy trung bình theo số lượng mẫu N giúp giá trị của hàm mất mát không phụ thuộc vào kích thước của tập dữ liệu, từ đó cho phép so sánh chất lượng mô hình giữa các tập dữ liệu khác nhau một cách hợp lý.
Ngoài ra, hệ số $\frac{1}{2}$ thường được đưa vào để đơn giản hóa biểu thức đạo hàm khi tối ưu hóa. Cụ thể, khi lấy đạo hàm của hàm mất mát theo tham số $\mathbf{w}$, hệ số 2 sinh ra từ phép đạo hàm của bình phương sẽ triệt tiêu với $\frac{1}{2}$, giúp biểu thức gradient trở nên gọn gàng hơn.

Trước khi xây dựng nghiệm cho bài toán tối ưu hàm mất mát, ta thấy rằng hàm số này có thể được viết gọn lại dưới dạng ma trận, vector, và chuẩn như sau:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - \mathbf{x}_i^T\mathbf{w})^2 = \frac{1}{2N}\left\|\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} - \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}\mathbf{w}\right\|_2^2 = \frac{1}{2N}\|\mathbf{y} - \mathbf{X}^T\mathbf{w}\|_2^2 \tag{4.15}$$

với $\mathbf{y} = [y_1, y_2, \ldots, y_N]^T$, $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N]$. Như vậy $\mathcal{L}(\mathbf{w})$ là một hàm số liên quan tới bình phương của chuẩn ℓ_2.

4.2.3. Nghiệm của hồi quy tuyến tính

Nhận thấy rằng hàm mất mát $\mathcal{L}(\mathbf{w})$ khả vi đối với mọi $\mathbf{w}$. Do đó, nghiệm tối ưu của bài toán tối ưu có thể được xác định bằng cách tìm các điểm dừng của hàm mất mát, tức là giải phương trình gradient của $\mathcal{L}(\mathbf{w})$ theo $\mathbf{w}$ bằng không.

Gradient của hàm mất mát có dạng:

$$\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}) = \frac{1}{N}\mathbf{X}(\mathbf{X}^T\mathbf{w} - \mathbf{y}). \tag{4.16}$$

Đặt gradient bằng không, ta thu được phương trình:

$$\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}) = \mathbf{0} \quad \Leftrightarrow \quad \mathbf{X}\mathbf{X}^T\mathbf{w} = \mathbf{X}\mathbf{y}. \tag{4.17}$$

Nếu ma trận vuông $\mathbf{X}\mathbf{X}^T$ khả nghịch thì phương trình (4.17) có nghiệm duy nhất:

$$\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{X}\mathbf{y}. \tag{4.18}$$

Trong trường hợp ma trận $\mathbf{X}\mathbf{X}^T$ không khả nghịch, phương trình (4.17) có thể không có nghiệm duy nhất. Khi đó ta sử dụng *giả nghịch đảo* (pseudo-inverse) để tìm một nghiệm đặc biệt. Người ta có thể chứng minh rằng với mọi ma trận $\mathbf{X}$ luôn tồn tại duy nhất một vector $\mathbf{w}$ có chuẩn ℓ_2 nhỏ nhất sao cho sai số bình phương:

$$\|\mathbf{X}^T\mathbf{w} - \mathbf{y}\|_F^2 \tag{4.19}$$

đạt giá trị tối thiểu. Nghiệm này được cho bởi:

$$\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^\dagger \mathbf{X}\mathbf{y}, \tag{4.20}$$

trong đó $(\mathbf{X}\mathbf{X}^T)^\dagger$ là giả nghịch đảo của ma trận $\mathbf{X}\mathbf{X}^T$. Giả nghịch đảo luôn tồn tại ngay cả khi ma trận không vuông hoặc không khả nghịch. Khi ma trận là vuông và khả nghịch thì giả nghịch đảo trùng với nghịch đảo thông thường.

Vì vậy, nghiệm tổng quát của bài toán tối ưu (4.13) có thể được viết dưới dạng:

$$\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^\dagger \mathbf{X}\mathbf{y}. \tag{4.21}$$

Trong thực hành, việc tính giả nghịch đảo của ma trận thường được thực hiện thông qua các thư viện tính toán số, chẳng hạn như hàm `pinv()` trong thư viện NumPy của Python.

4.3. Ví dụ trên Python

4.3.1. Bài toán

Xét một ví dụ đơn giản có thể áp dụng hồi quy tuyến tính. Chúng ta sẽ so sánh nghiệm của bài toán khi giải theo phương trình (4.21) và nghiệm tìm được khi sử dụng thư viện `scikit-learn` của Python. Giả sử có dữ liệu cân nặng và chiều cao của 15 người trong Bảng 4.1. Dữ liệu của hai người có chiều cao 155 cm và 160 cm được tách ra làm tập kiểm tra, phần còn lại tạo thành tập huấn luyện.

Bảng 4.1: Dữ liệu chiều cao và cân nặng của 15 người

Chiều cao (cm)	**Cân nặng (kg)**	**Chiều cao (cm)**	**Cân nặng (kg)**	**Chiều cao (cm)**	**Cân nặng (kg)**
147	49	158	54	173	63
150	50	160	56	175	64
153	51	163	58	178	66
155	52	165	59	180	67
168	60	170	72	183	68

Bài toán đặt ra là liệu có thể xây dựng một mô hình để dự đoán cân nặng của một người dựa trên chiều cao của họ hay không. Quan sát dữ liệu thực tế cho thấy giữa chiều cao và cân nặng thường tồn tại mối quan hệ thuận chiều, tức là những người có chiều cao lớn hơn thường có xu hướng có cân nặng lớn hơn. Do đó, mối quan hệ giữa hai đại lượng này có thể được xấp xỉ bằng một hàm tuyến tính. Vì vậy, hồi quy tuyến tính là một mô hình phù hợp để mô tả và dự đoán cân nặng dựa trên chiều cao.

4.3.2. Hiển thị dữ liệu trên đồ thị

Trước tiên, ta khai báo dữ liệu huấn luyện.

```
from __future__ import print_function
import numpy as np
import matplotlib.pyplot as plt

# Du lieu chieu cao (don vi: cm)
# Moi hang tuong ung voi mot mau du lieu
X = np.array([[147, 150, 153, 158, 163, 165, 168, 170,
               173, 175, 178, 180, 183]]).T

# Du lieu can nang tuong ung (don vi: kg)
# Day la bien muc tieu can du doan trong bai toan hoi quy
y = np.array([49, 50, 51, 54, 58, 59, 60, 62, 63, 64, 66, 67, 68])
```

Các điểm dữ liệu được minh hoạ bởi các điểm hình tròn trong Hình 4.1. Ta thấy rằng dữ liệu được sắp xếp gần như theo một đường thẳng, vậy mô hình hồi quy tuyến tính sau đây có khả năng cho kết quả tốt, với `w_0` là hệ số điều chỉnh b:

$$(\text{cân nặng}) = \texttt{w_1}^*(\text{chiều cao}) + \texttt{w_0}$$

4.3.3. Nghiệm theo công thức

Tiếp theo, ta tìm các hệ số `w_1` và `w_0` dựa vào công thức (4.21). Giả nghịch đảo của một ma trận `A` trong Python được tính bằng `numpy.linalg.pinv(A)`.

```
# Tao ma tran Xbar (them cot 1 vao truoc ma tran X)
one = np.ones((X.shape[0], 1))
Xbar = np.concatenate((one, X), axis = 1)  # moi hang la mot mau du lieu

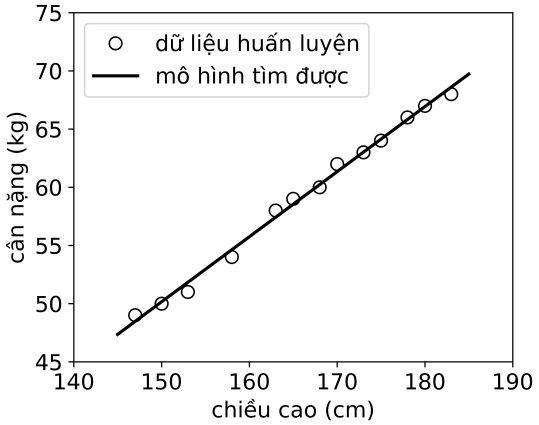
# Tinh cac tham so cua mo hinh hoi quy tuyen tinh
A = np.dot(Xbar.T, Xbar)
b = np.dot(Xbar.T, y)

# Tinh vector trong so w bang cong thuc nghich dao gia
w = np.dot(np.linalg.pinv(A), b)

# Tach cac he so cua mo hinh
w_0, w_1 = w[0], w[1]
```

Đường thẳng mô tả mối quan hệ giữa đầu vào và đầu ra được minh hoạ trong Hình 4.1. Ta thấy rằng các điểm dữ liệu nằm khá gần đường thẳng dự đoán. Vậy mô hình hồi quy tuyến tính hoạt động tốt với tập dữ liệu huấn luyện. Bây giờ, chúng ta sử dụng mô hình này để dự đoán dữ liệu trong tập kiểm tra.

```
# Du doan can nang tu chieu cao 155 cm va 160 cm
y1 = w_1*155 + w_0
y2 = w_1*160 + w_0
# In ket qua du doan
print('Chieu cao 155 cm, can nang thuc te 52 kg, du doan %.2f kg.' % (y1))
print('Chieu cao 160 cm, can nang thuc te 56 kg, du doan %.2f kg.' % (y2))
```



Hình 4.1. Minh hoạ dữ liệu và đường thẳng xấp xỉ tìm được bởi hồi quy tuyến tính

Kết quả:

```
Input chieu cao: 155 cm, can nang thuc te: 52 kg, can nang du doan: 52.94 kg
Input chieu cao: 160 cm, can nang thuc te: 56 kg, can nang du doan: 55.74 kg
```

Chúng ta có nhận xét rằng kết quả dự đoán ra tương đối giống so với giá trị thực tế.

4.3.4. Sử dụng thư viện scikit-learn

Tiếp theo, chúng ta sẽ sử dụng thư viện scikit-learn để tìm nghiệm.

```
from sklearn import datasets, linear_model

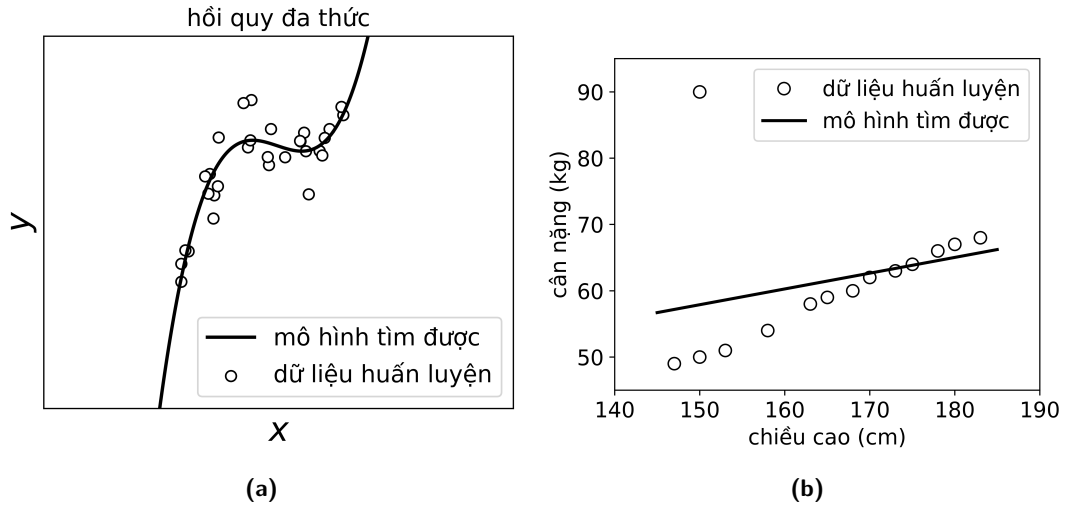
# Khoi tao mo hinh hoi quy tuyen tinh
model = linear_model.LinearRegression()

# Huan luyen mo hinh voi du lieu X (chieu cao) va y (can nang)
# Trong scikit-learn, moi mau du lieu duoc bieu dien boi mot hang
model.fit(X, y)

# In ra cac tham so cua mo hinh sau khi huan luyen
# coef_      : he so goc w1 cua duong hoi quy
# intercept_ : he so dich chuyen w0
print("Ket qua tu scikit-learn:")
print("w1 =", model.coef_[0])
print("w0 =", model.intercept_)

# So sanh voi ket qua tinh toan thu cong truoc do
print("Ket qua tinh toan thu cong:")
print("w1 =", w[1])
print("w0 =", w[0])
```

Chúng ta thấy rằng hai kết quả thu được là như nhau.



Hình 4.2. (a) Hồi quy đa thức bậc ba (b) Hồi quy tuyến tính nhạy cảm với nhiễu.

4.4. Thảo luận

4.4.1. Hồi quy đa thức

Hàm số $y \approx f(\mathbf{x}) = \mathbf{x}^T\mathbf{w} + b$ là một hàm tuyến tính theo cả $\mathbf{w}$ và $\mathbf{x}$. Hồi quy tuyến tính có thể áp dụng cho các mô hình chỉ cần tuyến tính theo $\mathbf{w}$. Ví dụ

$$y \approx w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 \sin(x_2) + w_5 x_1 x_2 + w_0 \tag{4.22}$$

là một hàm tuyến tính theo $\mathbf{w}$ nhưng không tuyến tính theo $\mathbf{x}$. Bài toán này vẫn có thể được giải bằng hồi quy tuyến tính. Với mỗi vector đặc trưng $\mathbf{x} = [x_1, x_2]^T$, ta tính vector đặc trưng mới $\tilde{\mathbf{x}} = [x_1, x_2, x_1^2, \sin(x_2), x_1 x_2]^T$ rồi áp dụng hồi quy tuyến tính với dữ liệu mới này. Tuy nhiên, việc tìm ra các hàm số $\sin(x_2)$ hay $x_1 x_2$ là tương đối không tự nhiên. *Hồi quy đa thức* thường được sử dụng nhiều hơn với các vector đặc trưng mới có dạng $[1, x_1, x_1^2, \dots]^T$ (đặc trưng đa thức). Một ví dụ về hồi quy đa thức bậc 3 được thể hiện trong Hình 4.2a.

4.4.2. Hạn chế của hồi quy tuyến tính

Một trong những hạn chế quan trọng của hồi quy tuyến tính là mô hình này *rất nhạy cảm với nhiễu* (noise) và các giá trị ngoại lai (outliers). Trong ví dụ về mối quan hệ giữa chiều cao và cân nặng ở trên, giả sử trong tập dữ liệu xuất hiện thêm một cặp dữ liệu bất thường như $(150\,\text{cm}, 90\,\text{kg})$. Giá trị này nằm rất xa so với xu hướng chung của dữ liệu.

Do hồi quy tuyến tính thường được huấn luyện bằng cách tối thiểu hóa *sai số bình phương trung bình* (Mean Squared Error – MSE), các sai số lớn sẽ bị khuếch đại do phép bình phương. Vì vậy, chỉ cần một vài điểm dữ liệu ngoại lai cũng có thể làm thay đổi đáng kể đường hồi quy thu được (xem Hình 4.2b).

Một cách tiếp cận để khắc phục vấn đề này là loại bỏ các điểm nhiễu trước khi huấn luyện mô hình. Tuy nhiên, trong thực tế việc xác định và loại bỏ các điểm dữ liệu ngoại lai có thể khá phức tạp, đặc biệt khi dữ liệu có kích thước lớn hoặc nhiều chiều.

Một hướng tiếp cận khác hiệu quả hơn là sử dụng các *hàm mất mát có khả năng kháng nhiễu* (robust loss functions). Một trong những hàm mất mát phổ biến là *mất mát Huber* (Huber loss). Hàm mất mát này kết hợp ưu điểm của hai loại hàm mất mát:

- Với sai số nhỏ, Huber loss hoạt động giống sai số bình phương (L2 loss), giúp quá trình tối ưu ổn định.
- Với sai số lớn, Huber loss chuyển sang dạng sai số tuyệt đối (L1 loss), nhờ đó giảm ảnh hưởng của các điểm dữ liệu ngoại lai.

Nhờ đặc điểm này, hồi quy tuyến tính sử dụng mất mát Huber, thường được gọi là *hồi quy Huber* (Huber Regression), có khả năng kháng nhiễu tốt hơn so với hồi quy tuyến tính truyền thống.

Ngoài Huber loss, một số phương pháp khác cũng được sử dụng để tăng khả năng kháng nhiễu của mô hình hồi quy, chẳng hạn như:

- **Least Absolute Deviations (L1 regression):** tối thiểu hóa tổng sai số tuyệt đối thay vì sai số bình phương.
- **RANSAC (Random Sample Consensus):** lựa chọn ngẫu nhiên các tập con dữ liệu để tìm mô hình phù hợp nhất, đồng thời loại bỏ các điểm dữ liệu ngoại lai.
- **Theil–Sen estimator:** ước lượng hệ số hồi quy dựa trên trung vị của nhiều ước lượng tuyến tính khác nhau, giúp giảm ảnh hưởng của các giá trị ngoại lai.

Một hạn chế quan trọng khác của hồi quy tuyến tính là *khả năng biểu diễn của mô hình còn hạn chế*. Mô hình này giả định rằng mối quan hệ giữa biến đầu vào và biến đầu ra có dạng tuyến tính. Tuy nhiên, trong nhiều bài toán thực tế, mối quan hệ giữa các biến có thể phức tạp và phi tuyến.

Một cách mở rộng hồi quy tuyến tính là sử dụng các đặc trưng phi tuyến, ví dụ: $x_1^2, sin(x_2), x_1x_2$

Việc bổ sung các đặc trưng này giúp mô hình tuyến tính có thể biểu diễn các mối quan hệ phức tạp hơn. Tuy nhiên, khi số chiều của dữ liệu tăng lên, số lượng đặc

trưng cần xây dựng có thể tăng rất nhanh, dẫn đến hiện tượng *bùng nổ số chiều* (curse of dimensionality).

Ngoài ra, hồi quy tuyến tính còn gặp một số hạn chế khác như:

- **Đa cộng tuyến (multicollinearity):** khi các biến đầu vào có tương quan cao với nhau, việc ước lượng các hệ số hồi quy trở nên không ổn định.
- **Giả định tuyến tính mạnh:** mô hình giả định rằng mối quan hệ giữa các biến là tuyến tính, điều này không phải lúc nào cũng đúng trong dữ liệu thực tế.
- **Phương sai thay đổi (heteroscedasticity):** khi phương sai của sai số thay đổi theo giá trị của biến đầu vào, hiệu quả của mô hình có thể bị suy giảm.

Do những hạn chế này, trong nhiều bài toán phức tạp, các mô hình học máy khác như cây quyết định, random forest, mạng nơ-ron nhân tạo hoặc các mô hình kernel thường được sử dụng để thay thế hoặc bổ sung cho hồi quy tuyến tính.

4.5. Hồi quy Ridge và Lasso

Như đã trình bày ở phần trước, hồi quy tuyến tính có một số hạn chế như nhạy cảm với nhiễu và hiện tượng đa cộng tuyến giữa các biến đầu vào. Một kỹ thuật phổ biến để khắc phục các vấn đề này là sử dụng *điều chuẩn* (regularization). Ý tưởng chính của điều chuẩn là bổ sung thêm một thành phần phạt vào hàm mất mát nhằm hạn chế độ lớn của các tham số mô hình.

Giả sử mô hình hồi quy tuyến tính có dạng:

$$y = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d \tag{4.23}$$

Trong đó $w_0, w_1, \ldots, w_d$ là các tham số cần học từ dữ liệu.

Trong hồi quy tuyến tính thông thường, các tham số được tìm bằng cách tối thiểu hóa hàm mất mát bình phương:

$$L(w) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{4.24}$$

Trong các phương pháp điều chuẩn, hàm mất mát này được bổ sung thêm một thành phần phạt phụ thuộc vào độ lớn của các tham số w.

4.5.1. Hồi quy Ridge

Hồi quy Ridge (Ridge Regression) là phương pháp điều chuẩn sử dụng chuẩn L_2 của vector tham số. Hàm mất mát của Ridge regression có dạng:

$$L(w) = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2 \tag{4.25}$$

Trong đó:

- λ là hệ số điều chuẩn (regularization parameter)
- $\sum w_j^2$ là chuẩn L_2 của vector tham số

Thành phần điều chuẩn này có tác dụng làm giảm độ lớn của các hệ số hồi quy, từ đó giúp mô hình ổn định hơn và giảm hiện tượng overfitting.

Nếu biểu diễn theo dạng ma trận, nghiệm của Ridge regression có thể được viết như sau:

$$w = (X^T X + \lambda I)^{-1} X^T y \tag{4.26}$$

Trong đó I là ma trận đơn vị.

4.5.2. Hồi quy Lasso

Khác với Ridge regression, hồi quy Lasso (Least Absolute Shrinkage and Selection Operator) sử dụng chuẩn L_1 của vector tham số:

$$L(w) = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{d} |w_j| \tag{4.27}$$

Điểm đặc biệt của Lasso regression là nó có thể làm cho một số hệ số w_j trở thành 0. Điều này tương đương với việc loại bỏ các đặc trưng không quan trọng khỏi mô hình. Vì vậy, Lasso regression thường được sử dụng như một phương pháp *lựa chọn đặc trưng* (feature selection).

4.5.3. So sánh Ridge và Lasso

- Ridge sử dụng chuẩn L_2 và làm giảm độ lớn của các hệ số nhưng hiếm khi đưa hệ số về đúng bằng 0.

- Lasso sử dụng chuẩn L_1 và có khả năng đưa một số hệ số về 0, do đó giúp loại bỏ các đặc trưng không cần thiết.
- Ridge thường phù hợp khi tất cả các đặc trưng đều có ảnh hưởng đến biến đầu ra.
- Lasso phù hợp khi chỉ có một số ít đặc trưng thực sự quan trọng.

4.5.4. Ví dụ minh họa bằng Python

Dưới đây là ví dụ sử dụng thư viện `scikit-learn` để áp dụng Ridge regression và Lasso regression cho dữ liệu chiều cao và cân nặng.

```
import numpy as np
from sklearn.linear_model import Ridge, Lasso

# Du lieu chieu cao (cm)
X = np.array([[147,150,153,158,163,165,168,170,
               173,175,178,180,183]]).T

# Du lieu can nang (kg)
y = np.array([49,50,51,54,58,59,60,62,63,64,66,67,68])

# Ridge Regression
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X, y)

print("Ridge Regression")
print("w1 =", ridge_model.coef_[0])
print("w0 =", ridge_model.intercept_)

# Lasso Regression
lasso_model = Lasso(alpha=0.1)
lasso_model.fit(X, y)

print("Lasso Regression")
print("w1 =", lasso_model.coef_[0])
print("w0 =", lasso_model.intercept_)
```

Sau khi huấn luyện, ta có thể sử dụng các mô hình này để dự đoán cân nặng dựa trên chiều cao:

```
h = np.array([[160]])

print("Du doan bang Ridge:", ridge_model.predict(h))
print("Du doan bang Lasso:", lasso_model.predict(h))
```

4.6. Kết luận

Trong chương này, chúng ta đã nghiên cứu một trong những mô hình cơ bản và quan trọng nhất trong học máy, đó là mô hình hồi quy tuyến tính (Linear Regression). Mô hình này được sử dụng để mô tả mối quan hệ giữa một biến phụ thuộc và một hoặc nhiều biến độc lập thông qua một hàm tuyến tính.

Chúng ta đã bắt đầu bằng việc xây dựng mô hình hồi quy tuyến tính đơn giản và hồi quy tuyến tính đa biến. Sau đó, chương đã trình bày cách xác định các tham số của mô hình thông qua phương pháp tối thiểu hóa sai số bình phương trung bình (Mean Squared Error). Các tham số của mô hình có thể được tìm bằng nghiệm giải tích (Closed-form solution) hoặc bằng các phương pháp tối ưu hóa như Gradient Descent.

Tiếp theo, chương đã phân tích một số hạn chế của hồi quy tuyến tính, đặc biệt là hiện tượng quá khớp (overfitting) khi số lượng đặc trưng lớn hoặc khi các đặc trưng có tương quan mạnh với nhau. Để khắc phục vấn đề này, các phương pháp điều chuẩn (regularization) đã được giới thiệu, bao gồm Ridge Regression và Lasso Regression. Hai phương pháp này bổ sung thêm các thành phần điều chuẩn vào hàm mất mát nhằm hạn chế độ lớn của các tham số mô hình, từ đó giúp cải thiện khả năng tổng quát hóa.

Ngoài ra, chương cũng đã minh họa việc triển khai các mô hình hồi quy tuyến tính bằng ngôn ngữ lập trình Python thông qua thư viện `scikit-learn`. Các ví dụ thực hành giúp người học hiểu rõ hơn cách xây dựng, huấn luyện và đánh giá các mô hình hồi quy trong thực tế.

Nhìn chung, hồi quy tuyến tính là một mô hình đơn giản nhưng có ý nghĩa nền tảng trong học máy. Nhiều mô hình phức tạp hơn trong học sâu và thống kê hiện đại đều được xây dựng dựa trên các ý tưởng cơ bản của phương pháp này.

4.7. Câu hỏi ôn tập

Câu hỏi lý thuyết

a. Trình bày khái niệm của mô hình hồi quy tuyến tính. Mục tiêu của bài toán hồi quy tuyến tính là gì?

b. Viết biểu thức toán học của mô hình hồi quy tuyến tính với d biến đầu vào.

c. Trình bày hàm mất mát thường được sử dụng trong hồi quy tuyến tính. Tại sao sai số bình phương trung bình (MSE) lại được sử dụng phổ biến?

d. Giải thích phương pháp tìm nghiệm giải tích (Normal Equation) trong hồi quy tuyến tính.

e. Trình bày nguyên lý của thuật toán Gradient Descent trong việc tối ưu các tham số của mô hình hồi quy.

f. Giải thích hiện tượng quá khớp (overfitting) trong hồi quy tuyến tính. Tại sao hiện tượng này thường xảy ra khi số chiều dữ liệu lớn?

g. So sánh Ridge Regression và Lasso Regression. Hai phương pháp này khác nhau như thế nào trong cách điều chuẩn các tham số của mô hình?

Bài tập thực hành

h. Cho tập dữ liệu gồm các cặp giá trị sau:

$$(1,2),\ (2,3),\ (3,5),\ (4,4),\ (5,6)$$

   a. Xây dựng mô hình hồi quy tuyến tính $y = w_0 + w_1 x$.

   b. Tính các tham số w_0 và w_1 bằng phương pháp bình phương tối thiểu.

i. Cho hàm mất mát của Ridge Regression:

$$J(\mathbf{w}) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$$

   a. Giải thích ý nghĩa của tham số λ.

   b. Điều gì xảy ra khi λ rất lớn hoặc rất nhỏ?

   c. Ridge Regression giúp giảm hiện tượng overfitting như thế nào?

j. Cho hàm mất mát của Lasso Regression:

$$J(\mathbf{w}) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{d} |w_j|$$

   a. So sánh thành phần điều chuẩn của Lasso với Ridge.

   b. Giải thích tại sao Lasso có khả năng thực hiện lựa chọn đặc trưng (feature selection).

   c. Trong trường hợp nào nên sử dụng Lasso thay vì Ridge?

Chương 5

K láng giềng gần nhất

Trong nhiều bài toán học máy, mục tiêu của chúng ta là dự đoán nhãn của một điểm dữ liệu mới dựa trên các mẫu dữ liệu đã quan sát trước đó. Một cách tiếp cận tự nhiên cho vấn đề này là giả định rằng các điểm dữ liệu có đặc trưng giống nhau thường thuộc cùng một lớp.

Thuật toán *k-Nearest Neighbors* (KNN) được xây dựng dựa trên ý tưởng trực quan này. Thay vì xây dựng một mô hình học phức tạp từ dữ liệu huấn luyện, KNN đưa ra dự đoán cho một điểm dữ liệu mới bằng cách xem xét các điểm dữ liệu gần nó nhất trong tập huấn luyện và sử dụng thông tin của các điểm này để quyết định nhãn của điểm cần dự đoán.

5.1. Giới thiệu

5.1.1. Một câu chuyện vui

Có một sinh viên của khoa Công nghệ thông tin tại ICTU chuẩn bị thi môn Học máy. Vì môn này được phép mang tài liệu vào phòng thi nên cậu ta quyết định không dành nhiều thời gian để học kỹ lý thuyết. Thay vào đó, cậu dành thời gian sưu tầm thật nhiều tài liệu: slide bài giảng của giảng viên, các bài tập thực hành trên lớp, bài tập lớn của khóa trước, tài liệu trên GitHub, thậm chí cả ảnh chụp bảng của bạn bè trong lớp.

Sau một thời gian chuẩn bị, cậu sinh viên đã thu thập được một thư mục tài liệu rất lớn trong máy tính của mình. Trong đó có hàng trăm file PDF, notebook Python và các đoạn code mẫu. Cậu nghĩ rằng: "Không cần phải nhớ nhiều, chỉ cần có đủ tài liệu là được. Khi gặp bài nào thì tìm lại trong tài liệu là xong."

Đến ngày thi, cậu mở máy tính, đọc đề bài và bắt đầu tìm kiếm trong thư mục tài liệu của mình. Câu 1 gần giống với một ví dụ trong notebook nên cậu nhanh chóng tìm được lời giải và làm đúng. Câu 2 cũng khá giống một bài trong slide bài giảng, chỉ khác một vài con số nên cậu chỉnh sửa một chút và cũng giải được.

Đến câu 3, đề bài không hoàn toàn giống với bất kỳ ví dụ nào trước đó. Cậu bắt đầu mở nhiều tài liệu khác nhau để tìm xem có bài nào "gần giống nhất" với đề bài hay không. Cuối cùng, cậu chọn một ví dụ có cách giải tương đối giống và áp dụng lại phương pháp đó.

Câu 4 còn khó hơn. Có vài ví dụ khá giống nhau nhưng mỗi ví dụ lại đưa ra một cách giải hơi khác. Cậu phải so sánh các ví dụ đó, xem ví dụ nào "gần giống nhất" với bài toán trong đề thi rồi quyết định làm theo cách giải của ví dụ đó.

Sau khi kết thúc bài thi, cậu nhận ra rằng mình gần như không học trước điều gì cả. Mỗi khi gặp một câu hỏi mới, cậu chỉ tìm trong tập tài liệu xem bài nào giống nhất rồi làm theo cách của bài đó.

Thực ra, cách làm của cậu sinh viên này lại rất giống với một thuật toán trong Machine Learning, được gọi là *k-Nearest Neighbors* (KNN). Thuật toán này không xây dựng một mô hình phức tạp từ trước. Thay vào đó, khi cần đưa ra dự đoán cho một dữ liệu mới, thuật toán sẽ tìm trong tập dữ liệu huấn luyện những điểm dữ liệu "gần nhất" với điểm mới này, rồi dựa vào các điểm đó để đưa ra kết quả dự đoán.

Chính vì lý do này, KNN thường được xếp vào nhóm các phương pháp *lazy learning* (học lười), bởi vì phần lớn công việc được thực hiện trong giai đoạn dự đoán thay vì trong quá trình huấn luyện.

5.1.2. Khái niệm

Thuật toán *k láng giềng gần nhất* (k-Nearest Neighbors – KNN) là một trong những phương pháp học có giám sát đơn giản và trực quan nhất trong học máy. Ý tưởng cơ bản của thuật toán là dự đoán đầu ra cho một điểm dữ liệu mới dựa trên thông tin của k điểm dữ liệu "gần nhất" với nó trong tập dữ liệu huấn luyện.

Khác với nhiều thuật toán học máy khác, KNN không xây dựng một mô hình tường minh trong quá trình huấn luyện. Thay vào đó, toàn bộ dữ liệu huấn luyện được lưu trữ lại và sử dụng trực tiếp khi cần đưa ra dự đoán. Vì vậy, phần lớn chi phí tính toán của thuật toán xảy ra trong giai đoạn dự đoán (prediction phase). Chính vì đặc điểm này, KNN thường được xếp vào nhóm các phương pháp *lazy learning* (học lười), trái ngược với các phương pháp *eager learning* như hồi quy tuyến tính hay cây quyết định, trong đó mô hình được xây dựng ngay từ giai đoạn huấn luyện.

Trong KNN, khái niệm "gần nhất" được xác định thông qua một *hàm khoảng cách* (distance metric). Một số khoảng cách thường được sử dụng bao gồm khoảng cách Euclid, khoảng cách Manhattan và khoảng cách Minkowski. Ví dụ, khoảng cách Euclid giữa hai điểm dữ liệu x_i và x_j trong không gian d chiều được xác định bởi:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^{d} (x_{ik} - x_{jk})^2} \tag{5.1}$$

Khi một điểm dữ liệu mới x xuất hiện, thuật toán KNN sẽ thực hiện các bước sau:

a. Tính khoảng cách giữa x và tất cả các điểm dữ liệu trong tập huấn luyện.

b. Lựa chọn k điểm dữ liệu có khoảng cách nhỏ nhất với x.

c. Sử dụng thông tin của các điểm lân cận này để đưa ra dự đoán.

Tùy thuộc vào loại bài toán, cách đưa ra dự đoán của KNN có thể khác nhau:

- **Bài toán phân loại:** nhãn của điểm dữ liệu mới được xác định bằng cách lấy *bỏ phiếu đa số* (majority voting) từ k điểm lân cận gần nhất.
- **Bài toán hồi quy:** giá trị đầu ra được tính bằng *trung bình* của giá trị đầu ra của k điểm lân cận.

Một tham số quan trọng của thuật toán là giá trị k. Nếu k quá nhỏ, mô hình có thể nhạy cảm với nhiễu trong dữ liệu. Ngược lại, nếu k quá lớn, mô hình có thể trở nên quá đơn giản và làm mất đi các đặc trưng cục bộ của dữ liệu. Vì vậy, việc lựa chọn giá trị k phù hợp đóng vai trò quan trọng trong hiệu quả của thuật toán.

5.2. Phân tích toán học

Trong thuật toán KNN, không tồn tại một hàm mất mát hay bài toán tối ưu nào cần giải trong giai đoạn huấn luyện. Thay vào đó, toàn bộ dữ liệu huấn luyện được lưu trữ và các phép tính chỉ được thực hiện khi cần đưa ra dự đoán cho một điểm dữ liệu mới. Chính vì vậy, chi phí tính toán của KNN chủ yếu nằm ở giai đoạn kiểm tra.

Vì KNN đưa ra quyết định dựa trên các điểm dữ liệu gần nhất, hai vấn đề quan trọng cần được xem xét là:

- Cách định nghĩa khoảng cách giữa các điểm dữ liệu.
- Cách tính toán các khoảng cách này một cách hiệu quả.

Định nghĩa khoảng cách

Mỗi điểm dữ liệu được biểu diễn bởi một vector đặc trưng trong không gian d chiều. Khoảng cách giữa hai điểm dữ liệu chính là khoảng cách giữa hai vector tương ứng. Trong nhiều bài toán, khoảng cách được sử dụng phổ biến nhất là khoảng cách Euclid.

Khoảng cách Euclid giữa hai điểm $\mathbf{x}_i, \mathbf{z} \in \mathbb{R}^d$ được định nghĩa bởi:

$$d(\mathbf{z}, \mathbf{x}_i) = \|\mathbf{z} - \mathbf{x}_i\|_2 \tag{5.2}$$

Trong thực tế, người ta thường sử dụng bình phương khoảng cách Euclid để tránh phép tính căn bậc hai:

$$\|\mathbf{z} - \mathbf{x}_i\|_2^2 = (\mathbf{z} - \mathbf{x}_i)^T(\mathbf{z} - \mathbf{x}_i) = \|\mathbf{z}\|_2^2 + \|\mathbf{x}_i\|_2^2 - 2\mathbf{x}_i^T\mathbf{z} \tag{5.3}$$

Biến đổi này có ý nghĩa quan trọng về mặt tính toán. Khi so sánh các khoảng cách, việc sử dụng bình phương khoảng cách không làm thay đổi thứ tự của các giá trị khoảng cách. Hơn nữa, nếu tập huấn luyện được biểu diễn bởi ma trận $\mathbf{X} \in \mathbb{R}^{d\times N}$ thì các giá trị $\|\mathbf{x}_i\|_2^2$ có thể được tính trước và lưu trữ. Khi đó, việc tính khoảng cách chỉ còn lại phép tính tích vô hướng $\mathbf{x}_i^T\mathbf{z}$, giúp giảm đáng kể chi phí tính toán.

Tính khoảng cách giữa hai tập dữ liệu

Trong nhiều bài toán thực tế, tập kiểm tra không chỉ có một điểm dữ liệu mà bao gồm nhiều điểm tạo thành ma trận $\mathbf{Z}$. Khi đó cần tính khoảng cách giữa từng cặp điểm trong hai tập dữ liệu $\mathbf{X}$ và $\mathbf{Z}$.

Giả sử:

$$\mathbf{X} \in \mathbb{R}^{d\times N}, \qquad \mathbf{Z} \in \mathbb{R}^{d\times M}$$

Ta cần tính ma trận khoảng cách kích thước $N \times M$. Nếu N và M đều lớn, số lượng phép tính có thể rất lớn. Vì vậy, trong thực hành người ta thường sử dụng các thư viện tối ưu hóa sẵn như `scipy` hoặc `scikit-learn`. Ví dụ:

```
from sklearn.metrics.pairwise import pairwise_distances
D = pairwise_distances(Z, X)
```

Ma trận D chứa toàn bộ khoảng cách giữa các điểm trong tập kiểm tra và tập huấn luyện.

Ví dụ minh họa

Giả sử ta có ba điểm dữ liệu huấn luyện biểu diễn bởi hai đặc trưng:

$$\mathbf{x}_1 = (1, 2), \quad \mathbf{x}_2 = (2, 1), \quad \mathbf{x}_3 = (4, 3)$$

Một điểm dữ liệu mới cần phân loại là:

$$\mathbf{z} = (2, 2)$$

Khoảng cách Euclid từ $\mathbf{z}$ đến từng điểm huấn luyện được tính như sau:

$$d(\mathbf{z}, \mathbf{x}_1) = \sqrt{(2-1)^2 + (2-2)^2} = 1$$

$$d(\mathbf{z}, \mathbf{x}_2) = \sqrt{(2-2)^2 + (2-1)^2} = 1$$

$$d(\mathbf{z}, \mathbf{x}_3) = \sqrt{(2-4)^2 + (2-3)^2} = \sqrt{5}$$

Như vậy, hai điểm gần nhất với $\mathbf{z}$ là $\mathbf{x}_1$ và $\mathbf{x}_2$. Nếu sử dụng $k = 2$, thuật toán KNN sẽ dựa vào nhãn của hai điểm này để đưa ra dự đoán cho điểm $\mathbf{z}$.

5.2.1. Một số loại khoảng cách phổ biến

5.2.2. Một số loại khoảng cách phổ biến

Trong thuật toán KNN, khái niệm "gần nhất" phụ thuộc trực tiếp vào cách định nghĩa khoảng cách giữa các điểm dữ liệu. Tùy thuộc vào đặc điểm của dữ liệu và bài toán cụ thể, nhiều loại khoảng cách khác nhau có thể được sử dụng. Dưới đây là một số khoảng cách phổ biến trong học máy.

Khoảng cách Euclid

Khoảng cách Euclid là loại khoảng cách được sử dụng phổ biến nhất. Khoảng cách này tương ứng với khoảng cách hình học giữa hai điểm trong không gian Euclid.

Giả sử $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ và $\mathbf{y} = (y_1, y_2, \ldots, y_d)$ là hai vector trong $\mathbb{R}^d$. Khoảng cách Euclid giữa hai vector này được định nghĩa bởi:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2} \tag{5.4}$$

Khoảng cách này thường được sử dụng khi các đặc trưng của dữ liệu có cùng đơn vị đo và có ý nghĩa hình học rõ ràng.

Khoảng cách Manhattan

Khoảng cách Manhattan (còn gọi là khoảng cách L_1) được tính bằng tổng giá trị tuyệt đối của sai khác giữa các thành phần của hai vector. Khoảng cách này được định nghĩa bởi:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} |x_i - y_i| \tag{5.5}$$

Tên gọi Manhattan xuất phát từ việc khoảng cách này tương tự như khoảng cách di chuyển trên các con phố vuông góc trong thành phố Manhattan, nơi người ta chỉ có thể di chuyển theo các trục ngang và dọc.

Khoảng cách Minkowski

Khoảng cách Minkowski là dạng tổng quát của nhiều loại khoảng cách khác nhau. Khoảng cách này được định nghĩa bởi:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{d} |x_i - y_i|^p \right)^{\frac{1}{p}} \tag{5.6}$$

Trong đó p là một tham số điều chỉnh. Một số trường hợp đặc biệt của khoảng cách Minkowski bao gồm:

- $p = 1$: khoảng cách Manhattan.

- $p = 2$: khoảng cách Euclid.

Khoảng cách Cosine

Trong nhiều bài toán xử lý văn bản hoặc dữ liệu có số chiều lớn, người ta thường quan tâm đến góc giữa các vector hơn là khoảng cách hình học giữa chúng. Khi đó, khoảng cách Cosine thường được sử dụng.

Độ tương tự Cosine giữa hai vector được định nghĩa bởi:

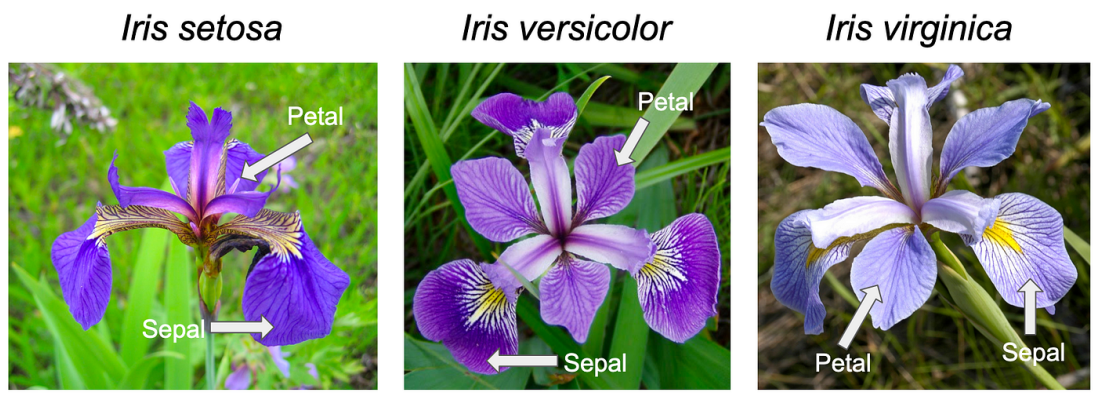
$$\cos(\theta) = \frac{\mathbf{x}^T\mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} \tag{5.7}$$

Từ đó, khoảng cách Cosine được xác định bởi:

$$d(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x}^T\mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} \tag{5.8}$$

Khoảng cách này đặc biệt hữu ích trong các bài toán như phân loại văn bản, tìm kiếm tài liệu và hệ thống gợi ý.

5.3. Thực hành phân loại hoa IRIS



Hình 5.1. Ba loại hoa trong bộ cơ sở dữ liệu hoa Iris.

5.3.1. Bộ cơ sở dữ liệu hoa Iris

Bộ dữ liệu hoa Iris (https://www.kaggle.com/datasets/saurabh00007/iriscsv) là một trong những bộ dữ liệu kinh điển trong lĩnh vực học máy và thống kê. Bộ dữ liệu này được giới thiệu lần đầu bởi nhà thống kê Ronald Fisher vào năm

1936 và thường được sử dụng để minh họa các bài toán phân loại và khai phá dữ liệu.

Bộ dữ liệu bao gồm 150 mẫu hoa Iris, được chia thành ba loài khác nhau: *Iris setosa*, *Iris versicolor* và *Iris virginica*. Mỗi loài có 50 mẫu dữ liệu. Mỗi mẫu không phải là một hình ảnh của bông hoa mà được biểu diễn bằng một vector đặc trưng gồm bốn thuộc tính số, được đo bằng centimet.

Cụ thể, mỗi mẫu dữ liệu trong tập Iris bao gồm các đặc trưng sau:

- **Sepal length (cm)**: chiều dài của đài hoa. Đài hoa là phần bao bọc và bảo vệ nụ hoa trước khi hoa nở.
- **Sepal width (cm)**: chiều rộng của đài hoa.
- **Petal length (cm)**: chiều dài của cánh hoa. Đây là phần màu sắc nổi bật của bông hoa.
- **Petal width (cm)**: chiều rộng của cánh hoa.

Do đó, mỗi điểm dữ liệu trong bộ Iris có thể được biểu diễn bởi một vector bốn chiều:

$$\mathbf{x} = (x_1, x_2, x_3, x_4) \tag{5.9}$$

trong đó:

$$x_1 = \text{sepal length}, \quad x_2 = \text{sepal width}, \quad x_3 = \text{petal length}, \quad x_4 = \text{petal width}$$

Nhãn của mỗi điểm dữ liệu tương ứng với loài hoa Iris:

$$y \in \{\text{setosa},\ \text{versicolor},\ \text{virginica}\}$$

Như vậy, bài toán đặt ra là: dựa trên bốn đặc trưng hình học của bông hoa, xây dựng một mô hình học máy để dự đoán loài hoa Iris tương ứng. Đây là một ví dụ điển hình của bài toán *phân loại đa lớp* (multiclass classification).

Hình 5.1 minh họa ba loài hoa trong bộ dữ liệu Iris. Cần lưu ý rằng trong các thí nghiệm học máy, dữ liệu đầu vào không phải là hình ảnh của bông hoa mà là các vector đặc trưng bốn chiều được mô tả ở trên.

5.3.2. Thực hành

Trong phần này, 150 điểm dữ liệu được tách thành tập huấn luyện và tập kiểm tra. KNN dựa vào trông tin trong tập huấn luyện để dự đoán mỗi dữ liệu trong tập kiểm tra tương ứng với loại hoa nào. Kết quả này được đối chiếu với đầu ra thực sự để đánh giá hiệu quả của KNN.

Trước tiên, chúng ta cần khai báo vài thư viện. Bộ dữ liệu hoa Iris có sẵn trong thư viện scikit-learn.

```
from __future__ import print_function
import numpy as np
from sklearn import neighbors, datasets
from sklearn.model_selection import train_test_split # Ham tach du lieu
    train/test
from sklearn.metrics import accuracy_score           # Ham danh gia do chinh
     xac

# Nap bo du lieu hoa Iris tu thu vien scikit-learn
iris = datasets.load_iris()

# X chua cac dac trung cua du lieu (sepal length, sepal width,
# petal length, petal width)
iris_X = iris.data

# y chua nhan cua tung mau du lieu (0, 1, 2 tuong ung voi
# ba loai hoa: setosa, versicolor, virginica)
iris_y = iris.target
```

Tiếp theo, 130 mẫu dữ liệu được lấy ra ngẫu nhiên tạo thành tập huấn luyện, 20 mẫu còn lại được dùng để kiểm tra.

```
print('Labels:', np.unique(iris_y))

# Chia du lieu train va test
np.random.seed(7)
X_train, X_test, y_train, y_test = train_test_split(
    iris_X, iris_y, test_size=130)
print('Training size:', X_train.shape[0], ', test size:', X_test.shape[0])
```

```
Labels: [0 1 2]
Training size: 20 , test size: 130
```

Dòng `np.random.seed(7)` để đảm bảo kết quả chạy ở các lần khác nhau là giống nhau. Có thể thay 7 bằng một số tự nhiên 32 bit bất kỳ.

Kết quả với 1NN

Tới đây, ta trực tiếp sử dụng thư viện scikit-learn cho KNN. Xét ví dụ đầu tiên với $K = 1$.

```
model = neighbors.KNeighborsClassifier(n_neighbors = 1, p = 2)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy of 1NN: %.2f %%" %(100*accuracy_score(y_test, y_pred)))
```

Kết quả:

```
Accuracy of 1NN: 92.31 %
```

Kết quả thu được là 92.31% (tỉ lệ số mẫu được phân loại chính xác trên tổng số mẫu). Ở đây, `n_neighbors = 1` chỉ ra rằng chỉ điểm gần nhất được lựa chọn, tức `k = 1`, `p = 2` chính là dùng chuẩn ℓ_2 để tính khoảng cách.

Kết quả với 7NN

Như đã đề cập, 1NN rất dễ gây ra quá khớp. Để hạn chế việc này, ta có thể tăng lượng điểm lân cận lên, ví dụ 7 điểm, kết quả được xác định dựa trên đa số.

```
model = neighbors.KNeighborsClassifier(n_neighbors = 7, p = 2)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy of 7NN with major voting: %.2f %%"\
     %(100*accuracy_score(y_test, y_pred)))
```

Kết quả:

```
Accuracy of 7NN with major voting: 93.85 %
```

Nhận thấy rằng khi sử dụng nhiều điểm lân cận hơn, độ chính xác đã tăng lên. Phương pháp dựa trên đa số trong lân cận còn được gọi là *bầu chọn đa số*.

Đánh trọng số cho các điểm lân cận

Trong kỹ thuật bầu chọn đa số phía trên, các điểm trong bảy điểm gần nhất đều có vai trò như nhau và giá trị "lá phiếu" của mỗi điểm này cũng như nhau. Cách bầu chọn này có thể thiếu công bằng vì các điểm gần hơn nên có tầm ảnh hưởng lớn hơn. Để thực hiện việc này, ta chỉ cần đánh trọng số khác nhau cho từng điểm trong bảy điểm gần nhất này. Cách đánh trọng số phải thoả mãn điều kiện điểm lân cận hơn được đánh trọng số cao hơn. Một cách đơn giản là lấy nghịch đảo của khoảng cách tới điểm lân cận. Trong trường hợp tồn tại khoảng cách bằng không, tức điểm kiểm tra trùng với một điểm huấn luyện, ta trực tiếp lấy đầu ra của điểm huấn luyện đó.

Để thực hiện việc này trong scikit-learn, ta chỉ cần gán `weights = 'distance'`. Giá trị mặc định của `weights` là `'uniform'`, tương ứng với việc coi tất cả các điểm lân cận có giá trị bằng nhau như trong bầu chọn đa số.

```
model = neighbors.KNeighborsClassifier(n_neighbors = 7, p = 2, \
weights = 'distance')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy of 7NN (1/distance weights): %.2f %%" %(100*accuracy_score(
    y_test, y_pred)))
```

Kết quả:

```
Accuracy of 7NN (1/distance weights): 94.62 %
```

Độ chính xác tiếp tục được tăng lên.

KNN với trọng số tự định nghĩa

Ngoài hai cách đánh trọng số `weights = 'uniform'` và `weights = 'distance'`, scikit-learn còn cung cấp cách đánh trọng số tùy chọn. Ví dụ, một cách đánh trọng số phổ biến khác thường được dùng là

$$w_i = \exp\left(\frac{-\|\mathbf{z} - \mathbf{x}_i\|_2^2}{\sigma^2}\right)$$

trong đó w_i là trọng số của điểm gần thứ i ($\mathbf{x}_i$) của điểm dữ liệu đang xét $\mathbf{z}$, σ là một số dương. Hàm số này cũng thỏa mãn điều kiện điểm càng gần $\mathbf{x}$ thì trọng số càng cao (cao nhất bằng 1). Với hàm số này, ta có thể lập trình như sau:

```
def myweight(distances):
sigma2 = .4
return np.exp(-distances**2/sigma2)

model = neighbors.KNeighborsClassifier(
    n_neighbors = 7, p = 2, weights = myweight)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy of 7NN (customized weights): %.2f %%"\
      %(100*accuracy_score(y_test, y_pred)))
```

Kết quả:

```
Accuracy of 7NN (customized weights): 95.38 %
```

Kết quả tiếp tục tăng lên một chút. Với từng bài toán, chúng ta có thể thay các thuộc tính của KNN bằng các giá trị khác nhau và chọn ra giá trị tốt nhất thông qua xác thực chéo.

5.4. Ưu điểm và hạn chế của thuật toán KNN

Thuật toán k láng giềng gần nhất (KNN) là một trong những phương pháp học máy đơn giản và trực quan nhất. Tuy nhiên, giống như nhiều thuật toán học máy khác, KNN cũng có những ưu điểm và hạn chế nhất định. Việc hiểu rõ các đặc điểm này giúp lựa chọn thuật toán phù hợp với từng bài toán cụ thể.

5.4.1. Ưu điểm của KNN

Một số ưu điểm nổi bật của thuật toán KNN bao gồm:

- **Đơn giản và dễ hiểu:** Ý tưởng của KNN rất trực quan: một điểm dữ liệu mới sẽ có nhãn giống với các điểm dữ liệu gần nó nhất. Vì vậy thuật toán này thường được sử dụng như một phương pháp cơ bản để minh họa các khái niệm trong học máy.
- **Không cần giai đoạn huấn luyện phức tạp:** KNN không xây dựng một mô hình tường minh trong quá trình huấn luyện. Thay vào đó, dữ liệu huấn luyện được lưu trữ và sử dụng trực tiếp trong giai đoạn dự đoán.
- **Có thể áp dụng cho nhiều loại bài toán:** KNN có thể được sử dụng cho cả hai loại bài toán phổ biến:
  - bài toán phân loại (classification)
  - bài toán hồi quy (regression)
- **Không giả định phân phối dữ liệu:** Khác với một số phương pháp thống kê, KNN không yêu cầu giả định về phân phối xác suất của dữ liệu.

5.4.2. Nhược điểm của KNN

Bên cạnh những ưu điểm trên, KNN cũng tồn tại một số hạn chế quan trọng:

- **Chi phí tính toán lớn khi dự đoán:** Để dự đoán nhãn của một điểm dữ liệu mới, thuật toán cần tính khoảng cách từ điểm đó đến toàn bộ các điểm trong tập huấn luyện. Nếu tập dữ liệu lớn, thời gian tính toán có thể trở nên rất đáng kể.
- **Yêu cầu bộ nhớ lớn:** Vì toàn bộ dữ liệu huấn luyện phải được lưu trữ, KNN có thể tiêu tốn nhiều bộ nhớ khi làm việc với các tập dữ liệu lớn.

- **Nhạy cảm với dữ liệu nhiễu:** Nếu tập dữ liệu có nhiều điểm nhiễu hoặc các điểm ngoại lai (outliers), kết quả dự đoán của KNN có thể bị ảnh hưởng đáng kể.

- **Ảnh hưởng của chuẩn hóa dữ liệu:** Khoảng cách giữa các điểm dữ liệu phụ thuộc vào thang đo của các đặc trưng. Nếu các đặc trưng có đơn vị đo khác nhau, những đặc trưng có giá trị lớn hơn có thể chi phối kết quả tính khoảng cách.

- **Lời nguyền số chiều (curse of dimensionality):** Khi số chiều của dữ liệu tăng lên, khoảng cách giữa các điểm dữ liệu có xu hướng trở nên ít phân biệt hơn, làm giảm hiệu quả của KNN.

5.4.3. Khi nào nên sử dụng KNN

KNN thường hoạt động tốt trong các trường hợp sau:

- Tập dữ liệu có kích thước nhỏ hoặc trung bình.

- Số chiều của dữ liệu không quá lớn.

- Mối quan hệ giữa các lớp dữ liệu có tính cục bộ (local structure), nghĩa là các điểm gần nhau có xu hướng thuộc cùng một lớp.

- Khi cần một phương pháp đơn giản để xây dựng mô hình cơ sở (baseline) cho bài toán.

5.4.4. Khi nào không nên sử dụng KNN

Trong một số trường hợp, KNN không phải là lựa chọn phù hợp:

- Khi tập dữ liệu rất lớn, vì chi phí tính toán trong giai đoạn dự đoán sẽ cao.

- Khi dữ liệu có số chiều rất lớn, vì khoảng cách giữa các điểm trở nên kém ý nghĩa.

- Khi dữ liệu có nhiều nhiễu hoặc nhiều điểm ngoại lai.

- Khi yêu cầu thời gian dự đoán nhanh trong các hệ thống thời gian thực.

Trong thực tế, để cải thiện hiệu quả của KNN, người ta thường kết hợp với các kỹ thuật tiền xử lý dữ liệu như chuẩn hóa đặc trưng (feature scaling), giảm chiều dữ liệu (dimensionality reduction) hoặc sử dụng các cấu trúc dữ liệu như KD-tree hay Ball-tree để tăng tốc việc tìm kiếm các điểm lân cận.

5.5. Kết luận

Trong chương này, chúng ta đã nghiên cứu thuật toán *k-Nearest Neighbors* (KNN), một trong những phương pháp phân loại và hồi quy đơn giản nhưng hiệu quả trong học máy. Khác với nhiều mô hình học máy khác, KNN không xây dựng một mô hình tường minh trong quá trình huấn luyện mà lưu trữ toàn bộ dữ liệu huấn luyện và đưa ra dự đoán dựa trên các điểm dữ liệu gần nhất.

Nguyên lý cơ bản của KNN là: một điểm dữ liệu mới sẽ được gán nhãn dựa trên nhãn của k điểm dữ liệu gần nhất trong tập huấn luyện theo một thước đo khoảng cách nhất định. Trong chương này, chúng ta đã tìm hiểu một số loại khoảng cách phổ biến như khoảng cách Euclid, khoảng cách Manhattan và khoảng cách Minkowski.

Bên cạnh đó, chương cũng đã trình bày cách lựa chọn tham số k và ảnh hưởng của tham số này đến kết quả phân loại. Nếu k quá nhỏ, mô hình có thể bị ảnh hưởng bởi nhiễu trong dữ liệu. Ngược lại, nếu k quá lớn, mô hình có thể trở nên quá tổng quát và mất đi khả năng phân biệt giữa các lớp.

Chúng ta cũng đã phân tích ưu điểm và hạn chế của thuật toán KNN. Ưu điểm nổi bật của phương pháp này là tính đơn giản, dễ cài đặt và không yêu cầu giả định về phân phối của dữ liệu. Tuy nhiên, KNN có nhược điểm là chi phí tính toán lớn trong giai đoạn dự đoán và có thể hoạt động kém hiệu quả khi số chiều của dữ liệu quá lớn.

Cuối cùng, chương đã minh họa việc triển khai thuật toán KNN bằng ngôn ngữ lập trình Python thông qua thư viện `scikit-learn`. Việc thực hành lập trình giúp người học hiểu rõ hơn cách áp dụng thuật toán vào các bài toán phân loại và hồi quy trong thực tế.

5.6. Câu hỏi ôn tập

Câu hỏi lý thuyết

a. Trình bày nguyên lý hoạt động của thuật toán k-Nearest Neighbors (KNN).

b. Thuật toán KNN được xếp vào loại thuật toán học máy nào? Giải thích tại sao KNN được gọi là thuật toán "lazy learning".

c. Trình bày vai trò của tham số k trong thuật toán KNN. Điều gì xảy ra nếu giá trị k quá nhỏ hoặc quá lớn?

d. Khoảng cách giữa các điểm dữ liệu được sử dụng như thế nào trong KNN? Hãy trình bày công thức của khoảng cách Euclid.

e. Nêu một số loại khoảng cách phổ biến được sử dụng trong thuật toán KNN.

f. Phân tích ưu điểm và hạn chế của thuật toán KNN.

g. Trong những trường hợp nào thuật toán KNN thường hoạt động hiệu quả? Khi nào không nên sử dụng KNN?

Bài tập thực hành

h. Cho các điểm dữ liệu sau trong không gian hai chiều:

$$(1,2),\ (2,3),\ (3,3),\ (6,5),\ (7,7)$$

Giả sử có một điểm dữ liệu mới $(4,4)$.

a. Tính khoảng cách Euclid từ điểm mới đến các điểm trong tập dữ liệu.

b. Với $k = 3$, hãy xác định ba điểm gần nhất.

i. Cho tập dữ liệu phân loại gồm hai lớp A và B với các điểm dữ liệu sau:

$$A:\ (1,1),\ (2,2),\ (2,1)$$

$$B:\ (6,5),\ (7,7),\ (8,6)$$

Hãy sử dụng thuật toán KNN với $k = 3$ để xác định lớp của điểm dữ liệu mới $(3,2)$.

j. Viết chương trình Python sử dụng thư viện `scikit-learn` để thực hiện phân loại KNN trên bộ dữ liệu Iris. Hãy thực hiện các bước sau:

a. Chia dữ liệu thành tập huấn luyện và tập kiểm tra.

b. Huấn luyện mô hình KNN với $k = 5$.

c. Đánh giá độ chính xác của mô hình trên tập kiểm tra.

Chương 6

Bộ phân loại Naïve Bayes

Trong nhiều bài toán học máy, dữ liệu thường chứa các thông tin không chắc chắn và việc đưa ra quyết định cần dựa trên các mô hình xác suất. Thay vì chỉ dựa vào sự giống nhau giữa các điểm dữ liệu, một cách tiếp cận khác là ước lượng xác suất để một đối tượng thuộc về từng lớp khác nhau dựa trên các đặc trưng quan sát được.

Bộ phân loại *Naïve Bayes* được xây dựng dựa trên nguyên lý này. Phương pháp này sử dụng định lý Bayes để mô hình hóa mối quan hệ giữa các đặc trưng của dữ liệu và nhãn lớp tương ứng. Từ đó, mô hình có thể ước lượng xác suất hậu nghiệm của từng lớp và đưa ra quyết định phân loại bằng cách chọn lớp có xác suất lớn nhất.

Nhờ cấu trúc toán học đơn giản và khả năng xử lý hiệu quả các dữ liệu có số chiều lớn, Naïve Bayes đã trở thành một trong những phương pháp phân loại được sử dụng rộng rãi trong nhiều lĩnh vực như lọc thư rác, phân loại văn bản và phân tích dữ liệu trong thực tế.

6.1. Giới thiệu

6.1.1. Khái niệm

Naïve Bayes là một mô hình phân loại xác suất (probabilistic classifier) dựa trên định lý Bayes trong lý thuyết xác suất. Mô hình này được sử dụng rộng rãi trong nhiều bài toán phân loại như phân loại văn bản, lọc email spam và phân tích cảm xúc.

Tên gọi *Naïve Bayes* xuất phát từ hai thành phần chính của mô hình. Phần "Bayes" đề cập đến việc mô hình được xây dựng dựa trên *định lý Bayes*, cho phép

tính xác suất hậu nghiệm của một lớp dựa trên thông tin của các đặc trưng đầu vào.

Trong khi đó, từ "Naïve" (ngây ngô) xuất phát từ giả thiết đơn giản mà mô hình sử dụng. Cụ thể, Naïve Bayes giả định rằng các đặc trưng trong dữ liệu là *độc lập có điều kiện* với nhau khi biết lớp c. Nói cách khác, sự xuất hiện của một đặc trưng không ảnh hưởng đến sự xuất hiện của các đặc trưng khác khi lớp của dữ liệu đã được biết trước.

Mặc dù giả thiết độc lập này thường không hoàn toàn đúng trong dữ liệu thực tế, nhưng Naïve Bayes vẫn hoạt động khá hiệu quả trong nhiều bài toán thực tế. Nhờ cấu trúc đơn giản, mô hình có tốc độ huấn luyện và dự đoán rất nhanh, đồng thời có thể xử lý tốt các bài toán có tập dữ liệu lớn và số chiều cao.

6.1.2. Nhắc lại xác suất có điều kiện



Hình 6.1. Thomas Bayes (1701–1761), nhà toán học đặt nền móng cho định lý Bayes

Trước khi trình bày chi tiết về bộ phân loại Naïve Bayes, chúng ta cần ôn lại một số khái niệm cơ bản trong lý thuyết xác suất, đặc biệt là khái niệm *xác suất có điều kiện* và *định lý Bayes*.

Xác suất có điều kiện của một biến cố A khi biết biến cố B đã xảy ra được định nghĩa bởi:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \tag{6.1}$$

Trong đó:

- $P(A|B)$ là xác suất để biến cố A xảy ra khi biết B đã xảy ra.
- $P(A \cap B)$ là xác suất để cả hai biến cố A và B cùng xảy ra.
- $P(B)$ là xác suất xảy ra của biến cố B.

Từ định nghĩa trên, ta có thể suy ra một trong những kết quả quan trọng nhất trong xác suất học, được gọi là *định lý Bayes*. Định lý này được đặt theo tên của nhà toán học Thomas Bayes (Hình 6.1).

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{6.2}$$

Trong đó:

- $P(A)$ được gọi là *xác suất tiên nghiệm* (prior probability).
- $P(B|A)$ được gọi là *xác suất khả năng* (likelihood).
- $P(A|B)$ được gọi là *xác suất hậu nghiệm* (posterior probability).

Ví dụ minh họa.

Giả sử trong một lớp học có 60% sinh viên là nam và 40% sinh viên là nữ. Trong số các sinh viên nam, có 30% thích học môn Machine Learning, trong khi ở nhóm nữ con số này là 50%.

Giả sử chọn ngẫu nhiên một sinh viên và biết rằng sinh viên này thích môn Machine Learning. Ta muốn tính xác suất để sinh viên đó là nữ.

Áp dụng định lý Bayes:

$$P(\text{Nữ}|\text{Thích ML}) = \frac{P(\text{Thích ML}|\text{Nữ})P(\text{Nữ})}{P(\text{Thích ML})} \tag{6.3}$$

Trong đó:

$$P(\text{Thích ML}) = 0.3 \times 0.6 + 0.5 \times 0.4$$

Từ đó ta có thể tính được xác suất hậu nghiệm $P(\text{Nữ}|\text{Thích ML})$.

Ví dụ này minh họa cách định lý Bayes cho phép chúng ta suy ra xác suất của một nguyên nhân khi biết kết quả quan sát được. Ý tưởng này chính là nền tảng của mô hình Naïve Bayes trong học máy.

6.2. Bài toán phân loại trong Naïve Bayes

Xét một bài toán phân loại với C lớp khác nhau:

$$1, 2, \ldots, C.$$

Giả sử có một điểm dữ liệu $\mathbf{x} \in \mathbb{R}^d$. Mục tiêu của bài toán phân loại là xác định xác suất để điểm dữ liệu này thuộc về lớp c. Nói cách khác, ta cần tính xác suất hậu nghiệm:

$$p(y = c|\mathbf{x}) \tag{6.4}$$

Biểu thức này thường được viết gọn dưới dạng:

$$p(c|\mathbf{x})$$

tức là xác suất để đầu ra thuộc lớp c khi biết đầu vào là vector đặc trưng $\mathbf{x}$.

Nếu có thể tính được giá trị của $p(c|\mathbf{x})$ với mọi lớp c, ta có thể xác định nhãn của điểm dữ liệu bằng cách chọn lớp có xác suất lớn nhất:

$$c = \arg \max_{c \in \{1,\ldots,C\}} p(c|\mathbf{x}) \tag{6.5}$$

Tuy nhiên, việc tính trực tiếp xác suất hậu nghiệm $p(c|\mathbf{x})$ thường khá khó. Vì vậy, ta sử dụng định lý Bayes để biến đổi biểu thức trên:

$$c = \arg \max_{c} p(c|\mathbf{x}) \tag{6.6}$$

$$= \arg \max_{c} \frac{p(\mathbf{x}|c)p(c)}{p(\mathbf{x})} \tag{6.7}$$

Do $p(\mathbf{x})$ không phụ thuộc vào c, ta có thể bỏ qua thành phần này trong quá trình tối ưu. Khi đó bài toán trở thành:

$$c = \arg\max_{c} p(\mathbf{x}|c)p(c) \tag{6.8}$$

Trong biểu thức trên:

- $p(c)$ là xác suất tiên nghiệm của lớp c.
- $p(\mathbf{x}|c)$ là xác suất để quan sát được dữ liệu $\mathbf{x}$ khi biết dữ liệu thuộc lớp c.

Xác suất tiên nghiệm $p(c)$ thường được ước lượng bằng tần suất xuất hiện của lớp c trong tập dữ liệu huấn luyện, tức là:

$$p(c) = \frac{\text{số mẫu thuộc lớp } c}{\text{tổng số mẫu}}$$

Phương pháp ước lượng này tương ứng với *Maximum Likelihood Estimation (MLE)* và được sử dụng phổ biến trong thực tế.

6.2.1. Giả thiết độc lập trong Naïve Bayes

Thành phần còn lại trong biểu thức trên là $p(\mathbf{x}|c)$. Việc ước lượng phân phối này thường rất khó vì $\mathbf{x}$ là một vector ngẫu nhiên nhiều chiều. Nếu không có giả thiết bổ sung, ta cần một lượng dữ liệu huấn luyện rất lớn để có thể mô hình hóa chính xác phân phối này.

Để đơn giản hóa bài toán, Naïve Bayes đưa ra một giả thiết quan trọng: các đặc trưng của dữ liệu độc lập có điều kiện khi biết lớp c. Khi đó ta có:

$$p(\mathbf{x}|c) = p(x_1, x_2, \ldots, x_d|c) = \prod_{i=1}^{d} p(x_i|c) \tag{6.9}$$

Giả thiết này được gọi là *giả thiết độc lập có điều kiện* (conditional independence assumption). Mặc dù giả thiết này thường không hoàn toàn đúng trong thực tế, nhưng nó giúp đơn giản hóa đáng kể việc tính toán và trong nhiều trường hợp vẫn mang lại kết quả khá tốt.

Dựa trên giả thiết này, bộ phân loại Naïve Bayes xác định lớp của một điểm dữ liệu mới $\mathbf{x}$ theo công thức:

$$c = \arg\max_{c \in \{1,\ldots,C\}} p(c) \prod_{i=1}^{d} p(x_i|c) \tag{6.10}$$

Trong thực tế, khi số chiều d lớn, tích của nhiều xác suất nhỏ có thể dẫn tới sai số số học. Vì vậy, biểu thức trên thường được viết lại dưới dạng log:

$$c = \arg\max_c \left[\log p(c) + \sum_{i=1}^{d} \log p(x_i|c) \right] \tag{6.11}$$

Do hàm log là hàm đồng biến trên tập số dương, việc lấy log không làm thay đổi kết quả của phép tối ưu.

Nhờ giả thiết độc lập đơn giản này, mô hình Naïve Bayes có cấu trúc tính toán rất đơn giản. Thay vì phải ước lượng một phân phối xác suất đa chiều phức tạp cho toàn bộ vector đặc trưng $\mathbf{x}$, mô hình chỉ cần ước lượng các phân phối xác suất một chiều $p(x_i|c)$ cho từng đặc trưng riêng lẻ. Điều này giúp giảm đáng kể độ phức tạp của quá trình huấn luyện cũng như dự đoán. Nhờ những đặc điểm này, Naïve Bayes đặc biệt phù hợp với các bài toán có quy mô dữ liệu lớn và số chiều đặc trưng cao. Trong thực tế, mô hình này thường được áp dụng hiệu quả trong nhiều bài toán xử lý dữ liệu văn bản như phân loại tài liệu, lọc email spam, phân tích cảm xúc trong mạng xã hội, hay phân loại tin tức tự động. Mặc dù giả thiết độc lập giữa các đặc trưng là một giả thiết khá mạnh và không phải lúc nào cũng đúng trong thực tế, Naïve Bayes vẫn cho kết quả khá tốt trong nhiều ứng dụng thực tiễn nhờ khả năng tổng quát hóa tốt và tốc độ tính toán cao.

Do việc ước lượng các tham số chỉ dựa trên các tần suất xuất hiện trong dữ liệu huấn luyện, thời gian huấn luyện của Naïve Bayes thường rất nhanh, ngay cả khi kích thước tập dữ liệu lớn. Bên cạnh đó, quá trình dự đoán cho một điểm dữ liệu mới cũng chỉ bao gồm việc tính một số phép nhân hoặc phép cộng log xác suất, do đó chi phí tính toán tương đối thấp. Trong thực tế, cách tính $p(x_i|c)$ phụ thuộc vào loại dữ liệu. Ba biến thể phổ biến của Naïve Bayes bao gồm:

- Gaussian Naïve Bayes
- Multinomial Naïve Bayes
- Bernoulli Naïve Bayes

Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE) là một phương pháp phổ biến để ước lượng các tham số của một mô hình xác suất dựa trên dữ liệu quan sát được. Ý tưởng cơ bản của phương pháp này là tìm giá trị tham số sao cho xác suất sinh ra tập dữ liệu quan sát là lớn nhất.
Giả sử tập dữ liệu quan sát là $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ và các dữ liệu này được giả định là độc lập và tuân theo một phân phối xác suất $p(x|\theta)$ với tham số θ. Khi đó xác suất để toàn bộ tập dữ liệu xuất hiện (likelihood) được viết dưới dạng:

$$L(\theta) = \prod_{i=1}^{n} p(x_i|\theta) \tag{6.12}$$

Phương pháp MLE tìm giá trị tham số θ sao cho hàm likelihood đạt giá trị lớn nhất:

$$\hat{\theta} = \arg\max_{\theta} L(\theta) \tag{6.13}$$

Trong thực tế, để thuận tiện cho việc tính toán, người ta thường tối đa hóa log-likelihood:

$$\hat{\theta} = \arg\max_{\theta} \sum_{i=1}^{n} \log p(x_i|\theta) \tag{6.14}$$

Ví dụ. *Giả sử ta tung một đồng xu n lần và quan sát được k lần xuất hiện mặt ngửa. Nếu p là xác suất xuất hiện mặt ngửa, ước lượng MLE của p sẽ là:*

$$\hat{p} = \frac{k}{n} \tag{6.15}$$

tức là xác suất được ước lượng bằng tần suất xuất hiện của sự kiện trong dữ liệu quan sát.

6.2.2. Hàm mất mát và tối ưu trong Naïve Bayes

Trong nhiều mô hình học máy, quá trình huấn luyện thường được thực hiện bằng cách tối thiểu hóa một hàm mất mát thông qua các thuật toán tối ưu như Gradient Descent. Tuy nhiên, trong Naïve Bayes việc ước lượng tham số được thực hiện theo một cách đơn giản hơn, dựa trên nguyên lý *Maximum Likelihood Estimation (MLE).*

Giả sử tập dữ liệu huấn luyện gồm N mẫu:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\}$$

trong đó $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{id})$ là vector đặc trưng d chiều và y_i là nhãn lớp tương ứng.

Mục tiêu của mô hình là tìm các tham số θ sao cho xác suất sinh ra tập dữ liệu huấn luyện là lớn nhất. Xác suất này được gọi là *likelihood*:

$$L(\theta) = \prod_{i=1}^{N} p(\mathbf{x}_i, y_i|\theta) \tag{6.16}$$

Theo quy tắc xác suất ta có:

$$p(\mathbf{x}_i, y_i|\theta) = p(y_i|\theta)p(\mathbf{x}_i|y_i, \theta) \tag{6.17}$$

Dựa trên giả thiết độc lập của Naïve Bayes, ta có:

$$p(\mathbf{x}_i|y_i, \theta) = \prod_{j=1}^{d} p(x_{ij}|y_i, \theta) \tag{6.18}$$

Do đó hàm likelihood của toàn bộ tập dữ liệu có thể viết lại thành:

$$L(\theta) = \prod_{i=1}^{N} p(y_i) \prod_{j=1}^{d} p(x_{ij}|y_i) \tag{6.19}$$

Trong thực tế, để tránh các vấn đề tính toán với tích của nhiều số nhỏ, người ta thường tối đa hóa *log-likelihood* thay vì likelihood:

$$\ell(\theta) = \sum_{i=1}^{N} \left[\log p(y_i) + \sum_{j=1}^{d} \log p(x_{ij}|y_i) \right] \tag{6.20}$$

Quá trình huấn luyện Naïve Bayes tương đương với việc tìm tham số θ sao cho:

$$\theta^* = \arg\max_{\theta} \ell(\theta) \tag{6.21}$$

Nhờ giả thiết độc lập giữa các đặc trưng, việc tối đa hóa log-likelihood có thể được thực hiện rất nhanh bằng cách đếm tần suất xuất hiện của các đặc trưng trong dữ liệu huấn luyện. Điều này giúp Naïve Bayes có thời gian huấn luyện và dự đoán rất nhanh ngay cả với các tập dữ liệu có kích thước lớn.

Ví 6.1. **Naïve Bayes cho dữ liệu danh nghĩa**

Cho tập dữ liệu huấn luyện "Buy Computer" trong bảng sau.

Bảng 6.1: Dữ liệu cho bài toán phân loại Buy Computer

RID	Age	Income	Student	Credit Rating	Buys Computer
1	≤ 30	high	no	fair	no
2	≤ 30	high	no	excellent	no
3	31–40	high	no	fair	yes
4	> 40	medium	no	fair	yes
5	> 40	low	yes	fair	yes
6	> 40	low	yes	excellent	no
7	31–40	low	yes	excellent	yes
8	≤ 30	medium	no	fair	no
9	≤ 30	low	yes	fair	yes
10	> 40	medium	yes	fair	yes
11	≤ 30	medium	yes	excellent	yes
12	31–40	medium	no	excellent	yes
13	31–40	high	yes	fair	yes
14	> 40	medium	no	excellent	no

Hãy sử dụng bộ phân loại Naïve Bayes để dự đoán lớp của mẫu dữ liệu mới:

$$x = (age \leq 30,\ income = medium,\ student = yes,\ credit_rating = fair)$$

Lời giải.

Đầu tiên, ta cần tính xác suất tiên nghiệm của hai lớp $buys_computer = yes$ và $buys_computer = no$. Quan sát bảng dữ liệu huấn luyện gồm 14 mẫu, trong đó có 9 mẫu thuộc lớp *yes* và 5 mẫu thuộc lớp *no*. Do đó:

$$P(yes) = \frac{9}{14}, \qquad P(no) = \frac{5}{14} \tag{6.22}$$

Tiếp theo, ta tính các xác suất điều kiện của từng đặc trưng theo mỗi lớp.

Đối với lớp *yes*, trong 9 mẫu dữ liệu thuộc lớp này, ta có:

$$P(age \leq 30|yes) = \frac{2}{9}$$

$$P(income = medium|yes) = \frac{4}{9}$$

$$P(student = yes|yes) = \frac{6}{9}$$

$$P(credit_rating = fair|yes) = \frac{6}{9}$$

Theo công thức của Naïve Bayes, ta có:

$$\begin{aligned} P(yes|x) \propto P(yes)\, P(age \leq 30|yes)\, P(income = medium|yes) \\ \times P(student = yes|yes)\, P(credit_rating = fair|yes) \end{aligned} \tag{6.23}$$

Thay các giá trị đã tính được vào:

$$P(yes|x) = \frac{9}{14} \times \frac{2}{9} \times \frac{4}{9} \times \frac{6}{9} \times \frac{6}{9} = 0.0282$$

Tiếp theo, ta thực hiện tính toán tương tự cho lớp *buys_computer = no*. Trong 5 mẫu thuộc lớp này, ta có:

$$P(age \leq 30|no) = \frac{3}{5}$$

$$P(income = medium|no) = \frac{2}{5}$$

$$P(student = yes|no) = \frac{1}{5}$$

$$P(credit_rating = fair|no) = \frac{2}{5}$$

Do đó:

$$\begin{aligned} P(no|x) \propto P(no)\, P(age \leq 30|no)\, P(income = medium|no) \\ \times P(student = yes|no)\, P(credit_rating = fair|no) \end{aligned} \tag{6.24}$$

Thay các giá trị vào:

$$P(no|x) = \frac{5}{14} \times \frac{3}{5} \times \frac{2}{5} \times \frac{1}{5} \times \frac{2}{5} = 0.0068$$

So sánh hai giá trị xác suất hậu nghiệm:

$$P(yes|x) = 0.0282, \qquad P(no|x) = 0.0068$$

Vì $P(yes|x) > P(no|x)$ nên theo bộ phân loại Naïve Bayes, mẫu dữ liệu mới

$$(age \leq 30,\ income = medium,\ student = yes,\ credit_rating = fair)$$

được phân loại vào lớp:

$$\boxed{buys_computer = yes} \tag{6.25}$$

6.3. Thực hành

6.3.1. Cài đặt Naïve Bayes bằng Python (tính toán thủ công)

Để giúp sinh viên hiểu rõ cách hoạt động của Naïve Bayes, ta có thể tự cài đặt thuật toán dựa trên công thức xác suất đã trình bày ở phần trước. Ý tưởng chính là tính xác suất tiên nghiệm $P(c)$ và các xác suất điều kiện $P(x_i|c)$ từ dữ liệu huấn luyện, sau đó nhân các xác suất này lại để tìm xác suất hậu nghiệm cho từng lớp.

Đoạn chương trình Python sau minh họa cách thực hiện phương pháp này.

```
import pandas as pd

# Tao du lieu
data = {
    "age": ["<=30","<=30","31-40",">40",">40",">40","31-40","<=30",
            "<=30",">40","<=30","31-40","31-40",">40"],
    "income": ["high","high","high","medium","low","low","low","medium",
               "low","medium","medium","medium","high","medium"],
    "student": ["no","no","no","no","yes","yes","yes","no",
                "yes","yes","yes","no","yes","no"],
    "credit_rating": ["fair","excellent","fair","fair","fair","excellent",
                      "excellent","fair","fair","fair","excellent",
                      "excellent","fair","excellent"],
    "buys_computer": ["no","no","yes","yes","yes","no","yes","no",
                      "yes","yes","yes","yes","yes","no"]
}

df = pd.DataFrame(data)

# Mau can du doan
x = {
    "age": "<=30",
    "income": "medium",
    "student": "yes",
    "credit_rating": "fair"
}

def naive_bayes_classify(x, df):
    classes = df["buys_computer"].unique()
    probs = {}

    for c in classes:
        df_c = df[df["buys_computer"] == c]
        p = len(df_c) / len(df)

        for feature in x:
            p *= len(df_c[df_c[feature] == x[feature]]) / len(df_c)

        probs[c] = p

    return probs

result = naive_bayes_classify(x, df)

print("Posterior probabilities:", result)
prediction = max(result, key=result.get)
print("Prediction:", prediction)
```

Kết quả của chương trình cho thấy mẫu dữ liệu mới được phân loại vào lớp *yes*, phù hợp với kết quả tính toán bằng tay ở phần ví dụ trước.

6.3.2. Sử dụng thư viện scikit-learn

Trong thực tế, thay vì tự cài đặt thuật toán từ đầu, ta thường sử dụng các thư viện học máy đã được tối ưu sẵn như `scikit-learn`. Vì dữ liệu trong ví dụ này là dữ liệu dạng danh mục (categorical), các giá trị cần được mã hóa thành số trước khi đưa vào mô hình.

Đoạn chương trình sau minh họa cách huấn luyện mô hình `CategoricalNB` trong thư viện `scikit-learn`.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import CategoricalNB

data = {
    "age": ["<=30","<=30","31-40",">40",">40",">40","31-40","<=30",
            "<=30",">40","<=30","31-40","31-40",">40"],
    "income": ["high","high","high","medium","low","low","low","medium",
               "low","medium","medium","medium","high","medium"],
    "student": ["no","no","no","no","yes","yes","yes","no",
                "yes","yes","yes","no","yes","no"],
    "credit_rating": ["fair","excellent","fair","fair","fair","excellent",
                      "excellent","fair","fair","fair","excellent",
                      "excellent","fair","excellent"],
    "buys_computer": ["no","no","yes","yes","yes","no","yes","no",
                      "yes","yes","yes","yes","yes","no"]
}

df = pd.DataFrame(data)

encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le

X = df.drop("buys_computer", axis=1)
y = df["buys_computer"]

model = CategoricalNB()
model.fit(X, y)

x_new = {
    "age": "<=30",
    "income": "medium",
    "student": "yes",
    "credit_rating": "fair"
}
x_new_encoded = [encoders[col].transform([x_new[col]])[0] for col in X.
    columns]

prediction = model.predict([x_new_encoded])

print("Prediction:",
      encoders["buys_computer"].inverse_transform(prediction))
```

Kết quả thu được cũng dự đoán lớp của mẫu dữ liệu mới là *yes*. Điều này cho thấy mô hình Naïve Bayes trong `scikit-learn` cho kết quả phù hợp với phương pháp tính toán xác suất đã trình bày trước đó.

6.4. Kết luận

Trong phần này, chúng ta đã tìm hiểu về bộ phân loại Naïve Bayes, một trong những mô hình phân loại xác suất cơ bản và được sử dụng rộng rãi trong học máy. Mô hình này được xây dựng dựa trên định lý Bayes và giả thiết rằng các đặc trưng của dữ liệu độc lập có điều kiện với nhau khi biết lớp của dữ liệu.

Nhờ giả thiết độc lập đơn giản này, việc ước lượng các xác suất điều kiện trở nên dễ dàng và chi phí tính toán của thuật toán được giảm đáng kể. Thay vì phải mô hình hóa một phân phối xác suất đa chiều phức tạp, Naïve Bayes chỉ cần ước lượng các phân phối một chiều cho từng đặc trưng riêng lẻ. Điều này giúp thuật toán có tốc độ huấn luyện và dự đoán rất nhanh, đặc biệt khi làm việc với các tập dữ liệu có kích thước lớn.

Trong phần trình bày, chúng ta đã xem xét cách áp dụng định lý Bayes để xây dựng bộ phân loại, đồng thời phân tích vai trò của các thành phần như xác suất tiên nghiệm $P(c)$ và xác suất điều kiện $P(x_i|c)$. Thông qua ví dụ minh họa và bài tập thực hành, người học có thể hiểu rõ hơn cách tính các xác suất này từ dữ liệu huấn luyện và cách sử dụng chúng để đưa ra quyết định phân loại cho một mẫu dữ liệu mới.

Ngoài ra, phần thực hành bằng Python đã cho thấy cách triển khai Naïve Bayes trong thực tế, bao gồm cả việc cài đặt thuật toán từ các công thức xác suất cũng như sử dụng các thư viện học máy như `scikit-learn`. Những công cụ này giúp việc áp dụng thuật toán vào các bài toán thực tế trở nên thuận tiện và hiệu quả hơn.

Mặc dù giả thiết độc lập giữa các đặc trưng là một giả thiết khá mạnh và không phải lúc nào cũng hoàn toàn đúng trong thực tế, Naïve Bayes vẫn cho kết quả khá tốt trong nhiều ứng dụng, đặc biệt là trong các bài toán phân loại văn bản, lọc thư rác và phân tích dữ liệu có số chiều cao.

6.5. Câu hỏi ôn tập

Câu hỏi lý thuyết

a. Trình bày khái niệm của bộ phân loại Naïve Bayes. Tại sao mô hình này lại được gọi là "Naïve"?

b. Phát biểu định lý Bayes và giải thích ý nghĩa của các thành phần trong công thức:

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

c. Giải thích giả thiết độc lập có điều kiện giữa các đặc trưng trong mô hình Naïve Bayes. Giả thiết này giúp đơn giản hóa việc tính toán như thế nào?

d. Trong mô hình Naïve Bayes, xác suất tiên nghiệm $P(c)$ và xác suất điều kiện $P(x_i|c)$ thường được ước lượng bằng phương pháp nào? Hãy giải thích ngắn gọn nguyên lý của phương pháp đó.

e. Tại sao trong thực tế người ta thường sử dụng log-likelihood thay vì nhân trực tiếp các xác suất khi tính toán trong Naïve Bayes?

f. Trình bày ba biến thể phổ biến của Naïve Bayes và nêu đặc điểm của từng loại:

- Gaussian Naïve Bayes
- Multinomial Naïve Bayes
- Bernoulli Naïve Bayes

g. Nêu một số ưu điểm và hạn chế của mô hình Naïve Bayes. Trong những trường hợp nào mô hình này thường hoạt động hiệu quả?

Bài tập thực hành

h. Cho một tập dữ liệu huấn luyện gồm 12 mẫu, trong đó có 8 mẫu thuộc lớp *yes* và 4 mẫu thuộc lớp *no*.

   a. Tính xác suất tiên nghiệm $P(yes)$ và $P(no)$.

   b. Nếu trong 8 mẫu lớp *yes* có 5 mẫu có đặc trưng $x_1 = 1$, hãy tính $P(x_1 = 1|yes)$.

i. Giả sử trong một bài toán phân loại văn bản sử dụng Multinomial Naïve Bayes, một từ xuất hiện 15 lần trong các tài liệu thuộc lớp *Spam* và 5 lần trong các tài liệu thuộc lớp Ham. Tổng số từ trong lớp *Spam* là 100 và trong lớp Ham là 80.

   a. Tính xác suất $P(word|Spam)$ và $P(word|Ham)$.

   b. Giải thích ý nghĩa của hai xác suất này trong bài toán phân loại văn bản.

j. Cho một mẫu dữ liệu mới có vector đặc trưng $x = (x_1, x_2, x_3)$. Giả sử các giá trị sau đã được ước lượng từ dữ liệu huấn luyện:

$$P(c_1) = 0.6, \quad P(c_2) = 0.4$$

$$P(x_1|c_1) = 0.5, \quad P(x_2|c_1) = 0.6, \quad P(x_3|c_1) = 0.7$$

$$P(x_1|c_2) = 0.4, \quad P(x_2|c_2) = 0.5, \quad P(x_3|c_2) = 0.6$$

a. Tính $P(c_1|x)$ và $P(c_2|x)$ theo công thức Naïve Bayes (bỏ qua mẫu số $P(x)$).

b. Xác định lớp của điểm dữ liệu x.

Chương 7

Thuật toán học Perceptron

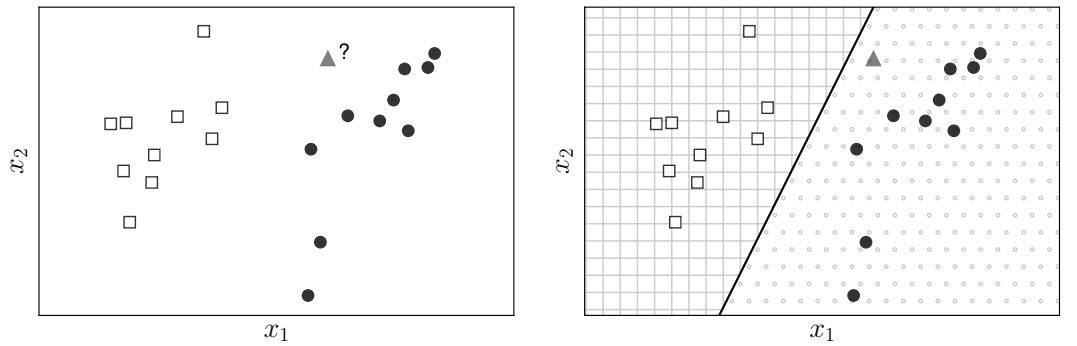
Thuật toán **Perceptron Learning Algorithm (PLA)** là một trong những mô hình học máy tuyến tính cơ bản, được đề xuất nhằm giải quyết các bài toán *phân loại nhị phân*. Mục tiêu của thuật toán là tìm một siêu phẳng tuyến tính có khả năng phân tách các mẫu dữ liệu thuộc hai lớp khác nhau trong không gian đặc trưng.

PLA thực hiện quá trình học thông qua cơ chế cập nhật lặp các tham số của mô hình. Cụ thể, mỗi khi một mẫu dữ liệu bị phân loại sai, vector trọng số sẽ được điều chỉnh để cải thiện khả năng phân tách của siêu phẳng. Nhờ cơ chế học đơn giản nhưng hiệu quả, perceptron được xem là nền tảng quan trọng cho sự phát triển của các mô hình mạng nơ-ron và nhiều phương pháp học sâu hiện đại.

7.1. Giới thiệu

Giả sử có hai tập dữ liệu hình vuông và tròn như được minh hoạ trong Hình 7.1a. Bài toán đặt ra là từ dữ liệu của hai tập được gán nhãn cho trước, hãy xây dựng một bộ phân loại có khả năng dự đoán nhãn của một điểm dữ liệu mới, chẳng hạn điểm hình tam giác màu xám.

Nếu coi mỗi vector đặc trưng là một điểm trong không gian nhiều chiều, bài toán phân loại có thể được coi như bài toán xác định nhãn của từng điểm trong không gian. Nếu coi mỗi nhãn chiếm một hoặc vài vùng trong không gian, ta cần đi tìm ranh giới giữa các vùng đó. Ranh giới đơn giản nhất trong không gian hai chiều là một đường thẳng, trong không gian ba chiều là một mặt phẳng, trong không gian nhiều chiều là một *siêu phẳng*. Những ranh giới phẳng này đơn giản vì chúng có thể được biểu diễn bởi một hàm số tuyến tính. Hình 7.1b minh họa một đường thẳng phân chia hai tập dữ liệu trong không gian hai chiều. Trong trường hợp này, điểm dữ liệu mới hình tam giác rơi vào cùng tập hợp với các điểm hình tròn.



(a) **(b)**

Hình 7.1. Bài toán phân loại nhị phân trong không gian hai chiều. (a) Cho hai tập dữ liệu được gán nhãn vuông và tròn, hãy xác định nhãn của điểm tam giác. (b) Ví dụ về một ranh giới phẳng phân chia hai tập hợp. Điểm tam giác được phân vào tập các điểm hình tròn.

PLA là một thuật toán đơn giản giúp tìm ranh giới siêu phẳng cho bài toán phân loại nhị phân trong trường hợp tồn tại siêu phẳng đó. Nếu hai tập dữ liệu có thể được phân chia hoàn toàn bằng một siêu phẳng, ta nói rằng hai tập đó *tách biệt tuyến tính* (linearly separable).

7.2. Thuật toán học perceptron

7.2.1. Quy tắc phân loại

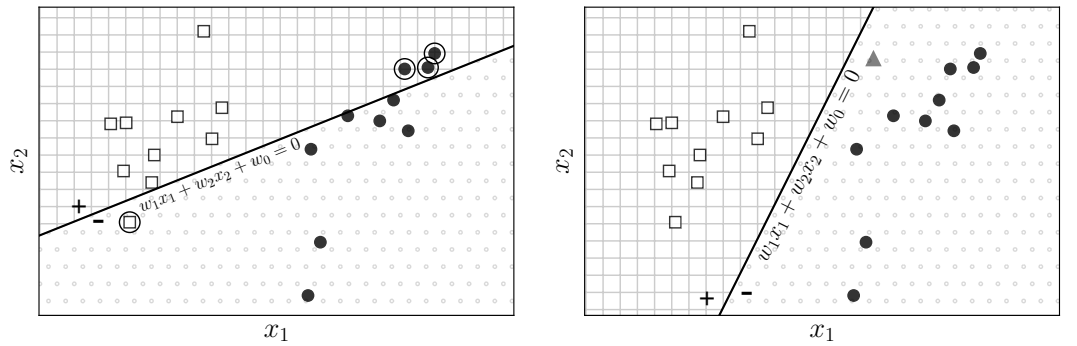
Giả sử $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N] \in \mathbb{R}^{d \times N}$ là ma trận chứa tập huấn luyện mà mỗi cột $\mathbf{x}_i$ là một điểm dữ liệu trong không gian d chiều. Các nhãn được lưu trong một vector hàng $\mathbf{y} = [y_1, y_2, \ldots, y_N] \in \mathbb{R}^{1 \times N}$ với $y_i = 1$ nếu $\mathbf{x}_i$ mang nhãn vuông và $y_i = -1$ nếu $\mathbf{x}_i$ mang nhãn tròn.

Tại một thời điểm, giả sử ranh giới là một siêu phẳng có phương trình

$$f_{\mathbf{w}}(\mathbf{x}) = w_1 x_1 + \cdots + w_d x_d + w_0 = \mathbf{x}^T \mathbf{w} + w_0 = 0 \tag{7.1}$$

với $\mathbf{w} \in \mathbb{R}^d$ là vector trọng số và w_0 là hệ số điều chỉnh. Bằng cách sử dụng thủ thuật gộp hệ số điều chỉnh (xem Mục **??**), ta có thể coi phương trình siêu phẳng là $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^T \mathbf{w} = 0$ với $\mathbf{x}$ ở đây được ngầm hiểu như vector đặc trưng mở rộng thêm một đặc trưng bằng một. Vector trọng số $\mathbf{w}$ chính là *vector pháp tuyến* của siêu phẳng $\mathbf{x}^T \mathbf{w} = 0$.

Trong không gian hai chiều, giả sử đường thẳng $w_1 x_1 + w_2 x_2 + w_0 = 0$ là nghiệm cần tìm như Hình 7.2a. Ta thấy rằng các điểm nằm cùng phía so với đường thẳng này làm cho hàm số $f_{\mathbf{w}}(\mathbf{x})$ mang cùng dấu. Nếu cần thiết, ta có thể đổi dấu của $\mathbf{w}$ để các điểm trên nửa mặt phẳng nền kẻ ô vuông mang dấu dương (+), các



(a) Đường thẳng phân chia không gây lỗi, mọi điểm được phân loại đúng.

(b) Đường thẳng phân chia gây ra lỗi tại các điểm được khoanh tròn.

Hình 7.2. Ví dụ về các đường thẳng trong không gian hai chiều: (a) một nghiệm của bài toán PLA, (b) đường thẳng không phân chia chính xác hai lớp.

điểm trên nửa mặt phẳng nền chấm mang dấu âm (-). Các dấu này tương đương với nhãn y của mỗi điểm dữ liệu. Như vậy, nếu $\mathbf{w}$ là một nghiệm của bài toán thì nhãn của một điểm dữ liệu mới $\mathbf{x}$ được xác định bởi

$$\text{label}(\mathbf{x}) = \begin{cases} 1 & \text{nếu } \mathbf{x}^T\mathbf{w} \geq 0 \\ -1 & \text{o.w.} \end{cases} \tag{7.2}$$

Vậy, $\text{label}(\mathbf{x}) = \text{sgn}(\mathbf{w}^T\mathbf{x})$ với sgn là hàm xác định dấu. Quy ước $\text{sgn}(0) = 1$.

7.2.2. Xây dựng hàm mất mát

Tiếp theo, chúng ta xây dựng một hàm mất mát theo tham số $\mathbf{w}$ bất kỳ. Vẫn trong không gian hai chiều, xét đường thẳng $w_1x_1 + w_2x_2 + w_0 = 0$ được cho như Hình 7.2b. Các điểm khoanh tròn là các điểm bị *phân loại lỗi*. Tham số $\mathbf{w}$ là một nghiệm của bài toán nếu nó không gây ra điểm bị phân loại lỗi nào. Như vậy, hàm đếm số lượng điểm bị phân loại lỗi có thể coi là hàm mất mát của mô hình. Ta sẽ tìm cách tối thiểu hàm số này.

Nếu một điểm $\mathbf{x}_i$ với nhãn y_i bị phân loại lỗi, ta có $\text{sgn}(\mathbf{x}^T\mathbf{w}) \neq y_i$. Vì hai giá trị này chỉ bằng 1 hoặc -1, ta phải có $y_i\text{sgn}(\mathbf{x}_i^T\mathbf{w}) = -1$. Như vậy, hàm đếm số lượng điểm bị phân loại lỗi có thể được viết dưới dạng

$$J_1(\mathbf{w}) = \sum_{\mathbf{x}_i \in \mathcal{M}} (-y_i\text{sgn}(\mathbf{x}_i^T\mathbf{w})) \tag{7.3}$$

trong đó $\mathcal{M}$ ký hiệu tập các điểm bị phân loại lỗi ứng với mỗi $\mathbf{w}$. Mục đích cuối cùng là đi tìm $\mathbf{w}$ sao cho mọi điểm trong tập huấn luyện đều được phân loại đúng, tức $J_1(\mathbf{w}) = 0$. Một điểm quan trọng cần lưu ý là hàm mất mát $J_1(\mathbf{w})$ rất khó được tối ưu vì sgn là một hàm rời rạc. Chúng ta cần tìm một hàm mất mát khác để việc tối ưu khả thi hơn. Xét hàm

$$J(\mathbf{w}) = \sum_{\mathbf{x}_i \in \mathcal{M}} (-y_i \mathbf{x}_i^T \mathbf{w}). \tag{7.4}$$

Trong hàm số này, hàm rời rạc sgn đã được lược bỏ. Ngoài ra, khi một điểm phân loại lỗi $\mathbf{x}_i$ nằm càng xa ranh giới, giá trị $-y_i \mathbf{x}_i^T \mathbf{w}$ sẽ càng lớn, khiến cho hàm mất mát cũng càng lớn. Lưu ý rằng hàm mất mát chỉ được tính trên các tập điểm bị phân loại lỗi $\mathcal{M}$, giá trị nhỏ nhất của hàm số này cũng bằng không nếu $\mathcal{M}$ là một tập rỗng. Vì vậy, $J(\mathbf{w})$ được cho là tốt hơn $J_1(\mathbf{w})$ vì nó *trừng phạt rất nặng những điểm lấn sâu sang lãnh thổ của tập còn lại*. Trong khi đó, $J_1(\mathbf{w})$ *trừng phạt* các điểm phân loại lỗi một lượng như nhau và đều bằng một, bất kể chúng gần hay xa ranh giới.

7.2.3. Tối ưu hàm mất mát

Tại một thời điểm, nếu chỉ quan tâm tới các điểm bị phân loại lỗi thì hàm số $J(\mathbf{w})$ khả vi tại mọi $\mathbf{w}$. Vậy ta có thể sử dụng GD hoặc SGD để tối ưu hàm mất mát này. Chúng ta sẽ giải quyết bài toán tối ưu hàm mất mát $J(\mathbf{w})$ bằng SGD. Nếu chỉ một điểm dữ liệu $\mathbf{x}_i$ bị phân loại lỗi, hàm mất mát và gradient của nó lần lượt là

$$J(\mathbf{w}; \mathbf{x}_i; y_i) = -y_i \mathbf{x}_i^T \mathbf{w}; \quad \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i; y_i) = -y_i \mathbf{x}_i \tag{7.5}$$

Quy tắc cập nhật $\mathbf{w}$ sử dụng SGD là

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(-y_i \mathbf{x}_i) = \mathbf{w} + \eta y_i \mathbf{x}_i \tag{7.6}$$

với η là tốc độ học. Trong PLA, η được chọn bằng 1. Ta có một quy tắc cập nhật rất gọn:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + y_i \mathbf{x}_i \tag{7.7}$$

Tiếp theo, ta thấy rằng

$$\mathbf{x}_i^T \mathbf{w}_{t+1} = \mathbf{x}_i^T (\mathbf{w}_t + y_i \mathbf{x}_i) \ = \mathbf{x}_i^T \mathbf{w}_t + y_i \|\mathbf{x}_i\|_2^2. \tag{7.8}$$

Nếu $\mathbf{x}_i$ bị phân loại lỗi và có nhãn đúng $y_i = 1$, ta có $\mathbf{x}_i^T \mathbf{w}_t < 0$. Cũng vì $y_i = 1$ nên $y_i \|\mathbf{x}_i\|_2^2 = \|\mathbf{x}_i\|_2^2 \geq 1$ (chú ý $\mathbf{x}_i$ là một vector đặc trưng mở rộng với một phần tử bằng một). Từ đó suy ra $\mathbf{x}_i^T \mathbf{w}_{t+1} > \mathbf{x}_i^T \mathbf{w}_t$. Nói cách khác, $-y_i \mathbf{x}_i^T \mathbf{w}_{t+1} < -y_i \mathbf{x}_i^T \mathbf{w}_t$. Điều tương tự cũng xảy ra với $y_i = -1$. Việc này chỉ ra rằng đường thẳng được mô tả bởi $\mathbf{w}_{t+1}$ có xu hướng khiến hàm mất mát tại điểm bị phân loại lỗi $\mathbf{x}_i$ giảm đi. *Chú ý rằng việc này không đảm bảo hàm mất mát tổng cộng sẽ giảm, vì rất có thể siêu thẳng mới sẽ làm cho một điểm lúc trước được phân loại đúng trở thành một điểm bị phân loại lỗi. Tuy nhiên, thuật toán này được đảm bảo sẽ hội tụ sau một số hữu hạn bước.* Thuật toán perceptron được tóm tắt dưới đây:

Thuật toán 7.1: Perceptron

a. Tại thời điểm $t = 0$, chọn ngẫu nhiên một vector trọng số $\mathbf{w}_0$.
b. Tại thời điểm t, nếu không có điểm dữ liệu nào bị phân loại lỗi, dừng thuật toán.
c. Giả sử $\mathbf{x}_i$ là một điểm bị phân loại lỗi, cập nhật

$$\mathbf{w}_{t+1} = \mathbf{w}_t + y_i \mathbf{x}_i$$

d. Thay đổi $t = t + 1$ rồi quay lại Bước 2.

7.2.4. Chứng minh hội tụ

Gọi $\mathbf{w}^*$ là một nghiệm của bài toán phân loại nhị phân. Nghiệm này luôn tồn tại khi hai tập dữ liệu tách biệt tuyến tính. Ta sẽ chứng minh bằng phản chứng Thuật toán 7.1 kết thúc sau một số hữu hạn bước.

Giả sử ngược lại, tồn tại một điểm xuất phát $\mathbf{w}$ khiến Thuật toán 7.1 chạy vô hạn bước. Trước hết ta thấy rằng, nếu $\mathbf{w}^*$ là nghiệm thì $\alpha\mathbf{w}^*$ cũng là nghiệm của bài toán với $\alpha > 0$ bất kỳ. Xét dãy số không âm $u_\alpha(t) = \|\mathbf{w}_t - \alpha\mathbf{w}^*\|_2^2$. Theo giả thiết phản chứng, luôn tồn tại một điểm bị phân loại lỗi khi dùng nghiệm $\mathbf{w}_t$. Giả sử đó là điểm $\mathbf{x}_i$ với nhãn y_i. Ta có

$$\begin{aligned} u_\alpha(t+1) &= \|\mathbf{w}_{t+1} - \alpha\mathbf{w}^*\|_2^2 \\ &= \|\mathbf{w}_t + y_i\mathbf{x}_i - \alpha\mathbf{w}^*\|_2^2 \\ &= \|\mathbf{w}_t - \alpha\mathbf{w}^*\|_2^2 + y_i^2\|\mathbf{x}_i\|_2^2 + 2y_i\mathbf{x}_i^T(\mathbf{w}_t - \alpha\mathbf{w}^*) \\ &< u_\alpha(t) \ + \|\mathbf{x}_i\|_2^2 - 2\alpha y_i\mathbf{x}_i^T\mathbf{w}^* \end{aligned} \tag{7.9}$$

Dấu nhỏ hơn ở dòng cuối xảy ra vì $y_i^2 = 1$ và $2y_i\mathbf{x}_i^T\mathbf{w}_t < 0$. Nếu tiếp tục đặt

$$\beta^2 = \max_{i=1,2,\ldots,N} \|\mathbf{x}_i\|_2^2 \geq 1, \quad \gamma = \min_{i=1,2,\ldots,N} y_i\mathbf{x}_i^T\mathbf{w}^*$$

và chọn $\alpha = \frac{\beta^2}{\gamma}$, ta sẽ có $0 \leq u_\alpha(t+1) < u_\alpha(t) + \beta^2 - 2\alpha\gamma = u_\alpha(t) - \beta^2$. Ta có thể chọn giá trị này vì (7.9) đúng với α bất kỳ. Điều này chỉ ra rằng nếu luôn có điểm bị phân loại lỗi thì dãy $u_\alpha(t)$ là một dãy giảm bị chặn dưới bởi 0, và phần tử sau kém phần tử trước ít nhất một lượng là $\beta^2 \geq 1$. Điều vô lý này chứng tỏ giả thiết phản chứng là sai. Nói cách khác, thuật toán perceptron hội tụ sau một số hữu hạn bước.

7.3. Ví dụ và minh hoạ trên Python

Thuật toán 7.1 có thể được triển khai như sau:

Quy tắc phân loại

Giả sử đã tìm được vector trọng số `w`, nhãn của các điểm dữ liệu `X` được xác định bằng hàm `predict(w, X)`:

```
import numpy as np
def predict(w, X):
    """
    predict label of each row of X, given w
    X: a 2-d numpy array of shape (N, d), each row is a datapoint
    w: a 1-d numpy array of shape (d)
    """
    return np.sign(X.dot(w))
```

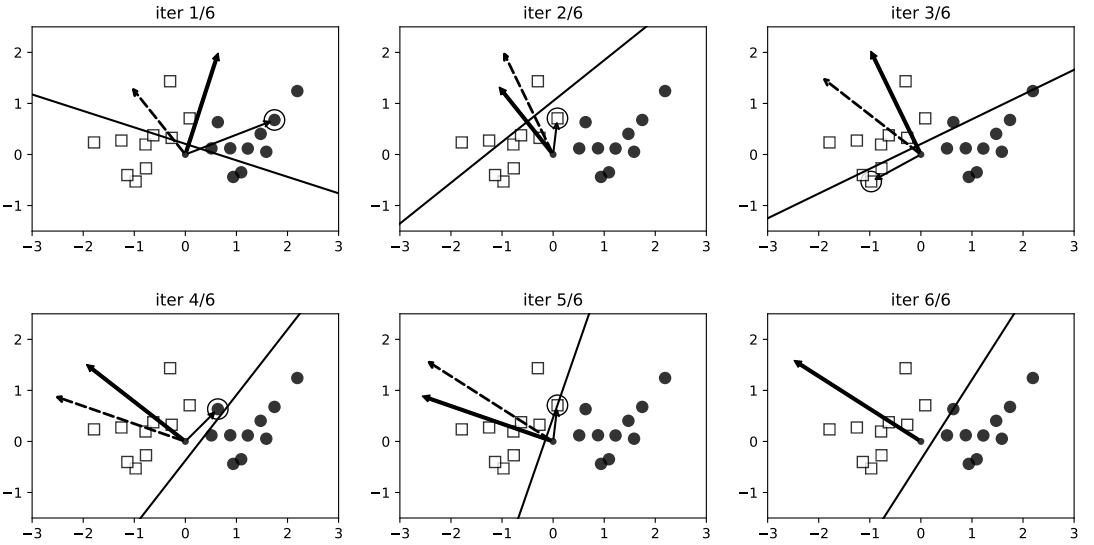
Thuật toán tối ưu hàm mất mát

Hàm `perceptron(X, y, w_init)` thực hiện thuật toán PLA với tập huấn luyện `X`, nhãn `y` và nghiệm ban đầu `w_init`.

```
def perceptron(X, y, w_init):
    """ Thuc hien thuat toan hoc Perceptron (PLA)
    X: ma tran numpy 2 chieu kich thuoc (N, d), moi hang la mot diem du lieu
    y: mang numpy 1 chieu kich thuoc (N), nhan cua tung diem du lieu, y[i] =
     1 hoac -1
    w_init: mang numpy 1 chieu kich thuoc (d), vector trong so ban dau
    """
    w = w_init
    while True:
        pred = predict(w, X)
        # tim cac chi so cua diem du lieu bi phan loai sai
        mis_idxs = np.where(np.equal(pred, y) == False)[0]
        # dem so diem bi phan loai sai
        num_mis = mis_idxs.shape[0]
        if num_mis == 0: # neu khong con diem nao bi phan loai sai
            return w
        # chon ngau nhien mot diem bi phan loai sai
        random_id = np.random.choice(mis_idxs, 1)[0]
        # cap nhat vector trong so
        w = w + y[random_id]*X[random_id]
    return w
```

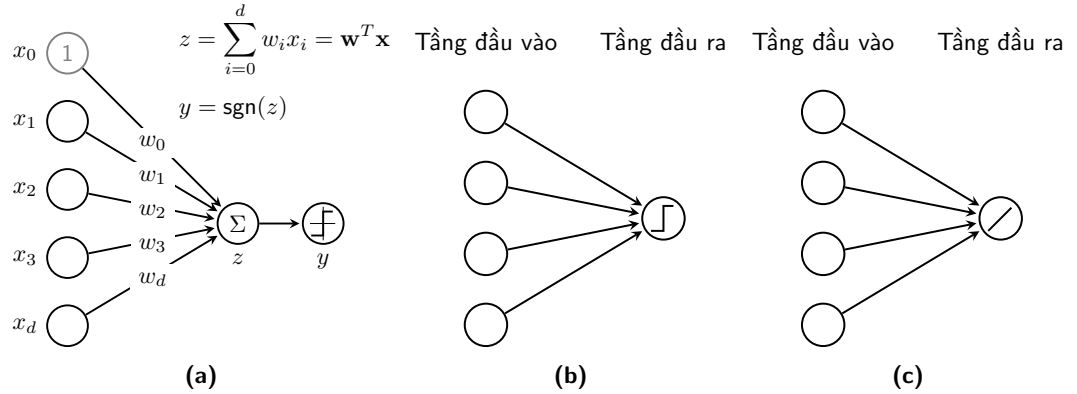
Áp dụng thuật toán vừa viết vào dữ liệu trong không gian hai chiều:

```
means = [[-1, 0], [1, 0]]
cov = [[.3, .2], [.2, .3]]
N = 10
X0 = np.random.multivariate_normal(means[0], cov, N)
X1 = np.random.multivariate_normal(means[1], cov, N)
X = np.concatenate((X0, X1), axis = 0)
y = np.concatenate((np.ones(N), -1*np.ones(N)))
Xbar = np.concatenate((np.ones((2*N, 1)), X), axis = 1)
w_init = np.random.randn(Xbar.shape[1])
w = perceptron(Xbar, y, w_init)
```



Hình 7.3. Minh hoạ thuật toán perceptron. Các điểm hình vuông có nhãn bằng 1, các điểm hình tròn có nhãn -1. Tại mỗi vòng lặp, đường thẳng là đường ranh giới. Vector pháp tuyến $\mathbf{w}_t$ của đường thẳng này là vector đậm nét liền. Điểm được khoanh tròn là một điểm bị phân loại lỗi $\mathbf{x}_i$. Vector mảnh nét liền thể hiện vector $\mathbf{x}_i$. Vector nét đứt thể hiện $\mathbf{w}_{t+1}$. Nếu $y_i = 1$ (một điểm hình vuông), vector nét đứt bằng tổng hai vector kia. Nếu $y_i = -1$, vector nét đứt bằng hiệu hai vector kia.

Mỗi nhãn có 10 phần tử, là các vector ngẫu nhiên lấy theo phân phối chuẩn có ma trận hiệp phương sai `cov` và vector kỳ vọng `means`. Hình 7.3 minh hoạ thuật toán học perceptron cho bài toán này. Nghiệm hội tụ chỉ sau sáu vòng lặp.



Hình 7.4. Biểu diễn perceptron và hồi quy tuyến tính dưới dạng mạng neuron. (a) perceptron đầy đủ, (b) perceptron thu gọn, (c) hồi quy tuyến tính thu gọn.

7.4. Mô hình mạng neuron đầu tiên

Mô hình **Perceptron** được xem là dạng đơn giản nhất của mạng nơ-ron nhân tạo và được lấy cảm hứng từ cấu trúc của một nơ-ron sinh học trong hệ thần kinh. Trong nơ-ron sinh học, các tín hiệu được tiếp nhận thông qua các nhánh *dendrites*, sau đó được tổng hợp tại thân tế bào và truyền đi thông qua sợi trục *axon*. Nếu tổng tín hiệu đầu vào vượt qua một ngưỡng nhất định thì nơ-ron sẽ phát ra tín hiệu đầu ra. Hình 7.5 minh họa sự tương đồng giữa cấu trúc của nơ-ron sinh học và mô hình perceptron trong mạng nơ-ron nhân tạo.

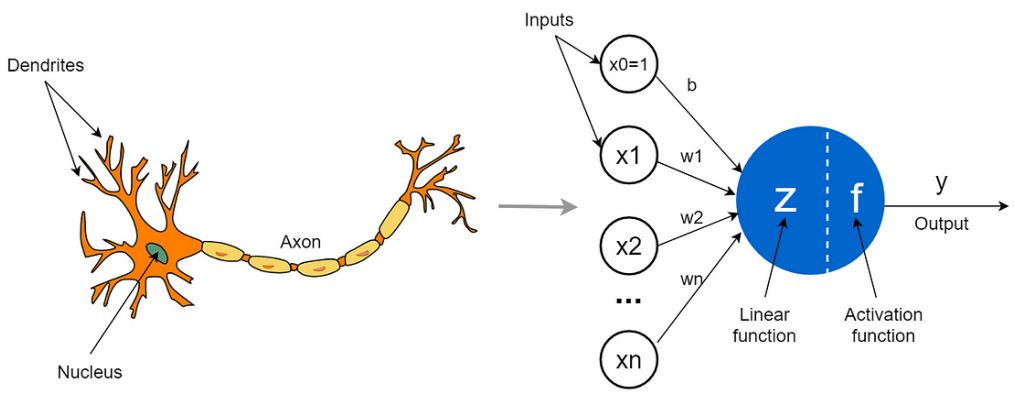
Dựa trên nguyên lý này, perceptron mô phỏng quá trình xử lý tín hiệu của nơ-ron bằng một mô hình toán học. Giả sử vector đầu vào của mô hình là:

$$\mathbf{x} = (x_1, x_2, \ldots, x_n) \tag{7.10}$$

mỗi đặc trưng đầu vào sẽ được gán với một trọng số tương ứng:

$$\mathbf{w} = (w_1, w_2, \ldots, w_n) \tag{7.11}$$

Ngoài ra mô hình còn có một hệ số chệch (bias) ký hiệu là b. Để thuận tiện trong biểu diễn, bias thường được viết dưới dạng một đầu vào cố định $x_0 = 1$ với trọng số tương ứng là b.



Hình 7.5. Mo hinh no-ron nhan tao don gian (Perceptron) duoc lay cam hung tu cau truc cua no-ron sinh hoc

Như minh họa trong Hình 7.5, các tín hiệu đầu vào $x_1, x_2, \ldots, x_n$ được nhân với các trọng số tương ứng $w_1, w_2, \ldots, w_n$ trước khi được tổng hợp tại nút tính toán

trung tâm. Quá trình này tương đương với việc tính tổng có trọng số của các đặc trưng đầu vào.

Perceptron tính tổng có trọng số của các tín hiệu đầu vào thông qua hàm tuyến tính:

$$z = b + \sum_{i=1}^{n} w_i x_i \tag{7.12}$$

Dưới dạng vector, biểu thức trên có thể viết gọn:

$$z = \mathbf{w}^T \mathbf{x} + b \tag{7.13}$$

Giá trị z thể hiện mức kích hoạt của nơ-ron nhân tạo. Giá trị này sau đó được đưa qua một hàm kích hoạt để tạo ra đầu ra của mô hình:

$$y = f(z) \tag{7.14}$$

Trong perceptron cổ điển, hàm kích hoạt thường được sử dụng là hàm dấu:

$$f(z) = \begin{cases} 1, & z \geq 0 \\ -1, & z < 0 \end{cases} \tag{7.15}$$

Do đó đầu ra của perceptron sẽ thuộc một trong hai lớp 1 hoặc -1, tương ứng với một bài toán phân loại nhị phân.

Quá trình học của perceptron được thực hiện bằng cách điều chỉnh vector trọng số khi mô hình phân loại sai một mẫu dữ liệu. Nếu một điểm dữ liệu $(\mathbf{x}_i, y_i)$ bị phân loại sai, vector trọng số sẽ được cập nhật theo quy tắc:

$$\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i \tag{7.16}$$

$$b \leftarrow b + y_i \tag{7.17}$$

Quy tắc cập nhật này giúp điều chỉnh siêu phẳng phân tách giữa hai lớp dữ liệu sao cho các điểm dữ liệu được phân loại chính xác hơn. Quá trình cập nhật được lặp lại cho đến khi không còn điểm dữ liệu nào bị phân loại sai hoặc đạt đến một số vòng lặp nhất định.

7.5. Kết luận

Trong chương này, chúng ta đã tìm hiểu một trong những mô hình học máy cơ bản nhất là **Perceptron Learning Algorithm (PLA)**. Thuật toán perceptron được xây dựng dựa trên ý tưởng mô phỏng hoạt động của nơ-ron sinh học và được xem là nền tảng cho sự phát triển của các mô hình mạng nơ-ron nhân tạo hiện đại.

Trước hết, chương đã trình bày cấu trúc của một nơ-ron nhân tạo, bao gồm các thành phần chính như các đặc trưng đầu vào, vector trọng số, hệ số chệch (bias) và hàm kích hoạt. Dựa trên các thành phần này, perceptron thực hiện việc tính tổng có trọng số của các đầu vào và sử dụng hàm kích hoạt để đưa ra quyết định phân loại.

Tiếp theo, chúng ta đã phân tích cơ chế học của thuật toán PLA. Khi một mẫu dữ liệu bị phân loại sai, vector trọng số của mô hình sẽ được cập nhật theo một quy tắc đơn giản nhằm điều chỉnh siêu phẳng phân tách giữa các lớp dữ liệu. Quá trình này được lặp lại cho đến khi tất cả các mẫu dữ liệu được phân loại chính xác hoặc đạt đến một số vòng lặp nhất định.

Mặc dù perceptron có cấu trúc đơn giản và dễ cài đặt, mô hình này chỉ hoạt động hiệu quả khi dữ liệu có thể được phân tách tuyến tính. Tuy nhiên, perceptron đóng vai trò quan trọng trong lịch sử phát triển của học máy và là nền tảng cho nhiều mô hình mạng nơ-ron phức tạp hơn được sử dụng trong học sâu hiện nay.

7.6. Câu hỏi ôn tập

a. Trình bày ý tưởng cơ bản của thuật toán Perceptron Learning Algorithm (PLA). Thuật toán này được lấy cảm hứng từ đặc điểm nào của nơ-ron sinh học?

b. Cho vector đầu vào

$$\mathbf{x} = (2, 1)$$

và vector trọng số ban đầu

$$\mathbf{w} = (1, -1), \quad b = 0$$

   a. Tính giá trị $z = \mathbf{w}^T\mathbf{x} + b$.

   b. Xác định đầu ra của perceptron sử dụng hàm kích hoạt dấu.

c. Cho tập dữ liệu phân loại nhị phân sau:

x_1	x_2	y
1	1	1
2	1	1
1	−1	−1
−1	−2	−1

Giả sử vector trọng số ban đầu là

$$\mathbf{w} = (0, 0), \quad b = 0$$

Hãy thực hiện một số bước cập nhật của thuật toán PLA cho đến khi tìm được một vector trọng số phân loại đúng các điểm dữ liệu.

d. Giả sử một perceptron có hàm quyết định

$$f(x) = \text{sign}(w_1 x_1 + w_2 x_2 + b)$$

với

$$w_1 = 2, \quad w_2 = -1, \quad b = -1$$

a. Viết phương trình siêu phẳng phân tách của mô hình.

b. Xác định lớp của điểm dữ liệu $x = (1, 2)$.

e. Giải thích khái niệm *linearly separable data*. Tại sao perceptron không thể học được các tập dữ liệu không phân tách tuyến tính? Hãy đưa ra một ví dụ minh họa.

Chương 8

Cây quyết định

Trong các chương trước, chúng ta đã nghiên cứu một số mô hình học máy tuyến tính như hồi quy tuyến tính và thuật toán perceptron. Tuy nhiên, trong nhiều bài toán thực tế, mối quan hệ giữa các đặc trưng đầu vào và nhãn đầu ra thường không tuân theo một cấu trúc tuyến tính đơn giản. Do đó, cần có những mô hình linh hoạt hơn có khả năng biểu diễn các quan hệ phi tuyến trong dữ liệu. Một trong những phương pháp phổ biến và trực quan để giải quyết vấn đề này là **cây quyết định** (Decision Tree).

Cây quyết định là một mô hình học máy có cấu trúc dạng cây, trong đó mỗi nút bên trong biểu diễn một phép kiểm tra trên một thuộc tính của dữ liệu, mỗi nhánh biểu diễn kết quả của phép kiểm tra đó và mỗi nút lá biểu diễn kết quả dự đoán cuối cùng của mô hình. Nhờ cấu trúc phân cấp này, cây quyết định có khả năng chia nhỏ không gian dữ liệu thành các vùng khác nhau và đưa ra quyết định phân loại hoặc dự đoán dựa trên các quy tắc suy luận rõ ràng.

Một trong những ưu điểm nổi bật của cây quyết định là tính trực quan và khả năng giải thích cao. Khác với nhiều mô hình học máy phức tạp khác, các quyết định của mô hình có thể được biểu diễn dưới dạng các luật *if–then* dễ hiểu. Điều này giúp người dùng dễ dàng theo dõi quá trình ra quyết định của mô hình cũng như phân tích vai trò của từng đặc trưng trong dữ liệu.

Trong bài toán phân loại, cây quyết định sẽ lần lượt lựa chọn các đặc trưng quan trọng nhất để phân chia tập dữ liệu thành các nhóm nhỏ hơn sao cho các mẫu trong cùng một nhóm có xu hướng thuộc cùng một lớp. Quá trình phân chia này được thực hiện lặp lại cho đến khi đạt được một tiêu chí dừng nhất định, chẳng hạn như tất cả các mẫu trong một nút đều thuộc cùng một lớp hoặc số lượng mẫu trong nút quá nhỏ để tiếp tục phân chia.

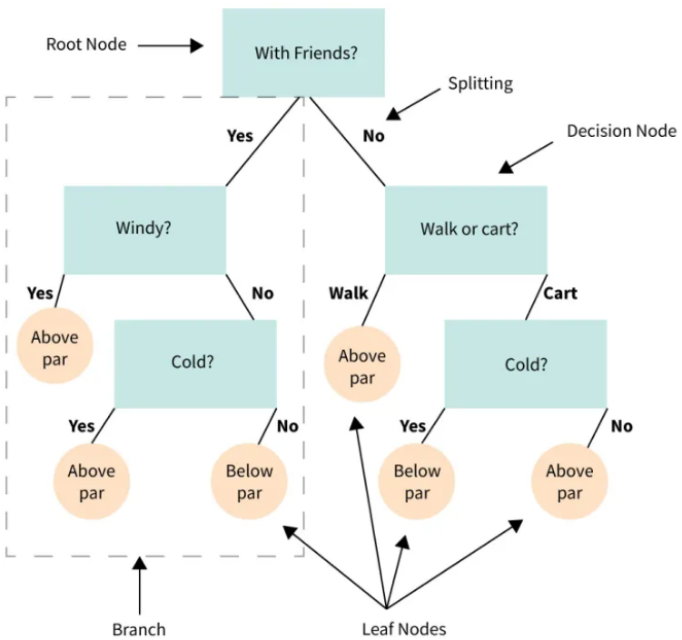
Việc xây dựng cây quyết định thường dựa trên các tiêu chí đo lường mức độ "tốt" của một phép phân chia dữ liệu. Một số tiêu chí phổ biến được sử dụng trong các thuật toán xây dựng cây quyết định bao gồm *Entropy*, *Information Gain* và *Gini Index*. Các tiêu chí này giúp lựa chọn thuộc tính tối ưu để phân tách dữ liệu tại mỗi nút của cây.

Trong chương này, chúng ta sẽ lần lượt tìm hiểu các khái niệm cơ bản của mô hình cây quyết định, cách xây dựng cây từ dữ liệu huấn luyện, các tiêu chí đánh giá phép phân chia dữ liệu cũng như các thuật toán phổ biến như ID3, C4.5 và CART.

8.1. Khái niệm

Cây quyết định (*Decision Tree*) là một mô hình học máy được sử dụng phổ biến trong các bài toán phân loại và hồi quy. Mô hình này biểu diễn quá trình ra quyết định dưới dạng một cấu trúc cây phân cấp, trong đó mỗi nút của cây tương ứng với một phép kiểm tra trên dữ liệu và mỗi nhánh biểu diễn kết quả của phép kiểm tra đó.

Về mặt trực quan, cây quyết định hoạt động giống như quá trình con người đưa ra quyết định dựa trên một chuỗi các câu hỏi. Mỗi câu hỏi sẽ chia tập dữ liệu thành các nhóm nhỏ hơn dựa trên giá trị của một thuộc tính. Quá trình này được lặp lại cho đến khi đạt được kết quả dự đoán cuối cùng.



Hình 8.1. Minh họa cấu trúc của một cây quyết định

Hình 8.1 minh họa cấu trúc cơ bản của một cây quyết định. Mô hình này được tổ chức dưới dạng một cấu trúc cây gồm nhiều nút và các nhánh kết nối giữa chúng. Mỗi nút trong cây tương ứng với một phép kiểm tra trên một thuộc tính của dữ liệu, trong khi mỗi nhánh biểu diễn kết quả của phép kiểm tra đó.

Thành phần đầu tiên của cây quyết định là **nút gốc** (root node). Đây là nút nằm ở vị trí cao nhất của cây và chứa toàn bộ tập dữ liệu ban đầu. Tại nút này, một thuộc tính sẽ được lựa chọn để thực hiện phép phân chia đầu tiên đối với dữ liệu. Trong Hình 8.1, nút gốc thực hiện phép kiểm tra *With Friends?*.

Từ nút gốc, dữ liệu được chia thành các **nhánh** (branches) dựa trên kết quả của phép kiểm tra. Mỗi nhánh tương ứng với một giá trị có thể của thuộc tính. Ví dụ, nếu câu hỏi *With Friends?* có hai giá trị *Yes* và *No*, thì cây sẽ tạo ra hai nhánh tương ứng với hai kết quả này.

Sau khi phân chia, dữ liệu tiếp tục được xử lý tại các **nút trung gian** (internal nodes hoặc decision nodes). Mỗi nút trung gian lại thực hiện một phép kiểm tra trên một thuộc tính khác của dữ liệu. Trong ví dụ trên, các nút trung gian bao gồm các câu hỏi như *Windy?*, *Walk or cart?* hoặc *Cold?*. Quá trình phân chia dữ liệu tiếp tục diễn ra cho đến khi đạt được kết quả dự đoán cuối cùng.

Các **nút lá** (leaf nodes) là các nút nằm ở cuối cây và không còn phát sinh thêm nhánh nào nữa. Mỗi nút lá tương ứng với một kết quả dự đoán của mô hình. Trong Hình 8.1, các nút lá đưa ra kết quả phân loại như *Above par* hoặc *Below par*.

Như vậy, cây quyết định thực hiện việc phân loại dữ liệu bằng cách lặp lại quá trình phân chia tập dữ liệu thành các nhóm nhỏ hơn dựa trên các thuộc tính khác nhau. Mỗi đường đi từ nút gốc đến một nút lá có thể được biểu diễn dưới dạng một tập hợp các luật *if–then*, giúp cho mô hình trở nên dễ hiểu và dễ giải thích.

8.2. Khái niệm về độ vẩn đục và độ tinh khiết

Trong quá trình xây dựng cây quyết định, một vấn đề quan trọng cần giải quyết là lựa chọn thuộc tính nào để thực hiện phép phân chia dữ liệu tại mỗi nút của cây. Nếu lựa chọn không phù hợp, cây quyết định có thể trở nên phức tạp nhưng lại không mang lại khả năng phân loại tốt. Do đó, cần có một tiêu chí để đánh giá mức độ "tốt" của mỗi phép phân chia dữ liệu.

Ý tưởng cơ bản của cây quyết định là chia tập dữ liệu ban đầu thành các tập con sao cho các mẫu dữ liệu trong cùng một nút có xu hướng thuộc cùng một lớp. Nói cách khác, sau mỗi lần phân chia, ta mong muốn các nhóm dữ liệu trở nên "đồng nhất" hơn về mặt nhãn lớp. Để đánh giá mức độ đồng nhất này, hai

khái niệm quan trọng được sử dụng là **độ tinh khiết** (purity) và **độ vẩn đục** (impurity).

Độ tinh khiết phản ánh mức độ mà các mẫu dữ liệu trong một nút thuộc về cùng một lớp. Một nút được xem là hoàn toàn tinh khiết khi tất cả các mẫu dữ liệu trong nút đều có cùng nhãn lớp. Khi đó, việc phân loại tại nút này trở nên rất rõ ràng và không cần tiếp tục phân chia thêm.

Ngược lại, **độ vẩn đục** phản ánh mức độ hỗn hợp của các lớp dữ liệu trong một nút. Nếu trong cùng một nút tồn tại nhiều mẫu thuộc các lớp khác nhau, nút đó được xem là có độ vẩn đục cao. Trong trường hợp này, cây quyết định cần tiếp tục thực hiện các phép phân chia để giảm sự hỗn hợp giữa các lớp dữ liệu.

Mục tiêu của thuật toán xây dựng cây quyết định là lựa chọn các phép phân chia sao cho độ vẩn đục của dữ liệu sau khi phân chia được giảm xuống nhiều nhất, hay tương đương với việc làm tăng độ tinh khiết của các nút con. Nói cách khác, mỗi bước phân chia của cây quyết định đều hướng tới việc tạo ra các nhóm dữ liệu ngày càng đồng nhất hơn.

Trong thực tế, để đo lường độ vẩn đục của một nút, các thuật toán cây quyết định thường sử dụng một số thước đo toán học như *Entropy* hoặc *Gini Index*. Các thước đo này sẽ được trình bày chi tiết trong các phần tiếp theo của chương.

8.2.1. Entropy

Entropy là một thước đo được sử dụng để đánh giá mức độ vẩn đục của dữ liệu tại một nút trong cây quyết định. Khái niệm này được vay mượn từ lý thuyết thông tin (*Information Theory*) và phản ánh mức độ không chắc chắn của một biến ngẫu nhiên.

Trong bối cảnh cây quyết định, entropy đo lường mức độ hỗn hợp của các lớp dữ liệu trong một tập dữ liệu. Nếu tất cả các mẫu dữ liệu thuộc cùng một lớp thì tập dữ liệu được xem là hoàn toàn tinh khiết và entropy đạt giá trị nhỏ nhất. Ngược lại, nếu các mẫu dữ liệu được phân bố đồng đều giữa các lớp thì entropy sẽ đạt giá trị lớn nhất.

Giả sử tại một nút của cây quyết định có K lớp dữ liệu khác nhau và xác suất để một mẫu thuộc lớp i là p_i. Khi đó entropy của nút được tính theo công thức:

$$Entropy(S) = -\sum_{i=1}^{K} p_i \log_2 p_i \tag{8.1}$$

Trong đó:

- S là tập dữ liệu tại nút đang xét
- p_i là xác suất để một mẫu dữ liệu thuộc lớp i
- K là số lớp trong bài toán phân loại

Ví dụ minh họa

Giả sử tại một nút của cây quyết định có 10 mẫu dữ liệu, trong đó:

- 6 mẫu thuộc lớp A
- 4 mẫu thuộc lớp B

Khi đó ta có:

$$p_A = \frac{6}{10} = 0.6, \qquad p_B = \frac{4}{10} = 0.4$$

Entropy của nút được tính như sau:

$$Entropy(S) = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4) \tag{8.2}$$

Kết quả:

$$Entropy(S) \approx 0.971$$

Giá trị entropy khá lớn cho thấy dữ liệu tại nút này vẫn còn khá hỗn hợp và cần tiếp tục được phân chia.

Công thức Entropy cần ghi nhớ

Để tính Entropy của một nút trong cây quyết định, trước hết cần xác định số lớp dữ liệu và tính xác suất p_i của từng lớp trong tập dữ liệu. Sau đó thay các giá trị này vào công thức

$$Entropy(S) = -\sum_{i=1}^{K} p_i \log_2 p_i$$

trong đó p_i là xác suất của lớp thứ i. Giá trị Entropy càng nhỏ thì dữ liệu tại nút càng tinh khiết; nếu tất cả các mẫu thuộc cùng một lớp thì Entropy $= 0$.

Information Gain - IG

Entropy đo lường mức độ vẩn đục của một tập dữ liệu. Trong khi đó, Information Gain (IG) đo lường mức độ giảm của entropy sau khi dữ liệu được phân chia theo một thuộc tính. Vì vậy, IG thực chất được tính dựa trên sự chênh lệch giữa entropy của tập dữ liệu ban đầu và entropy trung bình của các tập con sau khi phân chia. Information Gain đo lường mức độ giảm của entropy sau khi tập dữ liệu được phân chia theo một thuộc tính A. Công thức được xác định như sau:

$$IG(S, A) = Entropy(S) - \sum_{i=1}^{K} \frac{|S_i|}{|S|} Entropy(S_i)$$

Trong đó S là tập dữ liệu ban đầu và S_i là các tập con thu được sau khi phân chia theo thuộc tính A. Thuộc tính có Information Gain lớn nhất sẽ được chọn để thực hiện phép phân chia trong thuật toán ID3.

8.2.2. Gini Index

Gini Index là một thước đo khác được sử dụng để đánh giá độ vẩn đục của dữ liệu trong cây quyết định. Chỉ số này thường được sử dụng trong thuật toán CART (*Classification and Regression Trees*). Tương tự như entropy, Gini Index cũng phản ánh mức độ hỗn hợp của các lớp dữ liệu trong một nút.

Nếu tất cả các mẫu trong nút thuộc cùng một lớp thì Gini Index bằng 0, tức là nút hoàn toàn tinh khiết. Ngược lại, nếu các lớp dữ liệu phân bố đồng đều thì Gini Index sẽ có giá trị lớn hơn.

Công thức tính Gini Index của một nút được xác định như sau:

$$Gini(S) = 1 - \sum_{i=1}^{K} p_i^2 \tag{8.3}$$

Trong đó:

- S là tập dữ liệu tại nút
- p_i là xác suất để một mẫu dữ liệu thuộc lớp i
- K là số lớp trong tập dữ liệu

Ví dụ minh họa

Xét cùng tập dữ liệu ở ví dụ trước với 10 mẫu:

- 6 mẫu thuộc lớp A
- 4 mẫu thuộc lớp B

Ta có:

$$p_A = 0.6, \qquad p_B = 0.4$$

Gini Index được tính như sau:

$$Gini(S) = 1 - (0.6^2 + 0.4^2) \tag{8.4}$$

$$Gini(S) = 1 - (0.36 + 0.16) = 0.48$$

Giá trị này cho thấy tập dữ liệu vẫn chưa hoàn toàn tinh khiết và có thể tiếp tục được phân chia để cải thiện khả năng phân loại của mô hình.

Công thức Gini Index cần ghi nhớ

Chỉ số Gini được sử dụng để đo độ vẩn đục của một nút trong cây quyết định và được tính theo công thức:

$$Gini(S) = 1 - \sum_{i=1}^{K} p_i^2$$

trong đó p_i là xác suất để một mẫu dữ liệu thuộc lớp thứ i. Giá trị Gini càng nhỏ thì dữ liệu trong nút càng tinh khiết. Nếu tất cả các mẫu thuộc cùng một lớp thì $Gini = 0$.

8.3. Các thuật toán xây dựng cây quyết định

Để xây dựng cây quyết định từ dữ liệu huấn luyện, nhiều thuật toán đã được đề xuất trong lĩnh vực học máy. Các thuật toán này có cùng mục tiêu là lựa chọn thuộc tính tốt nhất để phân chia dữ liệu tại mỗi nút của cây, nhưng chúng khác nhau ở tiêu chí đánh giá phép phân chia cũng như cấu trúc của cây được tạo ra.

Hai thuật toán phổ biến nhất trong xây dựng cây quyết định là **ID3** (*Iterative Dichotomiser 3*) và **CART** (*Classification and Regression Trees*). Phần dưới đây sẽ trình bày nguyên lý hoạt động và các bước của hai thuật toán này.

8.3.1. Thuật toán ID3

Thuật toán ID3 được đề xuất bởi Ross Quinlan và là một trong những thuật toán cây quyết định đầu tiên được sử dụng rộng rãi trong các bài toán phân loại. ID3 sử dụng **Information Gain** làm tiêu chí để lựa chọn thuộc tính tốt nhất cho mỗi lần phân chia dữ liệu.

Ý tưởng chính của ID3 là lựa chọn thuộc tính giúp giảm entropy của tập dữ liệu nhiều nhất. Thuộc tính có Information Gain lớn nhất sẽ được chọn để tạo nút phân chia tiếp theo trong cây.

Các bước của thuật toán ID3

a. Bắt đầu với toàn bộ tập dữ liệu tại nút gốc.

b. Nếu tất cả các mẫu dữ liệu thuộc cùng một lớp thì tạo một nút lá với nhãn lớp đó.

c. Nếu không còn thuộc tính nào để phân chia thì tạo một nút lá với lớp xuất hiện nhiều nhất.

d. Với mỗi thuộc tính, tính *Information Gain*.

e. Chọn thuộc tính có *Information Gain* lớn nhất để phân chia dữ liệu.

f. Tạo các nhánh tương ứng với các giá trị của thuộc tính đã chọn.

g. Lặp lại quá trình trên cho từng tập dữ liệu con cho đến khi thỏa mãn điều kiện dừng.

Pseudo-code của thuật toán ID3

```
ID3(S, A)
Input:
    S - tap du lieu huan luyen
    A - tap cac thuoc tinh

Output:
    Cay quyet dinh

1. Neu tat ca cac mau trong S thuoc cung mot lop
       Tra ve mot nut la voi nhan lop do

2. Neu tap thuoc tinh A rong
       Tra ve nut la voi lop xuat hien nhieu nhat

3. Chon thuoc tinh a trong A co Information Gain lon nhat

4. Tao mot nut quyet dinh voi thuoc tinh a
```

```
5. Voi moi gia tri v cua thuoc tinh a:
       Tao nhanh tuong ung
       Tao tap du lieu con S_v

       Neu S_v rong
            Gan nut la voi lop pho bien nhat
       Nguoc lai
            Goi de quy ID3(S_v, A - {a})

6. Tra ve cay quyet dinh
```

8.3.2. Thuật toán CART

Thuật toán CART được đề xuất bởi Breiman và cộng sự, có thể được sử dụng cho cả bài toán phân loại và hồi quy. Trong bài toán phân loại, CART sử dụng **Gini Index** làm tiêu chí để lựa chọn phép phân chia dữ liệu.

Một đặc điểm quan trọng của CART là cây quyết định được xây dựng dưới dạng **cây nhị phân**, nghĩa là mỗi nút chỉ tạo ra hai nhánh con.

Các bước của thuật toán CART

a. Bắt đầu với toàn bộ tập dữ liệu tại nút gốc.

b. Với mỗi thuộc tính, tìm điểm phân chia tốt nhất.

c. Tính giá trị *Gini Index* cho từng phép phân chia.

d. Chọn phép phân chia làm giảm Gini Index nhiều nhất.

e. Chia dữ liệu thành hai tập con theo phép phân chia đã chọn.

f. Lặp lại quá trình trên cho từng nút con cho đến khi đạt điều kiện dừng.

Pseudo-code của thuật toán CART

```
CART(S)
Input:
    S - tap du lieu huan luyen

Output:
    Cay quyet dinh

1. Neu tat ca cac mau trong S thuoc cung mot lop
       Tra ve nut la

2. Tim phep chia du lieu tot nhat dua tren Gini Index

3. Chia du lieu thanh hai tap con:
```

```
       S_left va S_right

4. Tao nut quyet dinh voi phep chia da chon

5. Goi de quy:
       Left_subtree = CART(S_left)
       Right_subtree = CART(S_right)

6. Gan hai cay con vao nut hien tai

7. Tra ve cay quyet dinh
```

8.3.3. Điều kiện dừng trong quá trình xây dựng cây quyết định

Trong quá trình xây dựng cây quyết định, nếu thuật toán tiếp tục phân chia dữ liệu cho đến khi mỗi nút chỉ chứa một mẫu dữ liệu thì cây có thể trở nên rất sâu và phức tạp. Khi đó, mô hình có thể ghi nhớ quá chi tiết dữ liệu huấn luyện và dẫn đến hiện tượng *quá khớp* (*overfitting*). Điều này làm giảm khả năng tổng quát hóa của mô hình khi áp dụng cho dữ liệu mới.

Để hạn chế hiện tượng này, các thuật toán xây dựng cây quyết định thường áp dụng một số **điều kiện dừng** trong quá trình phát triển cây. Một số điều kiện dừng phổ biến bao gồm:

- Tất cả các mẫu dữ liệu trong một nút thuộc cùng một lớp.
- Số lượng mẫu dữ liệu trong nút nhỏ hơn một ngưỡng xác định trước.
- Không còn thuộc tính nào để tiếp tục phân chia dữ liệu.
- Độ sâu của cây đã đạt đến một giá trị tối đa.
- Giá trị giảm của độ vẩn đục (Entropy hoặc Gini) sau khi phân chia là rất nhỏ.

Việc thiết lập các điều kiện dừng hợp lý giúp hạn chế sự phát triển quá mức của cây quyết định và góp phần giảm hiện tượng quá khớp.

8.3.4. Phương pháp cắt tỉa cây quyết định

Một phương pháp phổ biến để giảm hiện tượng quá khớp trong cây quyết định là **cắt tỉa cây** (*pruning*). Ý tưởng cơ bản của phương pháp này là ban đầu xây dựng một cây quyết định tương đối đầy đủ trên tập dữ liệu huấn luyện (*training dataset*), sao cho mô hình có thể phân loại chính xác phần lớn các mẫu dữ liệu. Sau đó, các nhánh không cần thiết ở phần dưới của cây sẽ được loại bỏ dần nhằm đơn giản hóa mô hình.

Cụ thể, các nút lá ở mức thấp của cây có thể bị loại bỏ và nút cha trực tiếp của chúng sẽ được chuyển thành một nút lá mới. Việc cắt tỉa chỉ được thực hiện khi nó giúp cải thiện hoặc ít nhất không làm giảm độ chính xác của mô hình trên tập dữ liệu kiểm định (*validation dataset*). Quá trình này sẽ tiếp tục cho đến khi độ chính xác trên tập kiểm định không còn được cải thiện. Phương pháp này thường được gọi là **giảm thiểu sai số cắt tỉa** (*reduced error pruning*).

Ngoài ra, một phương pháp khác nhằm kiểm soát độ phức tạp của cây quyết định là sử dụng một hàm chi phí có thêm thành phần phạt đối với số lượng nút lá của cây. Giả sử cây quyết định cuối cùng phân chia tập dữ liệu thành K tập con tương ứng với các nút lá $S_1, S_2, \ldots, S_K$. Khi đó hàm chi phí của cây được xác định như sau:

$$L(\text{final_tree}) = \sum_{i=1}^{K} \frac{|S_i|}{N} H(S_i) + \lambda K \tag{8.5}$$

Trong đó:

- N là tổng số mẫu dữ liệu trong tập huấn luyện,
- $|S_i|$ là số lượng mẫu dữ liệu trong nút lá S_i,
- $H(S_i)$ là entropy của tập dữ liệu tại nút lá S_i,
- K là số lượng nút lá của cây,
- λ là tham số điều chỉnh mức độ phạt đối với số lượng nút lá.

Tham số λ thường được chọn là một giá trị dương tương đối nhỏ để kiểm soát độ phức tạp của mô hình. Giá trị λ càng lớn thì thành phần phạt đối với số lượng nút lá càng lớn, từ đó khuyến khích việc xây dựng các cây quyết định đơn giản hơn.

Trong thực tế, quá trình cắt tỉa thường bắt đầu từ một cây tương đối lớn có số lượng nút lá K khá nhiều. Sau đó, các nhánh của cây sẽ được loại bỏ dần sao cho mỗi lần cắt tỉa làm giảm giá trị của hàm chi phí nhiều nhất. Quá trình này tiếp tục cho đến khi việc cắt tỉa không còn giúp làm giảm hàm chi phí của mô hình.

8.4. Ví dụ xây dựng cây quyết định bằng Entropy và Information Gain

Xét tập dữ liệu trong Bảng 8.1 mô tả điều kiện thời tiết và quyết định có nên chơi thể thao hay không.

Bảng 8.1: Tập dữ liệu thời tiết

Day	Weather	Temperature	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

Mục tiêu là xây dựng cây quyết định để dự đoán biến *Play* dựa trên các thuộc tính Weather, Temperature, Humidity và Wind.

Bước 1: Tính Entropy của tập dữ liệu ban đầu

Trong tập dữ liệu có:

$$Yes = 5, \qquad No = 5$$

Xác suất:

$$p_{Yes} = \frac{5}{10} = 0.5$$

$$p_{No} = \frac{5}{10} = 0.5$$

Entropy của tập dữ liệu:

$$Entropy(S) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) \tag{8.6}$$

$$Entropy(S) = 1$$

Bước 2: Tính Information Gain cho thuộc tính Weather

Weather có 3 giá trị: Sunny, Cloudy, Rainy.

Sunny

3 mẫu:

Yes = 1, No = 2

$$Entropy(Sunny) = -\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3}$$

$$Entropy(Sunny) \approx 0.918$$

Cloudy

3 mẫu:

Yes = 3, No = 0

$$Entropy(Cloudy) = 0$$

Rainy

4 mẫu:

Yes = 1, No = 3

$$Entropy(Rainy) = -\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4}$$

$$Entropy(Rainy) \approx 0.811$$

Entropy sau khi phân chia theo Weather:

$$Entropy_{Weather} = \frac{3}{10}(0.918) + \frac{3}{10}(0) + \frac{4}{10}(0.811)$$

$$Entropy_{Weather} \approx 0.600$$

Information Gain:

$$IG(S, Weather) = 1 - 0.600 = 0.400 \tag{8.7}$$

Bước 3: Chọn thuộc tính làm nút gốc

Ta tính Information Gain cho các thuộc tính khác (Temperature, Humidity, Wind). Giả sử kết quả thu được:

$$IG(Weather) = 0.40$$

$$IG(Humidity) = 0.29$$

$$IG(Wind) = 0.05$$

$$IG(Temperature) = 0.02$$

Thuộc tính **Weather** có Information Gain lớn nhất nên được chọn làm **nút gốc** của cây quyết định.

Bước 4: Tiếp tục xây dựng các nhánh

Sau khi chọn Weather làm nút gốc:

- Nếu Weather = Cloudy → Play = Yes
- Nếu Weather = Sunny → tiếp tục phân chia theo thuộc tính khác
- Nếu Weather = Rainy → tiếp tục phân chia theo thuộc tính khác

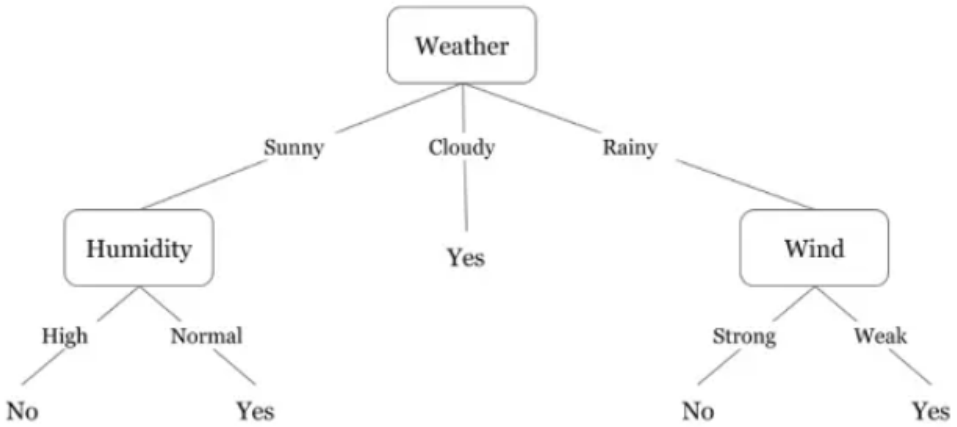
Quá trình tính Entropy và Information Gain tiếp tục được thực hiện trên từng tập con cho đến khi các nút trở nên tinh khiết.

Kết quả

Sau khi thực hiện các bước tính toán Entropy và Information Gain cho các thuộc tính trong tập dữ liệu, ta thu được cây quyết định hoàn chỉnh như minh họa trong Hình 8.2. Cây này cho thấy thuộc tính *Weather* được chọn làm nút gốc vì nó mang lại Information Gain lớn nhất. Từ nút gốc, dữ liệu tiếp tục được phân chia theo các thuộc tính *Humidity* và *Wind* cho đến khi các nút trở nên tinh khiết.

Cây quyết định trong Hình 8.2 có thể được sử dụng để dự đoán giá trị của biến *Play*. Mỗi đường đi từ nút gốc đến một nút lá tương ứng với một quy tắc quyết

định dạng *if–then*. Ví dụ, nếu *Weather* = *Sunny* và *Humidity* = *High* thì kết quả dự đoán là *Play* = *No*; ngược lại nếu *Weather* = *Sunny* và *Humidity* = *Normal* thì *Play* = *Yes*.



Hình 8.2. Cây quyết định được xây dựng từ tập dữ liệu thời tiết

8.5. Thực hành: Xây dựng mô hình cây quyết định bằng Python

Trong phần thực hành này, chúng ta sẽ xây dựng mô hình cây quyết định để phân loại dữ liệu bằng thư viện `scikit-learn` trong Python. Cụ thể, mô hình sẽ được huấn luyện trên bộ dữ liệu *Balance Scale* được cung cấp bởi kho dữ liệu UCI Machine Learning Repository.

Mục tiêu của bài thực hành là giúp người học hiểu rõ quy trình xây dựng mô hình học máy, bao gồm các bước: chuẩn bị dữ liệu, huấn luyện mô hình, dự đoán kết quả và đánh giá hiệu năng của mô hình.

8.5.1. Cài đặt thư viện và import các gói cần thiết

Trước hết cần cài đặt thư viện `scikit-learn` và import các thư viện cần thiết để xử lý dữ liệu, xây dựng mô hình và trực quan hóa cây quyết định.

```
pip install -U scikit-learn
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score,
    classification_report
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
```

8.5.2. Nạp bộ dữ liệu

Bộ dữ liệu được tải trực tiếp từ UCI Machine Learning Repository. Sau khi tải dữ liệu, ta hiển thị kích thước của tập dữ liệu và một số dòng đầu tiên để quan sát cấu trúc dữ liệu.

```
def importdata():
    url = "https://archive.ics.uci.edu/static/public/12/data.csv"
    balance_data = pd.read_csv(url, header=0)

    print("Dataset Length:", len(balance_data))
    print("Dataset Shape:", balance_data.shape)
    print("Dataset Head:\n", balance_data.head())

    return balance_data
```

8.5.3. Tách dữ liệu thành thuộc tính và nhãn

Sau khi tải dữ liệu, bước tiếp theo là tách dữ liệu thành hai phần:

- **X**: tập thuộc tính đầu vào
- **Y**: nhãn của dữ liệu

Dữ liệu sau đó được chia thành tập huấn luyện và tập kiểm tra.

```
def splitdataset(balance_data):

    X = balance_data.iloc[:, 1:5].values
    Y = balance_data.iloc[:, 0].values

    X_train, X_test, y_train, y_test = train_test_split(
        X, Y, test_size=0.3, random_state=100)

    return X, Y, X_train, X_test, y_train, y_test
```

8.5.4. Huấn luyện cây quyết định sử dụng Gini Index

Trong bước này, ta huấn luyện mô hình cây quyết định sử dụng tiêu chí *Gini Index* để lựa chọn phép phân chia dữ liệu.

```
def train_using_gini(X_train, y_train):
    clf_gini = DecisionTreeClassifier(
        criterion="gini",
        random_state=100,
        max_depth=3,
        min_samples_leaf=5)
    clf_gini.fit(X_train, y_train)
    return clf_gini
```

8.5.5. Huấn luyện cây quyết định sử dụng Entropy

Tương tự như trên, ta có thể huấn luyện cây quyết định sử dụng tiêu chí *Entropy* (Information Gain).

```
def train_using_entropy(X_train, y_train):

    clf_entropy = DecisionTreeClassifier(
        criterion="entropy",
        random_state=100,
        max_depth=3,
        min_samples_leaf=5)

    clf_entropy.fit(X_train, y_train)

    return clf_entropy
```

8.5.6. Dự đoán kết quả

Sau khi huấn luyện mô hình, ta sử dụng mô hình để dự đoán nhãn của các mẫu dữ liệu trong tập kiểm tra.

```
def prediction(X_test, clf_object):

    y_pred = clf_object.predict(X_test)
    print("Predicted Values:\n", y_pred)

    return y_pred
```

8.5.7. Đánh giá độ chính xác của mô hình

Hiệu năng của mô hình được đánh giá thông qua các chỉ số:

- Confusion Matrix
- Accuracy
- Precision, Recall và F1-score

```
def cal_accuracy(y_test, y_pred):

    print("\nConfusion Matrix:\n",
          confusion_matrix(y_test, y_pred))

    print("\nAccuracy:",
          accuracy_score(y_test, y_pred)*100)

    print("\nClassification Report:\n",
          classification_report(y_test, y_pred))
```

8.5.8. Trực quan hóa cây quyết định

Sau khi huấn luyện, ta có thể trực quan hóa cấu trúc của cây quyết định để quan sát cách mô hình phân chia dữ liệu.

```
def plot_decision_tree(clf_object, feature_names, class_names):

    plt.figure(figsize=(15,10))

    plot_tree(clf_object,
              filled=True,
              feature_names=feature_names,
              class_names=class_names,
              rounded=True)

    plt.show()
```

8.5.9. Chạy chương trình

Cuối cùng, ta thực hiện toàn bộ quy trình: nạp dữ liệu, huấn luyện mô hình, dự đoán và đánh giá kết quả.

```
if __name__ == "__main__":

    data = importdata()

    X, Y, X_train, X_test, y_train, y_test = splitdataset(data)

    print("\n----- Training Using Gini -----")
    clf_gini = train_using_gini(X_train, y_train)
    y_pred_gini = prediction(X_test, clf_gini)
    cal_accuracy(y_test, y_pred_gini)

    print("\n----- Training Using Entropy -----")
    clf_entropy = train_using_entropy(X_train, y_train)
    y_pred_entropy = prediction(X_test, clf_entropy)
    cal_accuracy(y_test, y_pred_entropy)
```

8.5.10. Trực quan hóa các cây quyết định

Sau khi huấn luyện mô hình bằng hai tiêu chí khác nhau (Gini Index và Entropy), chúng ta có thể trực quan hóa cấu trúc của các cây quyết định để quan sát cách mô hình thực hiện việc phân chia dữ liệu.

Việc trực quan hóa này giúp người học hiểu rõ hơn cách các thuộc tính được lựa chọn làm nút quyết định cũng như cách các điều kiện phân chia được hình thành trong cây.

```
# Visualize Decision Trees

plot_decision_tree(clf_gini,
                   ['X1', 'X2', 'X3', 'X4'],
                   ['L', 'B', 'R'])

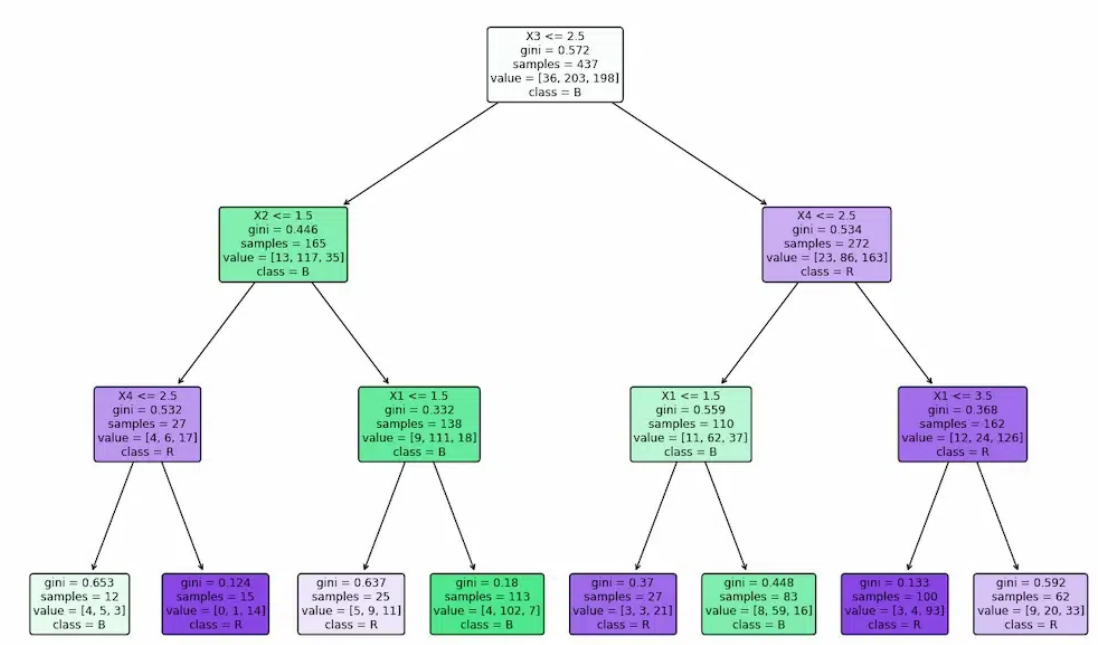
plot_decision_tree(clf_entropy,
                   ['X1', 'X2', 'X3', 'X4'],
                   ['L', 'B', 'R'])
```

Trong đó:

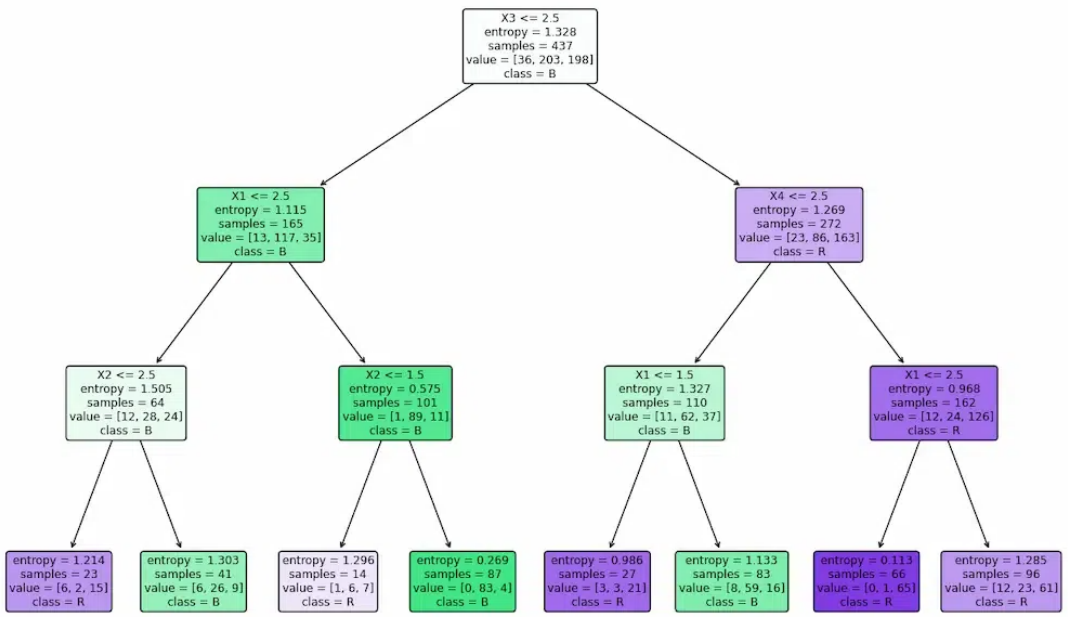
- `['X1', 'X2', 'X3', 'X4']` là tên của các thuộc tính đầu vào.
- `['L', 'B', 'R']` là các nhãn lớp trong bộ dữ liệu Balance Scale.

Kết quả thu được là hai cây quyết định tương ứng với hai tiêu chí phân chia khác nhau. Thông qua việc quan sát hai cây này, chúng ta có thể so sánh sự khác biệt giữa việc sử dụng *Gini Index* và *Entropy* trong quá trình xây dựng mô hình.

8.5.11. Kết quả trực quan hóa cây quyết định



Hình 8.3. Xây dựng cây sử dụng chỉ số Gini Index



Hình 8.4. Xây dựng cây sử dụng chỉ số Entropy

Kiến thức cần ghi nhớ

Decision Tree sử dụng các thước đo như Entropy hoặc Gini Index để đánh giá độ vẩn đục của dữ liệu. Thuộc tính được chọn để phân chia tại mỗi nút là thuộc tính làm giảm độ vẩn đục nhiều nhất.

8.6. Kết luận

Trong chương này, chúng ta đã nghiên cứu mô hình **cây quyết định** (Decision Tree), một trong những phương pháp học máy phổ biến và dễ diễn giải nhất trong các bài toán phân loại và hồi quy. Khác với các mô hình tuyến tính, cây quyết định xây dựng một cấu trúc dạng cây trong đó mỗi nút tương ứng với một phép kiểm tra trên một thuộc tính của dữ liệu và mỗi nút lá biểu diễn kết quả dự đoán cuối cùng.

Chương đã trình bày các khái niệm cơ bản liên quan đến cây quyết định, bao gồm cấu trúc của cây, các thành phần như nút gốc, nút trung gian và nút lá. Đồng thời, chúng ta cũng tìm hiểu các khái niệm quan trọng như *độ vẩn đục* và *độ tinh khiết* của dữ liệu, là cơ sở để đánh giá chất lượng của các phép phân chia dữ liệu trong quá trình xây dựng cây.

Tiếp theo, các thước đo phổ biến dùng để đánh giá mức độ vẩn đục của dữ liệu đã được trình bày, bao gồm **Entropy** và **Gini Index**. Từ đó, khái niệm

Information Gain được giới thiệu như một tiêu chí quan trọng giúp lựa chọn thuộc tính tối ưu để thực hiện phép phân chia dữ liệu trong thuật toán ID3.

Chương cũng đã trình bày hai thuật toán xây dựng cây quyết định phổ biến là **ID3** và **CART**. Bên cạnh đó, các vấn đề thực tế khi xây dựng cây quyết định như hiện tượng *quá khớp* và các phương pháp *cắt tỉa cây* (pruning) cũng được thảo luận nhằm cải thiện khả năng tổng quát hóa của mô hình.

Cuối cùng, phần thực hành đã minh họa cách xây dựng và đánh giá mô hình cây quyết định bằng Python, giúp người học hiểu rõ hơn quy trình áp dụng các phương pháp học máy vào bài toán thực tế.

8.7. Câu hỏi ôn tập

a. Trình bày cấu trúc của một cây quyết định. Phân biệt các thành phần nút gốc, nút trung gian và nút lá.

b. Giải thích khái niệm *độ vẩn đục* và *độ tinh khiết* trong cây quyết định. Tại sao các khái niệm này lại quan trọng trong quá trình xây dựng cây?

c. Trình bày công thức và ý nghĩa của thước đo **Entropy**. Trong trường hợp nào entropy đạt giá trị nhỏ nhất và lớn nhất?

d. Giải thích khái niệm **Information Gain**. Information Gain được sử dụng như thế nào trong thuật toán ID3?

e. So sánh hai tiêu chí **Entropy** và **Gini Index**. Nêu điểm giống và khác nhau giữa hai thước đo này.

f. **Bài tập thực hành**

Xét bộ dữ liệu giả định sau về quyết định đi dã ngoại của một nhóm sinh viên:

Weather	Wind	Humidity	Picnic
Sunny	Weak	High	No
Sunny	Strong	High	No
Cloudy	Weak	High	Yes
Rainy	Weak	Normal	Yes
Rainy	Strong	Normal	No
Cloudy	Strong	High	Yes

Hãy thực hiện các yêu cầu sau:

- Tính entropy của tập dữ liệu ban đầu.

- Tính Information Gain của các thuộc tính *Weather*, *Wind* và *Humidity*.
- Xác định thuộc tính nào nên được chọn làm nút gốc của cây quyết định.
- Vẽ cây quyết định tương ứng với kết quả thu được.

Chương 9

Hồi quy logistic

9.1. Giới thiệu

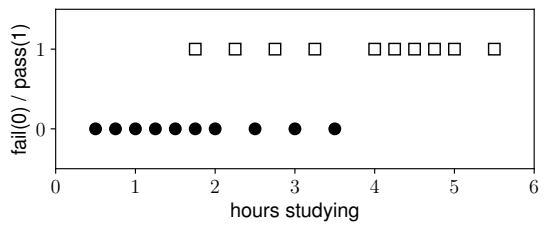
9.1.1. Nhắc lại hai mô hình tuyến tính

Hai mô hình tuyến tính đã thảo luận trong cuốn sách này, hồi quy tuyến tính và PLA, đều có thể viết chung dưới dạng $y = f(\mathbf{x}^T\mathbf{w})$ trong đó $f(s)$ là một hàm kích hoạt. Trong hồi quy tuyến tính $f(s) = s$, tích vô hướng $\mathbf{x}^T\mathbf{w}$ được trực tiếp sử dụng để dự đoán đầu ra y. Mô hình này phù hợp nếu ta cần dự đoán một đầu ra không bị chặn. PLA có đầu ra chỉ nhận một trong hai giá trị 1 hoặc -1 với hàm kích hoạt $f(s) = \text{sgn}(s)$ phù hợp với các bài toán phân loại nhị phân. Trong chương này, chúng ta sẽ thảo luận một mô hình tuyến tính với một hàm kích hoạt khác, thường được áp dụng cho các bài toán phân loại nhị phân. Trong mô hình này, đầu ra có thể được biểu diễn dưới dạng xác suất. Ví dụ, xác suất thi đỗ nếu biết thời gian ôn thi, xác suất ngày mai có mưa dựa trên những thông tin đo được trong ngày hôm nay,... Mô hình này có tên là *hồi quy logistic*. Mặc dù trong tên có chứa từ *hồi quy*, phương pháp này thường được sử dụng nhiều hơn cho các bài toán phân loại.

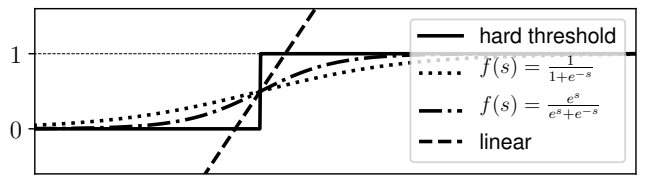
9.1.2. Một ví dụ nhỏ

Bảng 9.1: Thời gian ôn thi và kết quả thi của 20 sinh viên.

Số giờ	**Đậu?**	**Số giờ**	**Đậu?**	**Số giờ**	**Đậu?**	**Số giờ**	**Đậu?**
0.5	0	0.75	0	1	0	1.25	0
1.5	0	1.75	0	1.75	1	2	0
2.25	1	2.5	0	2.75	1	4	0
3.25	1	3.5	0	4	1	4.25	1
4.5	1	4.75	1	5	1	5.5	1



Hình 9.1. Ví dụ về kết quả thi dựa trên số giờ ôn tập. Trục hoành thể hiện thời gian ôn tập của mỗi sinh viên, trục tung gồm hai giá trị 0/fail (các điểm hình tròn) và 1/pass (các điểm hình vuông).



Hình 9.2. Một vài ví dụ về các hàm kích hoạt khác nhau.

Xét một ví dụ về quan hệ giữa thời gian ôn thi và kết quả của 20 sinh viên trong Bảng 9.1. Bài toán đặt ra là từ dữ liệu này hãy xây dựng mô hình đánh giá khả năng đỗ của một sinh viên dựa trên thời gian ôn tập. Dữ liệu trong Bảng 9.1 được mô tả trên Hình 9.1. Nhìn chung, thời gian học càng nhiều thì khả năng đỗ càng cao. Tuy nhiên, không có một ngưỡng thời gian học nào giúp phân biệt rạch ròi việc đỗ/trượt . Nói cách khác, dữ liệu của hai tập này là không tách biệt tuyến tính, và vì vậy PLA sẽ không hữu ích. Tuy nhiên, thay vì dự đoán chính xác hai giá trị đỗ/trượt, ta có thể dự đoán xác suất để một sinh viên thi đỗ dựa trên thời gian ôn thi.

9.1.3. Mô hình hồi quy logistic

Quan sát Hình 9.2 với các hàm kích hoạt $f(s)$ khác nhau.

- Đường nét đứt biểu diễn một hàm kích hoạt tuyến tính không phù hợp vì đầu ra không bị chặn. Có một cách đơn giản để đưa đầu ra về dạng bị chặn: nếu đầu ra nhỏ hơn không thì thay bằng không, nếu đầu ra lớn hơn một thì thay bằng một. Điểm phân chia, còn gọi là *ngưỡng*, được chọn là điểm có tung độ 0.5 trên đường thẳng này. Đây cũng không phải là một lựa chọn tốt. Giả sử có thêm một bạn sinh viên tiêu biểu ôn tập đến 20 giờ hoặc hơn thi đỗ. Lúc này ngưỡng tương ứng với mốc tung độ bằng 0.5 sẽ dịch nhiều về phía phải. Kéo theo đó, rất nhiều sinh viên thi đỗ được dự đoán là trượt. Rõ ràng đây là một mô hình không tốt. Nhắc lại rằng hồi quy tuyến tính rất nhạy cảm với nhiễu, ở đây là bạn sinh viên tiêu biểu đó.

- Đường nét liền tương tự với hàm kích hoạt của PLA. Ngưỡng dự đoán đỗ/trượt tại vị trí hàm số đổi dấu còn được gọi là *ngưỡng cứng*.

- Các đường nét chấm và chấm gạch phù hợp với bài toán đang xét hơn. Chúng có một vài tính chất quan trọng:

- Là các hàm số liên tục nhận giá trị thực, bị chặn trong khoảng $(0, 1)$.
- Nếu coi điểm có tung độ bằng 0.5 là ngưỡng, các điểm càng xa ngưỡng về bên trái có giá trị càng gần không, các điểm càng xa ngưỡng về bên phải có giá trị càng gần một. Điều này phù hợp với nhận xét rằng học càng nhiều thì xác suất đỗ càng cao và ngược lại.
- Hai hàm này có đạo hàm mọi nơi, điều này có thể có ích trong tối ưu.

Hàm sigmoid và tanh

Trong các hàm số có ba tính chất nói trên, hàm *sigmoid*:

$$f(s) = \frac{1}{1 + e^{-s}} \triangleq \sigma(s) \tag{9.1}$$

được sử dụng nhiều nhất, vì nó bị chặn trong khoảng $(0, 1)$ và:

$$\lim_{s \to -\infty} \sigma(s) = 0; \quad \lim_{s \to +\infty} \sigma(s) = 1. \tag{9.2}$$

Thú vị hơn:

$$\sigma'(s) = \frac{e^{-s}}{(1 + e^{-s})^2} = \frac{1}{1 + e^{-s}} \frac{e^{-s}}{1 + e^{-s}} = \sigma(s)(1 - \sigma(s)) \tag{9.3}$$

Với đạo hàm đơn giản, hàm sigmoid được sử dụng rộng rãi trong mạng neuron. Chúng ta sẽ sớm thấy hàm sigmoid được khám phá ra như thế nào.

Ngoài ra, hàm *tanh* cũng hay được sử dụng:

$$\tanh(s) = \frac{e^{s} - e^{-s}}{e^{s} + e^{-s}} = 2\sigma(2s) - 1. \tag{9.4}$$

Hàm số này nhận giá trị trong khoảng $(-1, 1)$.

Hàm sigmoid có thể được thực hiện trên Python như sau:

```
def sigmoid(S):
    """
    S: an numpy array
    return sigmoid function of each element of S
    """
    return 1/(1 + np.exp(-S))
```

9.2. Hàm mất mát và phương pháp tối ưu

9.2.1. Xây dựng hàm mất mát

Với các mô hình có hàm kích hoạt $f(s) \in (0, 1)$, ta có thể giả sử rằng xác suất để một điểm dữ liệu $\mathbf{x}_i$ có nhãn thứ nhất là $f(\mathbf{x}_i^T \mathbf{w})$ và nhãn còn lại là $1 - f(\mathbf{x}_i^T \mathbf{w})$:

$$p(y_i = 1|\mathbf{x}_i; \mathbf{w}) = f(\mathbf{x}_i^T \mathbf{w}) \tag{9.5}$$
$$p(y_i = 0|\mathbf{x}_i; \mathbf{w}) = 1 - f(\mathbf{x}_i^T \mathbf{w}) \tag{9.6}$$

trong đó $p(y_i = 1|\mathbf{x}_i; \mathbf{w})$ được hiểu là xác suất xảy ra sự kiện nhãn $y_i = 1$ khi biết tham số mô hình $\mathbf{w}$ và dữ liệu đầu vào $\mathbf{x}_i$. Mục đích là tìm các hệ số $\mathbf{w}$ sao cho $f(\mathbf{x}_i^T \mathbf{w}) \approx y_i$ với mọi điểm trong tập huấn luyện.

Ký hiệu $a_i = f(\mathbf{x}_i^T \mathbf{w})$, hai biểu thức (9.5) và (9.6) có thể được viết gọn lại:

$$p(y_i|\mathbf{x}_i; \mathbf{w}) = a_i^{y_i}(1 - a_i)^{1-y_i} \tag{9.7}$$

Biểu thức này tương đương với hai biểu thức (9.5) và (9.6) vì khi $y_i = 1$, thừa số thứ hai của vế phải sẽ bằng một, khi $y_i = 0$, thừa số thứ nhất sẽ bằng một. Để mô hình tạo ra dự đoán khớp với dữ liệu đã cho nhất, ta cần tìm $\mathbf{w}$ để xác xuất này đạt giá trị cao nhất.

Xét toàn bộ tập huấn luyện với ma trận dữ liệu $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N] \in \mathbb{R}^{d \times N}$ và vector nhãn tương ứng $\mathbf{y} = [y_1, y_2, \ldots, y_N]$. Ta cần giải bài toán tối ưu

$$\mathbf{w} = \arg\max_{\mathbf{w}} p(\mathbf{y}|\mathbf{X}; \mathbf{w}) \tag{9.8}$$

Đây chính là một bài toán MLE với tham số mô hình $\mathbf{w}$ cần được ước lượng. Ta có thể giải quyết bài toán này bằng cách giả sử các điểm dữ liệu độc lập nếu biết tham số mô hình. Đây cũng là giả sử thường được dùng khi giải các bài toán liên quan tới MLE:

$$p(\mathbf{y}|\mathbf{X}; \mathbf{w}) = \prod_{i=1}^{N} p(y_i|\mathbf{x}_i; \mathbf{w}) = \prod_{i=1}^{N} a_i^{y_i}(1 - a_i)^{1-y_i} \tag{9.9}$$

Lấy logarit tự nhiên, đổi dấu rồi lấy trung bình, ta thu được hàm số

$$J(\mathbf{w}) = -\frac{1}{N} \log p(\mathbf{y}|\mathbf{X}; \mathbf{w}) = -\frac{1}{N} \sum_{i=1}^{N} (y_i \log a_i + (1 - y_i) \log(1 - a_i)) \tag{9.10}$$

với chú ý rằng a_i là một hàm số của $\mathbf{w}$ và $\mathbf{x}_i$. Hàm số này là hàm mất mát của hồi quy logistic. Vì đã đổi dấu sau khi lấy logarit, ta cần tìm $\mathbf{w}$ để $J(\mathbf{w})$ đạt giá trị nhỏ nhất.

9.2.2. Tối ưu hàm mất mát

Bài toán tối ưu hàm mất mát của hồi quy logistic có thể được giải quyết bằng SGD. Tại mỗi vòng lặp, $\mathbf{w}$ được cập nhật dựa trên một điểm dữ liệu ngẫu nhiên. Hàm mất mát của hồi quy logistic với chỉ một điểm dữ liệu $(\mathbf{x}_i, y_i)$ và gradient của nó lần lượt là

$$J(\mathbf{w}; \mathbf{x}_i, y_i) = -(y_i \log a_i + (1 - y_i) \log(1 - a_i)) \tag{9.11}$$

$$\nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i, y_i) = -(\frac{y_i}{a_i} - \frac{1 - y_i}{1 - a_i})(\nabla_{\mathbf{w}} a_i) = \frac{a_i - y_i}{a_i(1 - a_i)}(\nabla_{\mathbf{w}} a_i) \tag{9.12}$$

ở đây ta đã sử dụng quy tắc chuỗi để tính gradient với $a_i = f(\mathbf{x}_i^T \mathbf{w})$. Để cho biểu thức này đơn giản, ta sẽ tìm hàm $a_i = f(\mathbf{x}_i^T \mathbf{w})$ sao cho mẫu số bị triệt tiêu.

Đặt $z = \mathbf{x}_i^T \mathbf{w}$, ta có

$$\nabla_{\mathbf{w}} a_i = \frac{\partial a_i}{\partial z_i}(\nabla_{\mathbf{w}} z_i) = \frac{\partial a_i}{\partial z_i}\mathbf{x}_i \tag{9.13}$$

Tạm thời bỏ qua các chỉ số i, ta đi tìm hàm số $a = f(z)$ sao cho

$$\frac{\partial a}{\partial z} = a(1 - a) \tag{9.14}$$

Nếu điều này xảy ra, mẫu số trong biểu thức (9.12) sẽ bị triệt tiêu. Phương trình vi phân này không quá phức tạp. Thật vậy, (9.14) tương đương với

$$\begin{aligned}
& \frac{\partial a}{a(1 - a)} = \partial z \\
\Leftrightarrow \quad & \left(\frac{1}{a} + \frac{1}{1 - a}\right) \partial a = \partial z \\
\Leftrightarrow \quad & \log a - \log(1 - a) = z + C \\
\Leftrightarrow \quad & \log \frac{a}{1 - a} = z + C \\
\Leftrightarrow \quad & \frac{a}{1 - a} = e^{z + C} \\
\Leftrightarrow \quad & a = e^{z + C}(1 - a) \\
\Leftrightarrow \quad & a = \frac{e^{z + C}}{1 + e^{z + C}} = \frac{1}{1 + e^{-z - C}} = \sigma(z + C)
\end{aligned}$$

với C là một hằng số. Chọn $C = 0$, ta được $a = f(\mathbf{x}^T \mathbf{w}) = \sigma(z)$. Đây chính là hàm sigmoid. Hồi quy logistic với hàm kích hoạt là hàm sigmoid được sử dụng phổ biến nhất. Mô hình này còn có tên là *hồi quy logistic sigmoid*. Khi nói hồi quy logistic, ta ngầm hiểu rằng đó chính là hồi quy logistic sigmoid.

Thay (9.13) và (9.14) vào (9.12) ta thu được

$$\nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i, y_i) = (a_i - y_i)\mathbf{x}_i = (\sigma(\mathbf{x}_i^T \mathbf{w}) - y_i)\mathbf{x}_i. \tag{9.15}$$

Từ đó, công thức cập nhật nghiệm cho hồi quy logistic sử dụng SGD là

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(a_i - y_i)\mathbf{x}_i = \mathbf{w} - \eta(\sigma(\mathbf{x}_i^T \mathbf{w}) - y_i)\mathbf{x}_i \tag{9.16}$$

với η là tốc độ học.

9.2.3. Hồi quy logistic với suy giảm trọng số

Một trong các kỹ thuật phổ biến giúp tránh quá khớp cho các mạng neuron là sử dụng *suy giảm trọng số* (*weight decay*). Đây là một kỹ thuật kiểm soát, trong đó một đại lượng tỉ lệ với bình phương chuẩn ℓ_2 của vector trọng số $\mathbf{w}$ được cộng vào hàm mất mát để kiểm soát độ lớn của các hệ số. Hàm mất mát trở thành

$$\bar{J}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\left(-y_i \log a_i - (1-y_i)\log(1-a_i) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2\right). \tag{9.17}$$

Công thức cập nhật $\mathbf{w}$ bằng SGD trong hồi quy logistic với suy giảm trọng số là:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\left((\sigma(\mathbf{x}_i^T\mathbf{w}) - y_i)\mathbf{x}_i + \lambda\mathbf{w}\right) \tag{9.18}$$

9.3. Triển khai thuật toán trên Python

Hàm ước lượng xác suất đầu ra cho mỗi điểm dữ liệu và hàm tính giá trị hàm mất mát với weight decay có thể được thực hiện như sau trong Python.

```
def prob(w, X):
    """
    X: a 2d numpy array of shape (N, d). N datatpoint, each with size d
    w: a 1d numpy array of shape (d)
    """
    return sigmoid(X.dot(w))

def loss(w, X, y, lam):
    """
    X, w as in prob
    y: a 1d numpy array of shape (N). Each elem = 0 or 1
    """
    a = prob(w, X)
    loss_0 = -np.mean(y*np.log(a) + (1-y)*np.log(1-a))
    weight_decay = 0.5*lam/X.shape[0]*np.sum(w*w)
    return loss_0 + weight_decay
```

Từ công thức (9.18), ta có thể thực hiện thuật toán tìm $\mathbf{w}$ cho hồi quy logistic như sau:

```
def logistic_regression(w_init, X, y, lam, lr = 0.1, nepoches = 2000):
    # lam: regulariza paramether, lr: learning rate, nepoches: # epoches
    N, d = X.shape[0], X.shape[1]
    w = w_old = w_init
    # store history of loss in loss_hist
    loss_hist = [loss(w_init, X, y, lam)]
    ep = 0
    while ep < nepoches:
        ep += 1
        mix_ids = np.random.permutation(N) # stochastic
        for i in mix_ids:
            xi = X[i]
            yi = y[i]
            ai = sigmoid(xi.dot(w))
            # update
            w = w - lr*((ai - yi)*xi + lam*w)
            loss_hist.append(loss(w, X, y, lam))
        if np.linalg.norm(w - w_old)/d < 1e-6:
            break
        w_old = w
    return w, loss_hist
```

9.3.1. Hồi quy logistic cho ví dụ đầu chương

Áp dụng vào bài toán dự đoán đỗ/trượt ở đầu chương:

```
np.random.seed(2)
X = np.array([[0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 1.75, 2.00, 2.25, 2.50,
2.75, 3.00, 3.25, 3.50, 4.00, 4.25, 4.50, 4.75, 5.00, 5.50]]).T
y = np.array([0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1])

# bias trick
Xbar = np.concatenate((X, np.ones((X.shape[0], 1))), axis = 1)
w_init = np.random.randn(Xbar.shape[1])
lam = 0.0001
w, loss_hist = logistic_regression(w_init, Xbar, y, lam, lr = 0.05, nepoches
    = 500)
print('Solution of Logistic Regression:', w)
print('Final loss:', loss(w, Xbar, y, lam))
```

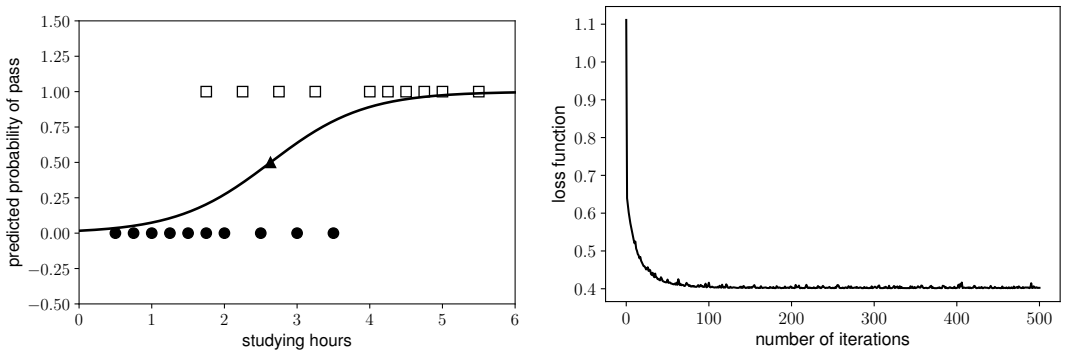
Kết quả:

```
Solution of Logistic Regression: [ 1.54337021 -4.06486702]
Final loss: 0.402446724975
```

Từ đây ta có thể rút ra xác suất thi đỗ dựa trên công thức:

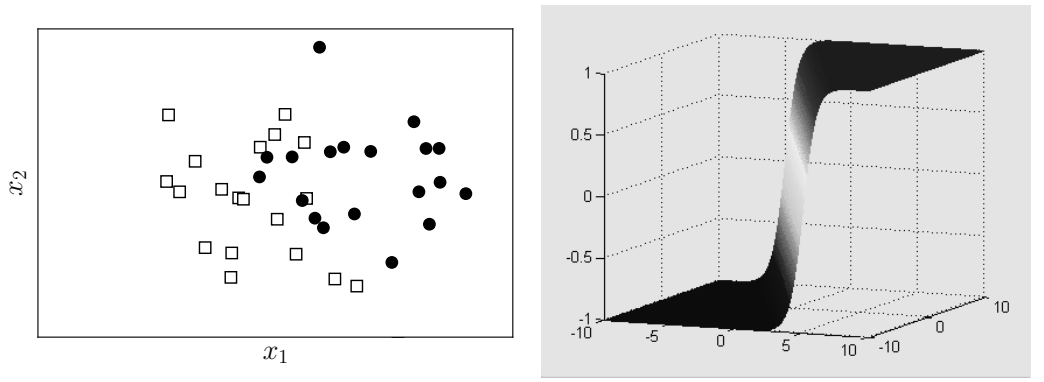
$$\texttt{probability_of_pass} \approx \texttt{sigmoid(1.54 * hours_of_studying - 4.06)}$$

Biểu thức này cũng chỉ ra rằng xác suất thi đỗ tăng khi thời gian ôn tập tăng, do sigmoid là một hàm đồng biến. Nghiệm của mô hình hồi quy logistic và giá trị hàm mất mát qua mỗi epoch được mô tả trên Hình 9.3.



(a) **(b)**

Hình 9.3. Nghiệm của hồi quy logistic cho bài toán dự đoán kết quả thi dựa trên thời gian học. (a) Đường nét liền thể hiện xác suất thi đỗ dựa trên thời gian học. Điểm tam giác thể hiện ngưỡng ra quyết định đỗ/trượt. Điểm này có thể thay đổi tuỳ vào bài toán. (b) Giá trị của hàm mất mát qua các vòng lặp. Hàm mất mát giảm nhanh và hội tụ sớm.



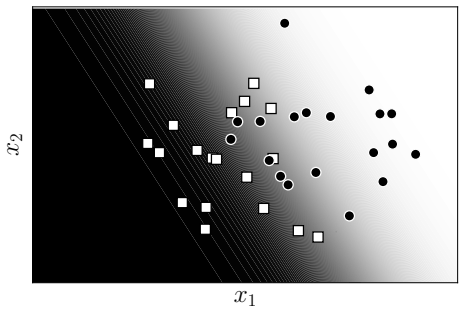
(a) Dữ liệu cho bài toán phân loại trong không gian hai chiều. **(b)** Đồ thị hàm sigmoid trong không gian hai chiều.

Hình 9.4. Ví dụ về dữ liệu và hàm sigmoid trong không gian hai chiều.

9.3.2. Ví dụ với dữ liệu hai chiều

Giả sử có hai tập dữ liệu vuông và tròn phân bố trên mặt phẳng như trong Hình 9.4a. Với dữ liệu đầu vào nằm trong không gian hai chiều, hàm sigmoid có dạng thác nước như trong Hình 9.4b.

Kết quả dự đoán xác suất đầu ra khi áp dụng mô hình hồi quy logistic được minh họa như Hình 9.5 với độ sáng của nền thể hiện xác suất điểm đó có nhãn tròn. Màu đen đậm thể hiện giá trị gần bằng không, màu trắng thể hiện giá trị rất gần bằng một.



Hình 9.5. Ví dụ về hồi quy logistic với dữ liệu hai chiều. Vùng màu càng đen thể hiện xác suất thuộc nhãn hình vuông càng cao. Vùng màu càng trắng thể hiện xác suất thuộc nhãn hình tròn càng cao. Vùng biên giữa hai nhãn (khu vực màu xám) thể hiện các điểm thuộc vào mỗi nhãn với xác suất thấp hơn.

Nếu phải lựa chọn một ranh giới thay vì xác suất, ta thấy các đường thẳng nằm trong khu vực màu xám là các lựa chọn hợp lý. Ta sẽ chứng minh ở phần sau rằng tập hợp các điểm có cùng xác suất đầu ra tạo thành một siêu phẳng.

9.4. Tính chất của hồi quy logistic

a. *Hồi quy logistic thực ra là một thuật toán phân loại.*

Mặc dù trong tên có từ *hồi quy*, hồi quy logistic được sử dụng nhiều trong các bài toán phân loại. Sau khi tìm được mô hình, việc xác định nhãn y cho một điểm dữ liệu $\mathbf{x}$ được xác định bằng việc so sánh hai giá trị:

$$P(y = 1|\mathbf{x}; \mathbf{w}); \quad P(y = 0|\mathbf{x}; \mathbf{w}) \tag{9.19}$$

Nếu giá trị thứ nhất lớn hơn, ta kết luận điểm dữ liệu có nhãn một và ngược lại. Vì tổng hai giá trị này luôn bằng một nên ta chỉ cần xác định $P(y = 1|\mathbf{x}; \mathbf{w})$ có lớn hơn 0.5 hay không.

b. *Đường ranh giới tạo bởi hồi quy logistic là một siêu phẳng.*

Thật vậy, giả sử những điểm có xác suất đầu ra lớn hơn 0.5 được gán nhãn một. Tập hợp các điểm này là nghiệm của bất phương trình:

$$P(y = 1|\mathbf{x}; \mathbf{w}) > 0.5 \Leftrightarrow \frac{1}{1 + e^{-\mathbf{x}^T\mathbf{w}}} > 0.5 \Leftrightarrow e^{-\mathbf{x}^T\mathbf{w}} < 1 \Leftrightarrow \mathbf{x}^T\mathbf{w} > 0$$

Nói cách khác, tập hợp các điểm được gán nhãn một tạo thành một nửa không gian $\mathbf{x}^T\mathbf{w} > 0$, tập hợp các điểm được gán nhãn không tạo thành nửa không gian còn lại. Ranh giới giữa hai nhãn là siêu phẳng $\mathbf{x}^T\mathbf{w} = 0$.

Vì vậy, hồi quy logistic được coi như một bộ phân loại tuyến tính.

c. *Hồi quy logistic không yêu cầu giả thiết tách biệt tuyến tính.*

Một điểm cộng của hồi quy logistic so với PLA là nó không cần giả thiết dữ liệu hai tập hợp là tách biệt tuyến tính. Tuy nhiên, ranh giới tìm được vẫn có

dạng tuyến tính. Vì vậy, mô hình này chỉ phù hợp với loại dữ liệu mà hai tập *gần* tách biệt tuyến tính.

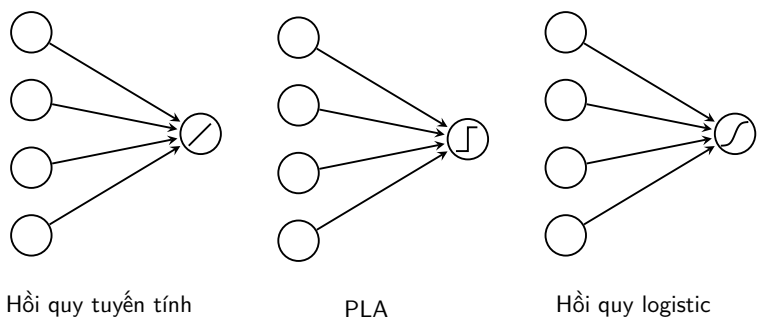
d. *Ngưỡng ra quyết định có thể thay đổi.*

Hàm dự đoán đầu ra của các điểm dữ liệu mới có thể được viết như sau:

```
def predict(w, X, threshold = 0.5):
"""
predict output for each row of X
X: a numpy array of shape (N, d), threshold: 0 < threshold < 1
return a 1d numpy array, each element is 0 or 1
"""
res = np.zeros(X.shape[0])
res[np.where(prob(w, X) > threshold)[0]] = 1
return res
```

Trong các ví dụ đã nêu, ngưỡng ra quyết định đều được lấy tại 0.5. Trong nhiều trường hợp, ngưỡng này có thể được thay đổi. Ví dụ, việc xác định các giao dịch là lừa đảo của một công ty tín dụng là rất quan trọng. Việc phân loại nhầm một giao dịch lừa đảo thành một giao dịch thông thường gây ra hậu quả nghiêm trọng hơn chiều ngược lại. Trong bài toán đó, ngưỡng phân loại có thể giảm xuống còn 0.3. Nghĩa là các giao dịch được dự đoán là lừa đảo với xác suất lớn hơn 0.3 sẽ được gán nhãn lừa đảo và cần được xử lý bằng các biện pháp khác.

e. Khi biểu diễn dưới dạng các mạng neuron, hồi quy tuyến tính, PLA và hồi quy logistic có thể được biểu diễn như trong Hình 9.6. Sự khác nhau chỉ nằm ở lựa chọn hàm kích hoạt.



Hình 9.6. Biểu diễn hồi quy tuyến tính, PLA, và hồi quy logistic dưới dạng neural network.

9.5. Bài toán phân biệt hai chữ số viết tay

Xét bài toán phân biệt hai chữ số không và một trong bộ cơ sở dữ liệu MNIST. Trong mục này, class `LogisticRegression` trong thư viện scikit-learn sẽ được sử dụng. Trước tiên, ta cần khai báo các thư viện và tải về bộ cơ sở dữ liệu MNIST:

```
import numpy as np
from sklearn.datasets import fetch_mldata
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
mnist = fetch_mldata('MNIST original', data_home='../../data/')
N, d = mnist.data.shape
print('Total {:d} samples, each has {:d} pixels.'.format(N, d))
```

Kết quả:

```
Total 70000 samples, each has 784 pixels.
```

Có tổng cộng 70000 điểm dữ liệu trong tập dữ liệu MNIST, mỗi điểm là một mảng 784 phần tử tương ứng với 784 pixel. Mỗi chữ số từ không đến chín chiếm khoảng mười phần trăm. Chúng ta sẽ lấy ra tất cả các điểm ứng với chữ số không và một, sau đó chọn ngẫu nhiên 2000 điểm làm tập kiểm tra, phần còn lại đóng vai trò tập huấn luyện.

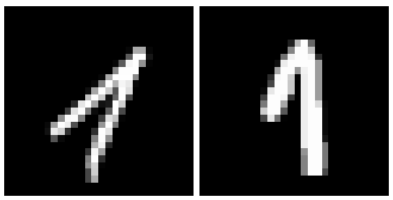
```
X_all = mnist.data
y_all = mnist.target
X0 = X_all[np.where(y_all == 0)[0]] # all digit 0
X1 = X_all[np.where(y_all == 1)[0]] # all digit 1
y0 = np.zeros(X0.shape[0]) # class 0 label
y1 = np.ones(X1.shape[0])  # class 1 label
X = np.concatenate((X0, X1), axis = 0) # all digits 0 and 1
y = np.concatenate((y0, y1)) # all labels
# split train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=2000)
```

Tiếp theo, ta xây dựng mô hình hồi quy logistic trên tập huấn luyện và dự đoán nhãn của các điểm trong tập kiểm tra. Kết quả này được so sánh với nhãn thực sự của mỗi điểm dữ liệu để tính độ chính xác của bộ phân loại:

```
model = LogisticRegression(C = 1e5) # C is inverse of lam
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy %.2f %%" % (100*accuracy_score(y_test, y_pred.tolist())))
```

Kết quả:

```
Accuracy 99.90 %
```



Hình 9.7. Các chữ số bị phân loại lỗi trong bài toán phân loại nhị phân với hai chữ số không và một.

Như vậy, gần 100% các ảnh được phân loại chính xác. Điều này dễ hiểu vì hai chữ số không và một khác nhau rất nhiều.

Tiếp theo, ta cùng đi tìm những ảnh bị phân loại sai và hiển thị chúng:

```
mis = np.where((y_pred - y_test) !=0)[0]
Xmis = X_test[mis, :]
from display_network import *
filename = 'mnist_mis.pdf'
with PdfPages(filename) as pdf:
plt.axis('off')
A = display_network(Xmis.T, 1, Xmis.shape[0])
f2 = plt.imshow(A, interpolation='nearest' )
plt.gray()
pdf.savefig(bbox_inches='tight')
plt.show()
```

Chỉ có hai chữ số bị phân loại lỗi được cho trên Hình 9.7. Trong đó, chữ số không bị phân loại lỗi là dễ hiểu vì nó trông rất giống chữ số một.

9.6. Bài toán phân loại đa lớp

Hồi quy logistic được áp dụng cho các bài toán phân loại nhị phân. Các bài toán phân loại thực tế có thể có nhiều hơn hai nhãn dữ liệu, được gọi là bài toán *phân loại đa lớp* (*multi-class classification*). Hồi quy logistic cũng có thể được áp dụng vào các bài toán này bằng một vài kỹ thuật.

Có ít nhất bốn cách áp dụng các bộ phân loại nhị phân vào bài toán phân loại đa lớp.

9.6.1. one-vs-one

Ta có thể xây dựng nhiều bộ phân loại nhị phân cho từng cặp hai nhãn dữ liệu. Bộ thứ nhất phân biệt nhãn thứ nhất và nhãn thứ hai, bộ thứ hai phân biệt nhãn thứ nhất và nhãn thứ ba,... Có tổng cộng $P = \frac{C(C-1)}{2}$ bộ phân loại nhị phân cần xây dựng với C là số lượng nhãn. Cách thực hiện này được gọi là *one-vs-one*.

Với một điểm dữ liệu kiểm tra, ta dùng tất cả P bộ phân loại để dự đoán nhãn của nó. Kết quả cuối cùng có thể được xác định bằng cách xem điểm dữ liệu đó

được gán nhãn nào nhiều nhất. Ngoài ra, nếu mỗi bộ phân loại có thể đưa ra xác suất giống hồi quy logistic, ta có thể tính tổng các xác suất mà điểm dữ liệu đó rơi vào mỗi nhãn. Chú ý rằng tổng các xác suất là P thay vì một bởi có P bộ phân loại khác nhau.

Cách làm này không lợi về tính toán vì số bộ phân loại phải huấn luyện tăng nhanh khi số nhãn tăng lên. Hơn nữa, điều không hợp lý xảy ra nếu một chữ số có nhãn bằng một được đưa vào bộ phân loại giữa hai nhãn chữ số năm và sáu.

9.6.2. Phân loại phân tầng

One-vs-one yêu cầu xây dựng $\frac{C(C-1)}{2}$ bộ phân loại khác nhau. Để giảm số bộ phân loại cần xây dựng, ta có thể dùng phương pháp *phân tầng*. Ý tưởng của phương pháp này có thể được thấy qua ví dụ sau.

Xét bài toán phân loại bốn chữ số {4, 5, 6, 7} trong MNIST. Vì chữ số 4 và 7 khá giống nhau, chữ số 5 và 6 khá giống nhau nên trước tiên ta xây dựng bộ phân loại giữa {4, 7} và {5, 6}. Sau đó xây dựng thêm hai bộ phân loại để xác định từng chữ số trong mỗi nhóm. Tổng cộng, ta cần ba bộ phân loại nhị phân so với sáu bộ như khi sử dụng one-vs-one.

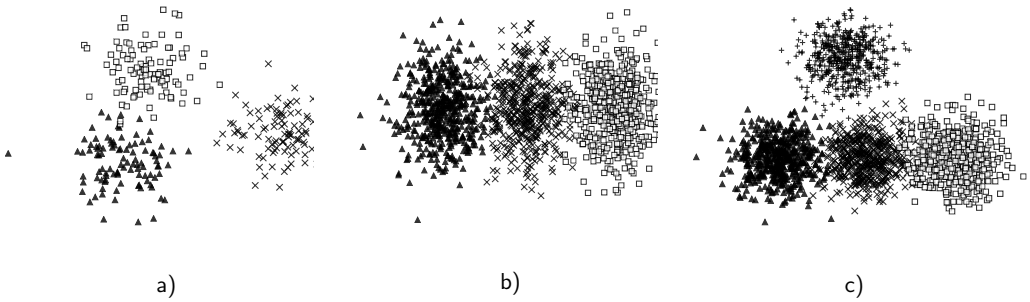
Có nhiều cách chia nhỏ tập dữ liệu ban đầu ra các cặp tập con. Cách phân tầng có ưu điểm là giảm số bộ phân loại nhị phân cần xây dựng. Tuy nhiên, cách làm này có một hạn chế lớn: nếu chỉ một bộ phân loại cho kết quả sai thì kết quả cuối cùng chắc chắn sẽ sai. Ví dụ, nếu một ảnh chứa chữ số 5 bị phân loại lỗi bởi bộ phân loại đầu tiên thì cuối cùng nó sẽ bị nhận nhầm thành 4 hoặc 7.

9.6.3. Mã hoá nhị phân

Có một cách tiếp tục giảm số bộ phân loại là *mã hoá nhị phân*. Trong phương pháp này, mỗi nhãn được mã hoá bởi một số nhị phân. Ví dụ, nếu có bốn nhãn thì chúng được mã hoá bởi 00, 01, 10, và 11. Số bộ phân loại nhị phân cần xây dựng chỉ là $m = \lceil \log_2(C) \rceil$ trong đó C là số nhãn, $\lceil a \rceil$ là số nguyên nhỏ nhất không nhỏ hơn a. Bộ phân loại đầu tiên giúp xác định bit đầu tiên của nhãn, bộ thứ hai xác định bit tiếp theo,.... Cách làm này sử dụng một số lượng nhỏ nhất các bộ phân loại nhị phân. Tuy nhiên, một điểm dữ liệu chỉ được phân loại đúng khi mọi bộ phân loại nhị phân dự đoán đúng bit tương ứng. Hơn nữa, nếu số nhãn không phải là lũy thừa của hai, mã nhị phân nhận được có thể không tương ứng với nhãn nào.

9.6.4. one-vs-rest

Kỹ thuật được sử dụng nhiều nhất là *one-vs-rest*. Cụ thể, C bộ phân loại nhị phân được xây dựng tương ứng với các nhãn. Bộ thứ nhất xác định một điểm có nhãn thứ nhất hay không, hoặc xác suất để một điểm có nhãn đó. Tương tự, bộ



Hình 9.8. Ví dụ về phân phối của các tập dữ liệu trong bài toán phân loại đa lớp.

thứ hai xác định điểm đó có nhãn thứ hai hay không hoặc xác xuất có nhãn thứ hai là bao nhiêu. Nhãn cuối cùng được xác định theo nhãn mà điểm đó rơi vào với xác suất cao nhất.

Hồi quy logistic trong thư viện scikit-learn có thể được áp dụng trực tiếp vào các bài toán phân loại đa lớp với kỹ thuật one-vs-rest. Với MNIST, ta có thể dùng hồi quy logistic kết hợp với one-vs-rest (mặc định) như sau:

```
X_train, X_test, y_train, y_test = \
train_test_split(X_all, y_all, test_size=10000)
model = LogisticRegression(C = 1e5) # C is inverse of lam
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy %.2f %%" % (100*accuracy_score(y_test, y_pred.tolist())))
```

Kết quả thu được tương đối thấp, khoảng 91.7%. Phương pháp KNN đơn giản hơn đã có độ chính xác khoảng 96%. Điều này chứng tỏ one-vs-rest không làm việc tốt trong trường hợp này.

9.7. Thảo luận

9.7.1. Kết hợp các phương pháp trên

Trong nhiều trường hợp, ta cần kết hợp nhiều kỹ thuật trong số bốn kỹ thuật đã đề cập. Xét ba ví dụ trong Hình 9.8.

- Hình 9.8a: Cả bốn phương pháp trên đây đều có thể áp dụng được.
- Hình 9.8b: One-vs-rest không phù hợp vì tập dữ liệu ở giữa và hợp của hai tập còn lại là không (gần) tách biệt tuyến tính. Lúc này, one-vs-one hoặc phân tầng phù hợp hơn.
- Tương tự như trên, trong trường hợp có ba tập dữ liệu nằm thẳng hàng như minh họa trong Hình 9.8c, phương pháp one-vs-rest sẽ không phù hợp. Nguyên

nhân là khi gộp hai trong ba tập dữ liệu lại thành một lớp đối nghịch với tập còn lại, hai nhóm dữ liệu này không còn (gần) tách biệt tuyến tính. Khi đó, bộ phân loại tuyến tính được xây dựng bởi one-vs-rest khó có thể tìm được một siêu phẳng phân chia hợp lý giữa hai nhóm dữ liệu.

Ngược lại, phương pháp one-vs-one vẫn có thể hoạt động hiệu quả trong tình huống này. Lý do là mỗi bộ phân loại nhị phân chỉ cần phân biệt giữa hai lớp dữ liệu tại một thời điểm. Nếu từng cặp tập dữ liệu có thể được tách biệt tuyến tính hoặc gần tách biệt tuyến tính, các bộ phân loại one-vs-one vẫn có thể tìm được ranh giới phân chia phù hợp cho từng cặp lớp. Kết quả cuối cùng được xác định dựa trên cơ chế bỏ phiếu hoặc tổng hợp xác suất từ các bộ phân loại này.

Phương pháp phân loại phân tầng cũng có thể được áp dụng nếu ta lựa chọn cách phân chia các nhãn một cách hợp lý. Ví dụ, nếu có bốn lớp dữ liệu và một lớp nằm tách biệt phía trên ba lớp còn lại, ta có thể xây dựng một bộ phân loại đầu tiên nhằm tách lớp phía trên ra khỏi ba lớp còn lại. Sau đó, ba lớp còn lại có thể tiếp tục được phân chia bằng các bộ phân loại khác, chẳng hạn như one-vs-one hoặc một cấu trúc phân tầng khác.

Trong nhiều trường hợp, việc kết hợp nhiều phương pháp phân loại có thể mang lại hiệu quả tốt hơn so với việc chỉ sử dụng một phương pháp duy nhất. Ví dụ, ta có thể dùng one-vs-rest để tách một nhóm dữ liệu rõ ràng ra khỏi các nhóm còn lại, sau đó áp dụng one-vs-one hoặc các kỹ thuật phân tầng để tiếp tục phân loại các nhóm còn lại. Tuy nhiên, khó khăn lớn nhất của cách tiếp cận này nằm ở việc xác định chiến lược phân nhóm ban đầu sao cho phù hợp với cấu trúc phân bố của dữ liệu. Nếu việc phân nhóm không hợp lý, sai sót ở các bước phân loại đầu tiên có thể lan truyền sang các bước tiếp theo và làm giảm độ chính xác của toàn bộ hệ thống phân loại.

Với bài toán phân loại đa lớp, nhìn chung các kỹ thuật sử dụng các bộ phân loại nhị phân ít mang lại hiệu quả.

Hồi quy Softmax

Hồi quy logistic được thiết kế cho các bài toán ***phân loại nhị phân****, tức là các bài toán chỉ có hai nhãn dữ liệu. Tuy nhiên, trong nhiều ứng dụng thực tế, số lượng nhãn có thể lớn hơn hai. Ví dụ, bài toán nhận dạng chữ số viết tay trong bộ dữ liệu MNIST có 10 lớp tương ứng với các chữ số từ 0 đến 9. Để xử lý các bài toán phân loại đa lớp như vậy, hồi quy logistic có thể được mở rộng thành* ***hồi quy softmax*** *(Softmax Regression), còn được gọi là multinomial logistic regression.*
Giả sử bài toán có C nhãn dữ liệu. Với mỗi nhãn j, ta gán một vector trọng số $\mathbf{w}_j$. Khi đó, xác suất để một điểm dữ liệu $\mathbf{x}$ thuộc lớp j được xác định bởi hàm softmax:

$$P(y = j|\mathbf{x}) = \frac{\exp(\mathbf{x}^T\mathbf{w}_j)}{\sum_{k=1}^{C}\exp(\mathbf{x}^T\mathbf{w}_k)}.$$

Hàm softmax có một số tính chất quan trọng:

- *Các giá trị xác suất luôn nằm trong khoảng $(0, 1)$.*
- *Tổng xác suất của tất cả các lớp luôn bằng 1:*

$$\sum_{j=1}^{C} P(y = j|\mathbf{x}) = 1.$$

- *Lớp dự đoán cho điểm dữ liệu $\mathbf{x}$ là lớp có xác suất lớn nhất:*

$$\hat{y} = \arg\max_j P(y = j|\mathbf{x}).$$

Tương tự như hồi quy logistic, các tham số của mô hình được ước lượng bằng cách tối thiểu hóa hàm mất mát ***cross-entropy****. Nếu $\mathbf{y}_i$ là vector nhãn dạng one-hot của điểm dữ liệu $\mathbf{x}_i$, hàm mất mát có dạng:*

$$J(\mathbf{W}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{ij}\log P(y = j|\mathbf{x}_i).$$

Ở đây $\mathbf{W}$ là ma trận trọng số gồm các vector $\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_C$. Việc tối ưu hàm mất mát có thể được thực hiện bằng các phương pháp gradient descent hoặc stochastic gradient descent tương tự như hồi quy logistic.
Như vậy, hồi quy softmax có thể được xem như một sự tổng quát hóa tự nhiên của hồi quy logistic từ bài toán hai lớp sang bài toán nhiều lớp. Trong nhiều mô hình học sâu hiện đại, đặc biệt là các mạng neuron cho bài toán phân loại, tầng cuối cùng thường sử dụng hàm softmax để chuyển các giá trị đầu ra thành các xác suất của từng lớp dữ liệu.

9.7.2. Biểu diễn dưới dạng mạng neuron

Lấy ví dụ bài toán có bốn nhãn dữ liệu {1, 2, 3, 4}; ta có thể biểu diễn các kỹ thuật đã được đề cập dưới dạng mạng neuron như trong Hình 9.9. Mỗi nút ở tầng đầu ra thể hiện đầu ra của một bộ phân loại nhị phân.

Các mạng neuron này đều có nhiều nút ở tầng đầu ra, vector trọng số $\mathbf{w}$ đã trở thành *ma trận trọng số* $\mathbf{W}$. Mỗi cột của $\mathbf{W}$ tương ứng với vector trọng số của một nút đầu ra. Các bộ phân loại nhị phân này có thể được xây dựng đồng thời. Nếu chúng là các bộ hồi quy logistic, công thức cập nhật theo SGD:

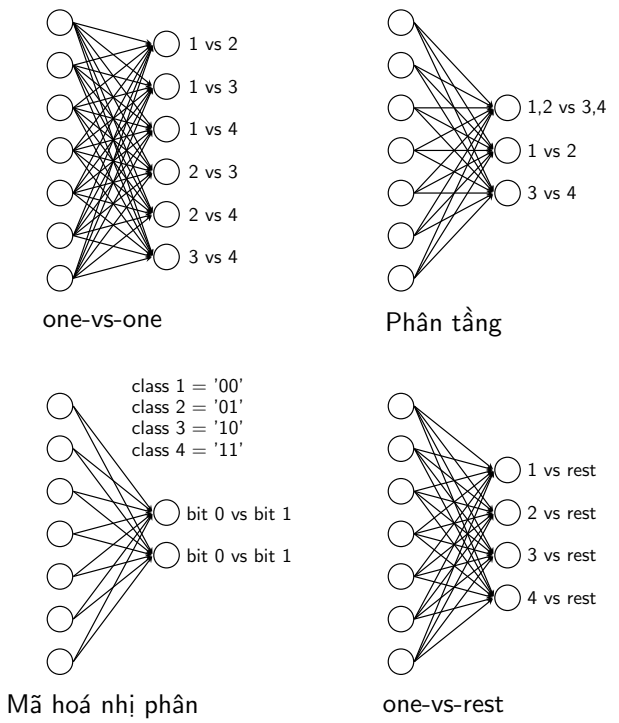
$$\mathbf{w} \leftarrow \mathbf{w} - \eta(a_i - y_i)\mathbf{x}_i \tag{9.20}$$

có thể được tổng quát thành

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\mathbf{x}_i(\mathbf{a}_i - \mathbf{y}_i)^T. \tag{9.21}$$

Với $\mathbf{W}, \mathbf{y}_i, \mathbf{a}_i$ lần lượt là ma trận trọng số, vector đầu ra thực sự và vector đầu ra dự đoán ứng với dữ liệu $\mathbf{x}_i$. Chú ý rằng vector $\mathbf{y}_i$ là một vector nhị phân, vector $\mathbf{a}_i$ gồm các phần tử nằm trong khoảng $(0, 1)$.

Chú ý: Số hạng thứ hai trong (9.21) không thể là $(\mathbf{a}_i - \mathbf{y}_i)\mathbf{x}_i^T$ vì ma trận này khác chiều với $\mathbf{W}$. Số hạng này cần là tích của hai vector: vector thứ nhất cần có cùng số hàng với $\mathbf{W}$, tức chiều của dữ liệu $\mathbf{x}_i$; vector thứ hai cần phù hợp với số cột của $\mathbf{W}$, tức số nút ở tầng đầu ra.



Hình 9.9. Mô hình neural network cho các kỹ thuật sử dụng các bộ phân loại nhị phân cho bài toán phân loại đa lớp.

9.8. Kết luận

Trong chương này, chúng ta đã nghiên cứu mô hình **hồi quy logistic** (Logistic Regression), một trong những phương pháp phân loại tuyến tính phổ biến trong học máy. Mặc dù trong tên có từ *hồi quy*, phương pháp này chủ yếu được sử dụng cho các bài toán **phân loại nhị phân** trong đó đầu ra được biểu diễn dưới dạng xác suất.

Chương đã bắt đầu bằng việc so sánh hồi quy logistic với hai mô hình tuyến tính đã học trước đó là hồi quy tuyến tính và PLA. Khác với hai mô hình này, hồi quy logistic sử dụng **hàm kích hoạt sigmoid** để biến đổi tích vô hướng $\mathbf{x}^T\mathbf{w}$ thành một giá trị nằm trong khoảng $(0, 1)$, từ đó có thể diễn giải như xác suất xảy ra của một lớp dữ liệu.

Tiếp theo, chúng ta đã xây dựng hàm mất mát của mô hình dựa trên phương pháp **ước lượng hợp lý cực đại** (Maximum Likelihood Estimation). Từ đó, hàm mất mát của hồi quy logistic được biểu diễn dưới dạng *cross-entropy*. Việc tối ưu hàm mất mát được thực hiện thông qua phương pháp **Stochastic Gradient Descent (SGD)**.

Chương cũng đã trình bày kỹ thuật **suy giảm trọng số** (weight decay) nhằm hạn chế hiện tượng quá khớp bằng cách bổ sung một thành phần chuẩn ℓ_2 vào hàm mất mát. Sau đó, mô hình hồi quy logistic đã được triển khai bằng Python và áp dụng vào các ví dụ thực tế như bài toán dự đoán kết quả thi dựa trên thời gian học và bài toán phân loại ảnh chữ số viết tay trong bộ dữ liệu MNIST.

Cuối cùng, chương đã thảo luận cách mở rộng hồi quy logistic cho các bài toán **phân loại đa lớp**. Các kỹ thuật phổ biến như *one-vs-one*, *one-vs-rest*, phân loại phân tầng và mã hóa nhị phân đã được giới thiệu, đồng thời chỉ ra rằng các kỹ thuật này có thể được biểu diễn dưới dạng các mạng neuron với nhiều nút ở tầng đầu ra.

Nhìn chung, hồi quy logistic là một mô hình đơn giản nhưng hiệu quả trong nhiều bài toán phân loại. Phương pháp này đóng vai trò quan trọng trong nền tảng của nhiều mô hình học máy và mạng neuron hiện đại.

9.9. Câu hỏi ôn tập

a. So sánh hồi quy tuyến tính, PLA và hồi quy logistic dưới góc nhìn của mô hình tuyến tính $y = f(\mathbf{x}^T\mathbf{w})$. Điểm khác nhau nằm ở đâu?

b. Trình bày các tính chất quan trọng của hàm kích hoạt **sigmoid**. Tại sao hàm sigmoid lại phù hợp với các bài toán phân loại nhị phân?

c. Từ giả thiết xác suất

$$p(y_i = 1|\mathbf{x}_i; \mathbf{w}) = \sigma(\mathbf{x}_i^T \mathbf{w})$$

hãy suy ra biểu thức của hàm mất mát trong hồi quy logistic.

d. Giải thích ý nghĩa của công thức cập nhật trọng số trong hồi quy logistic sử dụng SGD:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(\sigma(\mathbf{x}_i^T \mathbf{w}) - y_i)\mathbf{x}_i.$$

e. Trình bày vai trò của **weight decay** trong hồi quy logistic. Việc bổ sung thành phần $\lambda\|\mathbf{w}\|_2^2$ vào hàm mất mát giúp ích như thế nào trong việc tránh quá khớp?

f. **Bài tập thực hành**

Xét tập dữ liệu sau về thời gian ôn thi và kết quả thi của sinh viên:

Hours	Pass
0.5	0
1.0	0
1.5	0
2.0	0
2.5	1
3.0	1
3.5	1
4.0	1

Hãy thực hiện các yêu cầu sau:

- Xây dựng mô hình hồi quy logistic dự đoán xác suất thi đỗ dựa trên thời gian học.
- Cài đặt thuật toán bằng Python sử dụng hàm sigmoid và SGD.
- Vẽ đồ thị xác suất thi đỗ theo số giờ học.
- Xác định ngưỡng ra quyết định khi xác suất bằng 0.5.

Chương 10

Máy vector hỗ trợ - SVM

Máy vector hỗ trợ (*Support Vector Machine – SVM*) là một trong những thuật toán phân loại mạnh mẽ và được sử dụng rộng rãi trong học máy. Không giống như nhiều bộ phân loại tuyến tính khác, SVM tìm siêu phẳng phân chia dữ liệu sao cho khoảng cách tới các điểm dữ liệu gần nhất của hai lớp là lớn nhất. Nhờ tiêu chí tối đa hóa lề này, SVM thường có khả năng tổng quát hóa tốt đối với các dữ liệu chưa quan sát.

10.1. Mô hình máy vector hỗ trợ

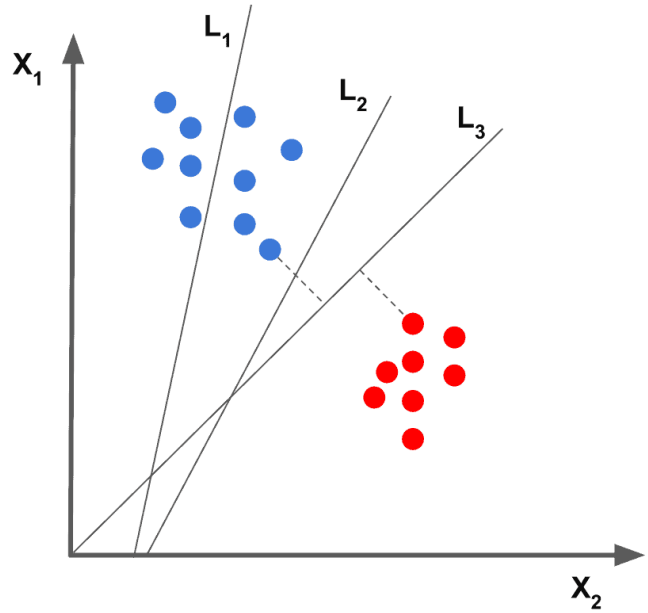
10.1.1. Giới thiệu

Máy vector hỗ trợ (Support Vector Machine – SVM) là một trong những thuật toán học máy có giám sát phổ biến nhất cho các bài toán phân loại. Ý tưởng cốt lõi của SVM là tìm một ranh giới quyết định (*decision boundary*) tối ưu để phân tách các lớp dữ liệu trong không gian đặc trưng.

Trong các chương trước, chúng ta đã nghiên cứu một số bộ phân loại tuyến tính như PLA và hồi quy logistic. Các mô hình này đều tìm một siêu phẳng để phân chia dữ liệu. Tuy nhiên, trong nhiều trường hợp, có thể tồn tại vô số siêu phẳng khác nhau phân loại đúng dữ liệu. Điều này đặt ra một câu hỏi quan trọng:

Trong tất cả các siêu phẳng phân chia được dữ liệu, đâu là siêu phẳng tốt nhất?

Máy vector hỗ trợ trả lời câu hỏi này bằng cách lựa chọn siêu phẳng phân chia sao cho khoảng cách từ siêu phẳng đó tới các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất. Tiêu chí này giúp mô hình có khả năng tổng quát hóa tốt hơn đối với các dữ liệu chưa nhìn thấy.



Hình 10.1. Ví dụ về nhiều siêu phẳng có thể phân chia hai lớp dữ liệu.

Xét ví dụ trong Hình 10.1. Hai lớp dữ liệu được biểu diễn bởi các điểm màu xanh và màu đỏ trong không gian hai chiều với hai thuộc tính x_1 và x_2. Ta có thể vẽ nhiều đường thẳng khác nhau (L_1, L_2, L_3) để phân chia hai lớp dữ liệu này.

Tuy nhiên, không phải mọi đường phân chia đều có chất lượng như nhau. Một số đường có thể nằm quá gần một lớp dữ liệu, khiến các điểm dữ liệu mới dễ bị phân loại sai. Điều này dẫn đến một câu hỏi quan trọng: *trong tất cả các siêu phẳng phân chia được dữ liệu, đâu là siêu phẳng tốt nhất?*

Máy vector hỗ trợ (SVM) trả lời câu hỏi này bằng cách lựa chọn siêu phẳng sao cho khoảng cách từ siêu phẳng đó tới các điểm dữ liệu gần nhất của hai lớp là lớn nhất. Khoảng cách này được gọi là *lề* (margin). Siêu phẳng có lề lớn nhất thường giúp mô hình có khả năng tổng quát hóa tốt hơn đối với dữ liệu chưa quan sát.

10.1.2. Siêu phẳng phân tách

Trong các bài toán phân loại tuyến tính, mục tiêu là tìm một ranh giới (*decision boundary*) để phân chia dữ liệu thành hai lớp khác nhau. Trong không gian hai chiều, ranh giới này là một đường thẳng; trong không gian ba chiều, đó là một mặt phẳng. Tổng quát hơn, trong không gian d chiều, ranh giới phân chia được gọi là một *siêu phẳng* (*hyperplane*).

Một siêu phẳng trong không gian $\mathbb{R}^d$ có dạng

$$\mathbf{w}^T\mathbf{x} + b = 0 \tag{10.1}$$

trong đó $\mathbf{w} = [w_1, w_2, \ldots, w_d]^T$ là vector trọng số và b là hệ số dịch chuyển. Vector $\mathbf{w}$ vuông góc với siêu phẳng này và quyết định hướng của mặt phân chia.

Siêu phẳng trên chia không gian đặc trưng thành hai nửa không gian:

$$\mathbf{w}^T\mathbf{x} + b > 0 \tag{10.2}$$

và

$$\mathbf{w}^T\mathbf{x} + b < 0 \tag{10.3}$$

Các điểm dữ liệu nằm ở hai phía của siêu phẳng sẽ được gán vào hai lớp khác nhau.

Giả sử tập dữ liệu huấn luyện gồm các cặp

$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)$$

trong đó $\mathbf{x}_i \in \mathbb{R}^d$ là vector đặc trưng và $y_i \in \{-1, 1\}$ là nhãn lớp. Một siêu phẳng được xem là phân chia đúng dữ liệu nếu với mọi điểm dữ liệu ta có

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) > 0, \quad i = 1, \ldots, N. \tag{10.4}$$

Điều kiện này đảm bảo rằng tất cả các điểm thuộc lớp $y_i = 1$ nằm về một phía của siêu phẳng và các điểm thuộc lớp $y_i = -1$ nằm về phía còn lại.

Sau khi tìm được siêu phẳng phân chia, nhãn của một điểm dữ liệu mới $\mathbf{x}$ có thể được dự đoán thông qua hàm dấu

$$\text{class}(\mathbf{x}) = \text{sgn}(\mathbf{w}^T\mathbf{x} + b). \tag{10.5}$$

Nếu $\mathbf{w}^T\mathbf{x}+b > 0$ thì điểm dữ liệu được gán vào lớp $+1$, ngược lại nếu $\mathbf{w}^T\mathbf{x}+b < 0$ thì điểm đó được gán vào lớp -1.

Tuy nhiên, trong nhiều trường hợp có thể tồn tại vô số siêu phẳng khác nhau phân chia chính xác hai lớp dữ liệu. Điều này đặt ra câu hỏi quan trọng: *siêu*

phẳng nào là tốt nhất? Máy vector hỗ trợ giải quyết vấn đề này bằng cách lựa chọn siêu phẳng sao cho khoảng cách từ siêu phẳng tới các điểm dữ liệu gần nhất của hai lớp là lớn nhất. Khái niệm này được gọi là *lề* (margin) và sẽ được trình bày chi tiết trong phần tiếp theo.

10.1.3. Lề phân loại (Margin)

Khi hai lớp dữ liệu là tách biệt tuyến tính, tồn tại vô số siêu phẳng có thể phân loại chính xác các điểm dữ liệu. Tuy nhiên, các siêu phẳng này không có khả năng tổng quát hóa giống nhau. Một tiêu chí quan trọng để lựa chọn siêu phẳng phù hợp là *lề* (margin).

Giả sử siêu phẳng phân tách có dạng

$$\mathbf{w}^T\mathbf{x} + b = 0 \tag{10.6}$$

Khoảng cách từ một điểm dữ liệu $\mathbf{x}_i$ đến siêu phẳng này được tính bởi

$$d_i = \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{\|\mathbf{w}\|_2} \tag{10.7}$$

Trong bài toán phân loại nhị phân với nhãn $y_i \in \{-1, 1\}$, khoảng cách có thể viết lại dưới dạng có dấu:

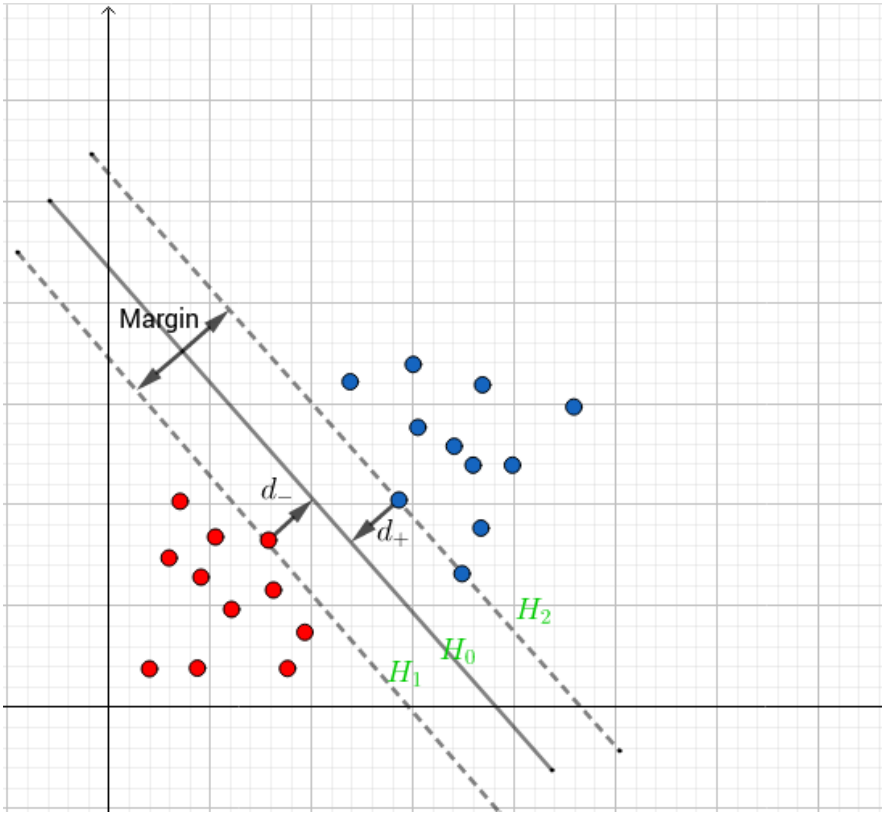
$$d_i = \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|_2} \tag{10.8}$$

Lề của bộ phân loại được định nghĩa là khoảng cách nhỏ nhất từ siêu phẳng phân tách đến các điểm dữ liệu trong tập huấn luyện:

$$\gamma = \min_i \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|_2} \tag{10.9}$$

Các điểm dữ liệu đạt giá trị nhỏ nhất trong biểu thức trên được gọi là *vector hỗ trợ* (support vectors). Những điểm này nằm gần siêu phẳng phân tách nhất và đóng vai trò quyết định trong việc xác định vị trí của siêu phẳng.

Hình 10.2 minh họa một siêu phẳng phân tách H_0 cùng với hai siêu phẳng song song H_1 và H_2 xác định vùng lề. Hai siêu phẳng biên này có phương trình



Hình 10.2. Minh họa siêu phẳng phân tách và vùng lề trong SVM.

$$\mathbf{w}^T\mathbf{x} + b = 1 \tag{10.10}$$

và

$$\mathbf{w}^T\mathbf{x} + b = -1 \tag{10.11}$$

Khoảng cách giữa hai siêu phẳng này chính là độ rộng của lề:

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|_2} \tag{10.12}$$

Một siêu phẳng có lề lớn tạo ra khoảng cách an toàn lớn hơn giữa hai lớp dữ liệu, giúp mô hình ít nhạy cảm hơn với nhiễu và cải thiện khả năng tổng quát hóa. Do đó, mục tiêu của máy vector hỗ trợ là tìm siêu phẳng phân tách sao cho lề đạt giá trị lớn nhất.

10.1.4. Bài toán tối ưu của SVM

Giả sử tập dữ liệu huấn luyện gồm N cặp dữ liệu

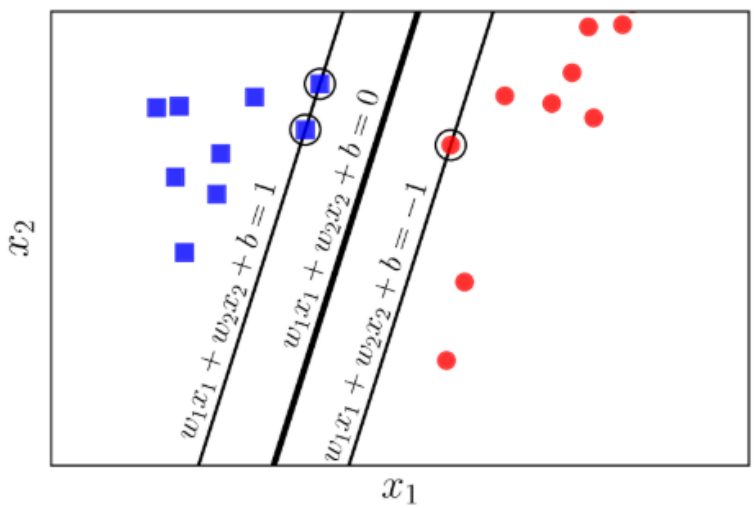
$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)$$

trong đó $\mathbf{x}_i \in \mathbb{R}^d$ là vector đặc trưng của điểm dữ liệu thứ i và y_i là nhãn tương ứng. Giá trị d biểu thị số chiều của không gian dữ liệu. Trong bài toán phân loại nhị phân, ta giả sử

$$y_i \in \{+1, -1\}$$

tương tự như cách ký hiệu đã sử dụng trong thuật toán PLA.

Để giúp dễ hình dung, trước tiên ta xét trường hợp dữ liệu trong không gian hai chiều. Các kết quả thu được hoàn toàn có thể tổng quát cho không gian nhiều chiều.



Hình 10.3. Minh họa siêu phẳng phân tách và vùng lề trong SVM. Hai đường song song $x^T w + b = 1$ và $x^T w + b = -1$ xác định vùng lề, trong khi đường ở giữa $x^T w + b = 0$ là siêu phẳng phân chia. Các điểm được khoanh tròn là các *vector hỗ trợ*.

Giả sử rằng các điểm vuông xanh thuộc lớp $+1$, các điểm tròn đỏ thuộc lớp -1 và siêu phẳng:

$$\mathbf{w}^T\mathbf{x} + b = 0 \tag{10.13}$$

là mặt phân chia giữa hai lớp dữ liệu. Không mất tính tổng quát, ta giả sử lớp $+1$ nằm về phía dương và lớp -1 nằm về phía âm của siêu phẳng này. Nếu xảy ra trường hợp ngược lại, ta chỉ cần đổi dấu của $\mathbf{w}$ và b.

Với một điểm dữ liệu bất kỳ $(\mathbf{x}_n, y_n)$, khoảng cách từ điểm đó tới siêu phẳng (10.13) được xác định bởi:

$$d_n = \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{\|\mathbf{w}\|_2}. \tag{10.14}$$

Biểu thức này luôn không âm vì theo giả thiết, y_n cùng dấu với $(\mathbf{w}^T\mathbf{x}_n + b)$.

Lề (margin) của bộ phân loại được định nghĩa là khoảng cách nhỏ nhất từ một điểm dữ liệu trong tập huấn luyện tới siêu phẳng phân chia:

$$\gamma = \min_n \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{\|\mathbf{w}\|_2}. \tag{10.15}$$

Ý tưởng cơ bản của máy vector hỗ trợ là tìm siêu phẳng phân chia sao cho lề này đạt giá trị lớn nhất. Khi đó bài toán tối ưu của SVM có thể viết dưới dạng:

$$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b} \left\{ \min_n \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{\|\mathbf{w}\|_2} \right\}. \tag{10.16}$$

Ta nhận thấy rằng nếu thay $\mathbf{w}$ bởi $k\mathbf{w}$ và b bởi kb với $k > 0$, siêu phẳng phân chia không thay đổi. Do đó, ta có thể chuẩn hóa các tham số sao cho các điểm nằm gần siêu phẳng nhất thỏa mãn:

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) = 1. \tag{10.17}$$

Khi đó với mọi điểm dữ liệu trong tập huấn luyện ta có:

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1, \quad n = 1, 2, \ldots, N. \tag{10.18}$$

Từ đây bài toán tối ưu (10.16) có thể được viết lại dưới dạng

$$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|_2} \quad \text{subject to } y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1. \tag{10.19}$$

Để thuận tiện cho việc tối ưu, bài toán trên thường được chuyển thành bài toán tương đương:

$$(\mathbf{w}, b) = \arg\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2 \tag{10.20}$$

với các ràng buộc:

$$1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0, \quad n = 1, 2, \ldots, N. \tag{10.21}$$

Tóm gọn bài toán tối ưu SVM

Bài toán tìm siêu phẳng phân tách có lề lớn nhất có thể viết gọn dưới dạng

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2$$

với các ràng buộc

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1, \quad n = 1, 2, \ldots, N.$$

Nghiệm của bài toán này xác định siêu phẳng phân chia

$$\mathbf{w}^T\mathbf{x} + b = 0$$

có lề lớn nhất giữa hai lớp dữ liệu. Bài toán (10.20) *là một bài toán quy hoạch toàn phương (quadratic programming) với hàm mục tiêu lồi và các ràng buộc tuyến tính. Nghiệm của bài toán này chính là siêu phẳng phân tách có lề lớn nhất giữa hai lớp dữ liệu.*

Giải bài toán tối ưu của SVM

Bài toán tối ưu trong (10.20) *là một bài toán quy hoạch toàn phương (quadratic programming). Trong thực tế, bài toán này có thể được giải bằng các thuật toán tối ưu tiêu chuẩn. Tuy nhiên, thay vì giải trực tiếp dạng bài toán ban đầu (primal problem), người ta thường chuyển sang giải bài toán đối ngẫu (dual problem). Việc chuyển sang dạng đối ngẫu giúp bài toán có cấu trúc thuận lợi hơn và cho phép khai thác một tính chất quan trọng của SVM.*
Một kết quả thú vị là nghiệm của bài toán tối ưu không phụ thuộc vào toàn bộ tập dữ liệu huấn luyện mà chỉ phụ thuộc vào một số điểm dữ liệu nằm gần siêu phẳng phân tách nhất. Những điểm này được gọi là vector hỗ trợ (support vectors).
Sau khi giải bài toán tối ưu, vector trọng số của siêu phẳng phân tách có thể được biểu diễn dưới dạng tổ hợp tuyến tính của các vector hỗ trợ:

$$\mathbf{w} = \sum_{i \in \mathcal{S}} \lambda_i y_i \mathbf{x}_i$$

trong đó $\mathcal{S}$ là tập các vector hỗ trợ và λ_i là các hệ số không âm thu được từ quá trình tối ưu.
Sau khi xác định được $\mathbf{w}$ và b, việc phân loại một điểm dữ liệu mới $\mathbf{x}$ trở nên rất đơn giản thông qua dấu của biểu thức

$$class(\mathbf{x}) = sgn(\mathbf{w}^T\mathbf{x} + b).$$

Như vậy, mặc dù bài toán tối ưu của SVM có vẻ phức tạp về mặt toán học, kết quả cuối cùng lại rất trực quan: mô hình chỉ cần một số ít điểm dữ liệu quan trọng (vector hỗ trợ) để xác định ranh giới phân loại.

10.2. Thực hành: Xây dựng bộ phân loại SVM trên Python

Trong phần này chúng ta sẽ xây dựng một bộ phân loại SVM bằng thư viện `scikit-learn`. Để thuận tiện cho việc trực quan hóa ranh giới phân loại, ta chỉ sử dụng hai đặc trưng đầu tiên của bộ dữ liệu *Breast Cancer Wisconsin*. Khi đó dữ liệu có thể được biểu diễn trên mặt phẳng hai chiều và ta có thể quan sát trực tiếp ranh giới phân loại do SVM tìm được.

10.2.1. Chương trình Python

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
```

```
from sklearn.inspection import DecisionBoundaryDisplay


def load_dataset():
    """
    Ham tai bo du lieu breast cancer
    Tra ve:
        X : ma tran dac trung
        y : nhan phan loai
        feature_names : ten cac dac trung
    """

    cancer = load_breast_cancer()

    # chi lay 2 dac trung dau tien de ve do thi 2 chieu
    X = cancer.data[:, :2]
    y = cancer.target
    feature_names = cancer.feature_names[:2]

    print("Dataset shape:", X.shape)
    print("Classes:", np.unique(y))

    return X, y, feature_names


def train_svm(X, y):
    """
    Ham huan luyen mo hinh SVM
    """

    # khoi tao mo hinh SVM voi kernel tuyen tinh
    model = SVC(kernel="linear", C=1)

    # huan luyen mo hinh
    model.fit(X, y)

    # in ra so support vectors
    print("So support vectors:",
          model.support_vectors_.shape[0])

    return model


def plot_decision_boundary(model, X, y, feature_names):
    """
    Ham ve ranh gioi phan loai cua SVM
    """

    plt.figure(figsize=(8,6))

    # ve vung phan loai
    DecisionBoundaryDisplay.from_estimator(
        model,
        X,
        response_method="predict",
        alpha=0.8,
        cmap="Pastel1",
        xlabel=feature_names[0],
```

```
        ylabel=feature_names[1],
    )

    # ve cac diem du lieu
    plt.scatter(X[:,0], X[:,1],
                c=y,
                s=30,
                edgecolors="k")

    # ve cac support vectors
    plt.scatter(model.support_vectors_[:,0],
                model.support_vectors_[:,1],
                s=120,
                facecolors="none",
                edgecolors="red",
                label="Support Vectors")

    plt.title("Decision Boundary cua Linear SVM")
    plt.legend()

    plt.show()


def main():

    # tai du lieu
    X, y, feature_names = load_dataset()

    # huan luyen mo hinh
    svm_model = train_svm(X, y)

    # ve ket qua
    plot_decision_boundary(svm_model, X, y, feature_names)


if __name__ == "__main__":
    main()
```

10.3. Kết luận

Trong chương này, chúng ta đã tìm hiểu về thuật toán *máy vector hỗ trợ* (Support Vector Machine – SVM), một trong những phương pháp phân loại tuyến tính mạnh và phổ biến trong học máy.

Trước hết, chương đã giới thiệu ý tưởng hình học của SVM thông qua bài toán tìm siêu phẳng phân tách hai lớp dữ liệu. Khác với nhiều thuật toán phân loại tuyến tính khác, SVM lựa chọn siêu phẳng sao cho khoảng cách từ siêu phẳng đến các điểm dữ liệu gần nhất của hai lớp là lớn nhất. Khoảng cách này được gọi là *lề* (margin) của bộ phân loại.

Tiếp theo, chúng ta đã xây dựng bài toán tối ưu tương ứng của SVM. Bài toán này có thể được đưa về dạng quy hoạch toàn phương với hàm mục tiêu là tối

thiểu hóa chuẩn của vector trọng số và các ràng buộc đảm bảo các điểm dữ liệu được phân loại đúng.

Một đặc điểm quan trọng của SVM là nghiệm của bài toán tối ưu không phụ thuộc vào toàn bộ tập dữ liệu huấn luyện mà chỉ phụ thuộc vào một số ít điểm dữ liệu nằm gần ranh giới phân loại nhất. Những điểm này được gọi là *vector hỗ trợ* (support vectors) và chúng quyết định vị trí của siêu phẳng phân tách.

Cuối cùng, chương cũng đã trình bày cách triển khai mô hình SVM trên Python bằng thư viện `scikit-learn` và trực quan hóa ranh giới phân loại cùng các vector hỗ trợ trên dữ liệu hai chiều.

10.4. Câu hỏi ôn tập

a. Trình bày ý tưởng cơ bản của thuật toán máy vector hỗ trợ (Support Vector Machine). Vì sao việc tối đa hóa lề (margin) giúp cải thiện khả năng tổng quát hóa của mô hình?

b. Giải thích khái niệm siêu phẳng phân tách (separating hyperplane). Trong không gian d chiều, phương trình tổng quát của siêu phẳng này có dạng như thế nào?

c. Lề (margin) của bộ phân loại SVM được định nghĩa như thế nào? Hãy viết công thức tính khoảng cách từ một điểm dữ liệu đến siêu phẳng phân tách.

d. Trình bày bài toán tối ưu của SVM dưới dạng quy hoạch toàn phương. Giải thích ý nghĩa của các ràng buộc trong bài toán này.

e. Vector hỗ trợ (support vectors) là gì? Vì sao nghiệm của SVM chỉ phụ thuộc vào một số ít điểm dữ liệu trong tập huấn luyện?

f. Kernel trong SVM là gì? Trình bày ý tưởng của *kernel trick*. Phân biệt hai loại kernel phổ biến là *linear kernel* và *RBF kernel*.

g. (Thực hành) Sử dụng thư viện `scikit-learn` trong Python để xây dựng một mô hình SVM phân loại trên bộ dữ liệu *Iris*.

- Chọn hai đặc trưng để trực quan hóa dữ liệu trên mặt phẳng hai chiều.
- Huấn luyện mô hình SVM với kernel tuyến tính.
- Vẽ ranh giới phân loại và xác định các vector hỗ trợ.

h. (Thực hành) Sử dụng bộ dữ liệu *Breast Cancer Wisconsin* trong `scikit-learn`.

- Huấn luyện hai mô hình SVM với kernel tuyến tính và kernel RBF.
- So sánh ranh giới phân loại của hai mô hình.

Chương 11

Phân cụm K-means

Trong nhiều bài toán học máy, dữ liệu không được gán nhãn sẵn mà chỉ bao gồm các quan sát chưa được phân loại. Khi đó, mục tiêu của chúng ta là tìm ra cấu trúc tiềm ẩn trong dữ liệu và nhóm các điểm dữ liệu có đặc điểm tương tự lại với nhau. Bài toán này được gọi là *phân cụm* (clustering).

Một trong những thuật toán phân cụm phổ biến và đơn giản nhất là *K-means*. Thuật toán này chia tập dữ liệu thành K cụm sao cho các điểm dữ liệu trong cùng một cụm có mức độ tương đồng cao, trong khi các điểm thuộc các cụm khác nhau có sự khác biệt lớn.

11.1. Giới thiệu

K-means là một thuật toán phân cụm phổ biến trong học máy không giám sát (*unsupervised learning*). Mục tiêu của thuật toán là chia tập dữ liệu thành K cụm sao cho các điểm dữ liệu trong cùng một cụm có mức độ tương đồng cao, trong khi các điểm thuộc các cụm khác nhau có sự khác biệt lớn.

Ý tưởng chính của K-means là biểu diễn mỗi cụm bằng một điểm trung tâm (*centroid*). Các điểm dữ liệu sẽ được gán vào cụm có tâm gần nhất theo một độ đo khoảng cách, thường là khoảng cách Euclid. Sau khi các điểm được gán vào các cụm, vị trí của các tâm cụm sẽ được cập nhật dựa trên giá trị trung bình của các điểm dữ liệu trong cụm đó.

Quá trình gán điểm dữ liệu vào các cụm và cập nhật tâm cụm được lặp lại nhiều lần cho đến khi các tâm cụm không còn thay đổi đáng kể hoặc khi đạt đến số vòng lặp tối đa. Kết quả cuối cùng là tập dữ liệu được phân thành K cụm với các tâm cụm tương ứng.

11.1.1. Ví dụ: Phân nhóm khách hàng

Một ứng dụng phổ biến của thuật toán K-means trong thực tế là phân nhóm khách hàng trong lĩnh vực kinh doanh. Giả sử một cửa hàng bán lẻ thu thập dữ liệu về khách hàng bao gồm hai đặc trưng: tổng số tiền chi tiêu trong năm và số lần mua hàng.

Mỗi khách hàng có thể được biểu diễn bằng một điểm dữ liệu trong không gian hai chiều. Khi áp dụng thuật toán K-means với $K = 3$, hệ thống có thể tự động chia khách hàng thành ba nhóm khác nhau, ví dụ:

- Nhóm khách hàng chi tiêu cao và mua hàng thường xuyên.
- Nhóm khách hàng chi tiêu trung bình.
- Nhóm khách hàng chi tiêu thấp và ít mua hàng.

Thông tin này rất hữu ích cho doanh nghiệp trong việc xây dựng chiến lược marketing, chẳng hạn như thiết kế chương trình khuyến mãi riêng cho từng nhóm khách hàng.

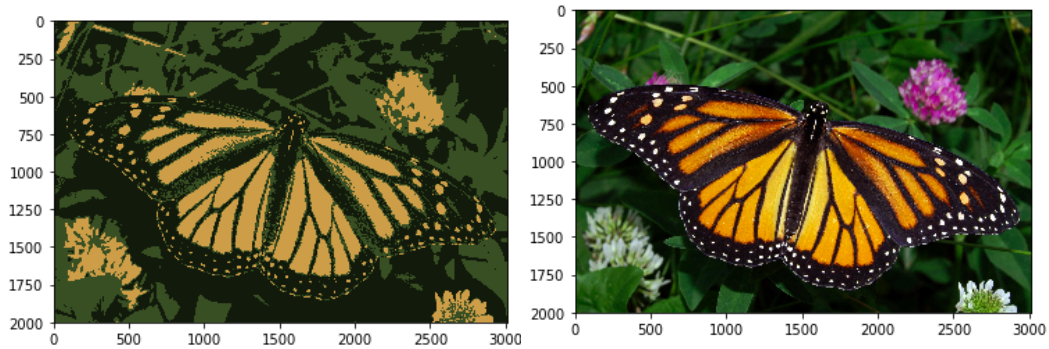
11.1.2. Ví dụ: Phân cụm ảnh (Image Segmentation)

Một ứng dụng quan trọng của thuật toán K-means trong thị giác máy tính là bài toán *phân cụm ảnh* (image segmentation). Mục tiêu của bài toán này là chia một bức ảnh thành nhiều vùng khác nhau sao cho các pixel trong cùng một vùng có đặc điểm tương tự nhau, ví dụ như màu sắc hoặc độ sáng.

Về bản chất, một bức ảnh số có thể được xem như một tập hợp các pixel. Mỗi pixel được mô tả bởi các giá trị màu sắc, chẳng hạn như ba thành phần trong hệ màu RGB. Khi đó, mỗi pixel có thể được xem như một điểm dữ liệu trong không gian ba chiều (R, G, B). Thuật toán K-means có thể được sử dụng để nhóm các pixel có màu sắc tương tự lại với nhau thành K cụm.

Quá trình phân cụm được thực hiện theo các bước sau:

- Chọn ngẫu nhiên K tâm cụm ban đầu trong không gian màu.
- Gán mỗi pixel của ảnh vào cụm có tâm gần nhất.
- Cập nhật lại tâm cụm bằng trung bình của các pixel trong cụm đó.
- Lặp lại hai bước trên cho đến khi các tâm cụm không thay đổi đáng kể.



(a) Ảnh sau khi phân cụm bằng K-means **(b)** Ảnh gốc

Hình 11.1. Ví dụ minh họa bài toán phân cụm ảnh bằng thuật toán K-means.

Kết quả của quá trình này là các pixel có màu sắc tương tự sẽ được nhóm lại với nhau. Nhờ đó, ảnh ban đầu có thể được chia thành các vùng màu rõ ràng hơn, giúp làm nổi bật cấu trúc và các đối tượng trong ảnh.

Hình 11.1 minh họa một ví dụ về phân cụm ảnh bằng thuật toán K-means. Hình bên phải là ảnh gốc của một con bướm (`kmeans3.png`). Sau khi áp dụng thuật toán K-means để phân cụm các pixel theo màu sắc, ta thu được ảnh ở bên trái (`kmeans1.png`). Có thể thấy rằng các vùng màu tương tự trong ảnh, chẳng hạn như cánh bướm, nền lá cây hoặc hoa, đã được nhóm lại thành các cụm riêng biệt.

Kỹ thuật phân cụm ảnh được sử dụng rộng rãi trong nhiều ứng dụng của thị giác máy tính như nhận dạng đối tượng, phân tích ảnh y tế, phát hiện biên hoặc tiền xử lý dữ liệu cho các mô hình học sâu.

11.2. Phân tích toán học

11.2.1. Hàm mất mát và bài toán tối ưu

Gọi $\mathbf{m}_k \in \mathbb{R}^d$ là tâm của cụm thứ k. Với mỗi điểm dữ liệu $\mathbf{x}_i$, ta ký hiệu $c_i \in \{1, 2, \ldots, K\}$ là chỉ số của cụm mà điểm đó được gán vào. Khi đó, khoảng cách giữa điểm dữ liệu và tâm cụm tương ứng được đo bằng khoảng cách Euclid

$$\|\mathbf{x}_i - \mathbf{m}_{c_i}\|_2^2. \tag{11.1}$$

Mục tiêu của thuật toán K-means là tìm các tâm cụm và cách gán điểm dữ liệu vào các cụm sao cho tổng sai số bình phương khoảng cách trong mỗi cụm là nhỏ nhất. Khi đó hàm mất mát có thể được viết dưới dạng

$$J(\mathbf{M}, \mathbf{c}) = \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{m}_{c_i}\|_2^2 \tag{11.2}$$

trong đó $\mathbf{M} = [\mathbf{m}_1, \mathbf{m}_2, \ldots, \mathbf{m}_K]$ là tập các tâm cụm và $\mathbf{c} = (c_1, c_2, \ldots, c_N)$ là vector chỉ số cụm của các điểm dữ liệu.

Nếu sử dụng cách biểu diễn nhãn dưới dạng ma trận $\mathbf{Y}$ với mã hoá one-hot như đã trình bày trước đó, hàm mất mát cũng có thể được viết dưới dạng

$$\mathcal{L}(\mathbf{Y}, \mathbf{M}) = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2. \tag{11.3}$$

Bài toán phân cụm K-means khi đó trở thành bài toán tối ưu

$$\begin{aligned} \min_{\mathbf{Y}, \mathbf{M}} \quad & \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \\ \text{thoả mãn:} \quad & y_{ij} \in \{0, 1\}, \quad \sum_{j=1}^{K} y_{ij} = 1. \end{aligned} \tag{11.4}$$

11.2.2. Giải bài toán tối ưu

Bài toán (11.4) là một bài toán tối ưu có ràng buộc với các biến rời rạc nên rất khó tìm nghiệm tối ưu toàn cục. Trong thực tế, thuật toán K-means sử dụng một chiến lược tối ưu lặp đơn giản bằng cách xen kẽ tối ưu hai nhóm biến $\mathbf{Y}$ và $\mathbf{M}$.

Cố định tâm cụm, cập nhật nhãn

Giả sử các tâm cụm $\mathbf{M}$ đã được xác định. Khi đó việc tìm nhãn cho từng điểm dữ liệu trở thành bài toán

$$c_i = \arg\min_k \|\mathbf{x}_i - \mathbf{m}_k\|_2^2. \tag{11.5}$$

Nói cách khác, mỗi điểm dữ liệu sẽ được gán vào cụm có tâm gần nó nhất. Quy tắc này tương đương với việc xác định các phần tử y_{ij} trong ma trận nhãn $\mathbf{Y}$.

Cố định nhãn, cập nhật tâm cụm

Ngược lại, nếu các nhãn của điểm dữ liệu đã được xác định, bài toán tìm tâm cụm mới trở thành

$$\mathbf{m}_k = \arg\min_{\mathbf{m}_k} \sum_{i:c_i=k} \|\mathbf{x}_i - \mathbf{m}_k\|_2^2. \tag{11.6}$$

Giải bài toán này bằng cách lấy đạo hàm theo $\mathbf{m}_k$ và cho bằng 0 ta thu được

$$\mathbf{m}_k = \frac{1}{N_k} \sum_{i:c_i=k} \mathbf{x}_i \tag{11.7}$$

trong đó N_k là số điểm dữ liệu thuộc cụm k.

Kết quả này cho thấy rằng *tâm của mỗi cụm chính là trung bình cộng của các điểm dữ liệu trong cụm đó*, và đây cũng chính là lý do thuật toán được gọi là *K-means*.

Tóm tắt thuật toán K-means

Giả sử ta có tập dữ liệu huấn luyện $\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(m)}\}$, trong đó $\mathbf{x}^{(i)} \in \mathbb{R}^n$ là vector đặc trưng của điểm dữ liệu thứ i. Bài toán phân cụm K-means nhằm tìm K tâm cụm $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_K$ và nhãn cụm $c^{(i)}$ cho mỗi điểm dữ liệu.
Thuật toán được thực hiện theo các bước lặp sau:
***Bước 1. Khởi tạo** ngẫu nhiên các tâm cụm*

$$\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^n.$$

Bước 2. Lặp cho đến khi hội tụ

- *Với mỗi điểm dữ liệu $\mathbf{x}^{(i)}$, gán nó vào cụm có tâm gần nhất:*

$$c^{(i)} = \arg\min_j \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2.$$

- *Với mỗi cụm j, cập nhật tâm cụm bằng trung bình của các điểm dữ liệu thuộc cụm đó:*

$$\boldsymbol{\mu}_j = \frac{\sum_{i=1}^m \mathbf{1}\{c^{(i)} = j\}\mathbf{x}^{(i)}}{\sum_{i=1}^m \mathbf{1}\{c^{(i)} = j\}}.$$

Thuật toán dừng khi các tâm cụm không còn thay đổi đáng kể hoặc khi việc gán điểm dữ liệu vào các cụm không thay đổi giữa hai lần lặp liên tiếp.

11.3. Lựa chọn số lượng cụm tối ưu

Trong thuật toán K-means, một vấn đề quan trọng là xác định số lượng cụm K phù hợp. Trong nhiều trường hợp thực tế, giá trị K không được biết trước và

cần phải được lựa chọn dựa trên cấu trúc của dữ liệu. Nếu K quá nhỏ, các cụm có thể chứa nhiều điểm dữ liệu khác biệt; ngược lại, nếu K quá lớn, dữ liệu có thể bị chia thành quá nhiều cụm nhỏ không có ý nghĩa. Vì vậy, việc lựa chọn số lượng cụm tối ưu là một bước quan trọng trong quá trình phân cụm.

Hai phương pháp phổ biến để lựa chọn số lượng cụm là *phương pháp Elbow* và *chỉ số Silhouette*.

11.3.1. Phương pháp Elbow

Phương pháp Elbow dựa trên việc quan sát sự thay đổi của tổng sai số bình phương trong các cụm khi thay đổi số lượng cụm K. Hàm mất mát của thuật toán K-means có dạng:

$$J(K) = \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{m}_{c_i}\|_2^2 \tag{11.8}$$

trong đó $\mathbf{m}_{c_i}$ là tâm của cụm chứa điểm dữ liệu $\mathbf{x}_i$.

Khi số lượng cụm K tăng, giá trị của hàm mất mát $J(K)$ sẽ giảm vì mỗi cụm chứa ít điểm dữ liệu hơn và các điểm sẽ gần tâm cụm hơn. Tuy nhiên, sau một giá trị K nhất định, tốc độ giảm của $J(K)$ sẽ chậm lại. Điểm mà tại đó đồ thị bắt đầu có dạng "khuỷu tay" (elbow) thường được xem là số lượng cụm hợp lý.

Quy trình áp dụng phương pháp Elbow:

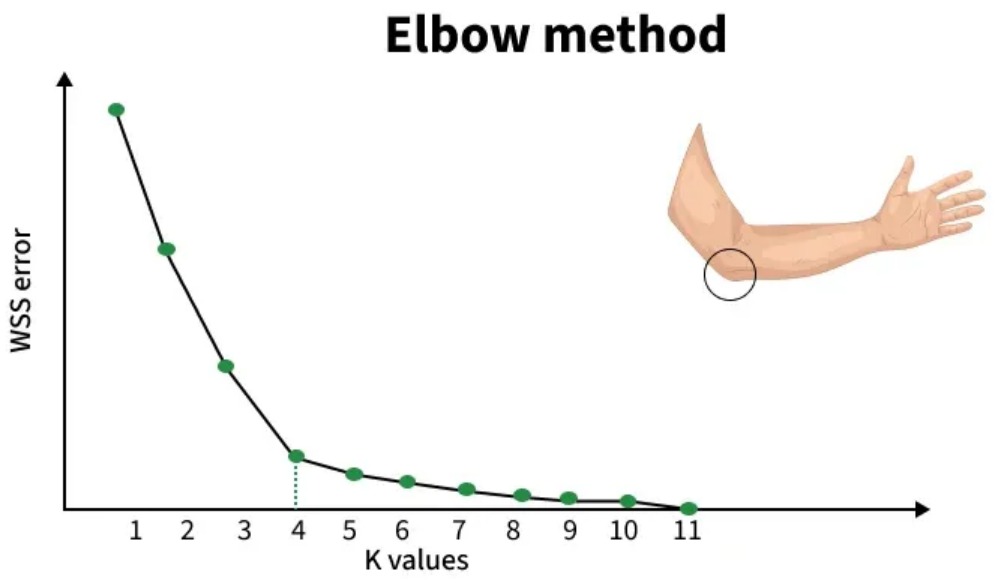
a. Chạy thuật toán K-means với nhiều giá trị khác nhau của K.

b. Tính giá trị hàm mất mát $J(K)$ cho mỗi giá trị.

c. Vẽ đồ thị $J(K)$ theo K.

d. Chọn giá trị K tại vị trí "khuỷu tay" của đồ thị.

Hình 11.2 minh họa nguyên lý của phương pháp *Elbow*. Trục ngang biểu diễn số lượng cụm K, trong khi trục đứng biểu diễn tổng sai số bình phương trong cụm (Within-Cluster Sum of Squares – WSS), tức tổng khoảng cách bình phương từ mỗi điểm dữ liệu tới tâm cụm của nó.



Hình 11.2. Minh hoạ phương pháp Elbow để lựa chọn số cụm K.

Khi số lượng cụm K tăng lên, giá trị WSS luôn giảm vì mỗi cụm chứa ít điểm dữ liệu hơn và các điểm dữ liệu sẽ gần với tâm cụm hơn. Tuy nhiên, sau một giá trị K nhất định, tốc độ giảm của WSS trở nên chậm hơn. Trên đồ thị, điểm chuyển tiếp này tạo thành một "góc gấp" giống như hình khuỷu tay của cánh tay người, do đó phương pháp này được gọi là *Elbow method*.

Trong Hình 11.2, ta có thể thấy rằng khi K tăng từ 1 đến khoảng 4, giá trị WSS giảm rất nhanh. Sau đó, khi K tiếp tục tăng, mức giảm của WSS không còn đáng kể. Vì vậy, giá trị $K = 4$ có thể được xem là số lượng cụm hợp lý cho tập dữ liệu trong ví dụ này.

Phương pháp Elbow có ưu điểm là trực quan và dễ áp dụng. Tuy nhiên, trong một số trường hợp đồ thị không xuất hiện "khuỷu tay" rõ ràng, khi đó cần kết hợp với các phương pháp khác như chỉ số Silhouette để đưa ra quyết định tốt hơn.

11.3.2. Chỉ số Silhouette

Một phương pháp khác để đánh giá chất lượng phân cụm là *chỉ số Silhouette*. Phương pháp này đo mức độ tương đồng của một điểm dữ liệu với các điểm trong cùng cụm so với các cụm khác.

Với mỗi điểm dữ liệu $\mathbf{x}_i$, ta định nghĩa:

- $a(i)$ là khoảng cách trung bình từ $\mathbf{x}_i$ đến tất cả các điểm khác trong cùng cụm.
- $b(i)$ là khoảng cách trung bình nhỏ nhất từ $\mathbf{x}_i$ đến các điểm thuộc cụm gần nhất khác với cụm của nó.

Chỉ số Silhouette của điểm dữ liệu $\mathbf{x}_i$ được tính như sau:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \tag{11.9}$$

Giá trị của $s(i)$ nằm trong khoảng $[-1, 1]$.

- Nếu $s(i)$ gần 1, điểm dữ liệu được phân cụm tốt.
- Nếu $s(i)$ gần 0, điểm dữ liệu nằm gần ranh giới giữa hai cụm.
- Nếu $s(i)$ âm, điểm dữ liệu có thể đã bị phân vào sai cụm.

Chỉ số Silhouette của toàn bộ dữ liệu được tính bằng trung bình của $s(i)$:

$$S = \frac{1}{N} \sum_{i=1}^{N} s(i) \tag{11.10}$$

Trong thực tế, ta chạy thuật toán K-means với nhiều giá trị khác nhau của K và chọn giá trị K cho chỉ số Silhouette trung bình lớn nhất. Giá trị này thường phản ánh cấu trúc phân cụm tốt nhất của dữ liệu.

11.4. Thực hành với Python

Trong phần này, chúng ta sẽ minh họa cách xác định số lượng cụm tối ưu trong thuật toán K-means bằng phương pháp Elbow. Thực hành được thực hiện bằng thư viện `scikit-learn` trong Python.

11.4.1. Bước 1: Import các thư viện cần thiết

Trước tiên, chúng ta cần import các thư viện cần thiết cho việc xử lý dữ liệu, xây dựng mô hình và trực quan hóa kết quả.

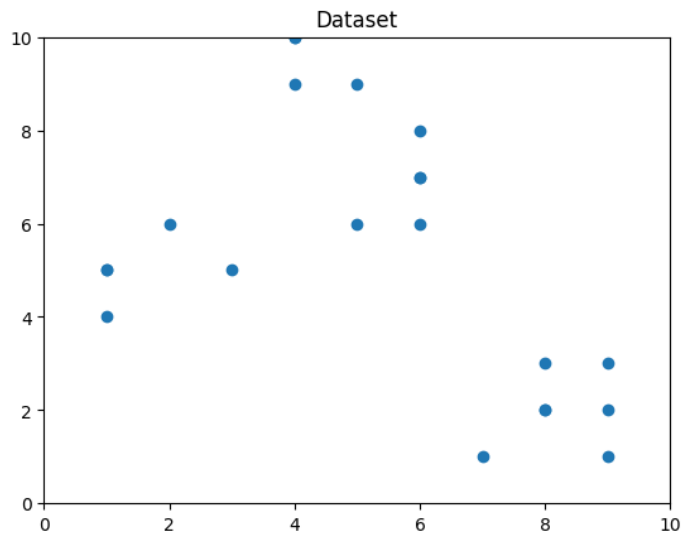
```
# Import cac thu vien can thiet
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
from scipy.spatial.distance import cdist
```

11.4.2. Bước 2: Tạo và trực quan hóa dữ liệu

Trong ví dụ này, chúng ta tạo một tập dữ liệu hai chiều đơn giản để minh họa quá trình phân cụm.

```
# Tao du lieu hai chieu don gian
x1 = np.array([3, 1, 1, 2, 1, 6, 6, 6, 5, 6,
               7, 8, 9, 8, 9, 9, 8, 4, 4, 5, 4])
x2 = np.array([5, 4, 5, 6, 5, 8, 6, 7, 6, 7,
               1, 2, 1, 2, 3, 2, 3, 9, 10, 9, 10])
# Ket hop hai mang thanh du lieu dau vao
X = np.array(list(zip(x1, x2))).reshape(len(x1), 2)
# Ham ve du lieu ban dau
def visualize_dataset(X):
    # Ve cac diem du lieu
    plt.scatter(X[:,0], X[:,1], marker='o')
    # Gioi han truc toa do
    plt.xlim([0,10])
    plt.ylim([0,10])
    # Dat tieu de va nhan truc
    plt.title("Dataset Visualization")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()
visualize_dataset(X)
```



Hình 11.3. Trực quan hóa dữ liệu

Quan sát đồ thị dữ liệu ban đầu, ta có thể dự đoán rằng dữ liệu có thể được chia thành khoảng ba cụm. Tuy nhiên, việc quan sát trực quan đôi khi không đủ chính xác, vì vậy cần sử dụng phương pháp Elbow để xác định giá trị K tối ưu.

11.4.3. Bước 3: Huấn luyện mô hình với nhiều giá trị K

Ở bước này, ta chạy thuật toán K-means với nhiều giá trị khác nhau của K và tính toán hai đại lượng quan trọng:

- **Distortion**: khoảng cách trung bình từ mỗi điểm dữ liệu tới tâm cụm gần nhất.
- **Inertia**: tổng bình phương khoảng cách từ các điểm dữ liệu tới tâm cụm của chúng.

```
# Ham tinh distortion va inertia cho nhieu gia tri K
def compute_elbow_metrics(X):

    distortions = []
    inertias = []

    mapping1 = {}
    mapping2 = {}

    K = range(1,10)

    for k in K:

        # Khoi tao mo hinh KMeans
        model = KMeans(n_clusters=k, random_state=42)
        model.fit(X)

        # Tinh distortion
        distortion = sum(
            np.min(cdist(X, model.cluster_centers_, 'euclidean'), axis=1)**2
        ) / X.shape[0]

        distortions.append(distortion)

        # Lay gia tri inertia
        inertia = model.inertia_
        inertias.append(inertia)

        mapping1[k] = distortion
        mapping2[k] = inertia

    return K, distortions, inertias, mapping1, mapping2


K, distortions, inertias, mapping1, mapping2 = compute_elbow_metrics(X)
```

11.4.4. Bước 4: Hiển thị kết quả và vẽ đồ thị Elbow

Sau khi tính toán các giá trị distortion và inertia, ta sẽ vẽ đồ thị để quan sát vị trí "khuỷu tay".

```
# Hien thi cac gia tri distortion
print("Distortion values:")
for key, val in mapping1.items():
    print(f"{key} : {val}")


# Ve do thi Elbow theo distortion
plt.plot(K, distortions, 'bx-')
plt.xlabel("Number of clusters (K)")
plt.ylabel("Distortion")
plt.title("Elbow Method using Distortion")
plt.show()
```

Tương tự, ta có thể vẽ đồ thị dựa trên giá trị inertia.

```
# Hien thi cac gia tri inertia
print("Inertia values:")
for key, val in mapping2.items():
    print(f"{key} : {val}")


# Ve do thi Elbow theo inertia
plt.plot(K, inertias, 'bx-')
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method using Inertia")
plt.show()
```

Quan sát đồ thị Elbow, ta sẽ tìm điểm mà tại đó đường cong bắt đầu giảm chậm lại. Điểm này thường được chọn làm số cụm tối ưu.

11.4.5. Bước 5: Quan sát kết quả phân cụm với các giá trị K khác nhau

Cuối cùng, ta có thể trực quan hóa kết quả phân cụm khi thay đổi số lượng cụm K.

```
# Ham ve ket qua phan cum voi cac gia tri K khac nhau
def visualize_clusters(X):

    k_range = range(1,5)

    for k in k_range:

        # Khoi tao mo hinh KMeans
        kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
```

```
        # Phan cum du lieu
        labels = kmeans.fit_predict(X)

        # Ve cac diem du lieu theo cum
        plt.scatter(X[:,0], X[:,1],
                    c=labels,
                    cmap='viridis',
                    marker='o',
                    edgecolor='k',
                    s=100)

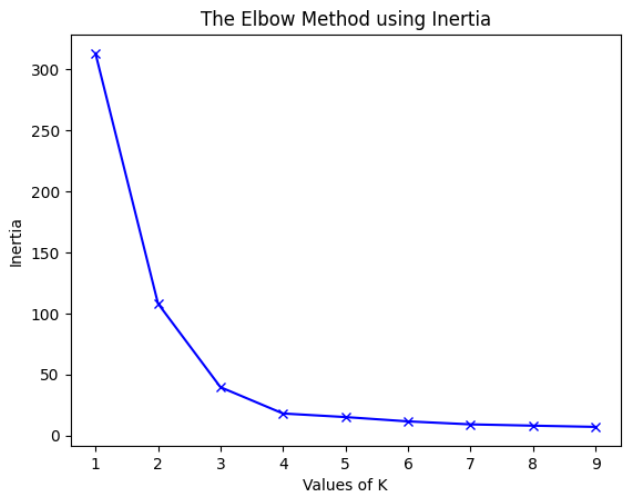
        # Ve tam cum
        plt.scatter(kmeans.cluster_centers_[:,0],
                    kmeans.cluster_centers_[:,1],
                    s=300,
                    c='red',
                    edgecolor='k',
                    label='Centroids')

        plt.title(f"K-means clustering (k = {k})")
        plt.xlabel("Feature 1")
        plt.ylabel("Feature 2")
        plt.legend()
        plt.grid()

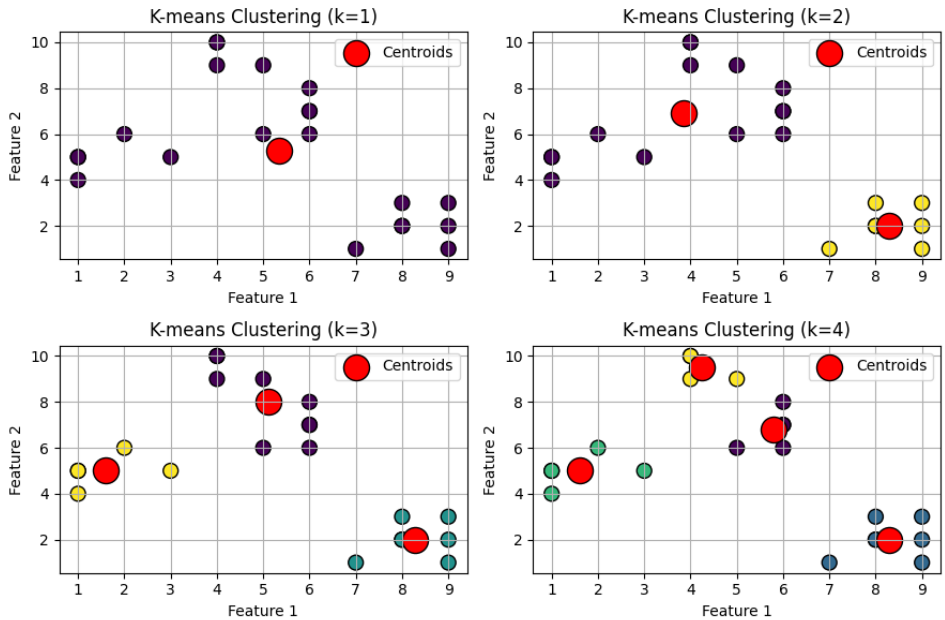
        plt.show()


visualize_clusters(X)
```

Thông qua các đồ thị này, ta có thể quan sát cách dữ liệu được phân cụm khi thay đổi giá trị K. Kết hợp với đồ thị Elbow, ta có thể lựa chọn số lượng cụm tối ưu cho tập dữ liệu.



Hình 11.4. Thay đổi K để xem đồ thị Elbow



Hình 11.5. Phân cụm với số lượng K khác nhau

11.5. Thực hành phân vùng ảnh bằng thuật toán K-means

Trong thị giác máy tính, *image segmentation* là quá trình chia một ảnh thành nhiều vùng khác nhau sao cho các pixel trong cùng một vùng có đặc trưng tương tự nhau (ví dụ màu sắc hoặc cường độ sáng). Một cách đơn giản để thực hiện điều này là sử dụng thuật toán phân cụm K-means.

Ý tưởng chính là xem mỗi pixel trong ảnh như một điểm dữ liệu trong không gian màu RGB. Thuật toán K-means sẽ nhóm các pixel có màu sắc tương tự vào cùng một cụm. Kết quả là ảnh sẽ được chia thành các vùng màu khác nhau.

Bước 1: Import các thư viện cần thiết

Trước tiên, chúng ta cần import các thư viện phục vụ cho việc xử lý ảnh và trực quan hóa dữ liệu.

```
# Import cac thu vien can thiet
import numpy as np
import matplotlib.pyplot as plt
import cv2
```

Bước 2: Đọc và hiển thị ảnh

Tiếp theo, chúng ta đọc ảnh từ file và hiển thị ảnh gốc.

```
# Ham doc va hien thi anh
def load_and_display_image(path):

    # Doc anh bang OpenCV
    image = cv2.imread(path)

    # Chuyen tu he mau BGR sang RGB
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Hien thi anh goc
    plt.imshow(image)
    plt.title("Original Image")
    plt.axis("off")
    plt.show()

    return image


# Doc anh tu file
image = load_and_display_image("butterfly.jpg")
```



Hình 11.6. Đọc ảnh gốc với OpenCV

Bước 3: Chuyển ảnh thành dữ liệu pixel

Thuật toán K-means làm việc với dữ liệu dạng ma trận hai chiều, vì vậy ta cần chuyển ảnh thành danh sách các pixel.

```
# Ham chuyen anh thanh danh sach pixel
def reshape_image(image):
```

```
    # Chuyen anh thanh ma tran pixel
    pixel_values = image.reshape((-1, 3))

    # Chuyen sang kieu float32 cho KMeans
    pixel_values = np.float32(pixel_values)

    return pixel_values


pixel_values = reshape_image(image)
```

Bước 4: Áp dụng thuật toán K-means

Sau khi có dữ liệu pixel, chúng ta tiến hành phân cụm bằng thuật toán K-means.

```
# Ham phan cum pixel bang KMeans
def kmeans_segmentation(pixel_values, k):

    # Dieu kien dung cua thuat toan
    criteria = (cv2.TERM_CRITERIA_EPS +
                cv2.TERM_CRITERIA_MAX_ITER,
                100, 0.85)

    # Thuc hien phan cum
    retval, labels, centers = cv2.kmeans(
        pixel_values,
        k,
        None,
        criteria,
        10,
        cv2.KMEANS_RANDOM_CENTERS
    )

    return labels, centers


labels, centers = kmeans_segmentation(pixel_values, 3)
```

Bước 5: Tái tạo ảnh sau khi phân cụm

Sau khi các pixel được gán vào các cụm, ta tái tạo lại ảnh bằng cách thay mỗi pixel bằng màu của tâm cụm tương ứng.

```
# Ham tao anh segmentation
def reconstruct_image(labels, centers, original_shape):

    # Chuyen cac tam cum ve dang so nguyen
    centers = np.uint8(centers)

    # Thay pixel bang mau tam cum
    segmented_data = centers[labels.flatten()]
```

```
    # Dua ve dang anh ban dau
    segmented_image = segmented_data.reshape(original_shape)

    return segmented_image


segmented_image = reconstruct_image(
    labels,
    centers,
    image.shape
)
```

Bước 6: Hiển thị kết quả phân vùng

Cuối cùng, chúng ta hiển thị ảnh sau khi đã được phân vùng.

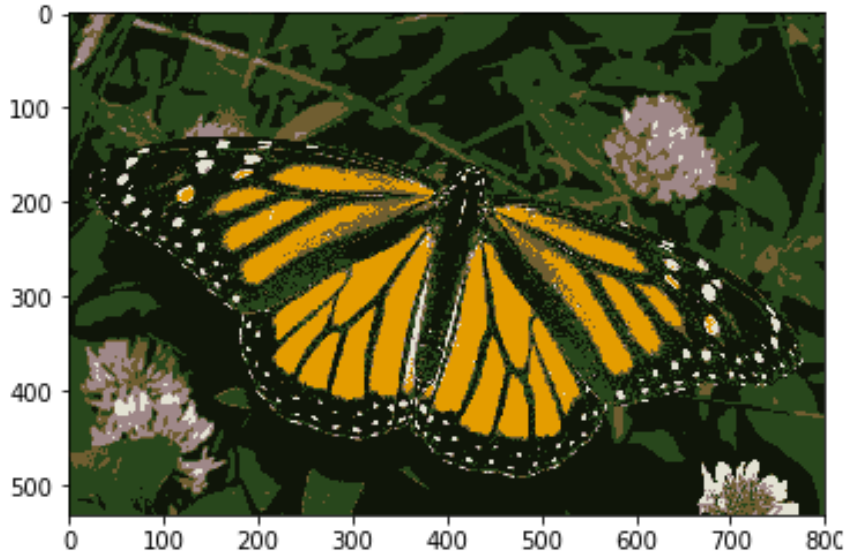
```
# Ham hien thi ket qua segmentation
def display_result(image):

    plt.imshow(image)
    plt.title("Segmented Image using KMeans")
    plt.axis("off")
    plt.show()


display_result(segmented_image)
```



Hình 11.7. Kết quả phân vùng với $K = 3$



Hình 11.8. Kết quả phân vùng với $K = 6$